

ZYNQ 开发平台 Linux 应用教程

2024/2/28 10:30:34



版权声明

Copyright ©2012-2018 芯驿电子科技（上海）有限公司

公司网址:

[Http://www.alinx.com.cn](http://www.alinx.com.cn)

技术论坛:

<http://www.heijin.org>

官方旗舰店:

<http://alinx.jd.com>

邮箱:

avic@alinx.com.cn

电话:

021-67676997

传真:

021-37737073

ALINX 微信公众号:



我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据论坛上大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程

文档修订记录:

版本	时间	描述
1.01	2024/1/12	2017v1.06 版本更新至 2023v1.01 版本

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 ZYNQ 的开发板！您对我们和我们产品的支持和信任,给我们增添了永往直前的信心和勇气。

“播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望，同时我们会在黑金动力社区 <http://www.heijin.org> 和大家一起讨论，一起学习，一起进步，一起成长。

通过前面的教程我们熟悉了 ZYNQ 开发的基本流程，学习了 Vitis 下 ZYNQ 程序的开发，但是很多应用可能需要使用嵌入式 Linux，由于 Linux 系统庞大，芯驿电子科技（上海）有限公司只能抛砖引玉，把最常用的知识点收集分享给大家。

目录

版权声明	2
前言	4
目录	5
基础篇	9
第一章 安装虚拟机和 Ubuntu 系统	10
1.1 虚拟机软件安装	10
1.2 Ubuntu 安装	11
1.2.1 安装系统	11
1.2.2 修改软件源服务器	16
1.2.3 设置 bash 为默认 sh	17
1.2.4 设置屏幕锁定时间	18
1.3 常见问题	19
1.3.1 虚拟机要求虚拟化支持	19
第二章 Ubuntu 安装 Linux 版 Vivado 软件	20
2.1 安装 Linux 版 Vivado	20
2.2 权限设置	24
2.3 安装下载器驱动	25
2.4 测试 Vivado	25
2.5 常见问题	27
2.5.1 Linux 下载器下载时提示被占用	27
2.5.2 适合 ZYNQ 的交叉编译器	28
第三章 Petalinux 工具安装	29
3.1 Petalinux 简介	29
3.2 安装必要的库	29
3.3 安装 Petalinux	31
第四章 NFS 服务软件安装	35
4.1 安装 NFS 服务	35
4.2 测试 NFS	37
4.3 常见问题	37
4.3.1 NFS 无法挂载	37
第五章 使用 Petalinux 定制 Linux 系统	39
5.1 Vivado 工程	错误! 未定义书签。
5.2 使用 Petalinux 建立工程	39
5.3 配置 Linux 内核	44

5.4 配置根文件系统	44
5.5 编译	52
5.6 生成 BOOT 文件	53
5.7 测试 Linux	53
5.8 常见问题	59
5.8.1 提示 “Bad FIT kernel image format!” 无法启动内核	59
5.8.2 无法保存文件和配置	59
第六章 使用 Vitis 开发 Linux 程序	61
6.1 使用 Vitis 建立 Linux 应用程序	61
6.2 通过 NFS 共享运行	67
6.3 通过 TCF-Agent 运行调试	错误! 未定义书签。
6.4 TCF-Agent 问题	错误! 未定义书签。
第七章 Linux 下 GPIO 实验	68
7.1 使用 SHELL 控制	68
7.2 使用 C 语言控制	69
7.2.1 如何确定 GPIO 的这个编号?	71
7.2.2 如何确定和物理 GPIO 的关系?	71
7.3 实验总结	71
第八章 Petalinux 下的 HDMI 显示	72
8.1 Petalinux 配置	72
8.2 配置 Linux 内核	74
8.3 修改设备树	77
8.4 编译测试 Petalinux 工程	79
8.5 常见问题	81
8.5.1 如何防止系统休眠	81
第九章 使用 Debian 桌面系统	82
9.1 Petalinux 配置	82
9.2 配置 Linux 内核	84
9.2.1 配置 USB WIFI 模块驱动	85
9.2.2 配置 USB 摄像头驱动	86
9.3 编译测试 Petalinux 工程	86
9.4 制作 SD 卡文件系统	87
9.4.1 SD 卡修改分区	87
9.4.2 同步根文件系统到 SD 卡 EXT4 分区	92
9.5 常见问题	94
第十章 制作 QSPI Flash 启动的 Linux	95
10.1 复制 Petalinux 工程	95
10.2 配置编译 Petalinux	95

应用篇	100
第十一章 QT 在 ZYNQ 上的应用	错误! 未定义书签。
11.1 QT Creator 安装	错误! 未定义书签。
11.2 QT 库交叉编译	错误! 未定义书签。
11.3 配置 ZYNQ 版 QT Kits	错误! 未定义书签。
11.4 创建 QT 测试工程	错误! 未定义书签。
11.5 运行 ZYNQ 版本 QT 程序	错误! 未定义书签。
11.6 常见问题	124
11.6.1 如何使用开发板 Debian 系统自带的 QT Creator	124
11.6.2 系统运行打印 I2C 错误信息	126
第十二章 OpenCV 应用 (USB 摄像头显示)	127
12.1 host 版 OpenCV 库编译	128
12.2 zynq 版 OpenCV 库编译	130
12.3 测试 host 版 OpenCV	131
12.4 测试 zynq 版 OpenCV	135
第十三章 双目摄像头模块 OpenCV 显示	140
13.1 vivado 工程简介	140
13.2 摄像头驱动	141
13.3 设备树配置	143
13.4 摄像头 Linux 应用程序	144
第十四章 触摸屏模块应用	148
14.1 vivado 工程简介	148
14.2 触摸屏驱动	149
14.3 内核驱动配置	150
14.4 设备树配置	152
14.5 触摸屏测试	152
第十五章 AXI DMA 读写测试	155
15.1 vivado 工程	155
15.2 petalinux 工程配置	156
15.3 运行测试	157
第十六章 基于 DMA 的 ADC 波形显示 (AN9238)	158
16.1 ADC 驱动	158
16.2 petalinux 工程配置	错误! 未定义书签。
16.3 设备树配置	159
16.4 应用程序	160
16.5 运行结果	160
16.6 常见问题	162
16.6.1 内核崩溃	162

第十七章 基于 DMA 的 ADC 波形显示 (AN706)	164
17.1 ADC 驱动	164
17.2 petalinux 工程配置	错误! 未定义书签。
17.3 设备树配置	165
17.4 应用程序	166
17.5 运行结果	166
第十八章 自动运行的 petalinux 应用程序	168
18.1 准备工作	168
18.2 startup 应用程序	168
18.3 使用 startup	错误! 未定义书签。
第十九章 附录: Linux 常用命令	170
19.1 文件和文件夹操作	170
19.1.1 创建一个文件夹	170
19.1.2 创建多层目录的文件夹	170
19.1.3 改变当前目录	170
19.1.4 复制文件	170
19.1.5 文件重命名	170
19.1.6 删除文件和文件夹	170
19.2 文件列表	171
19.3 压缩解压	171
19.3.1 创建一个压缩包	171
19.3.2 解压一个 tar.gz 文件	171
19.4 系统管理	171

基础篇

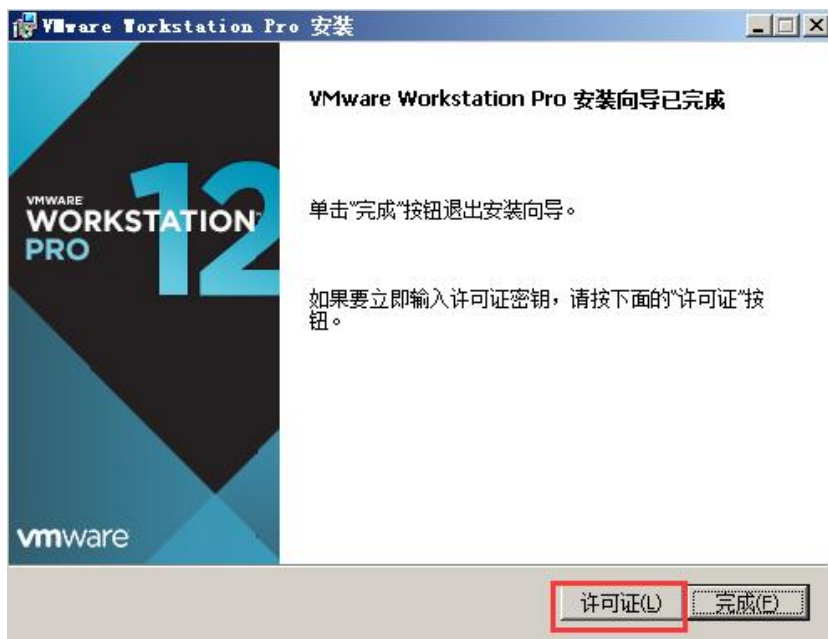
基础篇介绍安装 Linux 开发需要的相关软件，如何利用 Petalinu 定制 Linux 系统，以及 SD 卡和 QSPI 的启动等。

第一章 安装虚拟机和 Ubuntu 系统

后面的教程要涉及到嵌入式 Linux 开发, 一般需要一台 Linux 操作系统主机, 用来编译 u-boot 或者 Linux-kernel。在 Windows 操作系统的电脑上安装个虚拟机, 再在虚拟机上安装 Linux 操作系统是最便捷的方法了。

1.1 虚拟机软件安装

我们提供的虚拟机的安装软件版本为 VMware-workstation-full 12.1.1。用户可以在我们提供的资料里找到, 双击 “VMware-workstation-full-12.1.1-3770994.exe” 图标开始安装。因为安装比较简单, 所以具体的安装步骤我们这里不做具体介绍了, 用户只要按照默认安装项一直点 “Next” 按键来进行安装。安装完成的最后一个界面里, 我们需要选择许可证来输入一个 VMware12 的序列号。





安装完成后，桌面上有 VMware Workstation Pro 的图标。



1.2 Ubuntu 安装

1.2.1 安装系统

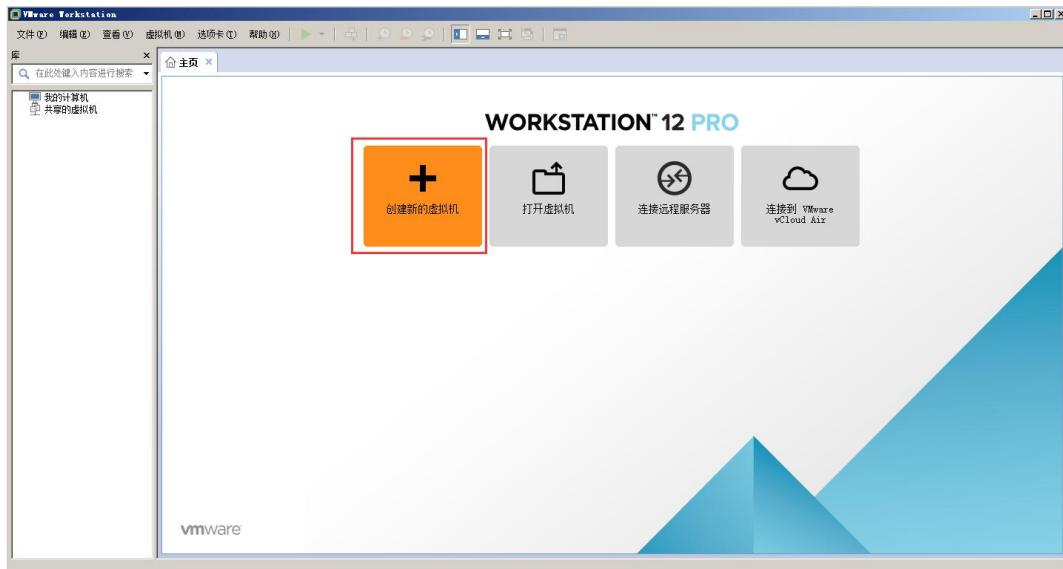
安装好虚拟机后，就要在虚拟机上安装 Linux 操作系统了。鉴于 ubuntu Linux 桌面操作系统的安装以及配置较为简单，所以我们选择了 ubuntu 桌面操作系统。

本教程使用 **Ubuntu 22.4.1** LTS 64 位操作系统 (ubuntu-22.4.1-desktop-amd64)

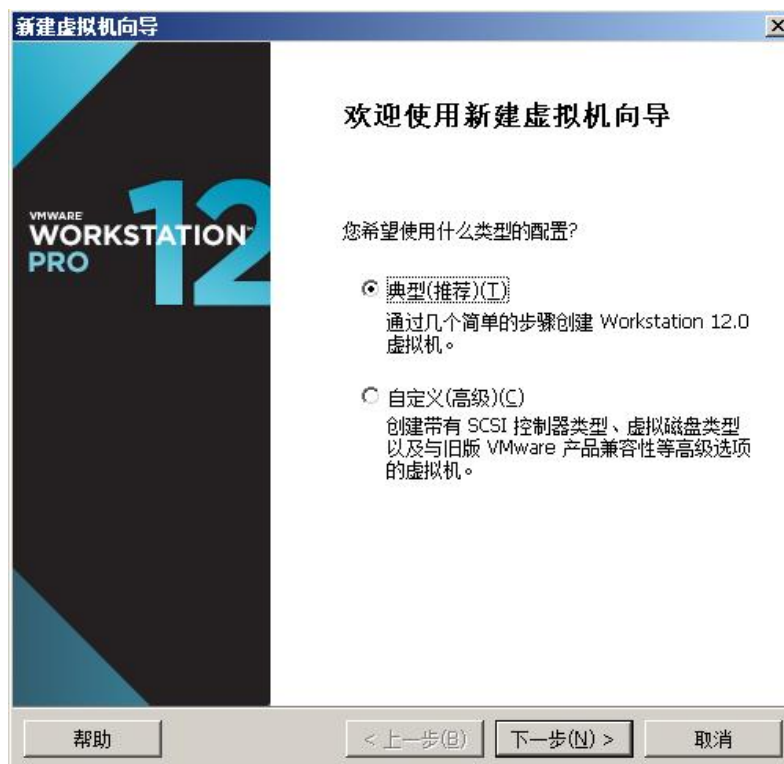
如果使用其他版本，可能有不可预料的错误，请保持版本一致，请不要升级系统。

ubuntu 安装步骤如下：

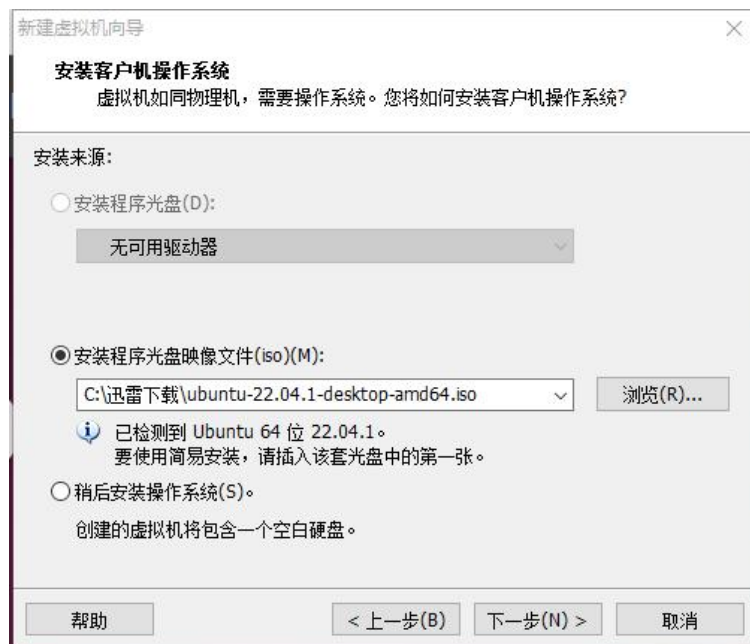
- 1) 双击桌面的 VMware Workstation Pro 的图标，然后在 VMware 工作界面上点击“创建新的虚拟机”图标。



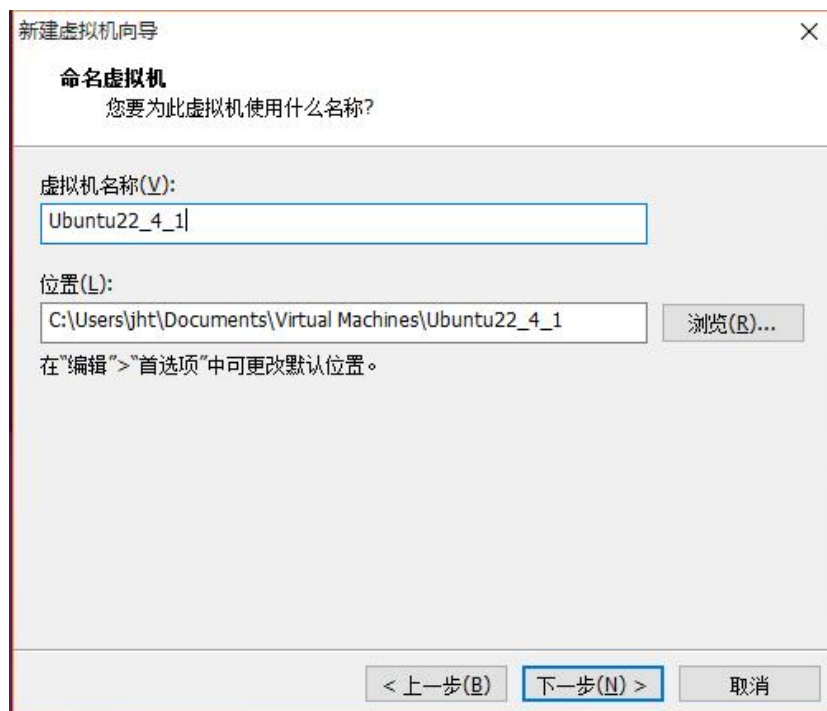
- 2) 选择典型，下一步。



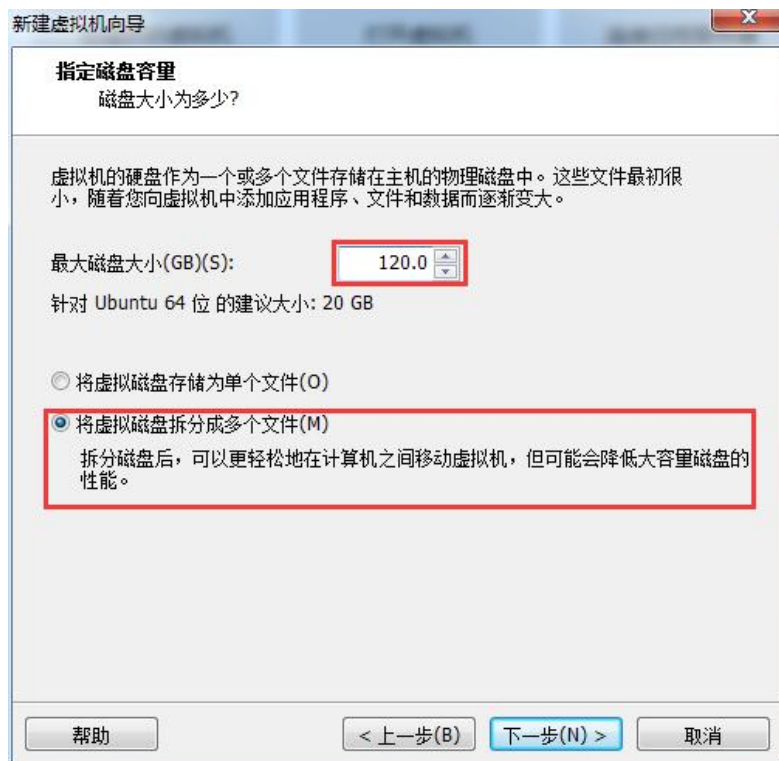
- 3) 选择"安装程序光盘映像文件(iso)"项, 然后点击浏览找到 ubuntu 的光盘映像文件 "ubuntu-22.4.1-desktop-amd64.iso" 。



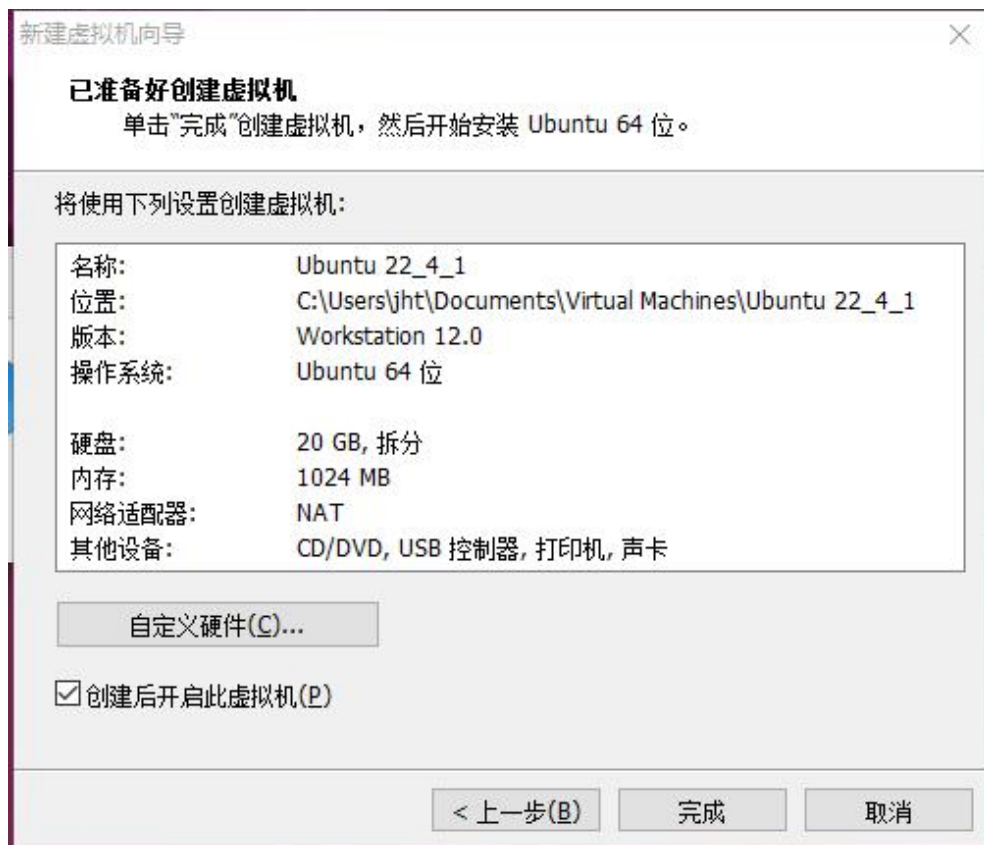
- 4) 虚拟机名称可以自己修改, 安装位置需要选择安装到硬盘空间比较充足的磁盘



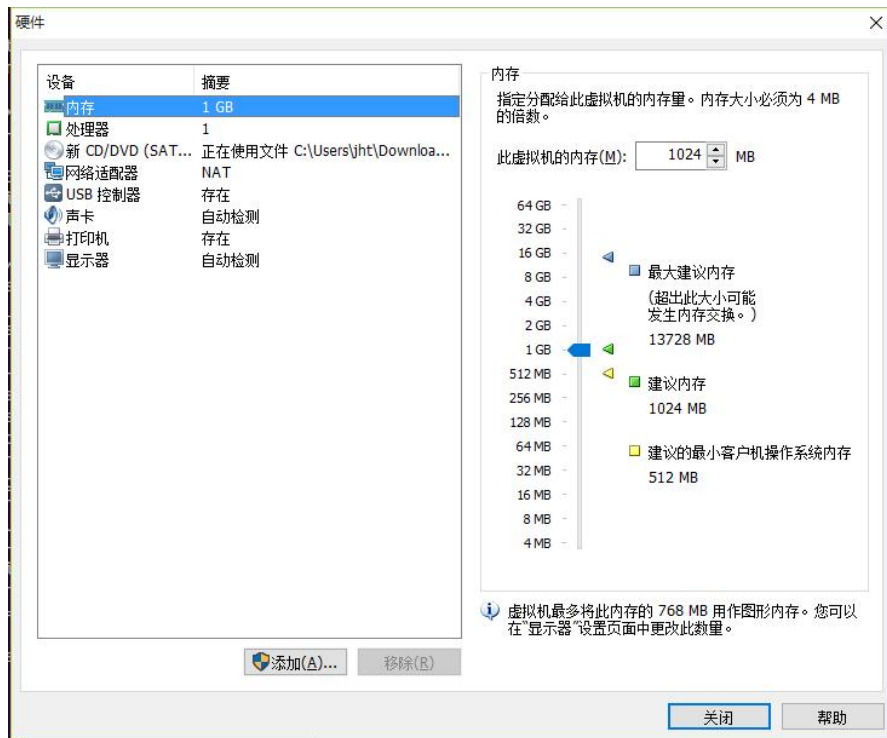
- 5) 设置最大的磁盘大小为 300G，我们需要在虚拟机里安装软件，这里预留空间大一些。用户可以根据自己的硬盘空间选择合适的空间尺寸，**建议大于等于 300G**。



- 6) 选择自定义硬件



- 7) 可以根据修改修改内存大小和处理器核心，网络适配器选项，网络连接选择桥接模式，内存建议设置为 8G，否则编译会出错。



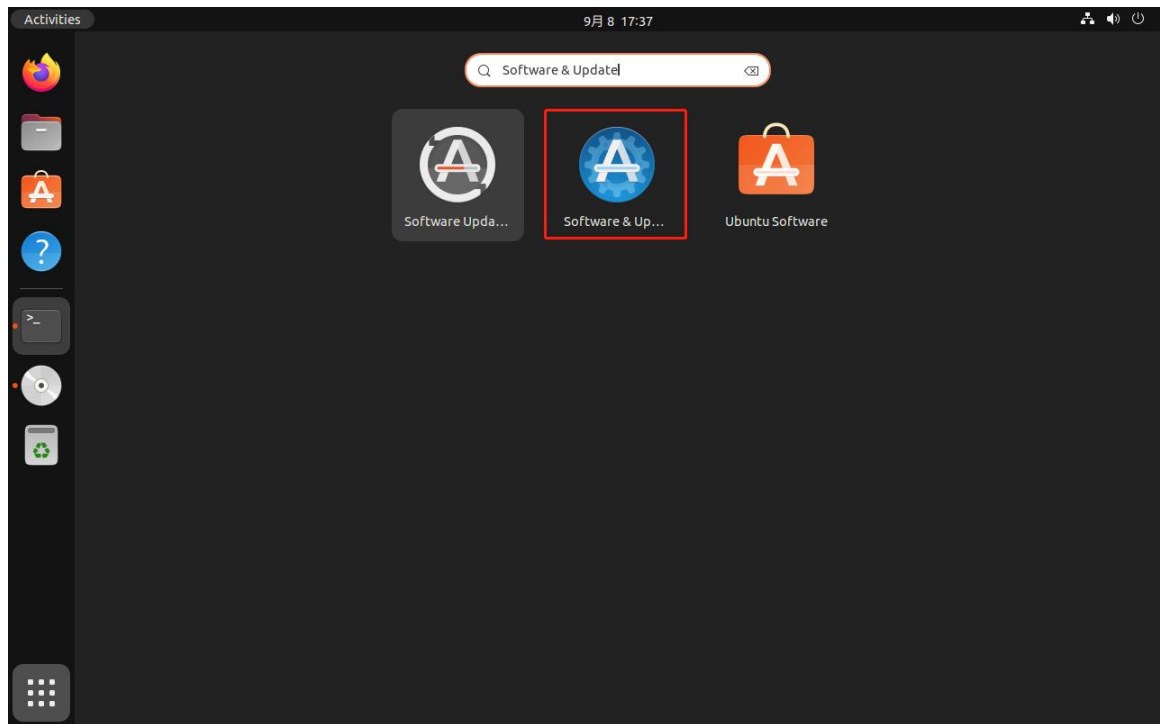
- 8) 点击完成就开始安装 Ubuntu 了



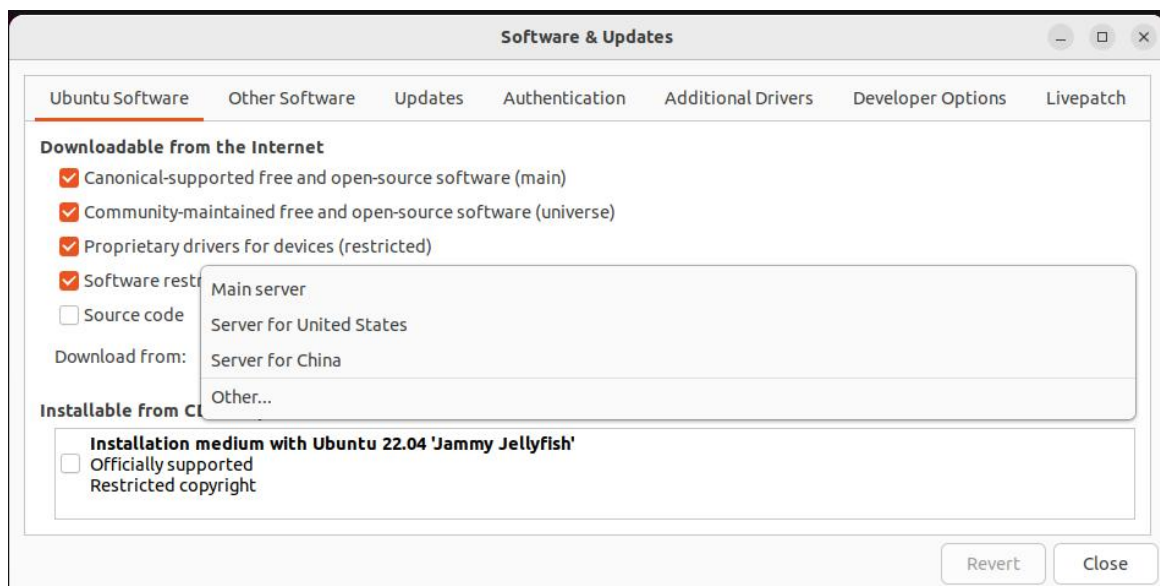
9) 安装完成以后进入系统

1.2.2 修改软件源服务器

- 1) 为了以后安装软件方便，我们要设置一下软件源，点击应用菜单，搜索 Software & Updates

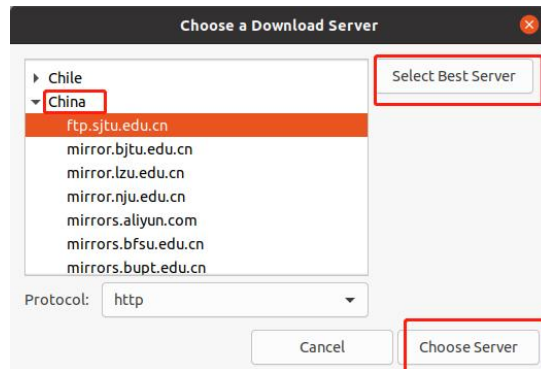


- 2) 在 “Software & Updates” 中选择 “Other...”

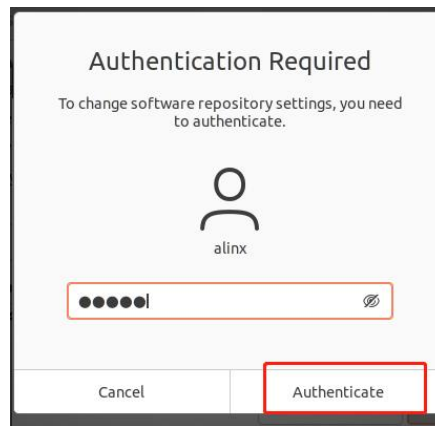


- 3) 点击 “Select Best Server” ，可以测试出一个最快的服务器，然后选择 “Choose

Server”，这些操作都是基于虚拟机能够连接互联网的情形。

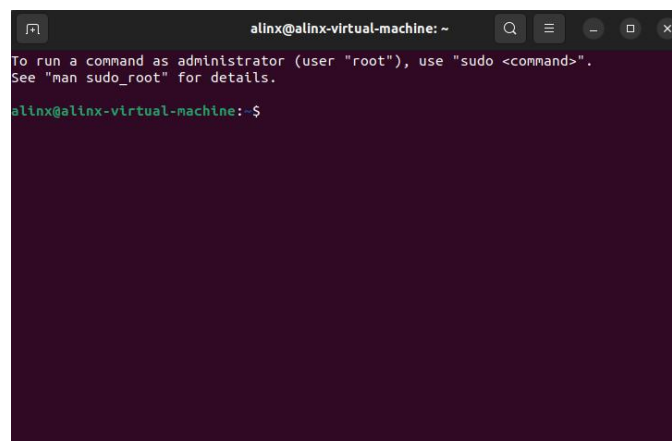


4) 输入密码，完成软件源修改



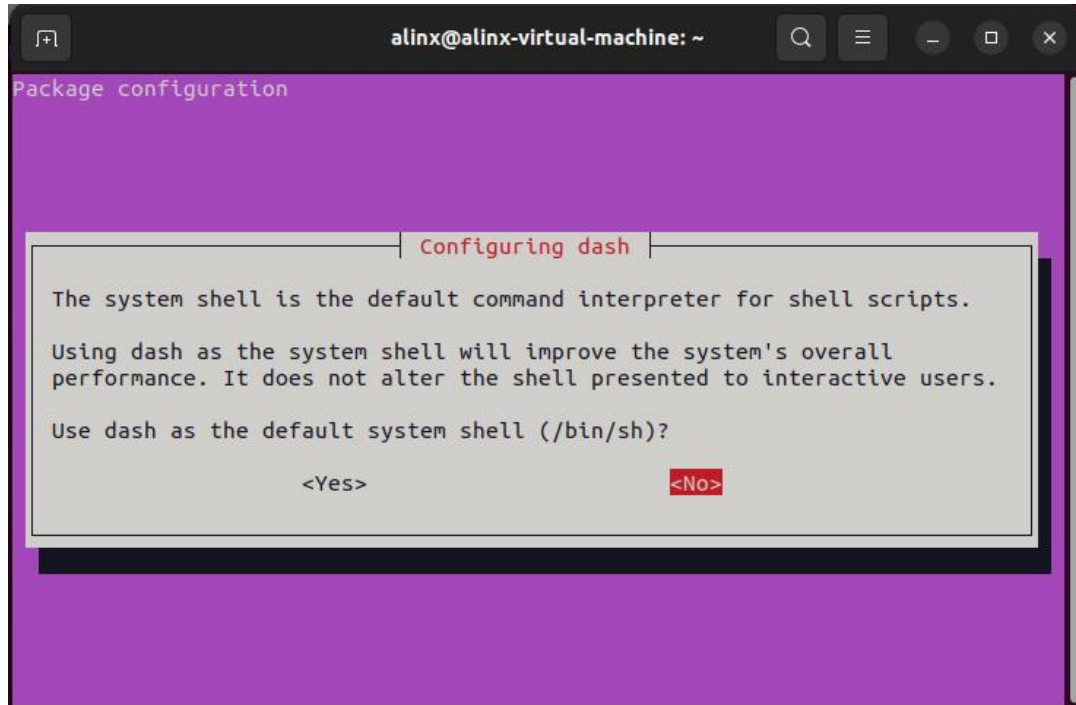
1.2.3 设置 bash 为默认 sh

1) Ctrl+Alt+T 打开终端



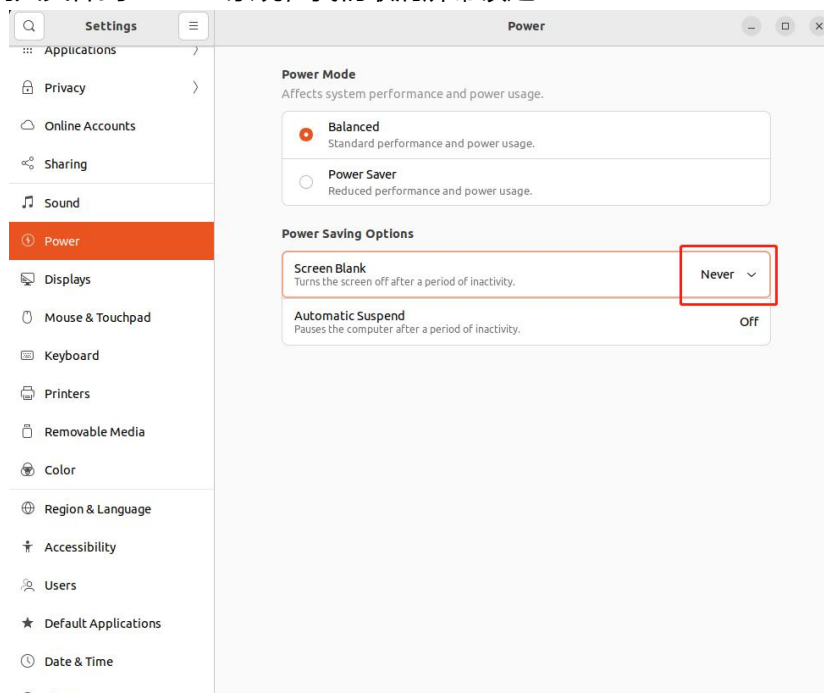
- 2) 输入命令, Configuring dash 选择 “No”, 回车确认

```
sudo dpkg-reconfigure dash
```



1.2.4 设置屏幕锁定时间

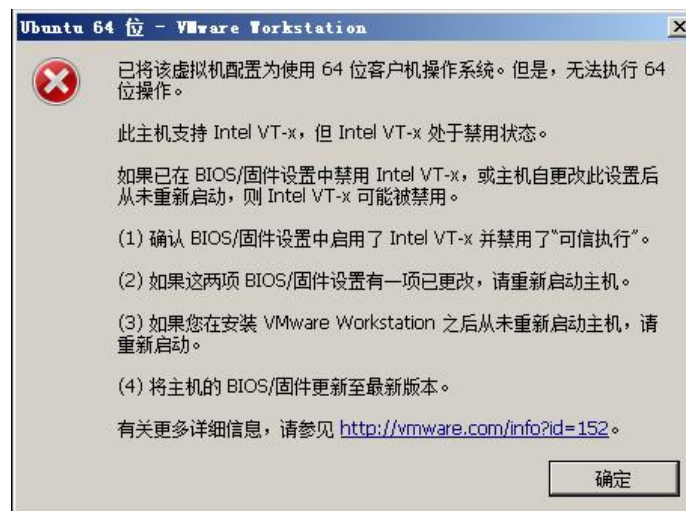
为了能复制大文件到 Ubuntu 系统, 我们取消屏幕锁定



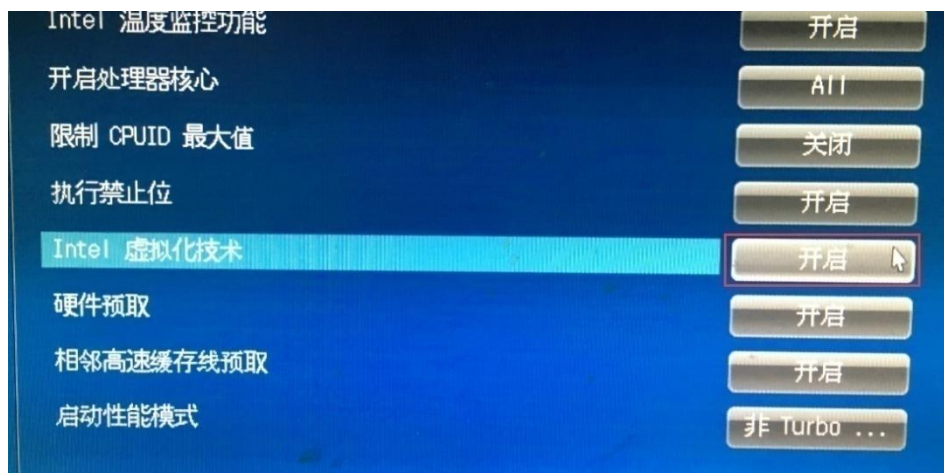
1.3 常见问题

1.3.1 虚拟机要求虚拟化支持

- 1) 如果安装 Ubuntu 弹出以下的错误信息框的话，用户需要重启电脑，进入 BIOS 里进行设置。



重启电脑后，进入到 BIOS 里，找到 Intel 虚拟化技术这一项，点击开启。不同的主板，可能名字不太一样。



第二章 Ubuntu 安装 Linux 版 Vitis 软件

虽然 Windows 下的 Vivado 软件可以解决大部分问题，可是偶尔我们还要使用 Linux 版本的 Vivado，特别是 Vitis，我们可以交叉编译很多应用程序。

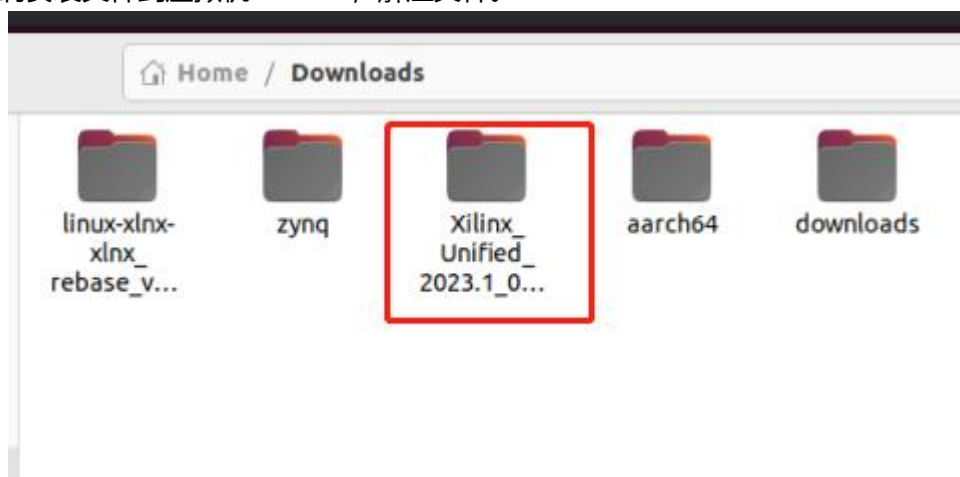
2.1 安装库

在终端输入下面的命令，如果没有安装的话，那么安装 vitis 就会一直卡住。

```
sudo apt install libncurses5  
sudo apt install libtinfo5  
sudo apt install libncurses5-dev libncursesw5-dev  
sudo apt install ncurses-compat-libs
```

2.2 安装 Linux 版 Vitis

- 1) 复制安装文件到虚拟机 Ubuntu，解压文件。



- 2) 使用终端进入解压后的

```
cd ~/Downloads/Xilinx_Unified_2023.1_0507_1903/
```

- 3) 运行命令

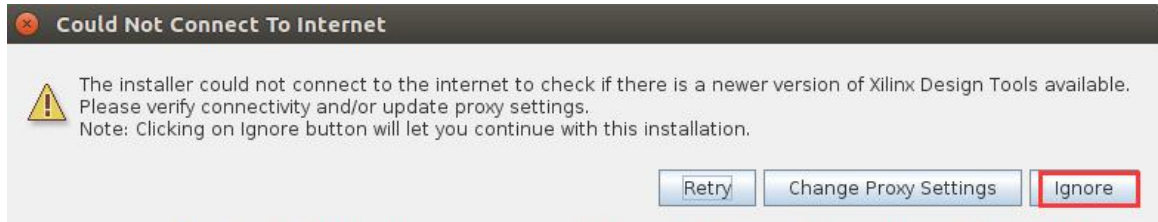
```
sudo chmod +x xsetup
```

```
alinx@alinx:~/Downloads$ cd ~/Downloads/Xilinx_Unified_2023.1_0507_1903/  
alinx@alinx:~/Downloads/Xilinx_Unified_2023.1_0507_1903$ sudo chmod +x xsetup
```

- 4) 运行命令，开始安装，**需要注意，这里是 sudo 安装**

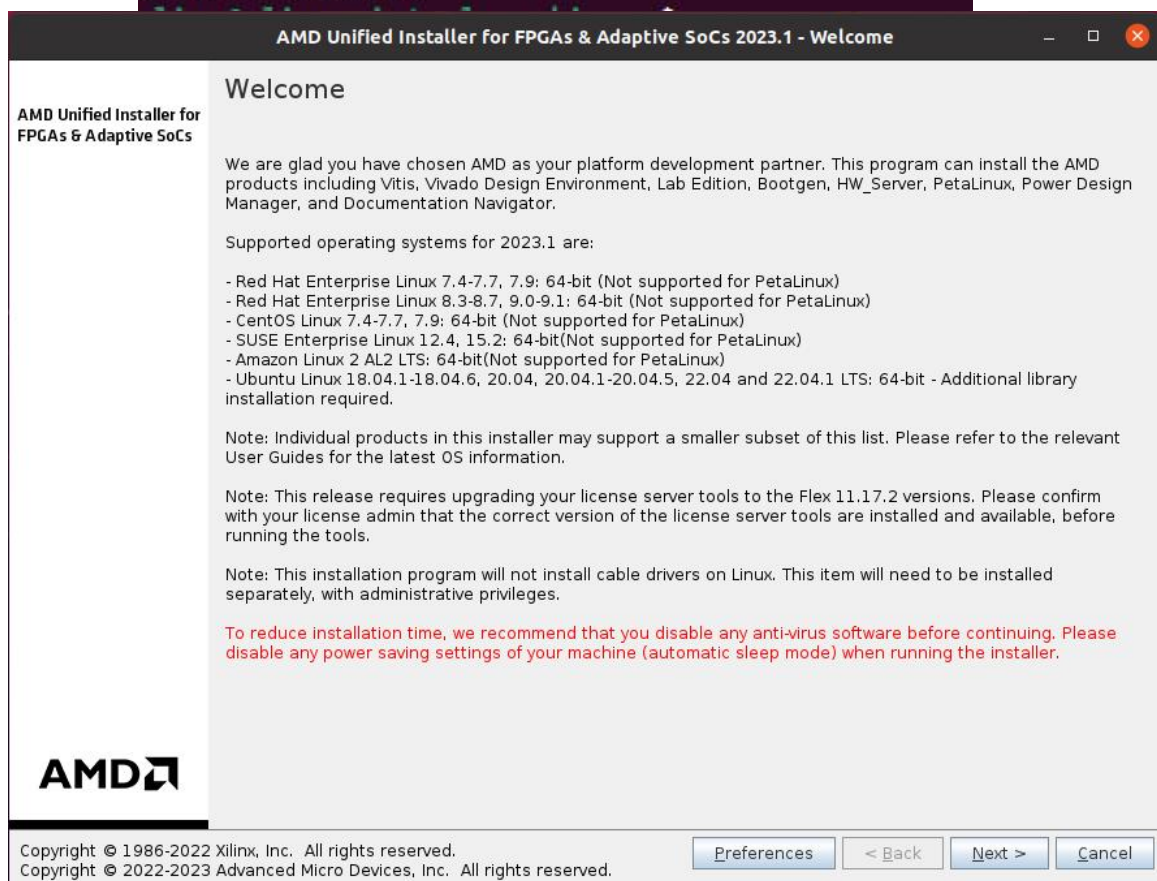
```
sudo ./xsetup
```

- 5) 如果弹出这些窗口，都点击 “Ignore”

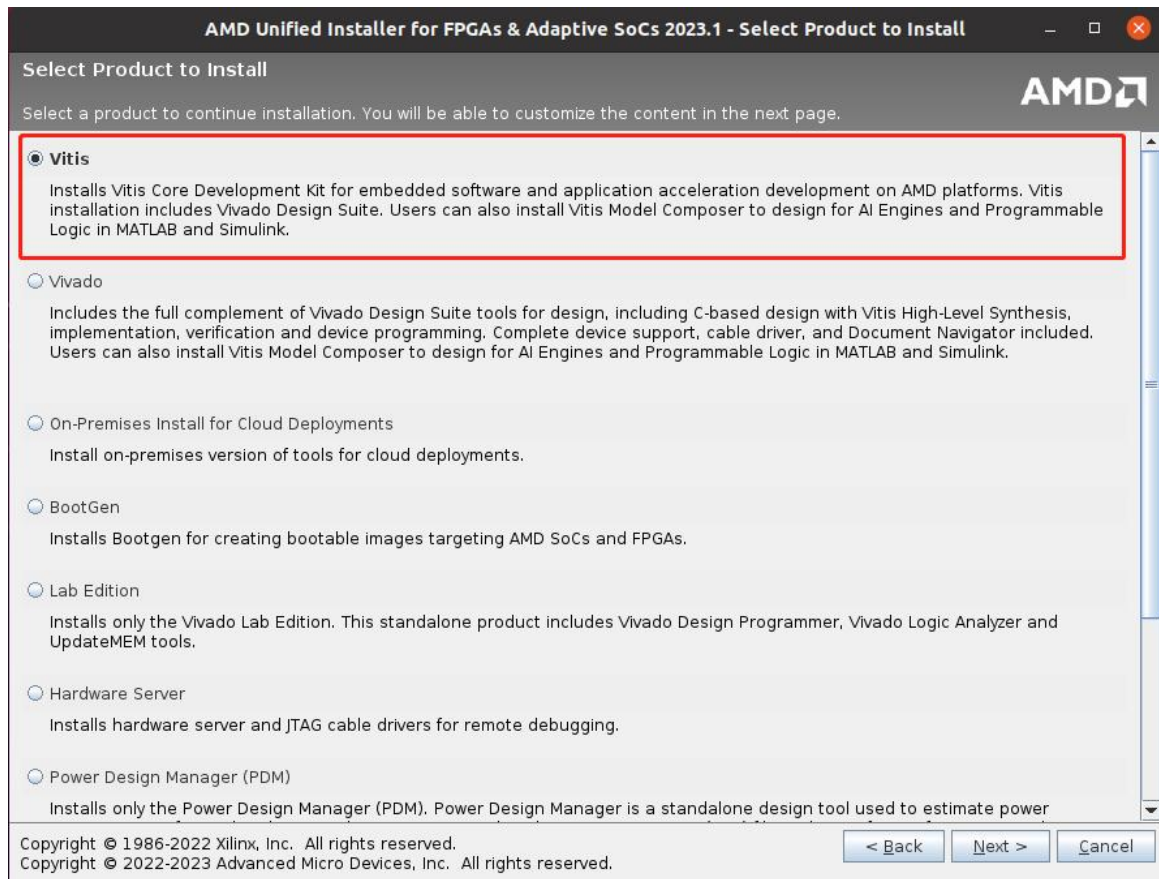


6) 安装过程要求我们关闭杀毒软件

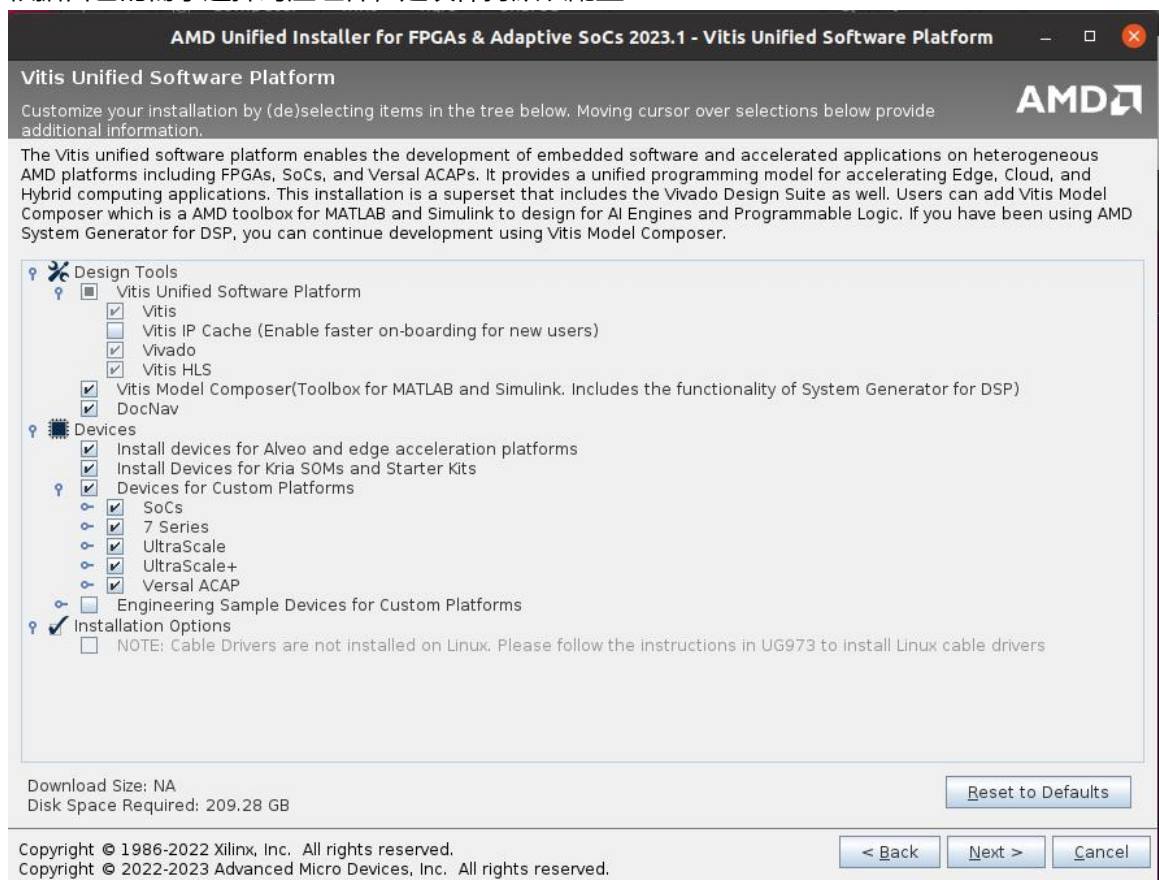
```
alinx@alinx-virtual-machine:~$ sudo ufw status
[sudo] password for alinx:
Status: inactive
```



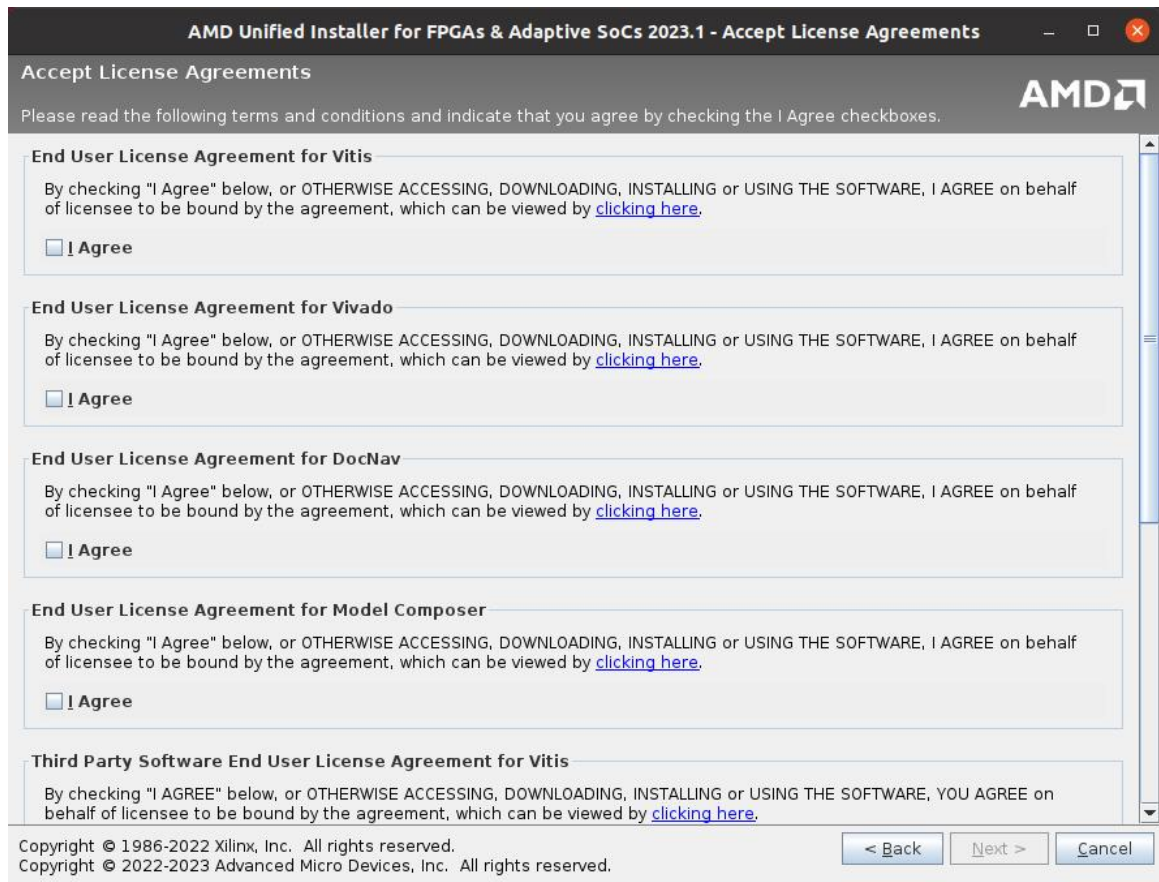
7) 选择 “Vitis”



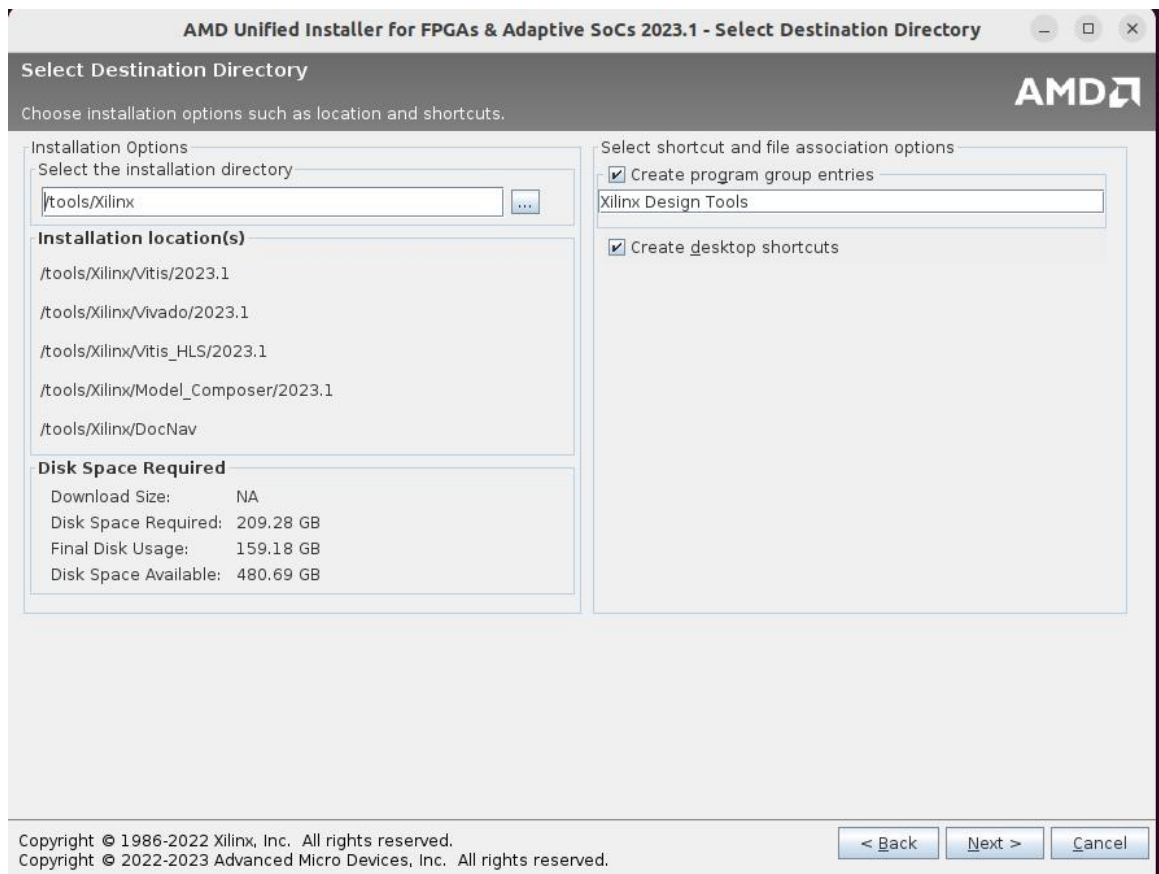
8) 根据自己的需求选择对应组件，建议保持默认配置



9) 勾选所有协议



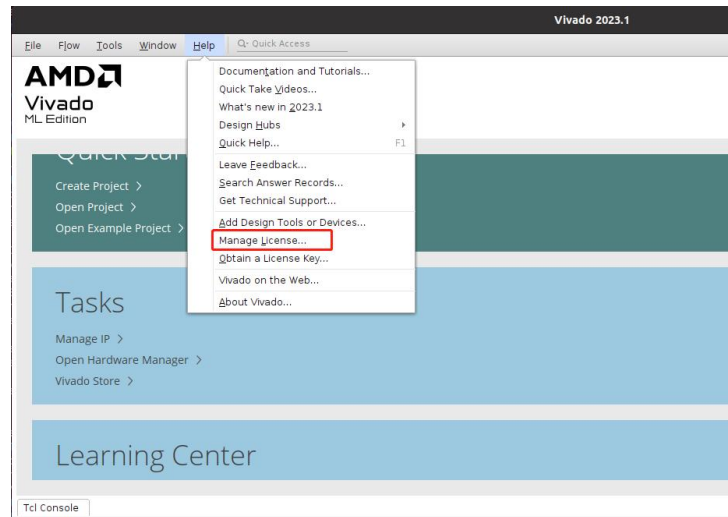
10) 安装路径使用默认路径



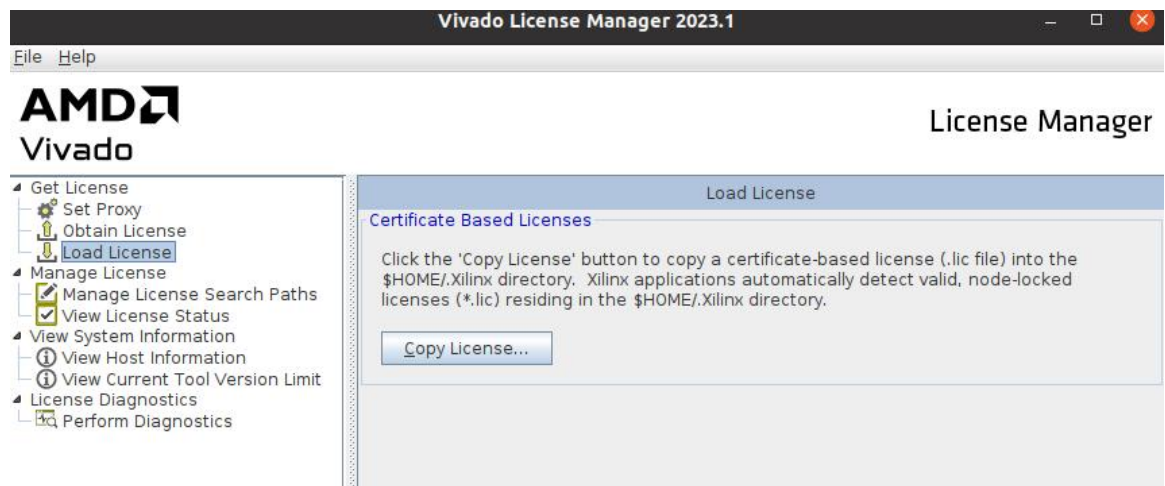
11) 运行 Vivado 软件

```
source /tools/Xilinx/Vivado/2023.1/settings64.sh  
vivado &
```

12) 软件启动以后在 “Help → Manage license...” 添加 license 文件



13) 先点击 “Load License” 再点击 Copy License 安装 “lic” 文件，lic 文件和 Windows 版本通用的，各种版本也是通用的



2.3 权限设置

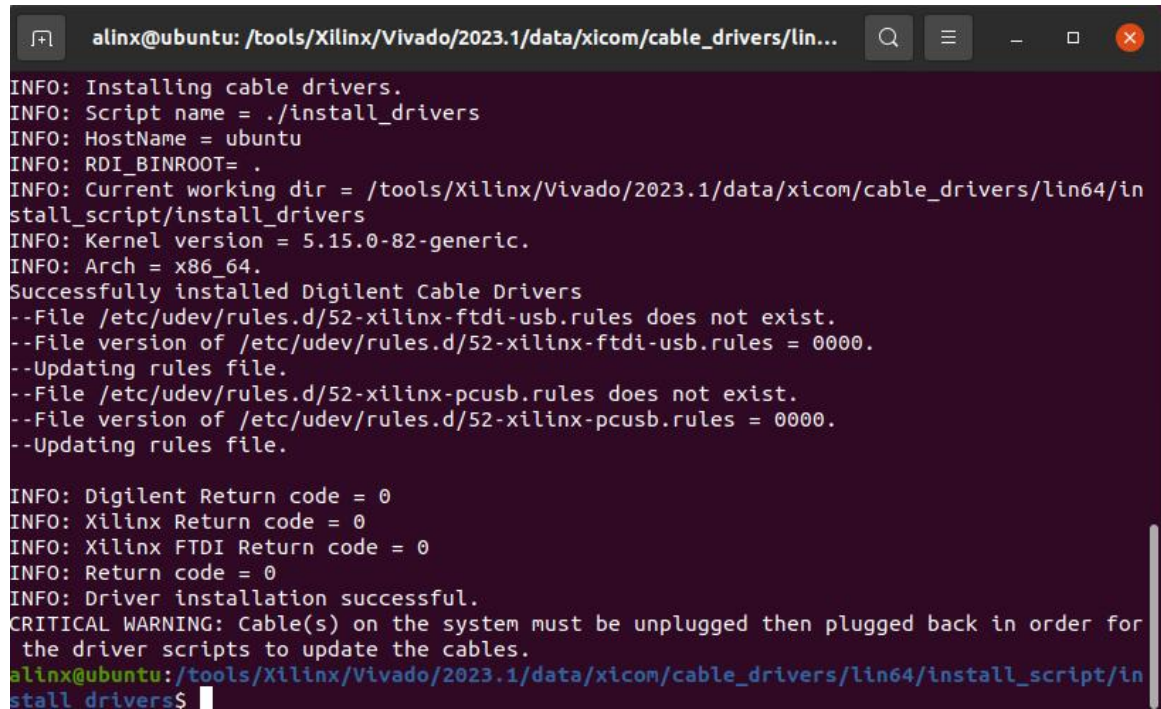
运行命令添加运行权限

```
sudo chmod 777 -R /tools/Xilinx/
```

2.4 安装下载器驱动

运行下列命令安装下载器驱动

```
cd /tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/install_script/install_drivers/  
sudo ./install_drivers
```

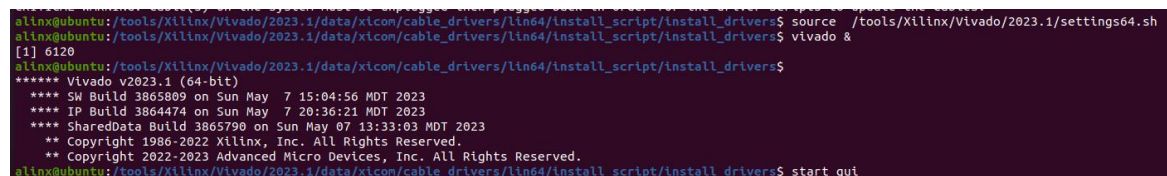


```
alinx@ubuntu: /tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin...  
INFO: Installing cable drivers.  
INFO: Script name = ./install_drivers  
INFO: HostName = ubuntu  
INFO: RDI_BINROOT= .  
INFO: Current working dir = /tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/in  
stall_script/install_drivers  
INFO: Kernel version = 5.15.0-82-generic.  
INFO: Arch = x86_64.  
Successfully installed Digilent Cable Drivers  
--File /etc/udev/rules.d/52-xilinx-ftdi-usb.rules does not exist.  
--File version of /etc/udev/rules.d/52-xilinx-ftdi-usb.rules = 0000.  
--Updating rules file.  
--File /etc/udev/rules.d/52-xilinx-pcusb.rules does not exist.  
--File version of /etc/udev/rules.d/52-xilinx-pcusb.rules = 0000.  
--Updating rules file.  
  
INFO: Digilent Return code = 0  
INFO: Xilinx Return code = 0  
INFO: Xilinx FTDI Return code = 0  
INFO: Return code = 0  
INFO: Driver installation successful.  
CRITICAL WARNING: Cable(s) on the system must be unplugged then plugged back in order for  
the driver scripts to update the cables.  
alinx@ubuntu:/tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/install_script/in  
stall_drivers$
```

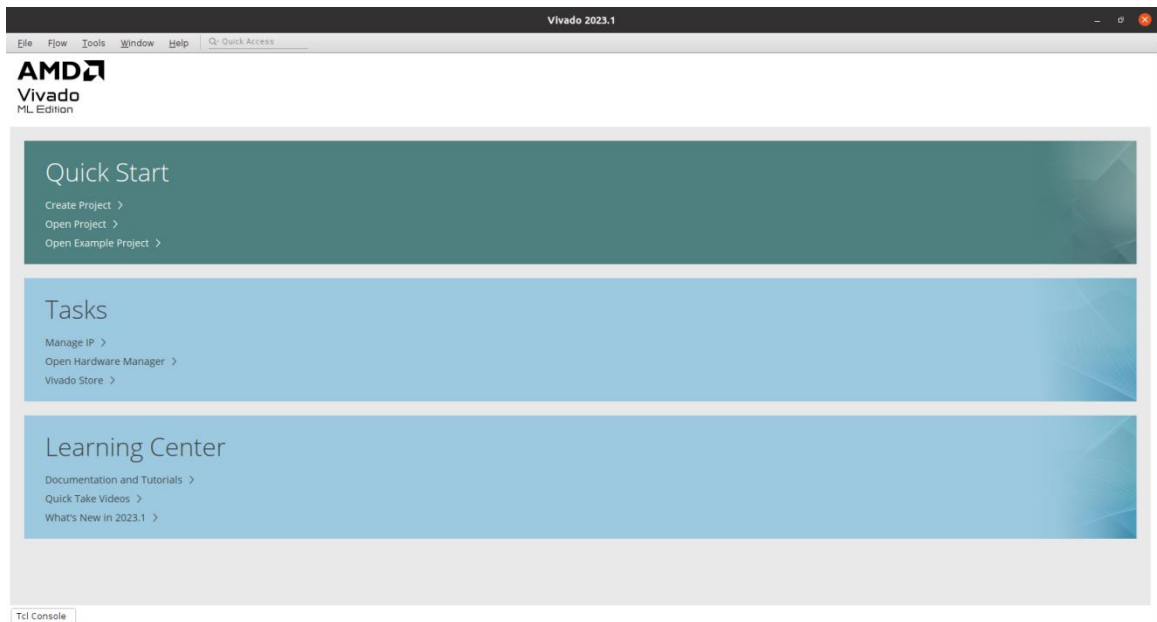
2.5 测试 Vivado

- 1) 运行下列命令，启动 Vivado

```
source /tools/Xilinx/Vivado/2023.1/settings64.sh  
vivado &
```



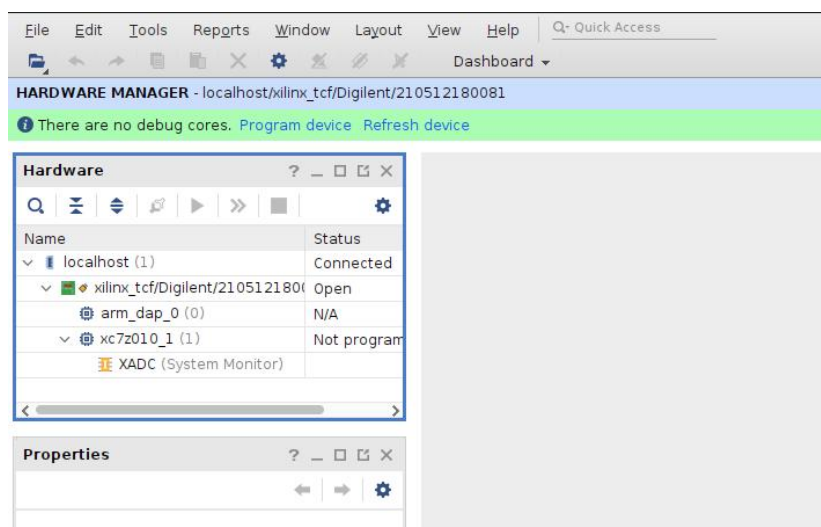
```
alinx@ubuntu:/tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/install_script/install_drivers$ source /tools/Xilinx/Vivado/2023.1/settings64.sh  
alinx@ubuntu:/tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/install_script/install_drivers$ vivado &  
(*) 6120  
alinx@ubuntu:/tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/install_script/install_drivers$  
***** Vivado v2023.1 (64-bit)  
**** SW Build 3865809 on Sun May 7 15:04:56 MDT 2023  
**** IP Build 3864474 on Sun May 7 20:36:21 MDT 2023  
**** SharedData Build 3865790 on Sun May 07 13:33:03 MDT 2023  
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.  
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.  
alinx@ubuntu:/tools/Xilinx/Vivado/2023.1/data/xicom/cable_drivers/lin64/install_script/install_drivers$ start_gui
```



2) 连接下载器到虚拟机



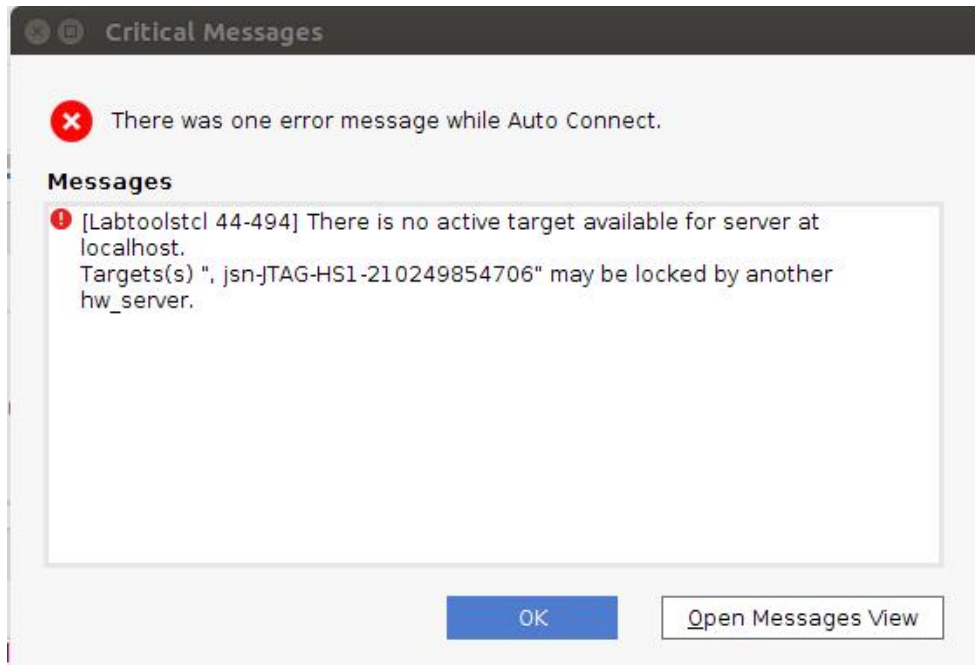
3) 连接开发板和下载器，使用“Open Hardware Manager”测试，正常情况下可以发现芯片，说明 Vivado 和下载器驱动安装成功



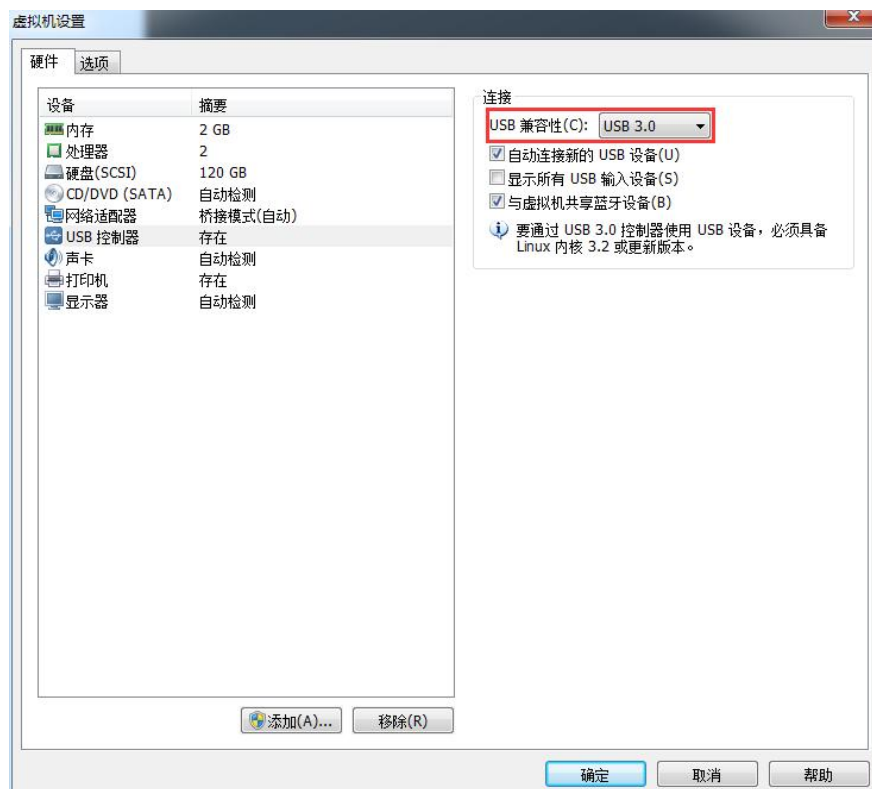
2.6 常见问题

2.6.1 Linux 下载器下载时提示被占用

- 1) 测试硬件时，能发现下载器，但是有个错误



- 2) 有些主板需要设置 USB 的兼容性，关闭虚拟机的 Ubuntu，设置 USB 兼容性到“USB 3.0”，再次尝试，如果还不能使用下载器，只能使用 Windows 版本下载了。



2.6.2 适合 ZYNQ 的交叉编译器

在安装 Vitis 软件里就包含了交叉编译器 arm-linux-gnueabi-hf-gcc。

```
source /tools/Xilinx/Vivado/2023.1/settings64.sh
arm-linux-gnueabi-hf-gcc -v
```

```
alinx@ubuntu:~$ source /tools/Xilinx/Vivado/2023.1/settings64.sh
alinx@ubuntu:~$ arm-linux-gnueabi-hf-gcc -v
Using built-in specs.
COLLECT_GCC=/tools/Xilinx/Vitis/2023.1/gnu/aarch32/lin/gcc-arm-linux-gnueabi/bin/../x86_64-petalinux-linux/usr/bin/arm-xilinx-linux-gnueabi/arm-xilinx-linux-gnueabi-gcc.real
COLLECT_LTO_WRAPPER=/tools/Xilinx/Vitis/2023.1/gnu/aarch32/lin/gcc-arm-linux-gnueabi/x86_64-petalinux-linux/usr/bin/arm-xilinx-linux-gnueabi/../../libexec/arm-xilinx-linux-gnueabi/g
cc/arm-xilinx-linux-gnueabi/12.2.0/lto-wrapper
Target: arm-xilinx-linux-gnueabi
Configured with: ../../../../../../work-shared/gcc-12.2.0-r0/gcc-12.2.0/configure --build=x86_64-linux --host=x86_64-petalinux-linux --target=arm-xilinx-linux-gnueabi --prefix=/usr/
local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr --exec_prefix=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr --bindir=/usr/local/oe-s
dk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/bin/arm-xilinx-linux-gnueabi --sbindir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/bin/arm-x
ilinx-linux-gnueabi --libexecdir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/libexec/arm-xilinx-linux-gnueabi --datadir=/usr/local/oe-sdk-hardcoded-bui
ldpath/sysroots/x86_64-petalinux-linux/usr/share --sysconfdir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/etc --sharedstatedir=/usr/local/oe-sdk-hardcoded-bui
ldpath/sysroots/x86_64-petalinux-linux/usr/share --localstatedir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/var --libdir=/usr/local/oe-sdk-hardcoded-buildpat
h/sysroots/x86_64-petalinux-linux/usr/lib/arm-xilinx-linux-gnueabi --includedir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/include --oldincludedir=/us
r/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/include --infodir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/share/info --mandi
r=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/usr/share/man --disable-silent-rules --disable-dependency-tracking --with-libtool-sysroot=/scratch/jenkins-BU
ILDS-eSDK-toolchain-2023.1/toolchain-27_Linux_zynq-generic/build/zynq-generic-x86_64/work/x86_64-nativesdk-petalinux-linux/gcc-cross-canadian-arm/12.2.0-r0/recipe-sysroot --with-gnu
ld --enable-shared --enable-languages=c,c++,fortran --enable-threads=posix --enable-multilib --enable-default-pie --enable-c99 --enable-long-long --enable-symvers=gnu --enable-ltbs
dxx-pch --program-prefix=arm-xilinx-linux-gnueabi- --without-local-prefix --disable-install-libiberty --disable-libssp --enable-libtln --enable-lto --disable-bootstrap --with-syst
em-zlib --with-linker-hash-style=gnu --enable-linker-build-id --with-ppino --with-cloog=no --enable-checking=release --enable-headers=global --without-lsl --with-gxx-include-dir
=/not/exist/usr/include/c++/12.2.0 --with-build-tline-tools=/scratch/jenkins-BUILDS-eSDK-toolchain-2023.1/toolchain-27_Linux_zynq-generic/build/zynq-generic-x86_64/work/x86_64-native
sdk-petalinux-linux/gcc-cross-canadian-arm/12.2.0-r0/recipe-sysroot-native/usr/arm-xilinx-linux-gnueabi/bin --with-sysroot=/not/exist --with-build-sysroot=/scratch/jenkins-BUILDS-eS
DK-toolchain-2023.1/toolchain-27_Linux_zynq-generic/build/zynq-generic-x86_64/work/x86_64-nativesdk-petalinux-linux/gcc-cross-canadian-arm/12.2.0-r0/recipe-sysroot --enable-standard
-branch-protection --with-plugin-ld=ld --enable-potson-system-directories --disable-static --enable-nls --with-glibc-version=2.28 --enable-initfini-array
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 12.2.0 (GCC)
```

第三章 Petalinux 工具安装

3.1 Petalinux 简介

petalinux 并不是一个特殊 Linux 内核，而是一套开发环境配置的工具，降低 uboot、内核、根文件系统的配置的工作量，可以从 Vivado 的导出硬件信息自动完成相关软件的配置。

这里要特意强调一下，petalinux 对系统版本和设置有严格要求，如果使用其他版本操作系统，使用中遇到问题请自行百度解决。

3.2 安装必要的库

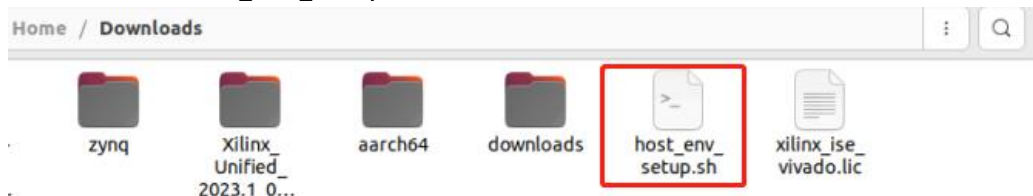
在 Xilinx 官方文档 UG1144

(<https://docs.xilinx.com/r/en-US/ug1144-petalinux-tools-reference-guide/Overview>)

中有详细安装方法，如果本教程安装失败，请参考 Xilinx 的 ug1144 文档进行安装。

我们运行下面 SHELL (host_env_setup.sh, 这个文件首先必须要有，而且要到文件所在目录运行，建议熟悉 Linux 教程基本操作，了解如何运行 Linux 命令等基础知识) 安装

- 1) 将我们提供的 host_env_setup.sh 复制到 Download 目录下



```
#!/bin/bash
set -x
script_dir=$(cd $(dirname ${BASH_SOURCE[0]}) && pwd)

# This script sets up a Ubuntu host to be able to create the image by
# installing all of the necessary files. It assumes a host with
# passwordless sudo

# Install a bunch of packages we need

read -d " PACKAGES <<EOT
iproute2
gawk
python3
python
```

```
build-essential
gcc
make
net-tools
libncurses5-dev
libssl-dev
flex
libselinux1
zlib1g-dev
gnupg
wget
diffstat
chrpath
socat
xterm
autoconf
libtool
tar
unzip
texinfo
gcc-multilib
automake
zlib1g:i386
screen
gzip
cpio
python3-git
python3-jinja2
xz-utils
g++
net-tools
build-essential
gawk
debianutils
iputils-ping
openssl
libstdc++-11-dev
cpp
patch
diffutils
tftpd-hpa

EOT

set -e
sudo apt-get update

if [[ $(lsb_release -rs) == "20.04" ]]; then
    echo "Install packages on Ubuntu 20.04..."
elif [[ $(lsb_release -rs) == "18.04" ]]; then
    echo "Install packages on Ubuntu 18.04..."
elif [[ $(lsb_release -rs) == "22.04" ]]; then
    echo "Install packages on Ubuntu 22.04..."
else
```

```
    echo "Error: current OS not supported."
    exit 1
fi
if [[ $(lsb_release -ds) < "Ubuntu 20.04.4" ]]; then
    sudo apt-get -y install bison pax python3-pip python3-pexpect libegl1-mesa libSDL1.2-dev
fi
sudo dpkg --add-architecture i386
sudo apt-get update
sudo apt-get install -y $PACKAGES

echo "Now install Vivado, Vitis, and Petalinux."
echo "Re-login to ensure the environment is properly set up."
```

- 2) 在文件所在目录下运行下面 SHELL，安装库

```
alinx@alinx:~/Downloads$ chmod +x host_env_setup.sh
alinx@alinx:~/Downloads$ sudo ./host_env_setup.sh
```

- 3) 配置 tftp server，如果不需要从 TFTP 启动，这一步可选

```
sudo -s
apt-get install tftpd-hpa
chmod a+w /var/lib/tftpboot/
reboot
```

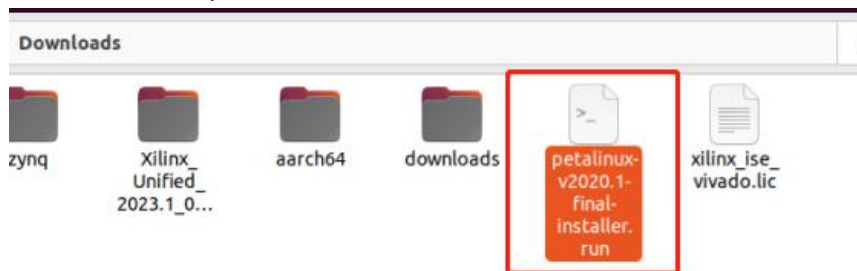
3.3 安装 Petalinux

- 1) 运行以下命令做安装准备，<your_user_name>为你的用户名，例如图中的 alinx

```
sudo -s
mkdir -p /opt/pkg/petalinux
chown <your_user_name> /opt/pkg/
chgrp <your_user_name> /opt/pkg/
chgrp <your_user_name> /opt/pkg/petalinux/
chown <your_user_name> /opt/pkg/petalinux/
exit
```

```
alinx@alinx-fpga: ~/Downloads
alinx@alinx-fpga:~/Downloads$ sudo -s
sudo: unable to resolve host alinx-fpga
[sudo] password for alinx:
root@alinx-fpga:~/Downloads# mkdir -p /opt/pkg/petalinux
root@alinx-fpga:~/Downloads# chown alinx /opt/pkg/
root@alinx-fpga:~/Downloads# chgrp alinx /opt/pkg/
root@alinx-fpga:~/Downloads# chown alinx /opt/pkg/petalinux/
root@alinx-fpga:~/Downloads# chgrp alinx /opt/pkg/petalinux/
root@alinx-fpga:~/Downloads# exit
exit
alinx@alinx-fpga:~/Downloads$
```

- 2) 将 petalinux-v2020.1-final-installer.run 至 ubuntu 系统下（笔者是复制到 Downloads 的 目录下）



- 3) 给安装文件添加运行权限

```
sudo chmod +x petalinux-v2023.1-05012318-installer.run
```

- 4) 开始安装

```
./petalinux-v2023.1-05012318-installer.run -d /opt/pkg/petalinux/
```

```
alinx@alinx:~/Downloads$ ./petalinux-v2020.1-final-installer.run -d /opt/pkg/petalinux/
```

- 5) 按回车查看协议内容

```
linux/
INFO: Checking installer checksum...
INFO: Extracting PetaLinux installer...

LICENSE AGREEMENTS

PetaLinux SDK contains software from a number of sources. Please review
the following licenses and indicate your acceptance of each to continue.

You do not have to accept the licenses, however if you do not then you may
not use PetaLinux SDK.

Use PgUp/PgDn to navigate the license viewer, and press 'q' to close
Press Enter to display the license agreements
```

- 6) 按 q 退出协议内容

```

XILINX, INC.
END USER LICENSE AGREEMENT FOR PETALINUX TOOLS

CAREFULLY READ THIS END USER LICENSE AGREEMENT FOR PETALINUX TOOLS ("AGREEMENT")
. BY CLICKING THE "ACCEPT" OR "AGREE" BUTTON, ENTERING <93>YES<94> OR <93>Y<94>
TO ACCEPT THIS AGREEMENT, OR OTHERWISE ACCESSING, DOWNLOADING, INSTALLING OR US
ING THE SOFTWARE, YOU AGREE ON BEHALF OF LICENSEE TO BE BOUND BY THIS AGREEMENT.

IF LICENSEE DOES NOT AGREE TO ALL OF THE TERMS AND CONDITIONS OF THIS AGREEMENT,
DO NOT CLICK THE "ACCEPT" OR "AGREE" BUTTON, ENTER <93>YES<94> OR <93>Y<94>, OR
ACCESS, DOWNLOAD, INSTALL OR USE THE SOFTWARE.

1.      Definitions

"Bitstream" means a machine-executable, binary form of a core used to program a
Xilinx Device.

"Licensee" means the individual, corporation or other legal entity who has downl
oaded and installed the Software.

"User" means a specific human being who is identified by Licensee as a person wh
o is authorized to use the applicable Software on behalf of Licensee. In cases
/tmp/tmp.Wt4jhy0kpU/.etc/license/Petalinux_EULA.txt

```

7) 按 y 同意协议内容

```

linux/
INFO: Checking installer checksum...
INFO: Extracting PetaLinux installer...

LICENSE AGREEMENTS

Petalinux SDK contains software from a number of sources. Please review
the following licenses and indicate your acceptance of each to continue.

You do not have to accept the licenses, however if you do not then you may
not use PetaLinux SDK.

Use PgUp/PgDn to navigate the license viewer, and press 'q' to close

Press Enter to display the license agreements
Do you accept Xilinx End User License Agreement? [y/N] > 

```

8) 在安装过程中会弹出 License, 按 "q" 退出, 然后按 "y" 同意。

```

WebTalk Terms and Conditions

By indicating I accept this WebTalk notice, I also confirm that I have read Sect
ion 13 of the terms and conditions above concerning WebTalk and have been afford
ed the opportunity to read the WebTalk FAQ posted at http://www.xilinx.com/webta
lk. I understand that I am able to disable WebTalk later if certain criteria de
scribed in Section 13(c) apply. If they don't apply, I can disable WebTalk by u
ninstalling the Software or using the Software on a machine not connected to the
internet. If I fail to satisfy the applicable criteria or if I fail to take th
e applicable steps to prevent such transmission of information, I agree to allow
Xilinx to collect the information described in Section 13(a) for the purposes d
escribed in Section 13(b).

/tmp/tmp.Wt4jhy0kpU/.etc/license/WebTalk_notice.txt (END)

```

```
INFO: Checking installer checksum...
INFO: Extracting PetaLinux installer...

LICENSE AGREEMENTS

PetaLinux SDK contains software from a number of sources. Please review
the following licenses and indicate your acceptance of each to continue.

You do not have to accept the licenses, however if you do not then you may
not use PetaLinux SDK.

Use PgUp/PgDn to navigate the license viewer, and press 'q' to close

Press Enter to display the license agreementsq
Do you accept Xilinx End User License Agreement? [y/N] > y
Do you accept Webtalk Terms and Conditions? [y/N] > y
Do you accept Third Party End User License Agreement? [y/N] > y
INFO: Checking installation environment requirements...
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
INFO: Installing PetaLinux...
█
```

第四章 NFS 服务软件安装

NFS (Network FileSystem, 网络文件系统) 是由 SUN 公司发展, 并于 1984 年推出的技术, 用于在不同机器, 不同操作系统之间通过网络互相分享各自的文件。NFS 设计之初就是为了在不同的系统间使用, 所以它的通讯协议设计与主机及操作系统无关。

NFS 分服务器和客户机, 当使用远端文件时只要用 mount 命令就可把远端 NFS 服务器上的文件系统挂载在本地文件系统之下, 操作远程文件与操作本地文件没有不同。NFS 服务器所共享文件或目录记录在/etc/exports 文件中。

嵌入式 Linux 开发中, 会经常使用 NFS, 目标系统通常作为 NFS 客户机使用, Linux 主机作为 NFS 服务器。在目标系统上通过 NFS, 将服务器的 NFS 共享目录挂载到本地, 可以直接运行服务器上的文件。在调试系统驱动模块以及应用程序, NFS 都是十分必要的, 并且 Linux 还支持 NFS 根文件系统, 能直接从远程 NFS root 启动系统, 这对嵌入式 Linux 根文件系统裁剪和集成也是十分有必要的。

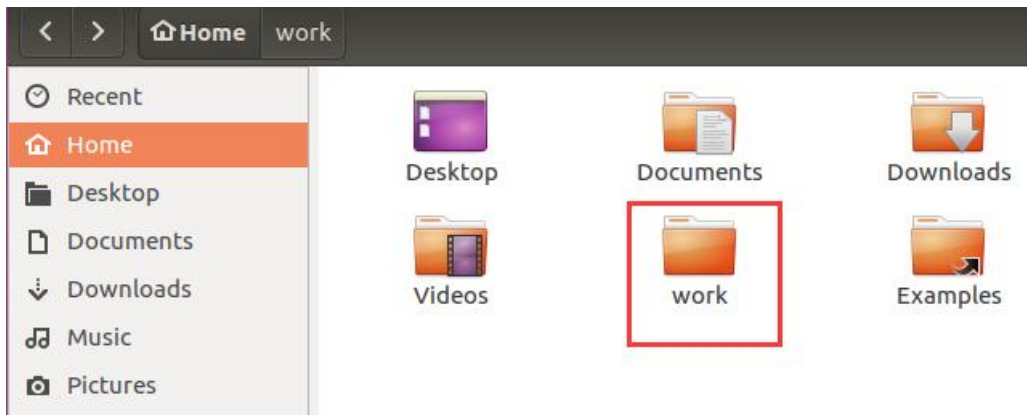
4.1 安装 NFS 服务

- 1) 通过下面的命令安装 NFS 服务器

```
sudo apt-get install nfs-kernel-server
```

```
alinx@ubuntu:~$ sudo apt-get install nfs-kernel-server
[sudo] password for alinx:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  keyutils libnfsidmap2 libtirpc1 nfs-common rpcbind
Suggested packages:
  open-iscsi watchdog
The following NEW packages will be installed:
  keyutils libnfsidmap2 libtirpc1 nfs-common nfs-kernel-server rpcbind
0 upgraded, 6 newly installed, 0 to remove and 325 not upgraded.
Need to get 467 kB of archives.
After this operation, 1,874 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

- 2) 新建一个 work 目录做为 NFS 的一个工作目录, 以后我们可以把交叉编译的程序放在这个目录里, 开发板可以很方便共享到这个目录里的文件。



- 3) 使用下面命令编辑/etc/exports 文件，配置 NFS 服务路径

```
sudo gedit /etc/exports
```

```
alinx@ubuntu:~$ sudo gedit /etc/exports
[sudo] password for alinx:

(gedit:60516): IBUS-WARNING **: The owner of /home/alinx/.config/ibus/bus is not root!
```

- 4) 在尾部添加/home/alinx/work *(rw, sync, no_root_squash, no_subtree_check) , 配置 /home/alinx/work 目录为 NFS 的一个工作目录。

A screenshot of a text editor window showing the contents of the /etc/exports file. The file contains several lines of configuration for NFS exports. The last line, line 11, has been added: /home/alinx/work *(rw, sync, no_root_squash, no_subtree_check).

```
1 # /etc/exports: the access control list for filesystems which may be exported
2 #           to NFS clients.  See exports(5).
3 #
4 # Example for NFSv2 and NFSv3:
5 # /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
6 #
7 # Example for NFSv4:
8 # /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
9 # /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
10 #
11 /home/alinx/work *(rw,sync,no_root_squash,no_subtree_check)
```

- 5) 执行下面命令重启 rpcbind 服务。nfs 是一个 RPC 程序，使用它前，需要映射好端口，通过 rpcbind 设定

```
sudo /etc/init.d/rpcbind restart
```

- 6) 执行下面命令重启 nfs 服务

```
sudo /etc/init.d/nfs-kernel-server restart
```

4.2 测试 NFS

- 1) 通过下面命令挂载 NFS，在本机将 NFS 工作路径挂载在/mnt 目录

```
sudo mount -t nfs 127.0.0.1:/home/alinx/work /mnt
```

- 2) 进入/mnt,新建一个 test 目录测试一下，可以在/home/alinx/work 目录同步看到 test 文件夹

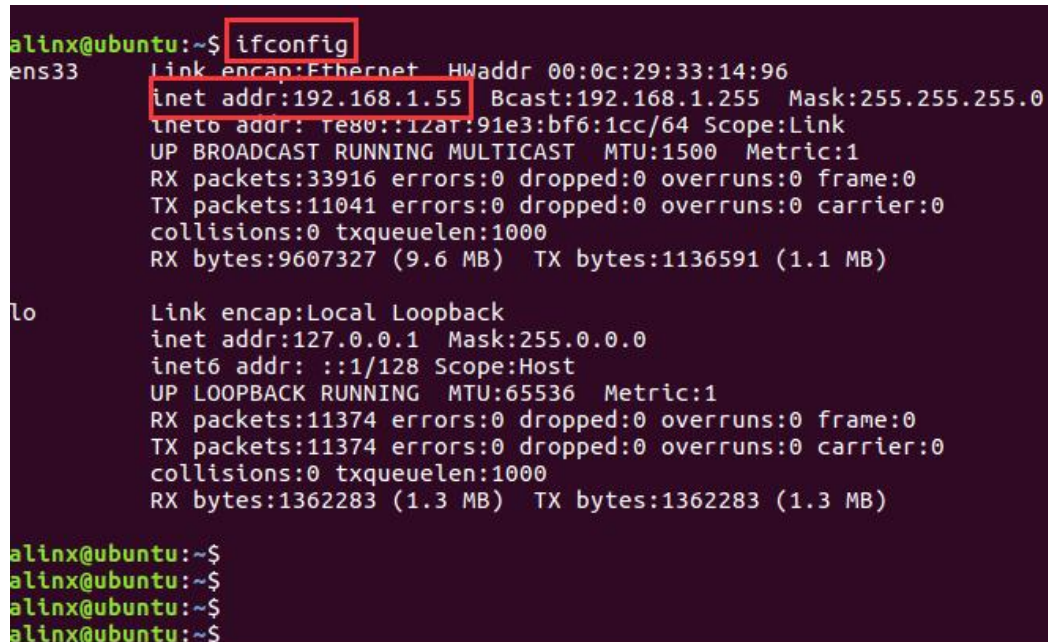
```
cd /mnt  
mkdir test
```

4.3 常见问题

4.3.1 NFS 无法挂载

首先确认虚拟机和开发板是不是一个网段？

使用 ifconfig 命令查看虚拟机 IP 地址，下图的例子中是 192.168.1.55，属于 192.168.1 网段，由于开发环境中有 DHCP 服务器存在，所以虚拟机的 IP 地址是自动分配的，因为网络环境不同，本教程不讲解如何配置网络，如果不会配置网络，咨询自己网络管理员。



```
alinx@ubuntu:~$ ifconfig  
ens33:  Link encap:Ethernet  HWaddr 00:0c:29:33:14:96  
        inet addr:192.168.1.55  Bcast:192.168.1.255  Mask:255.255.255.0  
        inet6 addr: fe80::12a7:91e3:bf6:1cc/64 Scope:Link  
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1  
        RX packets:33916 errors:0 dropped:0 overruns:0 frame:0  
        TX packets:11041 errors:0 dropped:0 overruns:0 carrier:0  
        collisions:0 txqueuelen:1000  
        RX bytes:9607327 (9.6 MB)  TX bytes:1136591 (1.1 MB)  
  
lo:      Link encap:Local Loopback  
        inet addr:127.0.0.1  Mask:255.0.0.0  
        inet6 addr: ::1/128 Scope:Host  
        UP LOOPBACK RUNNING  MTU:65536  Metric:1  
        RX packets:11374 errors:0 dropped:0 overruns:0 frame:0  
        TX packets:11374 errors:0 dropped:0 overruns:0 carrier:0  
        collisions:0 txqueuelen:1000  
        RX bytes:1362283 (1.3 MB)  TX bytes:1362283 (1.3 MB)  
  
alinx@ubuntu:~$  
alinx@ubuntu:~$  
alinx@ubuntu:~$  
alinx@ubuntu:~$
```

在串口终端中使用 ifconfig 命令查看开发板 IP 地址，下图的例子中是 192.168.1.46，属于 192.168.1 网段，如果没有 IP，或和开发板 IP 不同网段，请联系网络管理员。

```
root@zynq: ~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0a:35:00:1e:53
          inet addr:192.168.1.46  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::20a:35ff:fe00:1e53/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:37 errors:0 dropped:0 overruns:0 frame:0
          TX packets:37 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4597 (4.4 KiB)  TX bytes:3782 (3.6 KiB)
          Interrupt:29 Base address:0xb000

eth1      Link encap:Ethernet  HWaddr 00:0a:35:00:03:22
          UP BROADCAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:8 errors:0 dropped:0 overruns:0 frame:0
          TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:1104 (1.0 KiB)  TX bytes:1104 (1.0 KiB)

root@zynq: ~#
```

在串口终端中 ping 虚拟机，例程中 ping 192.168.1.55，这是因为虚拟机 IP 是 192.168.1.55，能够 ping 通才能正常挂载 NFS

```
root@zynq: ~# ping 192.168.1.55
PING 192.168.1.55 (192.168.1.55) 56(84) bytes of data.
64 bytes from 192.168.1.55: icmp_seq=1 ttl=64 time=0.862 ms
64 bytes from 192.168.1.55: icmp_seq=2 ttl=64 time=0.489 ms
64 bytes from 192.168.1.55: icmp_seq=3 ttl=64 time=0.779 ms
64 bytes from 192.168.1.55: icmp_seq=4 ttl=64 time=0.504 ms
64 bytes from 192.168.1.55: icmp_seq=5 ttl=64 time=0.574 ms
^C
--- 192.168.1.55 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4131ms
rtt min/avg/max/mdev = 0.489/0.641/0.862/0.153 ms
root@zynq: ~#
```

第五章 使用 Petalinux 定制 Linux 系统

实验 Vivado 工程为 “linux_base” ,在 vivado.zip 压缩包中。

实验 petalinux 工程相关文件夹为 “ax_peta”

前面的教程中我们搭建好了 Petalinux 环境，本教程主要演示如何使用 Petalinux。需要注意的是，本实验只有 Linux 主机可以连接互联网的情况下才能完成。

5.1 Vivado 工程

使用 Petalinux 可以非常方便地定制嵌入式 Linux 系统,只需要 Vivado 软件把硬件信息导出,然后 Petalinux 根据这些信息来配置 uboot, 内核、文件系统等, Vivado 工程建立的步骤在其他教程中有详细说明, 这里不再讲解, 这里可以用直接用我们提供的资料中的 xsa 文件。将 xsa 文件复制到虚拟机里。

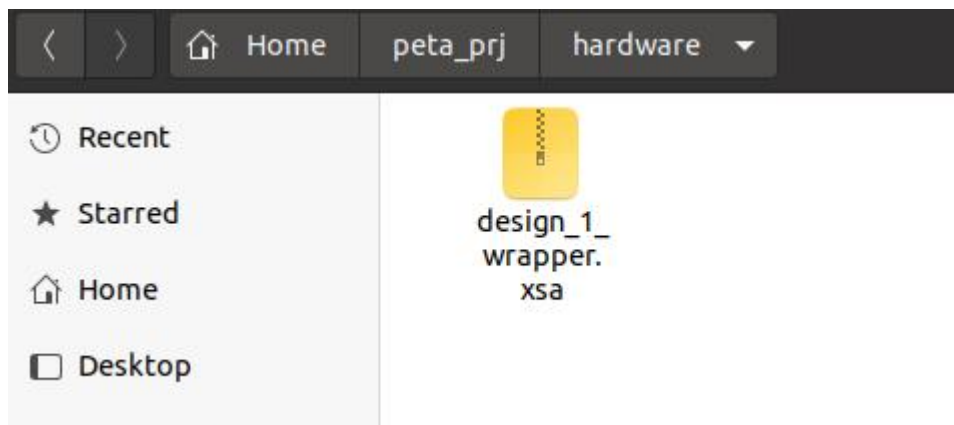
名称	修改日期	类型	大小
 design_1_wrapper.xsa	2023/12/29 14:10	XSA 文件	526 KB
 pl_config.tcl	2023/12/13 18:42	TCL 文件	13 KB

5.2 使用 Petalinux 建立工程

- 1) 在用户目录下, 创建一个工作目录, 路径中 “~” 表示用户 home 路径

```
mkdir -p ~/peta_prj/hardware
```

- 2) 将 vivado 导出 xsa 文件拷贝进 hardware 文件夹下



注：这里使用的是名为 ps_base 的 vivado 工程导出的 xsa 文件

- 3) 进入工程目录

```
cd ~/peta_prj/
```

- 4) 设置 petalinux 环境变量，运行下面命令

```
source /opt/pkg/petalinux/settings.sh
```

- 5) 运行下面命令设置 vivado 环境变量

```
source /tools/Xilinx/Vivado/2023.1/settings64.sh
```

- 6) 使用下面命令创建一个 petalinux 工程，工程名为 ax_peta，这个时候 petalinux 会自动创建一个名为 ax_peta 的工程。

```
petalinux-create -t project -n ax_peta --template zynq
```

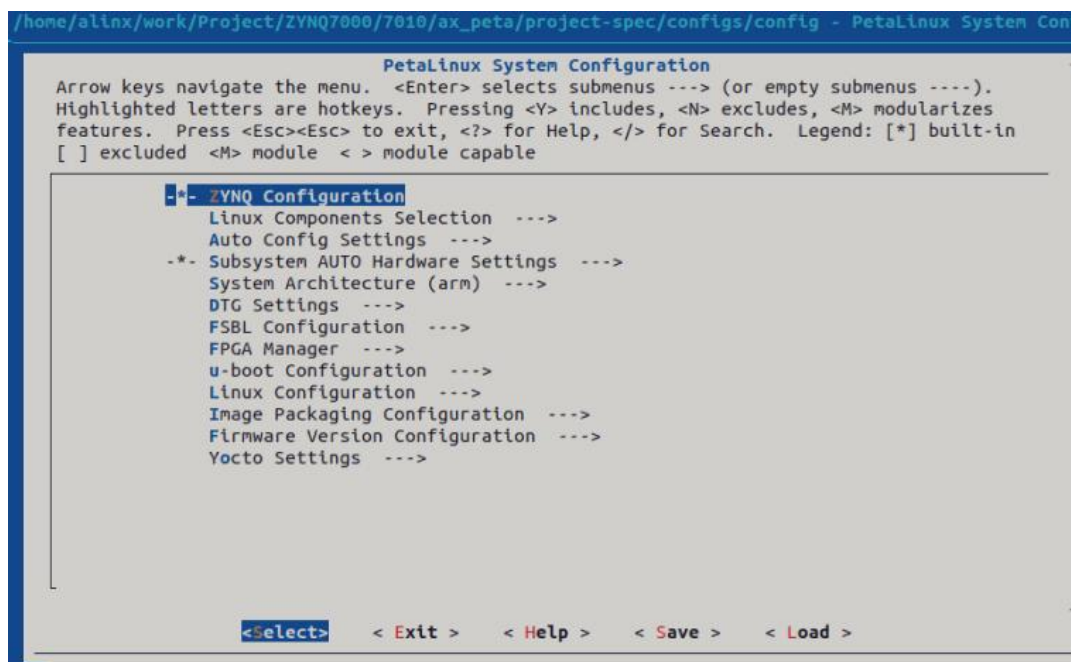
- 7) 使用下面的命令进入 petalinux 工作目录

```
cd ~/peta_prj/ax_peta
```

- 8) 配置 Petalinux 工程的硬件信息，硬件信息目录里只能有一个 xsa 文件

```
petalinux-config --get-hw-description ../hardware/
```

- 9) 在弹出一个窗口里可以配置 petalinux 工程，*如果配置过后想再次配置，可以运行命令“petalinux-config”来配置。*

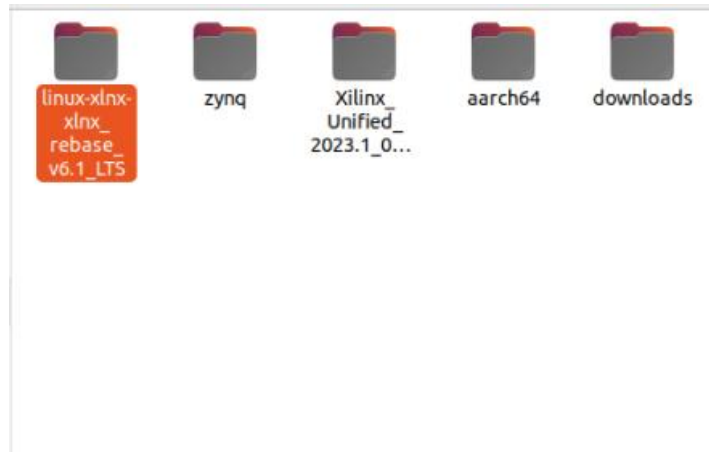


- 10) 在选项 Linux Components Selection 中可以配置 uboot 和 Linux 内核的来源，按回车键可进入下一个选择界面。uboot 和 Linux 内核默认是 github 上下载的，需要 Linux 主机连接互联网才能下载。本实验将对应的内核都下载到了本地。内核源码和 u-boot 源码由 xilinx 提供移植好的版本，可以到 xilinx 的 github 主页下载：<https://github.com/Xilinx>。在我们资料中提供了修改后的内核源码，而 linux 教程后续章节 内容都需要用到我们资料中的内核源码。

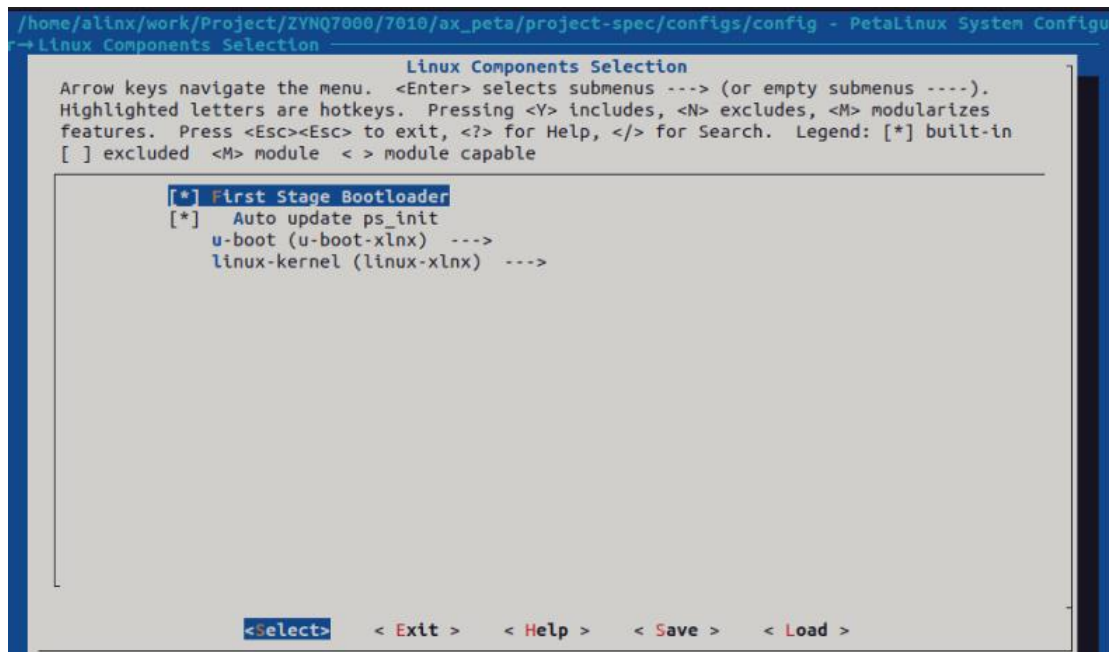
注意：①一定要使用和 petalinux 对应版本；②如果多个 petalinux 工程都需要使用本地源码，那么每一个 petalinux 工程都需要单独对应内核源码和 u-boot 源码，否则其中一个 petalinux 工程修改了源码，就会影响到其他的工程，教程后面的章节只需要将内核配置为本地的，uboot 配置保持不变。

名称	修改日期	类型	大小
downloads_2023.1_05010539.tar	2023/8/28 12:06	WinRAR 压缩文件	49,109,68...
linux-xlnx-xlnx_rebase_v6.1_LTS	2023/9/4 16:49	WinRAR ZIP 压缩...	264,767 KB
sstate_aarch64_2023.1_05010539.tar	2023/8/28 17:39	WinRAR 压缩文件	53,649,46...
u-boot-xlnx-xlnx_rebase_v2023.01	2023/9/4 16:45	WinRAR ZIP 压缩...	33,768 KB

将内核源码复制到虚拟机内解压。



- 11) 将 linux 内核的来源改为本地获取，并配置对应下载的离线包解压后的地址，uboot 保持默认。



```
/home/alinx/work/Project/ZYNQ7000/7010/ax_peta/project-spec/configs/config - PetaLinux System Configur
r->Linux Components Selection

Linux Components Selection
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

[*] First Stage Bootloader
[*] Auto update ps_init
    u-boot (u-boot-xlnx) --->
    linux-kernel (ext-local-src) --->
        External linux-kernel local source settings --->

<Select> < Exit > < Help > < Save > < Load >

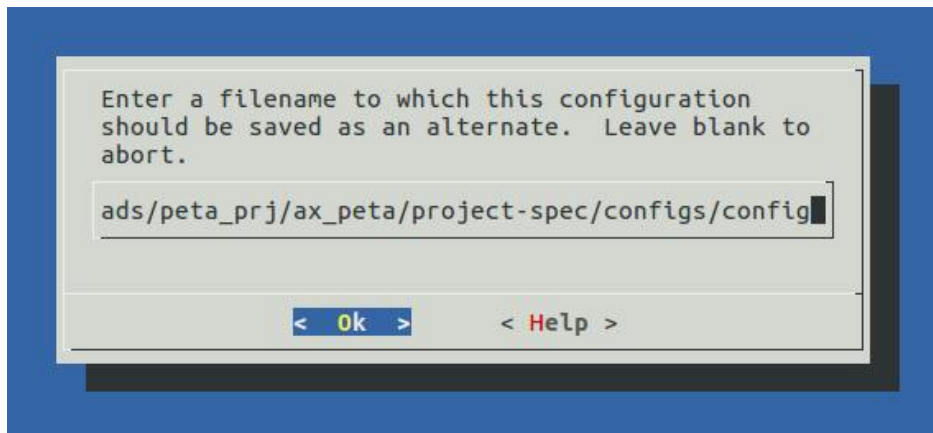
/home/alinx/work/Project/ZYNQ7000/7010/ax_peta/project-spec/configs/config - PetaLinux System Configur
r->Linux Components Selection ->External linux-kernel local source settings

External linux-kernel local source settings
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

(/home/alinx/Downloads/linux-xlnx-xlnx_rebase_v6.1.LTS) External linux-kernel loc
() External linux-kernel license checksum

<Select> < Exit > < Help > < Save > < Load >
```

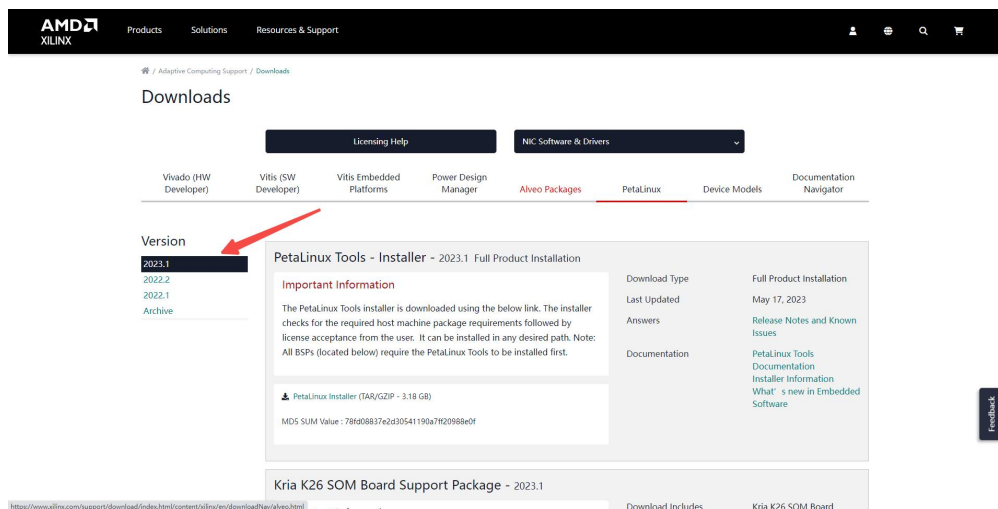
12) 配置完成后保存设置

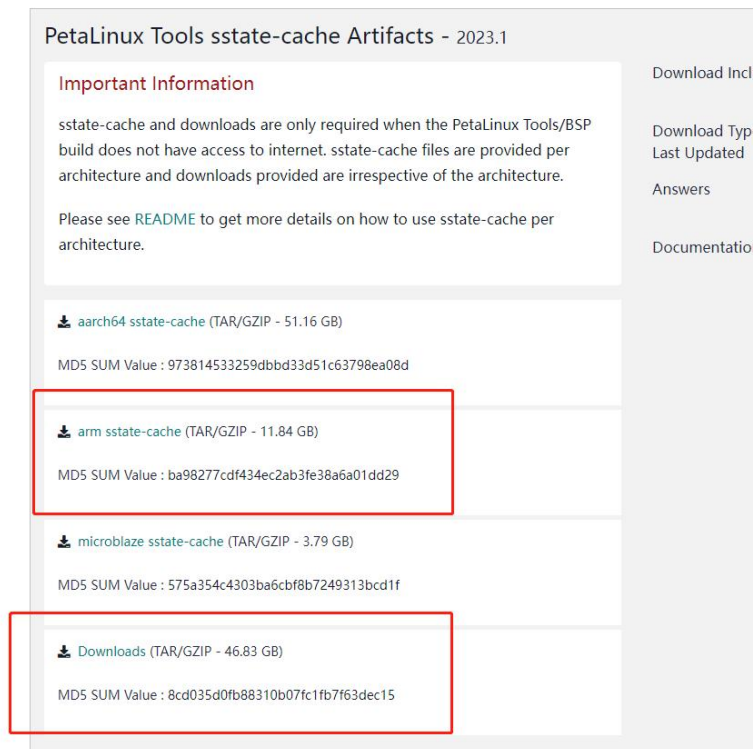


13) 退出配置界面等待一段时间

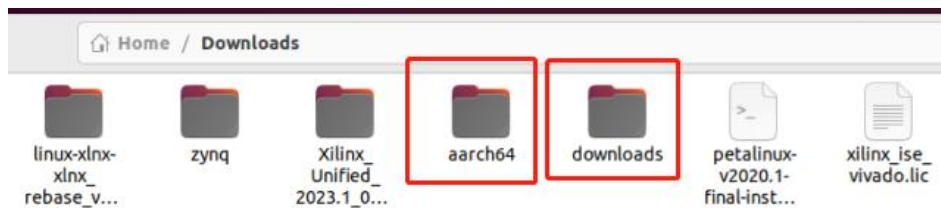
5.3 设置离线编译

Petalinux 编译工程默认需要联网下载一些资源，如果网络不佳会编译非常缓慢甚至编译失败。为了解决这个问题，我们可以预先下载两个离线资源包，然后设置 petalinux 离线编译这两个离线资源包也可以从 xilinx 官网获取：<https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-design-tools/2023-1.html> 下载下图中红框标出来的两个文件。注意，一定要下载 petalinux 版本对应的文件，我们使用的 petalinux 版本是 2023.1，就需要找到 2023.1 标签中的这两个文件下载。





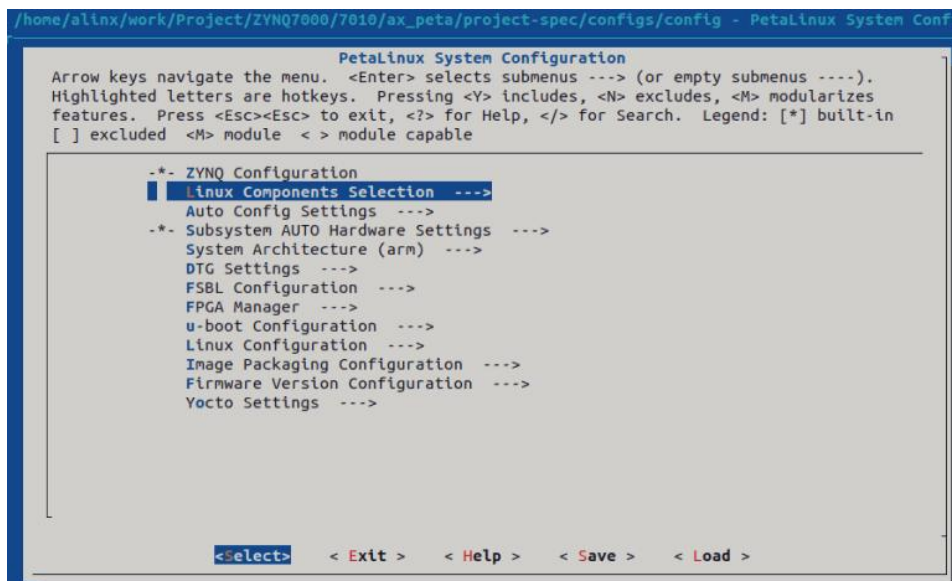
1. 将资源包拷贝到虚拟机中并解压。这里笔者解压在/home/Downloads/ 路径下，如下图：



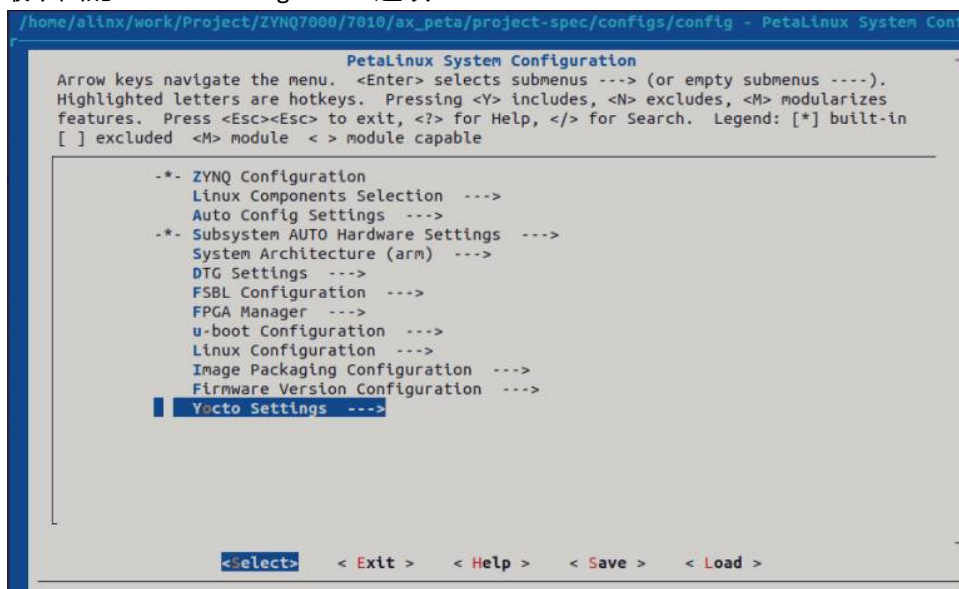
2. 在 petalinux 工程中打开终端，输入命令

```
petalinux-config
```

然后会弹出如下配置界面：



3. 选择最下面的“Yocto Settings --->”选项



4. 在“Yocto Settings --->”界面中，取消选择“Enable Network sstate feeds”，选择“”选项，修改成如下图的状态：

```
/home/alinx/work/Project/ZYNQ7000/7010/ax_peta/project-spec/configs/config - Petalinux System Config
r→ Yocto Settings

Yocto Settings
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu --->).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

(zynq-generic) YOCTO_MACHINE_NAME
TMPDIR Location --->
Devtool Workspace Location --->
(${PROOT}/images/linux) Petalinux Deploy Directory
() Petalinux Multiconfig Deploy Directory
Parallel thread execution --->
Add pre-mirror url --->
Local sstate feeds settings --->
[ ] Enable Network sstate feeds
[*] Enable BB NO NETWORK
[ ] Enable Buildtools Extended
User Layers --->

<Select> < Exit > < Help > < Save > < Load >
```

5. 设置“sstate_aarch64”包的路径，选择“Local sstate feeds settings --->”选项，配置相应的解压文件所在路径

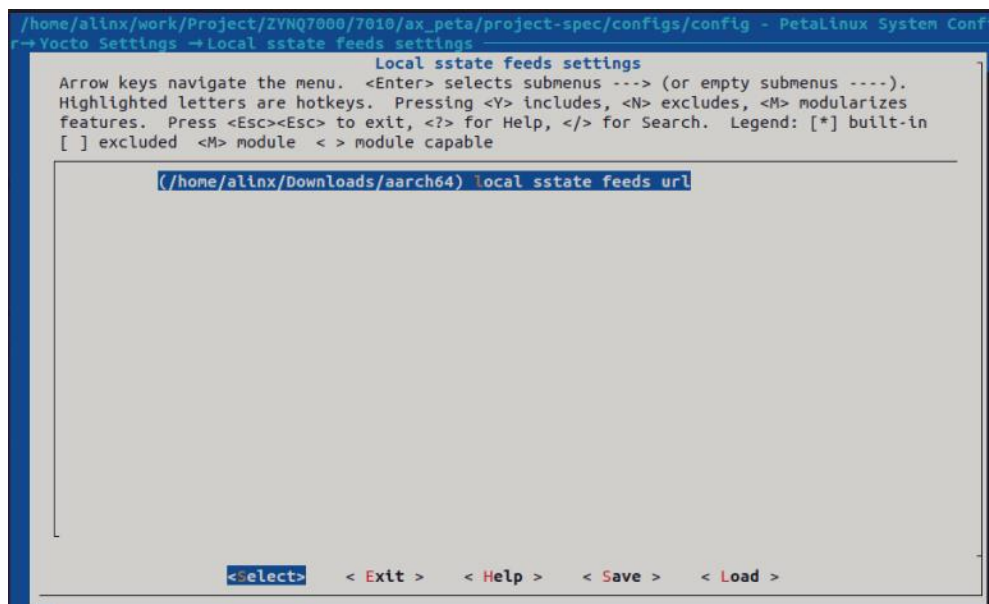
/home/alinx/Downloads/aarch64

```
/home/alinx/work/Project/ZYNQ7000/7010/ax_peta/project-spec/configs/config - Petalinux System Config
r→ Yocto Settings

Yocto Settings
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu --->).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

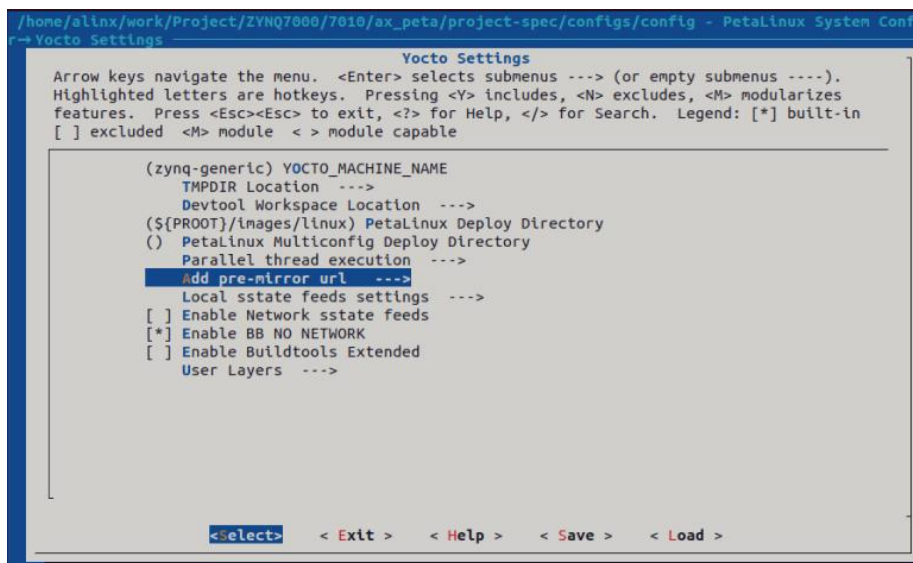
(zynq-generic) YOCTO_MACHINE_NAME
TMPDIR Location --->
Devtool Workspace Location --->
(${PROOT}/images/linux) Petalinux Deploy Directory
() Petalinux Multiconfig Deploy Directory
Parallel thread execution --->
Add pre-mirror url --->
[ ] Local sstate feeds settings --->
[ ] Enable Network sstate feeds
[*] Enable BB NO NETWORK
[ ] Enable Buildtools Extended
User Layers --->

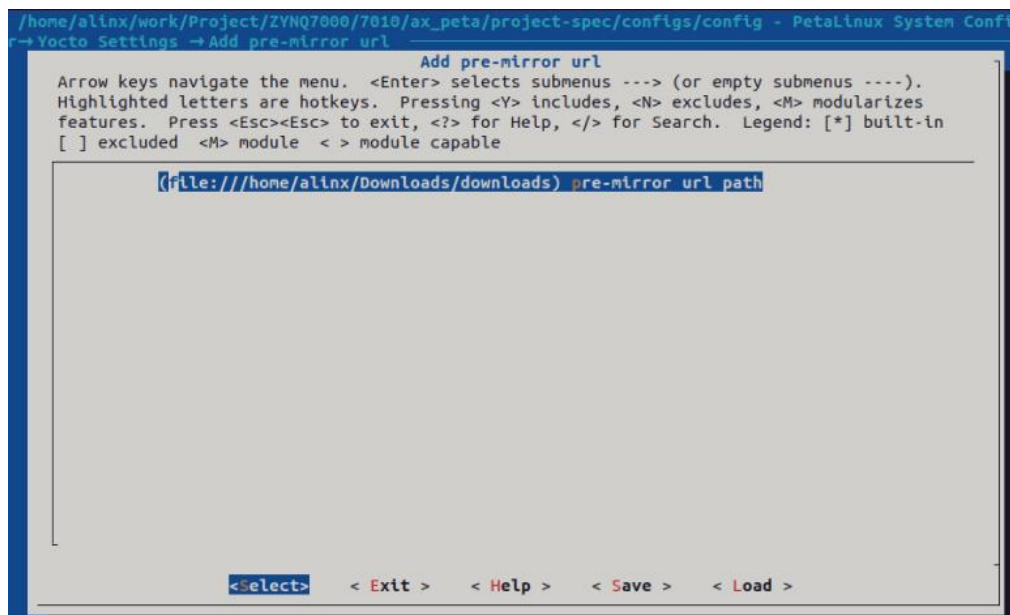
<Select> < Exit > < Help > < Save > < Load >
```



6. 设置“downloads”包的路径，先选择< Exit >选项返回到下图的界面，然后选择“Add pre-mirror url --->”选项

file:///home/alinx/Downloads/downloads





7. 选择< Save >选项，然后选择< Exit >选项退出配置界面即可

5.4 配置 Linux 内核

- 1) 使用下面命令配置内核，运行命令后又要等待很长一段时间

```
petalinux-config -c kernel
```

```

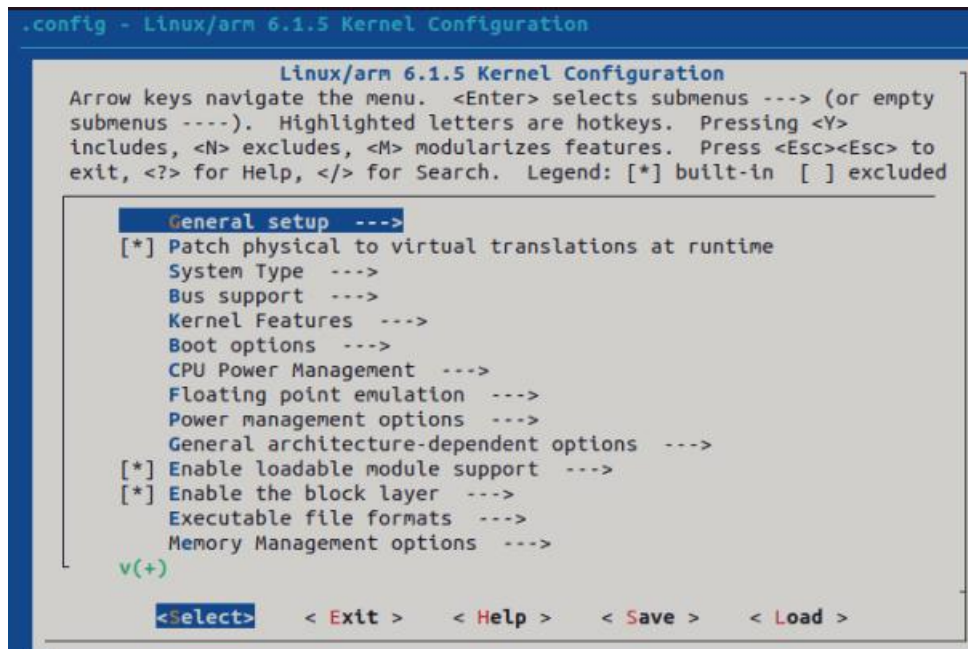
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
eta
~/Downloads/peta_prj/ax_peta
[INFO] generating u-boot configuration files

[INFO] generating kernel configuration files
[INFO] generating kconfig for Rootfs
Generate rootfs kconfig
[INFO] oldconfig rootfs
[INFO] generating petalinux-user-image.bb
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ petalinux-config -c kernel
[INFO] generating Kconfig for project

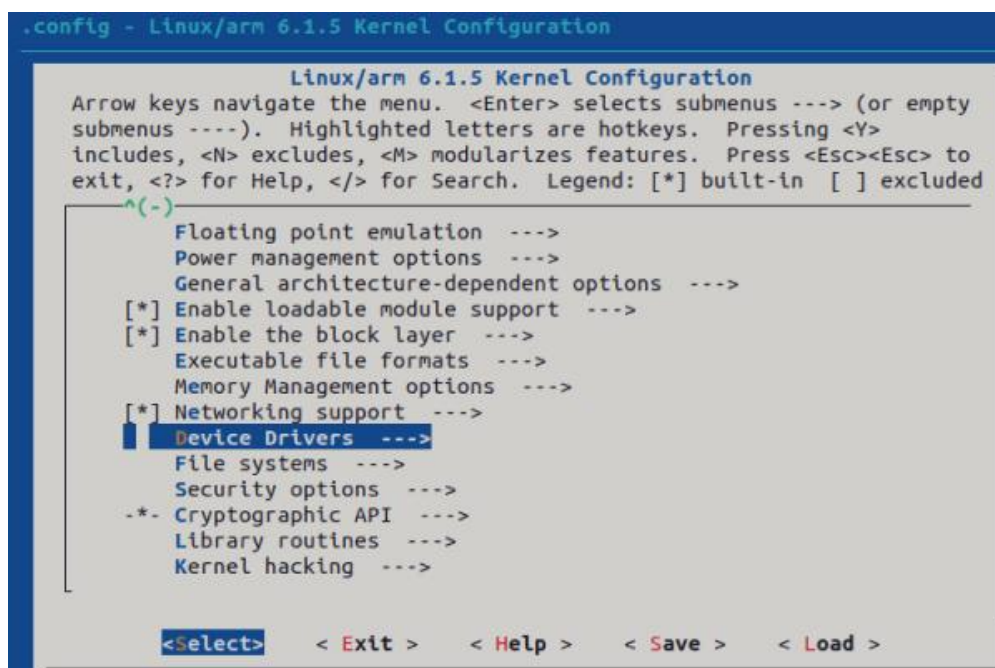
[INFO] sourcing bitbake
[INFO] generating plnxtool conf
[INFO] generating meta-plnx-generated layer
~/Downloads/peta_prj/ax_peta/build/misc/plnx-generated ~/Downloads/peta_prj/ax_p
eta
~/Downloads/peta_prj/ax_peta
[INFO] generating machine configuration
[INFO] configuring: kernel
[INFO] generating kernel configuration files
[INFO] bitbake virtual/kernel -c menuconfig
Parsing recipes: 14% |#####| ETA: 0:01:29
| ETA: 0:01:31

```

- 2) 等待一段时间后弹出配置内核的配置界面



- 3) 由于以太网的 phy 芯片驱动默认没有打开，需要配置驱动，在选项 Device Drivers 配置驱动



- 4) 然后到 Network device support > PHY Device support and infrastructure , 选中 Micrel PHYs, 按 Y 键。

```
.config - Linux/arm 6.1.5 Kernel Configuration
[...] Drivers > Network device support > PHY Device support and infrastructure
PHY Device support and infrastructure
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty
submenus ---). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded
^(-)
< > LSI ET1011C PHY
< * > Marvell Alaska PHYs
< > Marvell Alaska 10Gbit PHYs
< > Marvell 88X2222 PHY
< > Maxlinear Ethernet PHYs
< > MediaTek Gigabit Ethernet PHYs
< > Micrel PHYs
< * > Microchip PHYs
< > Microchip T1 PHYs
< > Microsemi PHYs
< > Motorcomm PHYs
< > National Semiconductor PHYs
< > NXP C45 TJA11XX PHYs
< > NXP TJA11xx PHYs support
v(+)
< Select > < Exit > < Help > < Save > < Load >
```

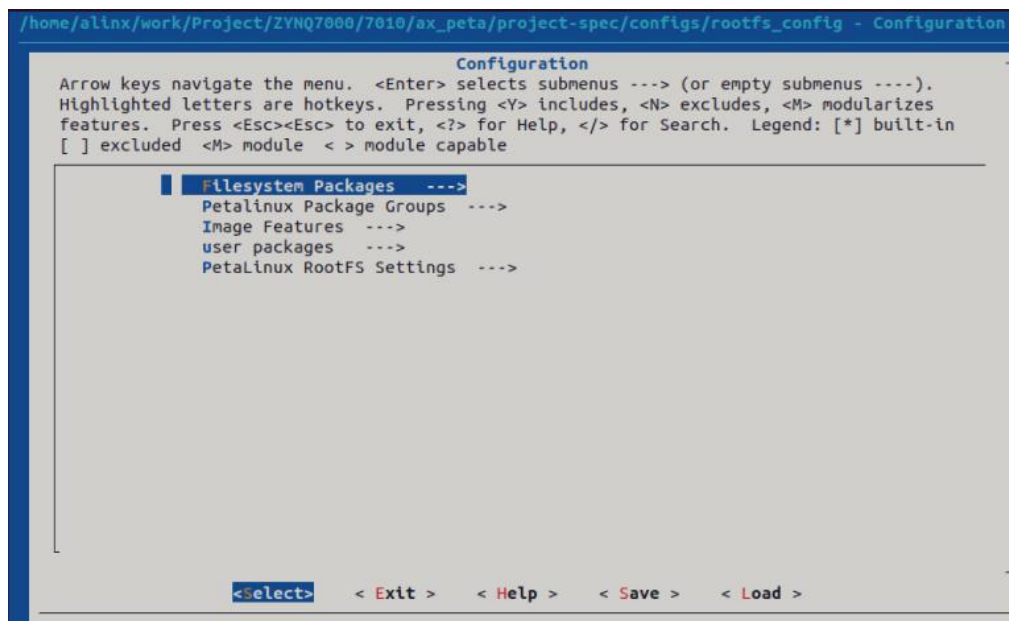
5) 其他的不需要再配置，保存配置并退出

```
.config - Linux/arm 6.1.5 Kernel Configuration
[...] Drivers > Network device support > PHY Device support and infrastructure
PHY Device support and infrastructure
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty
submenus ---). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded
^(-)
< > LSI ET1011C PHY
< * > Marvell Alaska PHYs
< > Marvell Alaska 10Gbit PHYs
< > Marvell 88X2222 PHY
< > Maxlinear Ethernet PHYs
< > MediaTek Gigabit Ethernet PHYs
< > Micrel PHYs
< * > Microchip PHYs
< > Microchip T1 PHYs
< > Microsemi PHYs
< > Motorcomm PHYs
< > National Semiconductor PHYs
< > NXP C45 TJA11XX PHYs
< > NXP TJA11xx PHYs support
v(+)
< Select > < Exit > < Help > < Save > < Load >
```

5.5 配置根文件系统

运行下面的命令配置根文件系统，可以根据需求来配置根文件系统，本实验保持默认配置。

```
petalinux-config -c rootfs
```



5.6 编译

- 1) 使用下面命令配置编译 uboot、内核、根文件系统、设备树等。

```
petalinux-build
```

注：即便设置了离线编译，但在构建工程时也需要保证 ubuntu 系统可以连接网络，petalinux 需要从 github 上下载一些源代码

- 2) 编译完成

```
[INFO] building project
[INFO] sourcing bitbake
INFO: bitbake petalinux-user-image
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3257 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2466 .bb files complete (2434 cached, 32 parsed). 3259 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:08
Checking sstate mirror object availability: 100% |#####| Time: 0:32:53
NOTE: Executing SetScene Tasks
NOTE: Executing RunQueue Tasks
fsbl-2017.4+gitAUTOINC+77448ae629-r0 do_compile: NOTE: fsbl: compiling from exte
rnal source tree /opt/pkg/petalinux/tools/hsm/data/embeddedsw
NOTE: Tasks Summary: Attempted 2444 tasks of which 1884 didn't need to be rerun
and all succeeded.
INFO: Copying Images from deploy to images
INFO: Creating images/linux directory
NOTE: Failed to copy built images to tftp dir: /tftpboot
[INFO] successfully built project
```

5.7 生成 BOOT 文件

运行下面命令生成 BOOT 文件，“--fpga”指合成 bit 文件到 BOOT.bin 中(注意：如果你的 xsa 文件中不包含 bitstream，就不要加这个参数)

```
petalinux-package --boot --u-boot --fsbl --fpga --force
```

```
alinx@ubuntu:~/peta_prj/ax_peta$ petalinux-package --boot --fsbl --fpga --u-boot --force
[INFO] Sourcing buildtools
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax_peta/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax_peta/project-spec/hw-description/design_1_wrapper.bit"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax_peta/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax_peta/images/linux/system.dtb"
INFO: Generating zynq binary package BOOT.BIN...

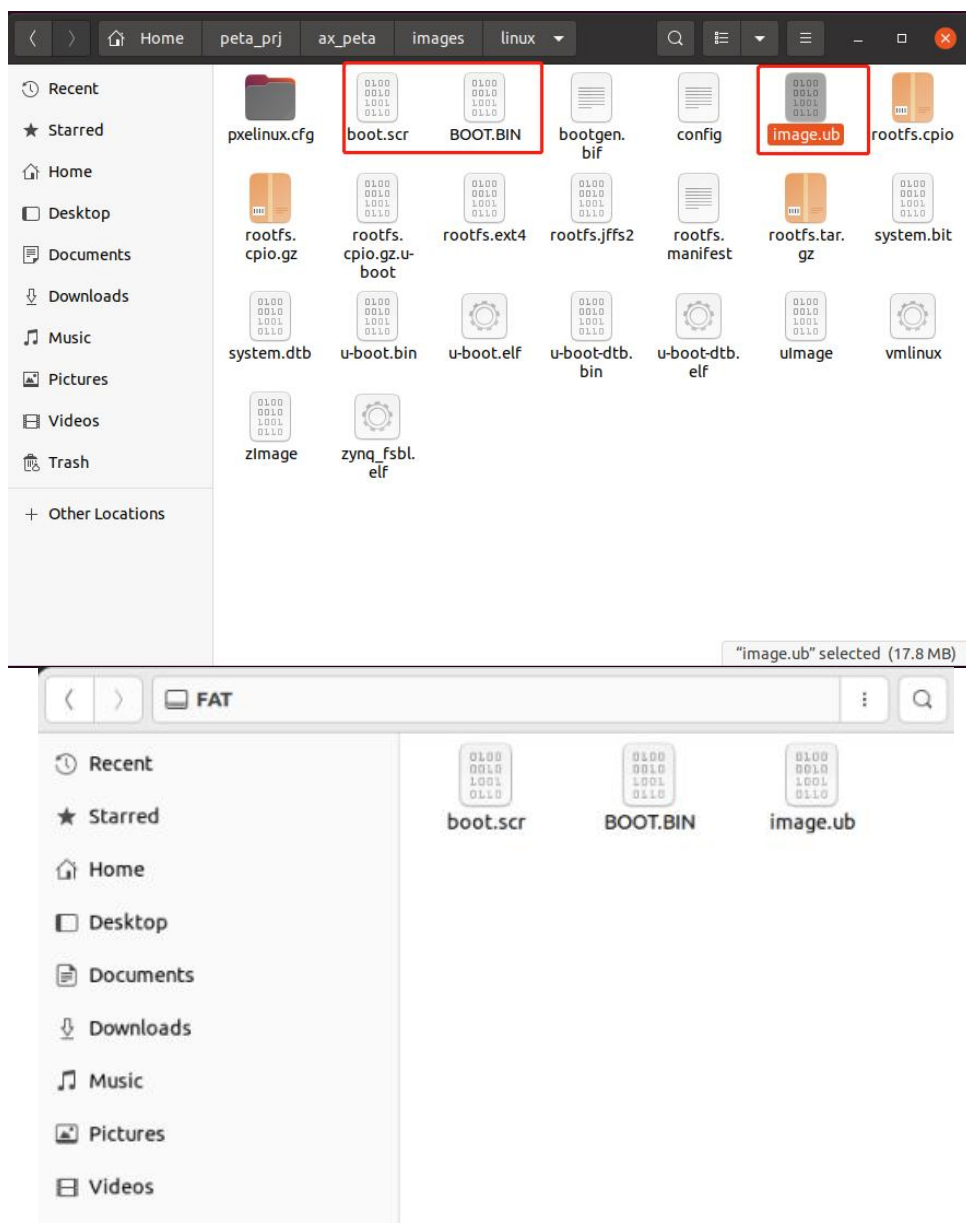
***** Bootgen v2023.1
**** Build date : Apr  7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition design_1_wrapper.bit.0 memory range
[INFO] : Bootimage generated successfully

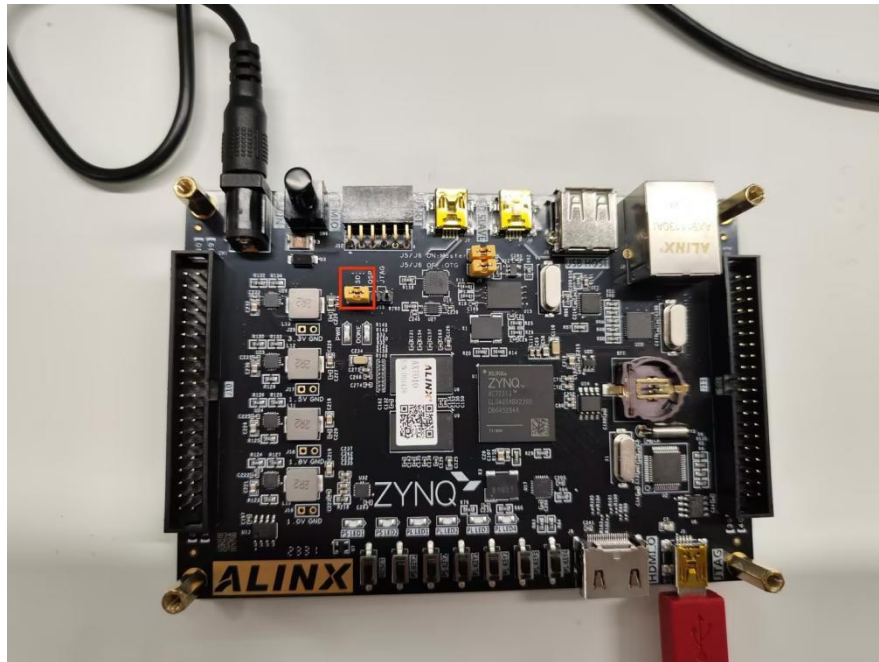
INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
alinx@ubuntu:~/peta_prj/ax_peta$
```

5.8 测试 Linux

- 1) 将 SD 卡格式为 FAT32 格式，可用 Ubuntu 系统上的 Disks 软件格式化 SD 卡，具体操作请参考第九章的 9.4 制作 SD 卡文件系统。
- 2) 将工程目录 images -> linux 目录中的 BOOT.BIN、boot.scr 及 image.ub 复制到 sd 卡的 FAT 分区。

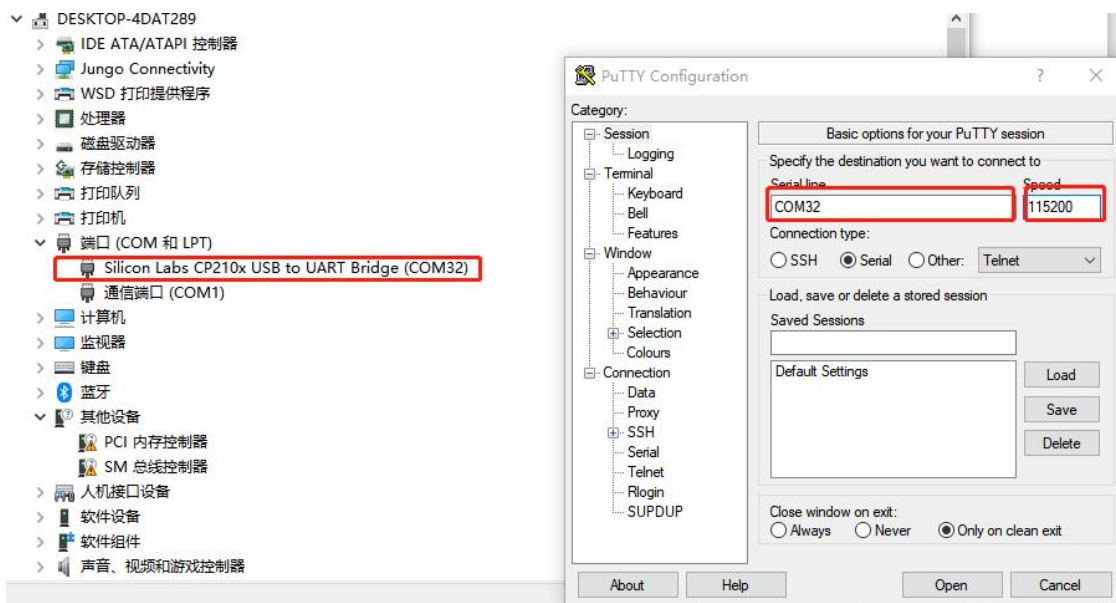


3) 将 SD 卡插入板卡内，跳线帽选择 SD 卡启动模式，板卡连接好串口。

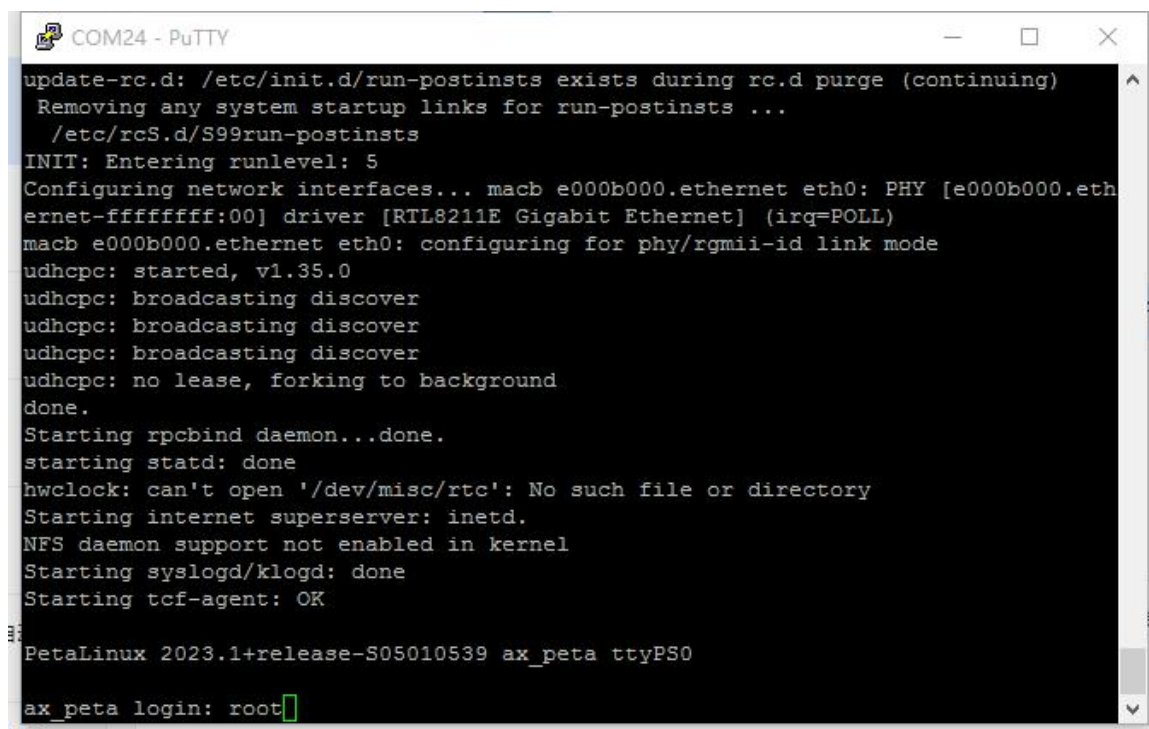


- 4) 解压资料中的 CP210x_Windows_Drivers 压缩包，根据当前系统安装相应的驱动，然后打开 putty 软件，根据设备管理器中的端口号，在软件中填入相关信息，设置好波特率。

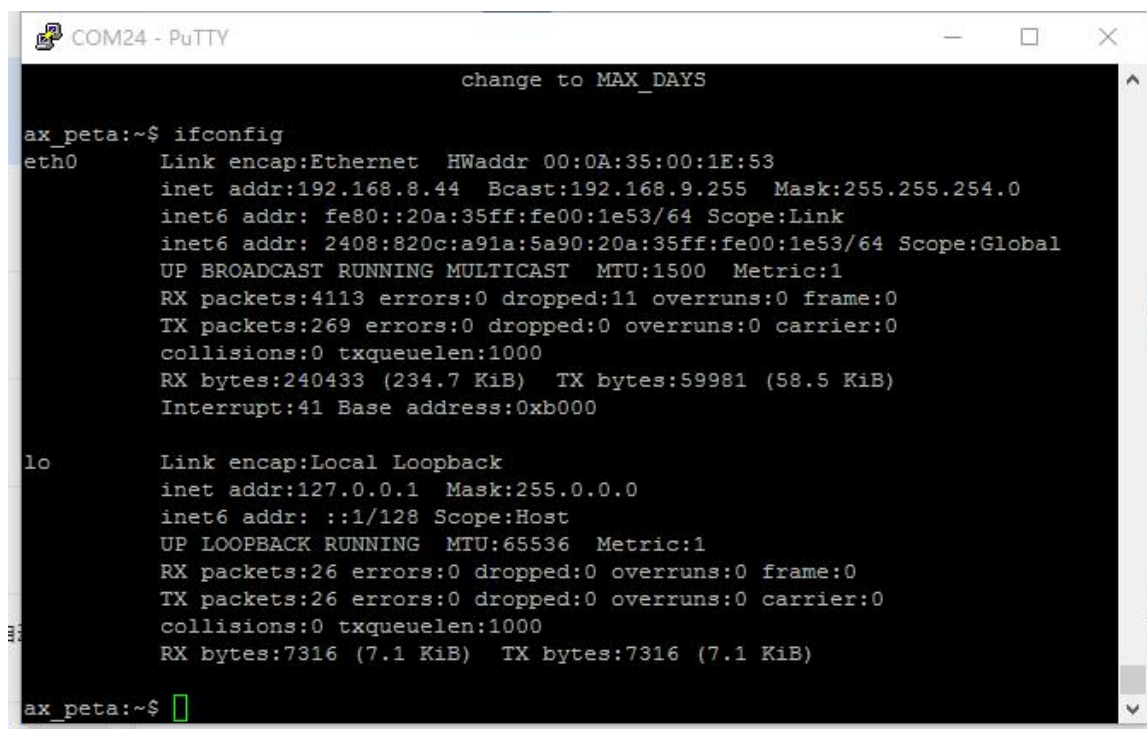
- x86
- x64
- slabvcp.inf
- slabvcp.cat
- SLAB_License_Agreement_VCP_Windows.txt
- dpinst.xml
- CP210xVCPInstaller_x86.exe
- CP210xVCPInstaller_x64.exe



5) 板卡插上电源，上电启动开发板



6) 使用 petalinux 登录，密码由用户设置，然后需要再次输入密码，成功后就可以进入系统控制台进行操作，插上网线后（路由器支持自动获取 IP），使用 ifconfig 命令可以看到网络状态。



```
change to MAX_DAYS

ax_peta:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0A:35:00:1E:53
          inet addr:192.168.8.44  Bcast:192.168.9.255  Mask:255.255.254.0
          inet6 addr: fe80::20a:35ff:fe00:1e53/64  Scope:Link
          inet6 addr: 2408:820c:a91a:5a90:20a:35ff:fe00:1e53/64  Scope:Global
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4113 errors:0 dropped:11 overruns:0 frame:0
          TX packets:269 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:240433 (234.7 KiB)  TX bytes:59981 (58.5 KiB)
          Interrupt:41 Base address:0xb000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128  Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:26 errors:0 dropped:0 overruns:0 frame:0
          TX packets:26 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7316 (7.1 KiB)  TX bytes:7316 (7.1 KiB)

ax_peta:~$
```

5.9 创建和使用 bsp 包

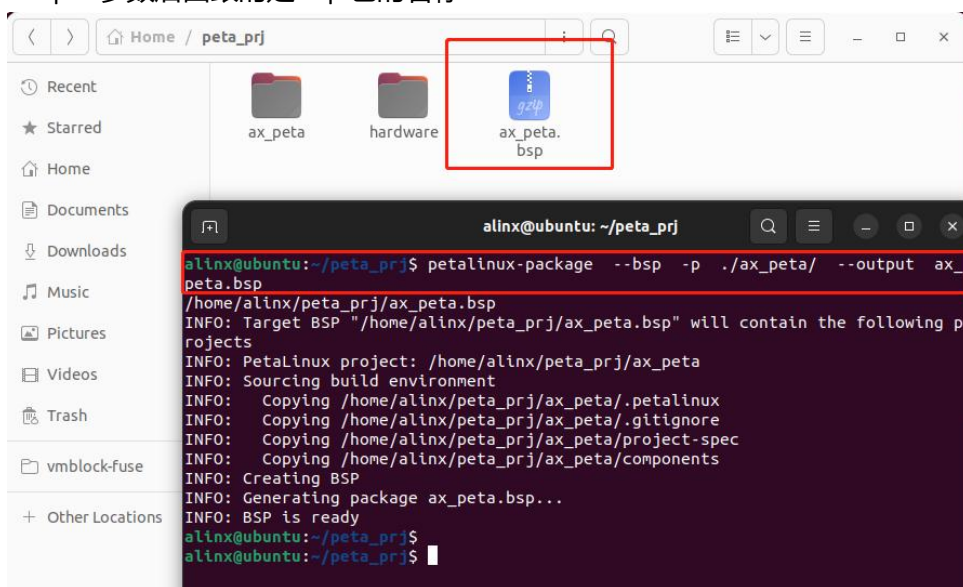
使用 petalinux 的 bsp 包,我们可以方便快速的提供我们验证好的 petalinux 工程给他人使用,或者是使用他人验证好的 petalinux 工程。只需要掌握创建和使用 bsp 包的两条命令即可。

1. 创建 bsp 包

使用下面的命令即可创建 petalinux 工程的 bsp 包到当前打开终端的路径中:

```
petalinux-package --bsp -p ./ax_peta/ --output ax_peta.bsp
```

- -p 参数后面的路径是 petalinux 工程的一级路径
- --output 参数后面跟的是 bsp 包的名称

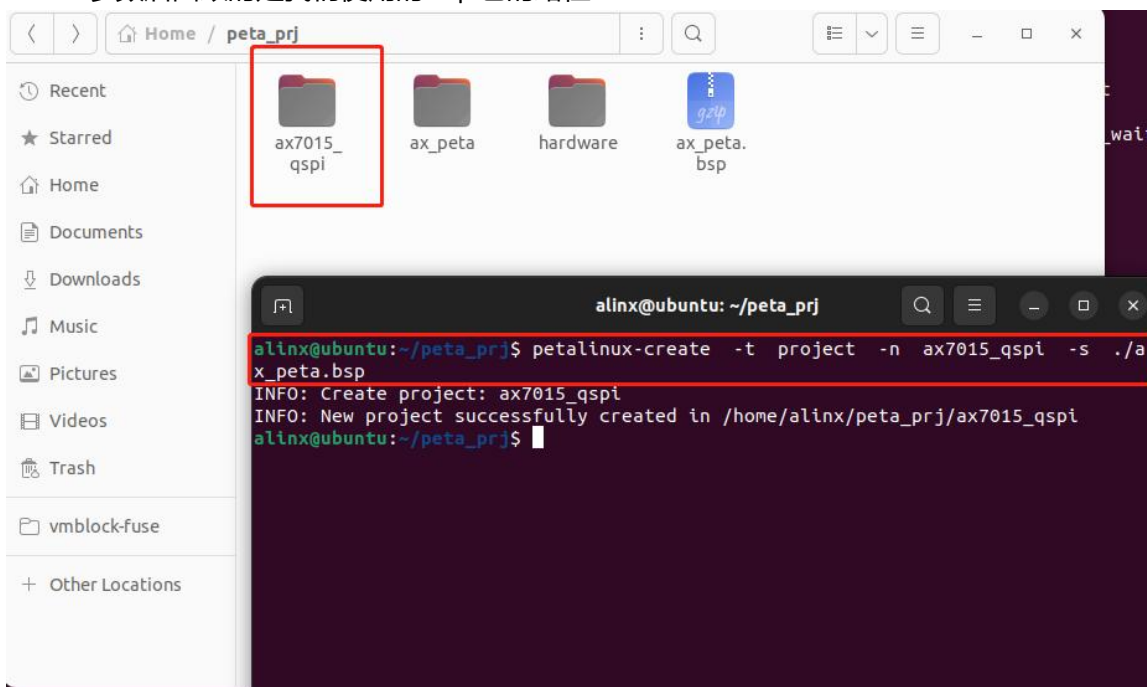


2. 使用 bsp 包

使用下面的命令即可用 bsp 包来创建 petalinux 工程到当前打开终端的路径中：

```
petalinux-create -t project -n ax7015_qspi -s ./ax_peta.bsp
```

- -n 参数后面跟的是 petalinux 工程的名称
- -s 参数后面跟的是我们使用的 bsp 包的路径



注：如果制作 bsp 包的工程可以编译成功，那使用 bsp 包制作的工程也可以直接编译。

```
[INFO] Generating workspace directory
^INFO: bitbake petalinux-image-minimal
NOTE: Started PRServer with DBfile: /home/alinx/peta_prj/ax7015_qspi/build/cache/prserv.
sqlite3, Address: 127.0.0.1:34897, PID: 6565
Loading cache: 100% |#####| Time: 0:0
0:08
Loaded 6269 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:0
0:05
Parsing of 4344 .bb files complete (4338 cached, 6 parsed). 6275 targets, 357 skipped, 1
masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
WARNING: /home/alinx/peta_prj/ax7015_qspi/components/yocto/layers/meta-xilinx/meta-xilin
x-core/recipes-bsp/u-boot/u-boot-xlnx_2023.1.bb:do_compile is tainted from a forced run0
0:02
Initialising tasks: 100% |#####| Time: 0:00:48
Sstate summary: Wanted 156 Local 156 Mirrors 0 Missed 0 Current 1364 (100% match, 100% c
omplete)
WARNING: The busybox:do_fetch sig is computed to be 16146b24d18a0700bfca13cdb5b9e2b8f48f
d710c6a24ce7f779c35c33ad65c0, but the sig is locked to c2ac5f0c29da84190d0f46f4f48dcf67e
f7255138a4424d0a00004f7f614a942 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 4109 tasks of which 4105 didn't need to be rerun and all
succeeded.

Summary: There were 2 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
alinx@ubuntu:~/peta_prj/ax7015_qspi$
```

5.10 常见问题

5.10.1 提示 “Bad FIT kernel image format!” 无法启动内核

```
U-Boot 2017.01 (Sep 05 2018 - 15:31:52 +0800)

Board: Xilinx Zynq
I2C: ready
DRAM: ECC disabled 1 GiB
MMC: sdhci_transfer_data: Error detected in status(0x208000)!
sdhci@e0100000: 0 (SD), sdhci@e0101000: 1 (eMMC)
SF: Detected w25q256 with page size 256 Bytes, erase size 4 KiB, total 32 MiB
*** Warning - bad CRC, using default environment

In: serial
Out: serial
Err: serial
Net: ZYNQ GEM: e000b000, phyaddr ffffffff, interface rgmii-id

Warning: ethernet@e000b000 (eth1) using random MAC address - 5a:51:56:3c:ab:16
eth1: ethernet@e000b000
U-BOOT for ax_peta

ethernet@e000b000 Waiting for PHY auto negotiation to complete..... TIMEOUT !
Hit any key to stop autoboot: 0
Device: sdhci@e0100000
Manufacturer ID: 41
OEM: 3432
Name: SD16G
Tran Speed: 50000000
Rd Block Len: 512
SD version 3.0
High Capacity: Yes
Capacity: 15 GiB
Bus Width: 4-bit
Erase Group Size: 512 Bytes
reading image.ub
Invalid FAT entry
4096 bytes read in 13 ms (307.6 KiB/s)
## Loading kernel from FIT Image at 10000000 ...
Bad FIT kernel image format!
ERROR: can't get kernel image!
Zynq>
```

解决办法：

将 sd 卡 fat32 分区再次格式化，重新放置启动文件。

5.10.2 无法保存文件和配置

因为 petalinux 默认文件系统是 RAM 类型，无法保存，后续教程中可设置为 SD 卡类型，数据就能保存到 SD 卡。

5.10.3 输入正确的 root 账户密码, 登不上系统, 可以使用 petalinux 用户登录, 然后再切换到 root 用户

```
ax_peta:~$ sudo -i

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

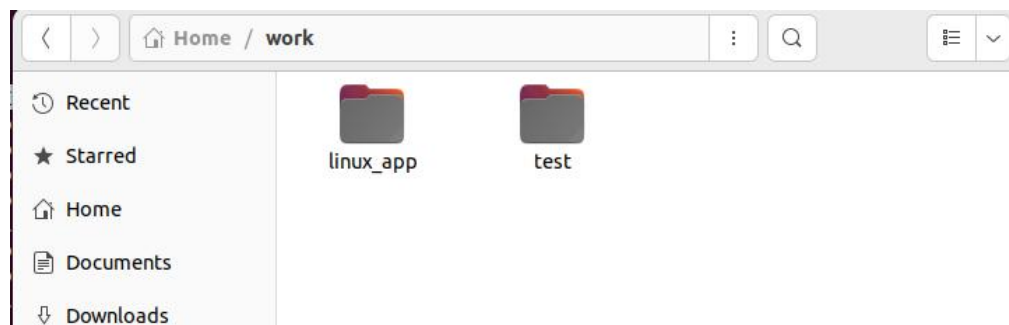
Password:
root@ax_peta:~# su petalinux
petalinux@ax_peta:/home/root$ cd
petalinux@ax_peta:~$ sudo -i
```

第六章 使用 Vitis 开发 Linux 程序

前面的教程中我们使用 petalinux 制作了一个嵌入式 Linux 系统, 本实验要做一个 Linux 应用程序, 然后在开发板上运行。本实验需要使用上面的实验中做好的 Linux 运行环境。

6.1 使用 Vitis 建立 Linux 应用程序

- 1) 在/home/alinx/work 建立目录 linux_app, 用于 Vitis 的工作空间



- 2) 设置 Vitis 环境变量, 运行 vitis

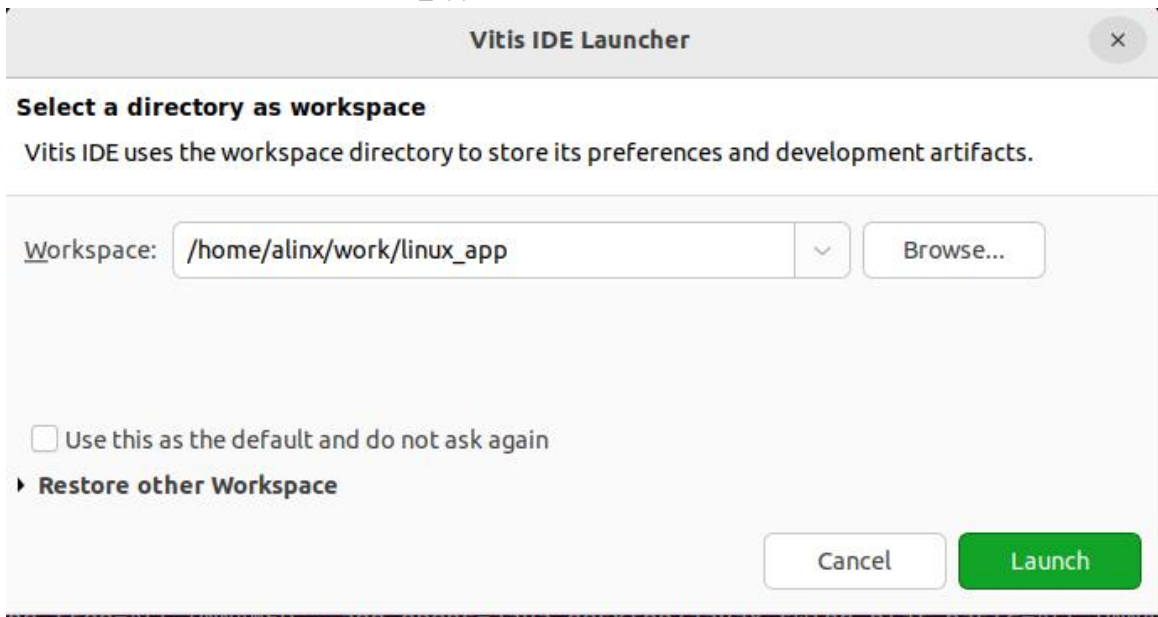
```
source /tools/Xilinx/Vitis/2023.1/settings64.sh
vitis
```

```
alinx@ubuntu:~/work$ source /tools/Xilinx/Vitis/2023.1/settings64.sh
alinx@ubuntu:~/work$ vitis

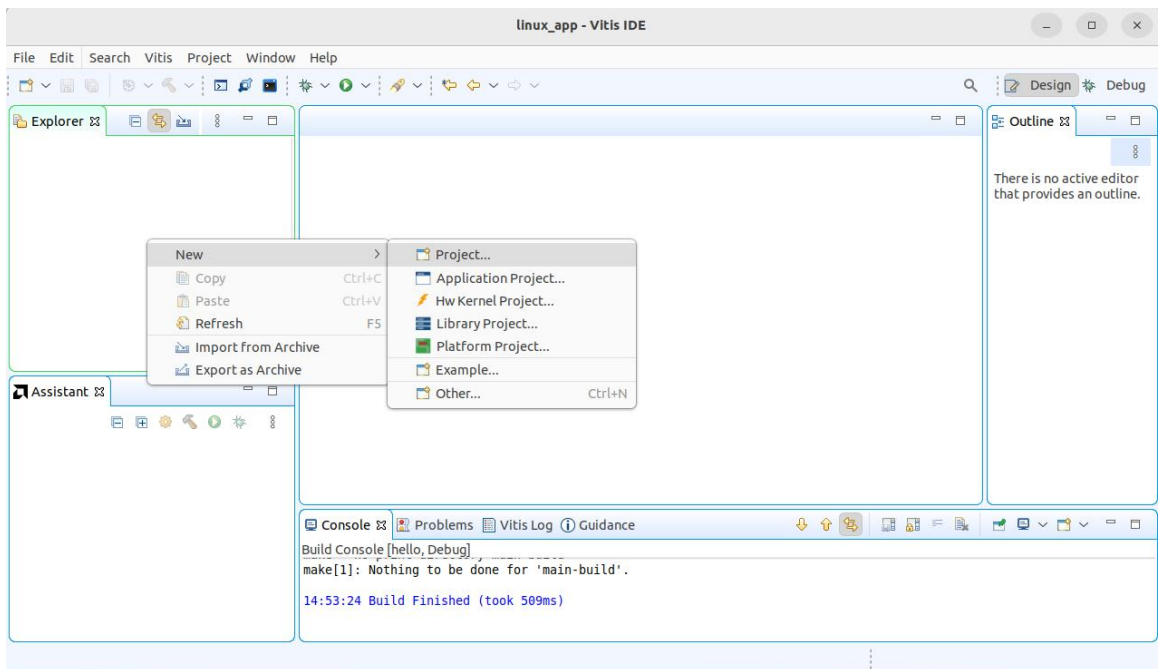
***** Vitis Development Environment
***** Vitis v2023.1 (64-bit)
**** SW Build 3860322 on 2023-05-04-06:32:46
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

Launching Vitis with command /tools/Xilinx/Vitis/2023.1/eclipse/lx64.o/eclipse
-vmargs -Xms64m -Xmx1024m -Dorg.eclipse.swt.internal.gtk.cairoGraphics=false -Do
sgl.configuration.area=@user.home/.Xilinx/Vitis/2023.1 --add-modules=ALL-SYSTEM
--add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.desktop/sun.SWING=AL
L-UNNAMED --add-opens=java.desktop/javafx.swing=ALL-UNNAMED --add-opens=java.desk
top/javafx.swing.tree=ALL-UNNAMED --add-opens=java.desktop/javafx.swing.plaf.bas
ic=ALL-UNNAMED --add-opens=java.desktop/javafx.swing.plaf.synth=ALL-UNNAMED --add-o
pens=java.desktop/com.sun.awt=ALL-UNNAMED --add-opens=java.desktop/sun.awt.X11=A
LL-UNNAMED &
```

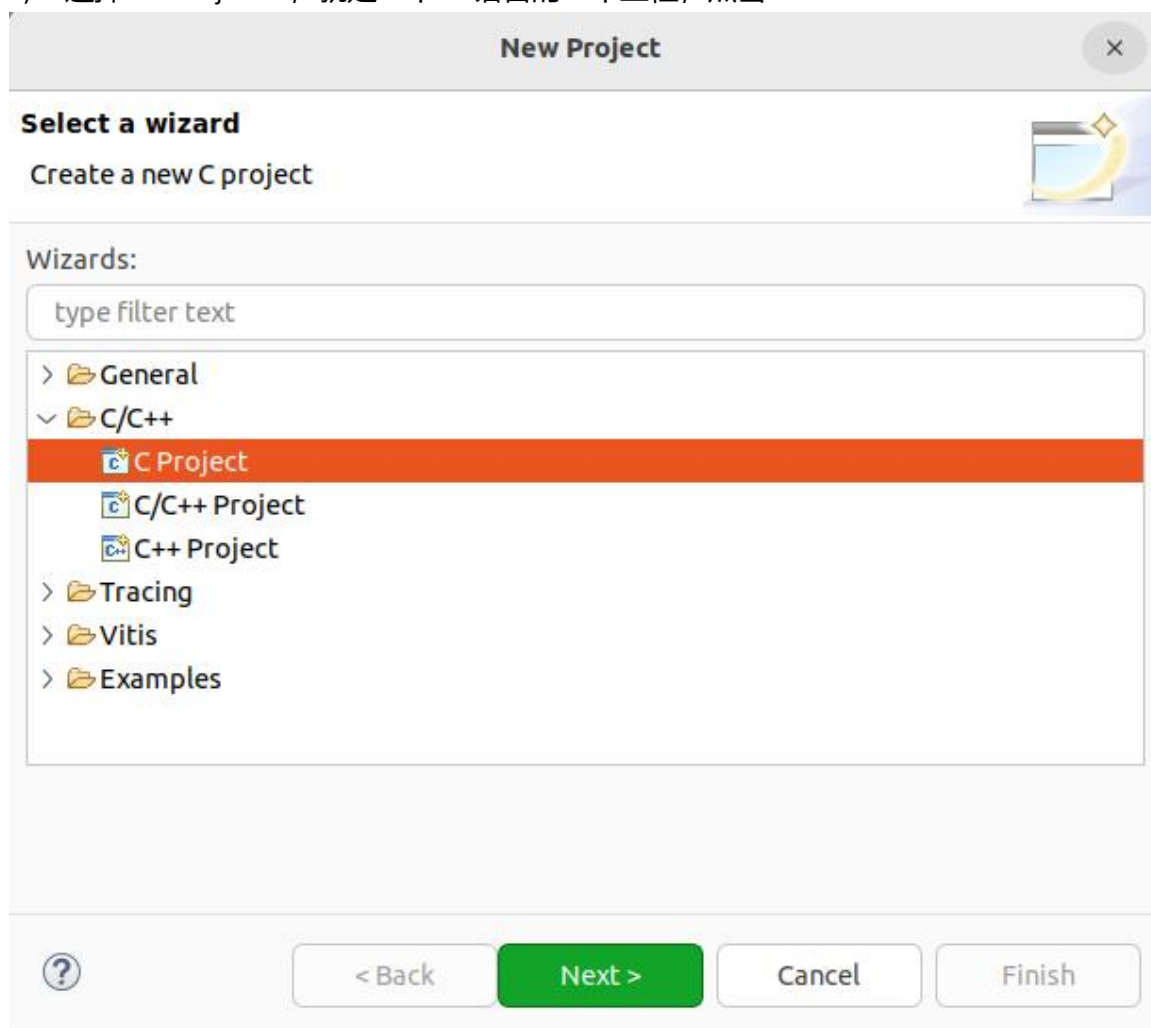
- 3) 工作空间选/home/alinx/work/linux_app



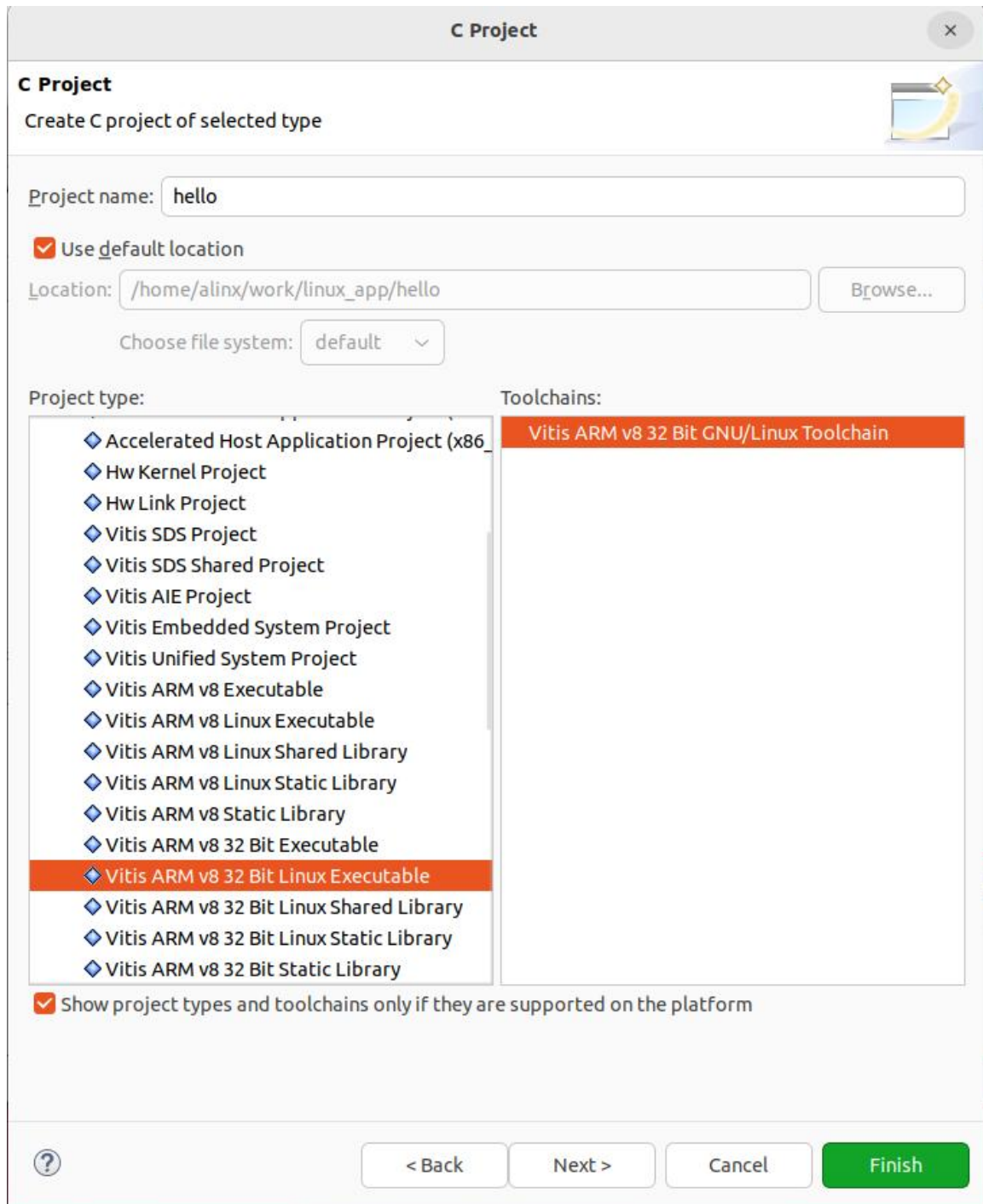
- 4) 关闭欢迎界面后进入到编辑界面，在 Explorer 空白处点击右键 “New->Project...”



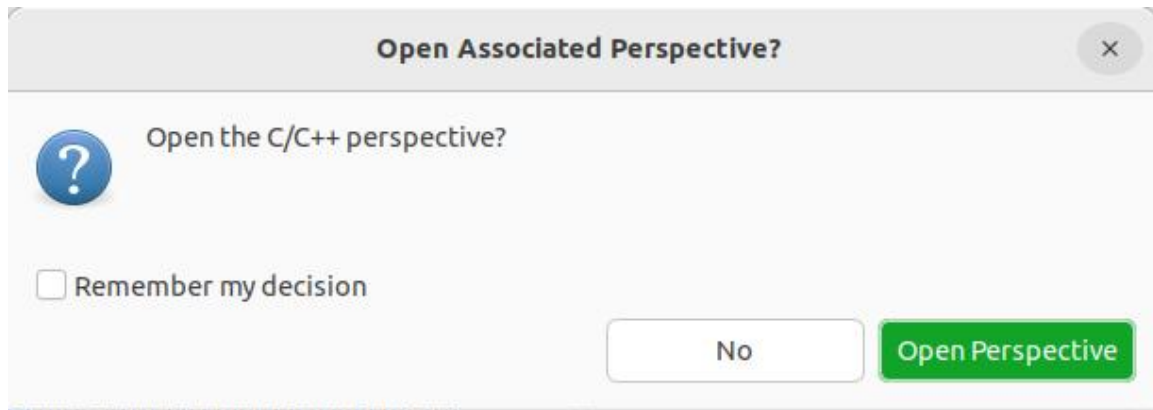
- 5) 选择 “C Project” ，就是一个 C 语言的一个工程，点击 “Next”



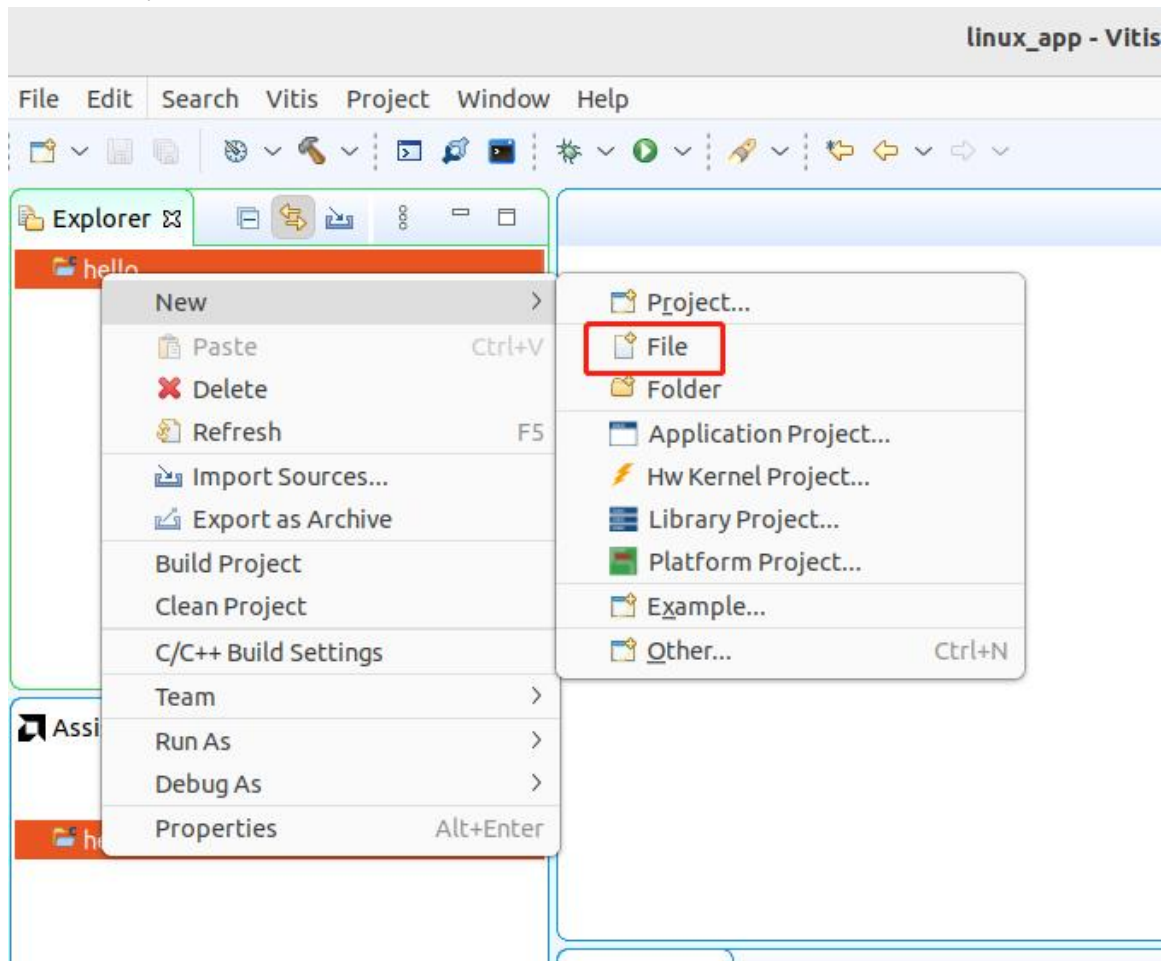
- 6) 工程名填写 “hello” ，工程类型选择 Others 文件夹里面的 “Xilinx ARM v8 32 Bit Linux Executable” ，然后点击 “Finish”



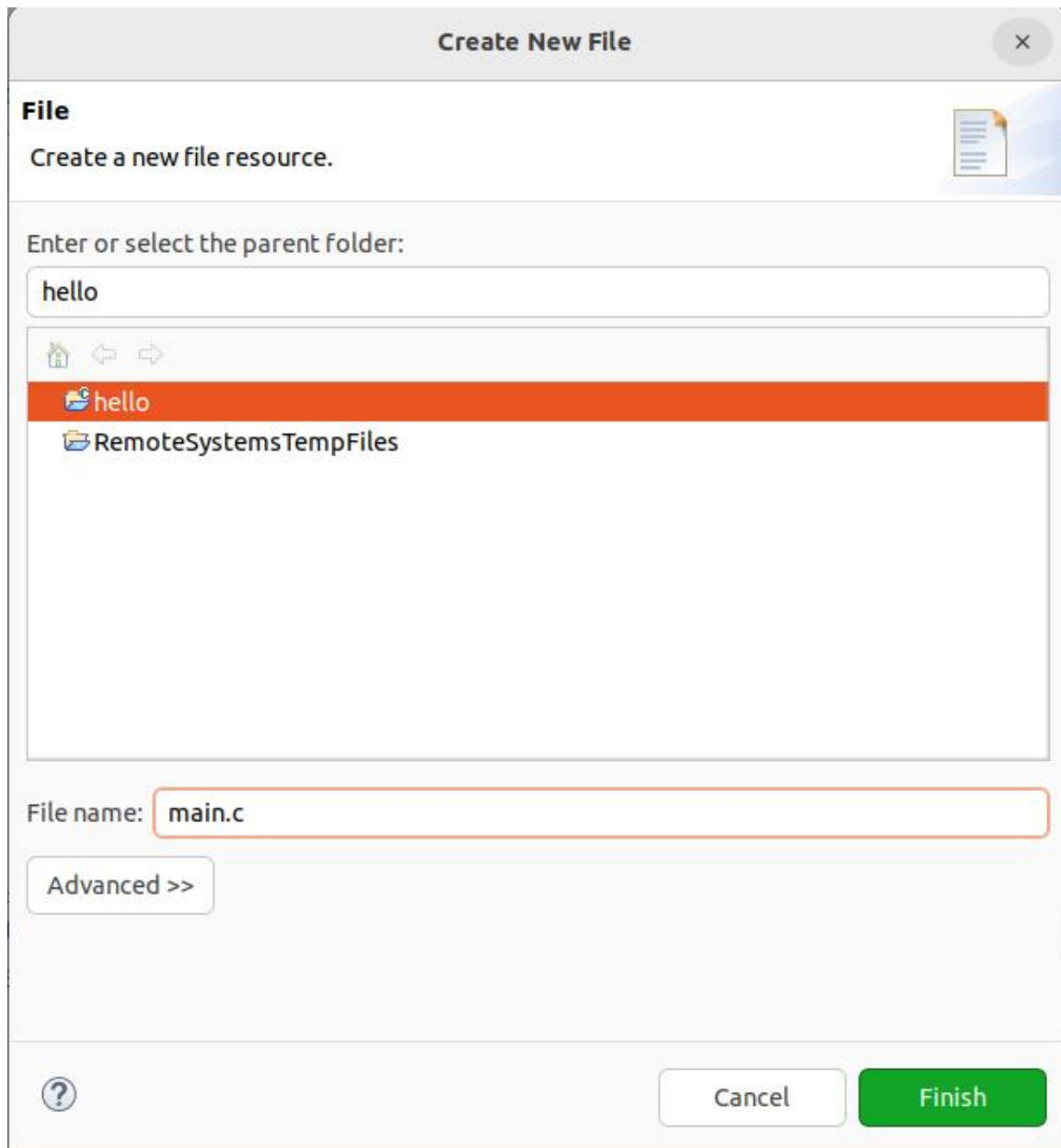
7) 如果弹出下面的信息，点 “No”



8) 在 “Explorer” 窗口中右击 hello 然后选择 “New > File”



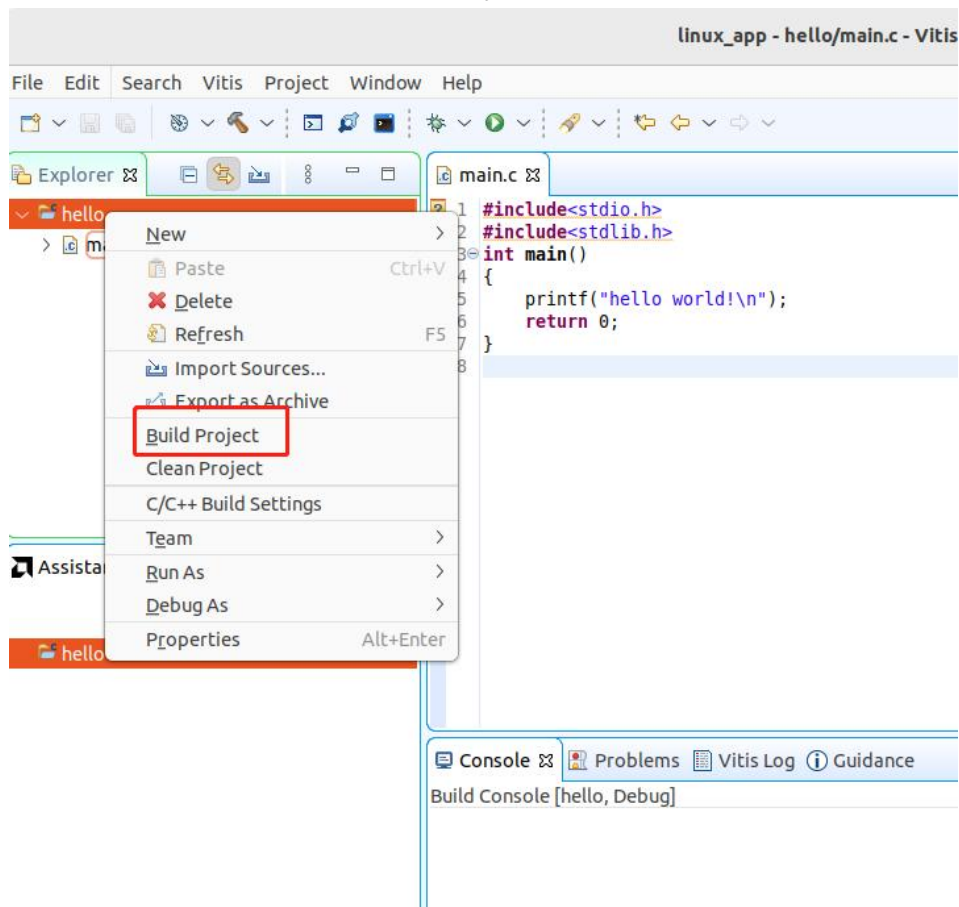
9) 弹出的窗口中，“File name:”后面填写文件名，这里命名为“main.c”，然后点“Finish”。



10) 编辑 main.c 文件，写一个简单的打印

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    printf("hello world!\n");
    return 0;
}
```

- 11) 在 “hello” 目录上右击，然后选择 “Build Project” 编译工程。



6.2 通过 NFS 共享运行

- 1) 开发板插上网线（PS、ETH1，需要路由器支持自动获取 IP），上电，挂载 Linux 主机 NFS，其中 192.168.1.77 为主机地址，/home/alinx/work 为 NFS 目录，/mnt 为开发板目录。这里要求主机和开发板在一个网段。

```
mount -t nfs -o nolock 192.168.8.86:/home/alinx/work /mnt
```

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.8.86:/home/alinx/work /mnt
root@ax_peta:~#
```

- 2) 进入/mnt/linux_app/hello/Debug 目录，运行 hello.elf，我们可以看到打印出 Hello World。

```
cd /mnt/linux_app/hello/Debug
./hello.elf
```

```
root@ax_peta:~# mount -t nfs -o nolock 192.168.8.86:/home/alinx/work /mnt
root@ax_peta:~# cd /mnt/linux_app/hello/Debug
root@ax_peta:/mnt/linux_app/hello/Debug# ./hello.elf
hello world!
root@ax_peta:/mnt/linux_app/hello/Debug#
```

第七章 Linux 下 GPIO 实验

前面的教程介绍如何使用 Vitis 编写一个 zynq 版本的 helloworld 实验, 本实验介绍如何控制 zynq 端外设, 实验使用 GPIO 来举例, ZYNQ 的 GPIO 可以分为 2 种, 一种是 PS 端自带的 GPIO, 一种是使用 PL 实现的 GPIO, 在建立 Vivado 工程时添加了 Xilinx 的 GPIO IP, 大部分 Xilinx 提供的 IP 核在 Linux 下都已经有了驱动, 而且很多默认配置都是可以用的, 像 AXI GPIO 驱动不需要在内核中再配置就可以使用。

在 <http://www.wiki.xilinx.com/Linux+Drivers> 页面我们可以找到所有 Linux 下 Xilinx 的驱动, 例如 GPIO 驱动如下图所示, 有些驱动给出了详细的用法。

GPIO	Zynq and Zynq Ultrascale+ MPSoC	GPIO Driver	Yes	drivers/gpio/gpio-zynq.c
GPIO	axi_gpio	AXI GPIO Driver	Yes	drivers/gpio/gpio-xilinx.c
HDMI Clks	SI5324 Clock Multiplier/Jitter Attenuator	CCF SI5324 Driver	No	hdmi-modules/clk/*

在 GPIO 驱动详细页面 <http://www.wiki.xilinx.com/Linux%20GPIO%20Driver> 中介绍了 GPIO 驱动使用范围, 设备树范例, 以及如何写程序。

7.1 使用 SHELL 控制

Linux 提供了强大的 SHELL 功能, 也是学习 Linux 必须掌握的技能, 对于不熟悉 Linux 的命令和 SHELL 是令人头疼的, 但是为了更好地学习 ZYNQ, 必须熟悉掌握 SHELL, 本教程不会去详细讲解 Linux 和 SHELL 的使用。

通过 `ls /sys/class/gpio` 命令可以查看 GPIO 编号

```
root@ax_peta:~# ls /sys/class/gpio/
export      gpiochip1016  gpiochip1020  gpiochip898  unexport
```

gpio_test.sh 文件内容如下, gpio_test 函数会根据参数来 export 一个 GPIO, 然后一个 for 循环 3 次, 每次先写 0 再写 1, 调用了 5 次 gpio_test, 依次点亮 5 个 LED, 其中 898 是 PS 端的, 其他是 PL 端的, (下一节中介绍如何确定 GPIO 的这个编号)。

```
#!/bin/sh
gpio_test() {
    gpio=$1
    echo $gpio > /sys/class/gpio/export
    echo out > /sys/class/gpio/gpio${gpio}/direction
    for i in $(seq 1 3)
    do
        echo 0 > /sys/class/gpio/gpio${gpio}/value
        sleep 1
        echo 1 > /sys/class/gpio/gpio${gpio}/value
        sleep 1
    done
}
```

```
    echo $gpio > /sys/class/gpio/unexport
}
gpio_test 898
gpio_test 1016
gpio_test 1017
gpio_test 1018
gpio_test 1019
```

我们可以通过挂载 NFS 来运行这个 SHELL。

如果 SHELL 不能运行，可以先添加运行权限，命令如下：

```
chmod +x gpio_test.sh
root@ax_peta:/mnt/linux_app/gpio# chmod +x gpio_test.sh
root@ax_peta:/mnt/linux_app/gpio# ./gpio_test.sh
```

7.2 使用 C 语言控制

大部分情况我们都需要使用 C 语言来控制外设，在 Xilinx 的 wiki 页面

<http://www.wiki.xilinx.com/GPIO%20User%20Space%20App> 我们找到一段 GPIO 测试代码，代码内容如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

// The specific GPIO being used must be setup and replaced thru
// this code. The GPIO of 898 is in the path of most the sys dirs
// and in the export write.
//
// Figuring out the exact GPIO was not totally obvious when there
// were multiple GPIOs in the system. One way to do is to go into
// the gpiochips in /sys/class/gpio and view the label as it should
// reflect the address of the GPIO in the system. The name of the
// the chip appears to be the 1st GPIO of the controller.
//
// The export causes the gpio898 dir to appear in /sys/class/gpio.
// Then the direction and value can be changed by writing to them.

// The performance of this is pretty good, using a nfs mount,
// running on open source linux,
// the GPIO can be toggled about every 1sec.
// The following commands from the console setup the GPIO to be
// exported, set the direction of it to an output and write a 1
// to the GPIO.
//
// bash> echo 898 > /sys/class/gpio/export
// bash> echo out > /sys/class/gpio/gpio898/direction
// bash> echo 1 > /sys/class/gpio/gpio898/value

// if sysfs is not mounted on your system, the you need to mount it
// bash> mount -t sysfs sysfs /sys

// the following bash script to toggle the gpio is also handy for
// testing
//
// while [ 1 ]; do
//   echo 1 > /sys/class/gpio/gpio898/value
//   echo 0 > /sys/class/gpio/gpio898/value
// done

// to compile this, use the following command
// gcc gpio.c -o gpio

// The kernel needs the following configuration to make this work.
//
// CONFIG_GPIO_SYSFS=y
// CONFIG_SYSFS=y
// CONFIG_EXPERIMENTAL=y
// CONFIG_GPIO_XILINX=y

int main()
{
```

```
int valuefd, exportfd, directionfd;

printf("GPIO test running...\n");

// The GPIO has to be exported to be able to see it
// in sysfs

exportfd = open("/sys/class/gpio/export", O_WRONLY);
if (exportfd < 0)
{
    printf("Cannot open GPIO to export it\n");
    exit(1);
}

write(exportfd, "898", 4);
close(exportfd);

printf("GPIO exported successfully\n");

// Update the direction of the GPIO to be an output
directionfd = open("/sys/class/gpio/gpio898/direction", O_RDWR);
if (directionfd < 0)
{
    printf("Cannot open GPIO direction it\n");
    exit(1);
}

write(directionfd, "out", 4);
close(directionfd);

printf("GPIO direction set as output successfully\n");

// Get the GPIO value ready to be toggled
valuefd = open("/sys/class/gpio/gpio898/value", O_RDWR);
if (valuefd < 0)
{
    printf("Cannot open GPIO value\n");
    exit(1);
}

printf("GPIO value opened, now toggling...\n");

// toggle the GPIO as fast a possible forever, a control c is needed
// to stop it

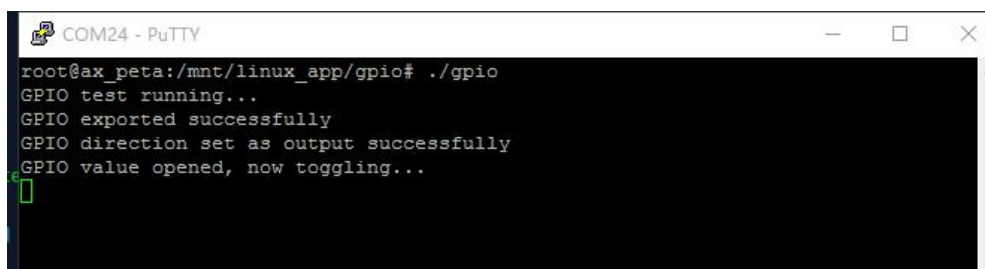
while (1)
{
    write(valuefd, "1", 2);
    sleep(1);
    write(valuefd, "0", 2);
    sleep(1);
}
}
```

这一次我们不再使用 vitis 来编译，源代码命名为 “gpio.c”，运行下面命令编译代码

```
source /tools/Xilinx/Vivado/2023.1/settings64.sh
arm-linux-gnueabihf-gcc gpio.c -o gpio
```

编译完成以后会生成一个 gpio 的文件，不像在 Windows，Linux 下对扩展名要求不是很严格，gpio 文件就是一个 elf 文件。

运行 gpio，可以看到 PS 端 LED 不断闪烁，说明这个 898 就是 PS 端第一个 LED。



7.2.1 如何确定 GPIO 的这个编号?

通过下面命令, 我们可以看到有 gpiochip898 gpiochip1016 gpiochip1020, 说明有三个 GPIO 控制器, 数字是控制器 GPIO 基数。

```
ls /sys/class/gpio
```

```
root@ax_peta:/mnt/linux_app/gpio# ls /sys/class/gpio
export      gpiochip1016  gpiochip898
gpio898     gpiochip1020  unexport
```

7.2.2 如何确定和物理 GPIO 的关系?

通过下面命令, 来确定 GPIO1016 和物理 GPIO 的关系, 可以看到这个 gpio 在设备树里的节点是 "gpio@41210000", 通过设备树的节点我们可以确定是哪一个物理 GPIO。

```
cat /sys/class/gpio/gpiochip1016/label
```

```
root@ax_peta:/mnt/linux_app/gpio# cat /sys/class/gpio/gpiochip1016/label
41210000.gpio
```

7.3 实验总结

本实验重点在于如何通过 Xilinx 给的资料来学习 ZYNQ, 技术资料更新较快, 只有紧跟芯片厂商提供的最新资料才能获取到最新最好的技术。后续教程中 PCIe 驱动、PL 端以太网驱动都是 Xilinx 提供, 这些资料都可以通过 wiki 获取。

如果使用一个非 xilinx 的 IP, 或者自己写的 IP, 那就要自己开发驱动程序, 这对没做过 Linux 驱动的开发人员来说是一个挑战, 所以我们尽可能使用 Xilinx 的 IP 来搭建系统, 优点就是不用开发 Linux 驱动, 缺点就是不够灵活, 如果 IP 有问题或者驱动有问题, 无法快速定位问题。

第八章 Petalinux 下的 HDMI 显示

实验 Vivado 工程为 “linux_base” ,在 vivado.zip 压缩包中。

实验 petalinux 工程相关文件夹为 “ax_peta_hdmi”

前面的教程中我们已经体验了使用 Petalinux 开发嵌入式 Linux 系统，但我们使用的功能仅是 Petalinux 冰山一角，Petalinux 的强大功能需要我们不断的探索，本实验讲解如何使用一个自己的内核来做 Linux，这样我们可以在内核里添加很多驱动，例如 HDMI 显示。

开发板使用的没有 HDMI 接口芯片，但是使用了 PFPA 去完成编码，芯驿电子将 Xilinx 提供的内核中加入了 HDMI 编码 IP 的驱动。**请使用其他版本软件开发者注意，本教程只提供修改版 linux-xlnx-xlnx_rebase_v6.1_LTS 内核，不提供其他版本修改版，也不提供修改说明。**

8.1 Petalinux 配置

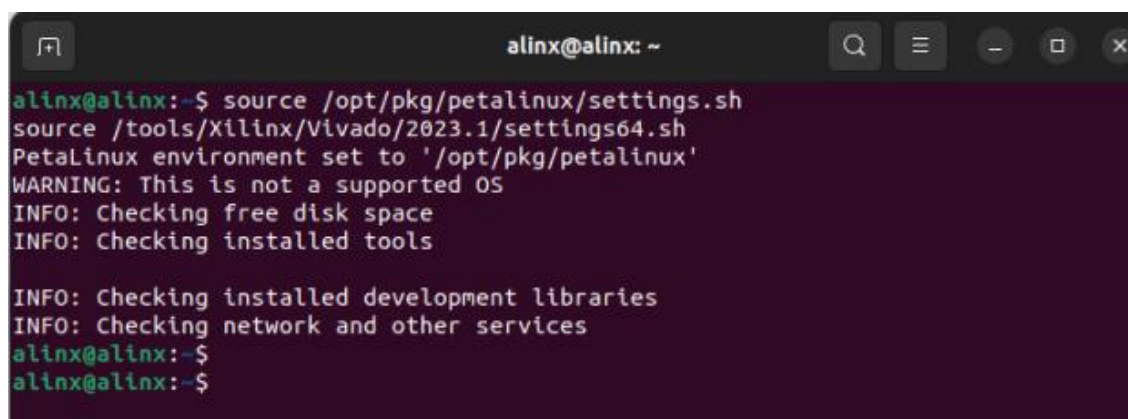
本实验还是在前面实验的 Petalinux 工程修改，需要先掌握前面的实验内容。

- 1) 打开终端，设置 petalinux 环境变量，运行下面命令

```
source /opt/pkg/petalinux/settings.sh
```

- 2) 运行下面命令设置 vivado 环境变量

```
source /tools/Xilinx/Vivado/2023.1/settings64.sh
```



```
alinx@alinx: ~  
alinx@alinx:~$ source /opt/pkg/petalinux/settings.sh  
source /tools/Xilinx/Vivado/2023.1/settings64.sh  
PetaLinux environment set to '/opt/pkg/petalinux'  
WARNING: This is not a supported OS  
INFO: Checking free disk space  
INFO: Checking installed tools  
  
INFO: Checking installed development libraries  
INFO: Checking network and other services  
alinx@alinx:~$  
alinx@alinx:~$
```

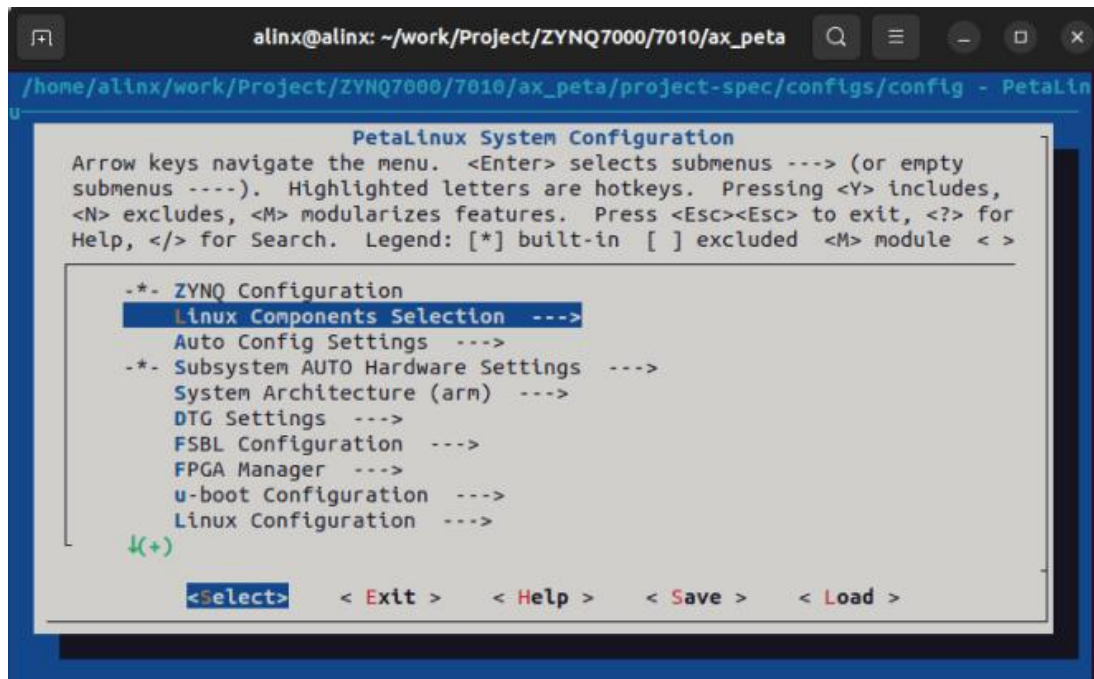
- 3) 将资料中第八章 HDMI 显示的 bsp 包复制到虚拟机里，运行命令，用 bsp 生成 PetaLinux 工程

```
petalinux-create -t project -n ax_peta -s ./hdmi.bsp
```

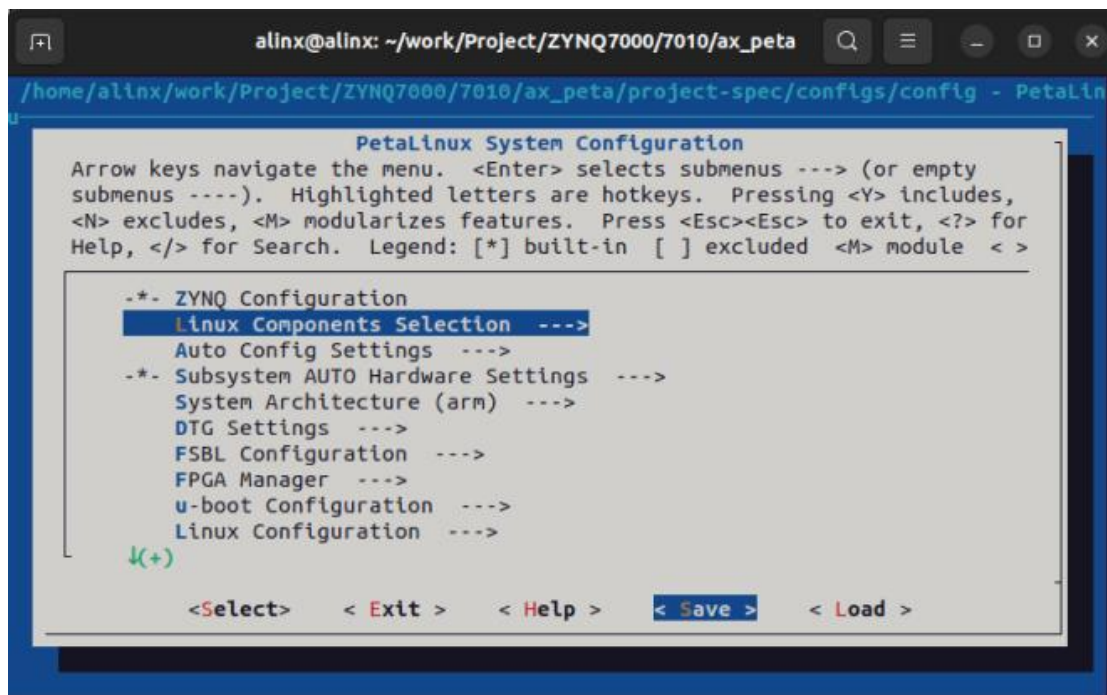
```
alinx@alinx:~/work/Project$ petalinux-create -t project -n ax_peta -s ./hdm1.bsp
INFO: Create project: ax_peta
INFO: New project successfully created in /home/alinx/work/Project/ax_peta
```

- 4) 控制台进入新生成的 ax_peta 目录，使用下面命令配置 petalinux 内核来源以及离线编译，具体配置过程参考第五章的 5.2 和 5.3，由于这里我们用的是 bsp 包，所以这些内容都已经配置好了，如果用户是用 xsa 文件生成的 Petalinux 工程，就需要重新配置这些项。

```
petalinux-config
```



- 5) 然后保存退出



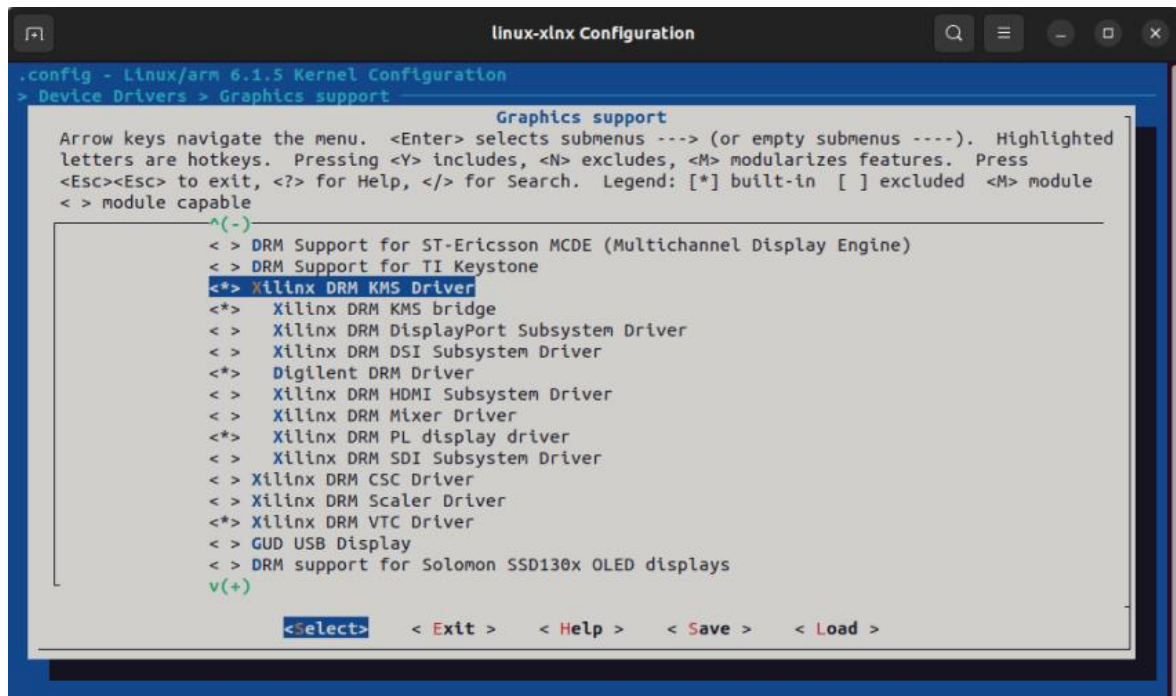
8.2 配置 Linux 内核

- 1) 运行下面的命令配置内核

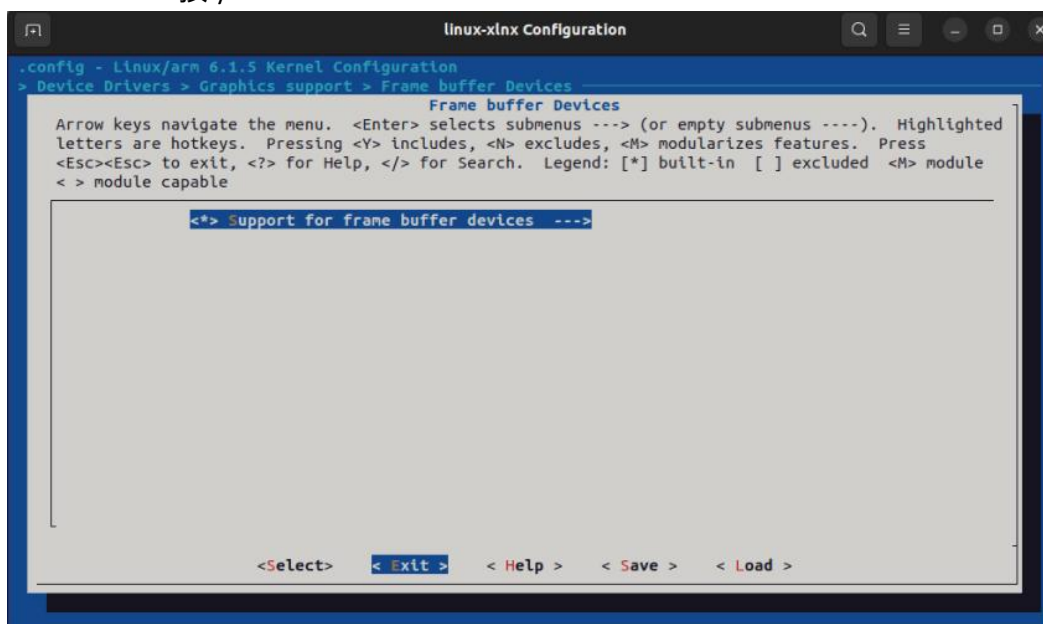
```
petalinux-config -c kernel
```

```
alinx@ubuntu:~/peta_prj/ax_peta$ petalinux-config -c kernel
[INFO] Sourcing buildtools
[INFO] Silentconfig project
[INFO] Silentconfig rootfs
[INFO] Generating workspace directory
[INFO] Configuring: kernel
[INFO] bitbake virtual/kernel -c cleansstate
NOTE: Started PRServer with DBfile: /home/alinx/peta_prj/ax_peta/build/cache/prs
erv.sqlite3, Address: 127.0.0.1:40095, PID: 40643
Loading cache: 100% |#####| Time: 0:00:03
Loaded 6269 entries from dependency cache.
```

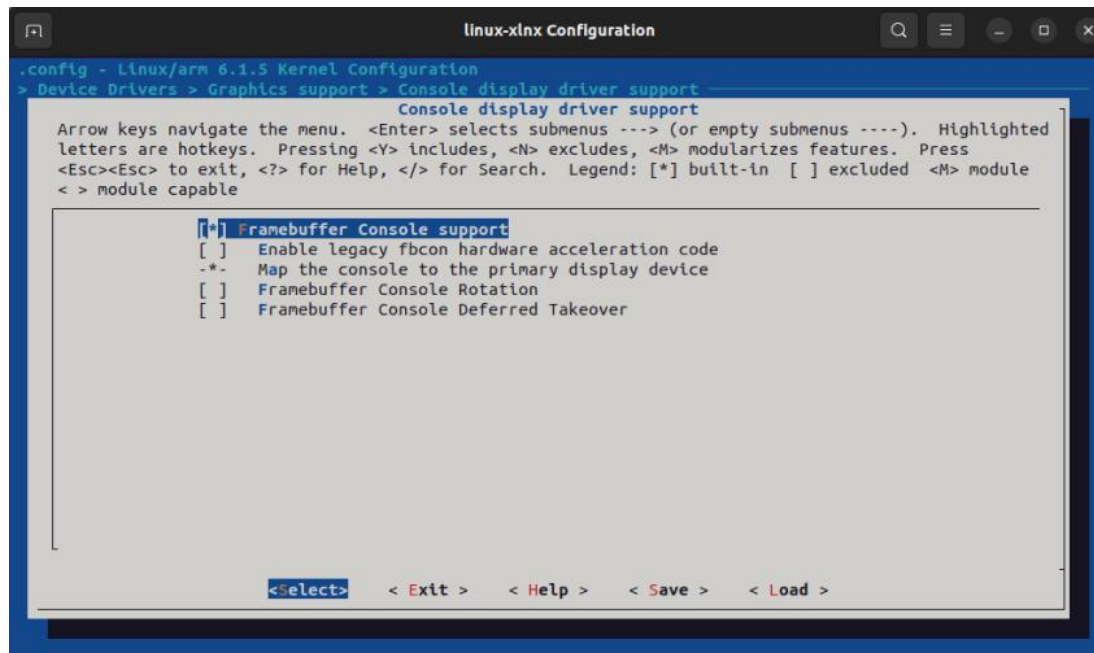
- 2) 在弹出的窗口中，进入 Device Drivers → Graphics support，分别选择以下 5 项
 - Xilinx DRM KMS Driver
 - Xilinx DRM KMS bridge
 - Digilent DRM Driver
 - Xilinx DRM PL display driver
 - Xilinx DRM VTC Driver



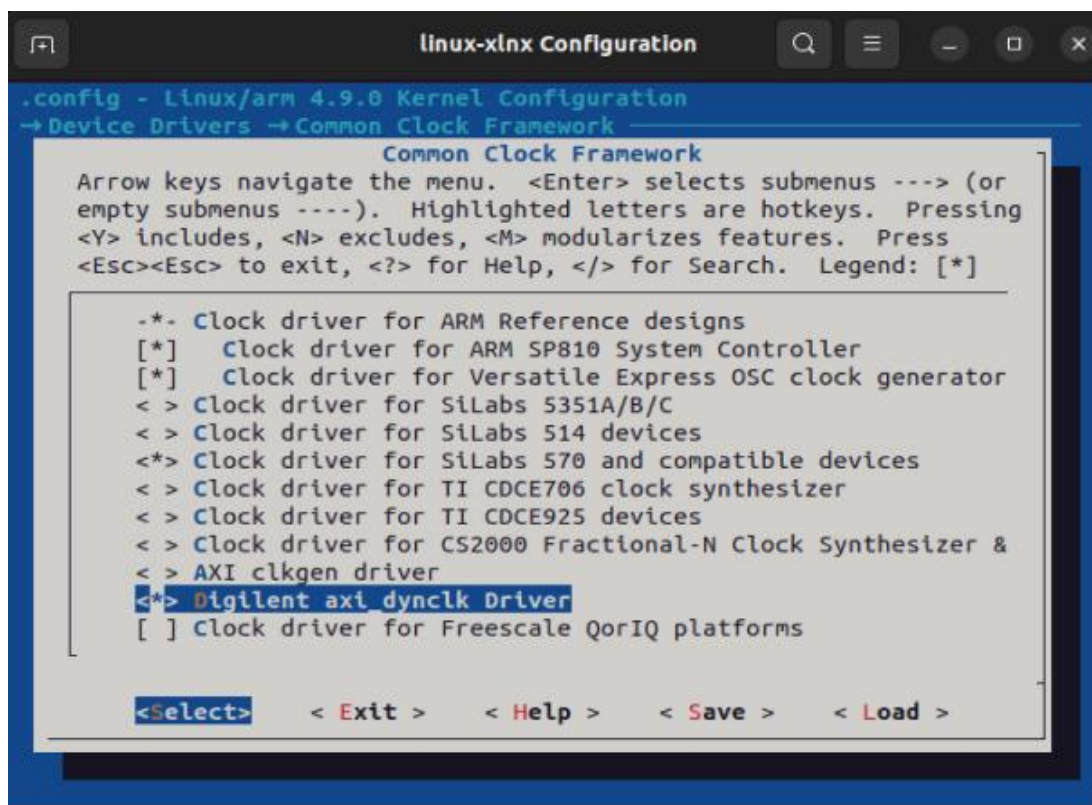
- 3) 在 Device Drivers → Graphics support → Frame buffer Devices 中选择 Support for frame buffer devices 按 y



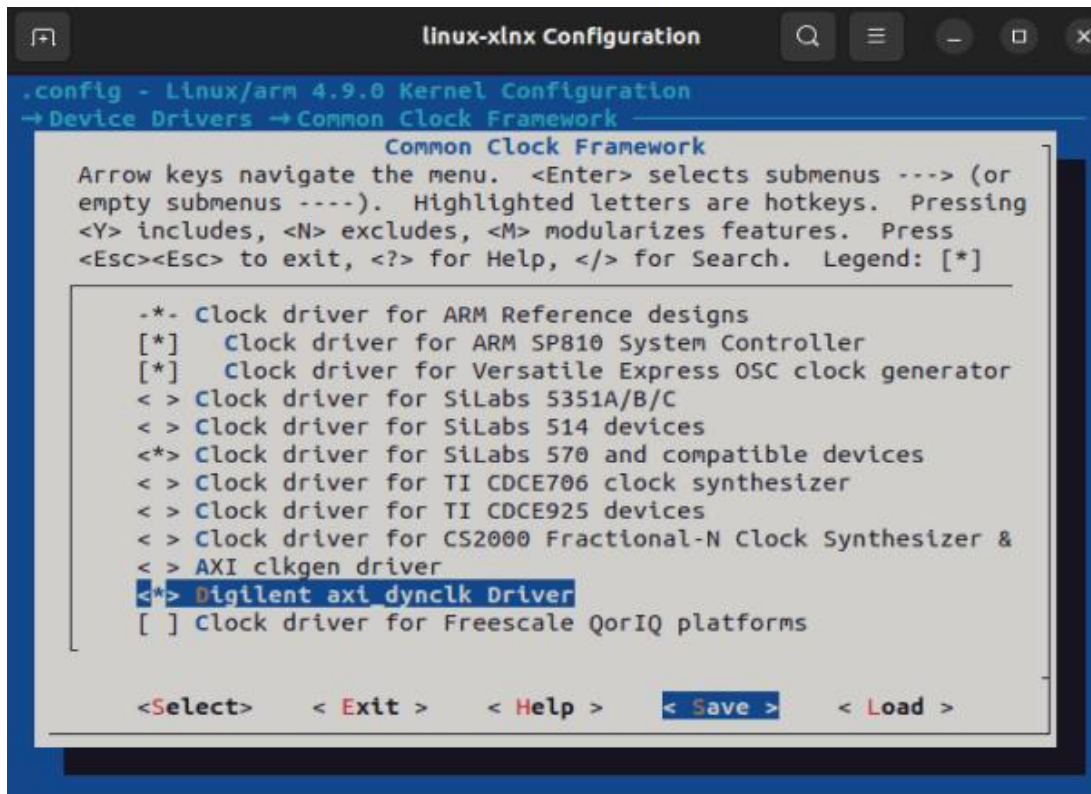
- 4) 在 Device Drivers → Graphics support → Console display driver support 选项中选择 Framebuffer Console support 按 y



- 5) 在 Device Drivers → Common Clock Framework 选项中选择 Digilent axi_dynclock Driver 按 y



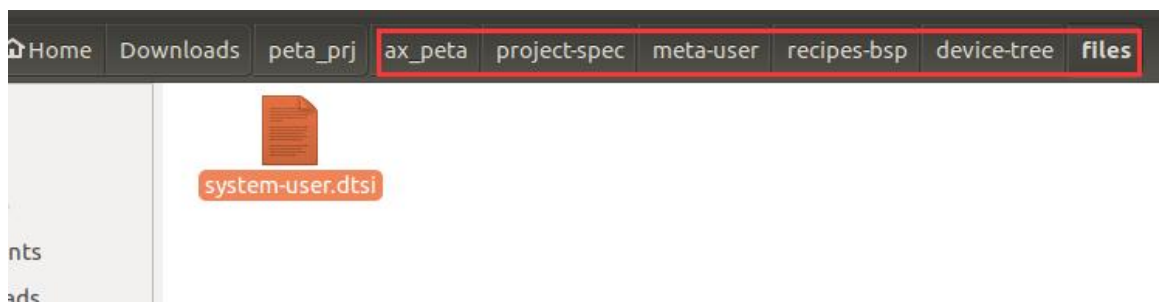
- 6) 保存并退出



8.3 修改设备树

设备树是描述设备信息的一种格式化文本，这种文本结构对类似于 XML 和 JSON，什么？XML 和 JSON 也不会？如果这些基础配置数据格式都没接触过学习设备树是要很长时间摸索的。每个驱动要求的设备树节点也是不同的，对于没接触过设备的开发人员，需要慢慢熟悉，在一次一次的使用种慢慢掌握。

- 1) 打开 petalinux 工程文件中的名为 system-user.dtsi 的文件



- 2) 修改设备树内容如下

```
/include/ "system-conf.dtsi"

/{
    model = "Zynq ALINX Development Board";
```

```
compatible = "alinx,zynq", "xlnx,zynq-7000";
aliases {
    ethernet0 = "&gem0";
    serial0 = "&uart1";
};

usb_phy0: usb_phy@0 {
    compatible = "ulpi-phy";
    #phy-cells = <0>;
    reg = <0xe0002000 0x1000>;
    view-port = <0x0170>;
    drv-vbus;
};

};

&i2c0 {
    clock-frequency = <100000>;
};

&usb0 {
    dr_mode = "host";
    usb-phy = <&usb_phy0>;
};

&sdhci0 {
    u-boot,dm-pre-reloc;
};

&uart1 {
    u-boot,dm-pre-reloc;
};

&flash0 {
    compatible = "micron,m25p80", "w25q256", "spi-flash";
};

&gem0 {
    phy-handle = <&ethernet_phy>;
    ethernet_phy: ethernet-phy@1 {
        reg = <1>;
        device_type = "ethernet-phy";
    };
};

&amba_pl {
    diligent_hdmi {
        compatible = "diligent,hdmi";

        clocks = <&axi_dyncclk_0>;
        clock-names = "clk";
```

```
digilent,edid-i2c = <&i2c0>;
digilent,fmax = <150000>;

port@0 {
    #address-cells = <1>;
    #size-cells = <0>;
    hdmi_ep: endpoint {
        remote-endpoint = <&pl_disp_ep>;
    };
};

xlnx_pl_disp {
    compatible = "xlnx,pl-disp";
    dmas = <&axi_vdma_0 0>;
    dma-names = "dma0";

    xlnx,vformat = "RG24";
    xlnx,bridge = <&v_tc_0>;
    port@0 {
        reg = <0>;
        pl_disp_ep: endpoint {
            remote-endpoint = <&hdmi_ep>;
        };
    };
};

};

&axi_dynclk_0 {
    compatible = "dglnt,axi-dynclk";
    #clock-cells = <0>;
    clocks = <&clkc 15>;
};

&v_tc_0 {
    compatible = "xlnx,bridge-v-tc-6.1";
    xlnx,pixels-per-clock = <1>;
};
```

8.4 编译测试 Petalinux 工程

- 1) 使用下面命令配置编译 uboot、内核、根文件系统、设备树等。

```
petalinux-build
```

```

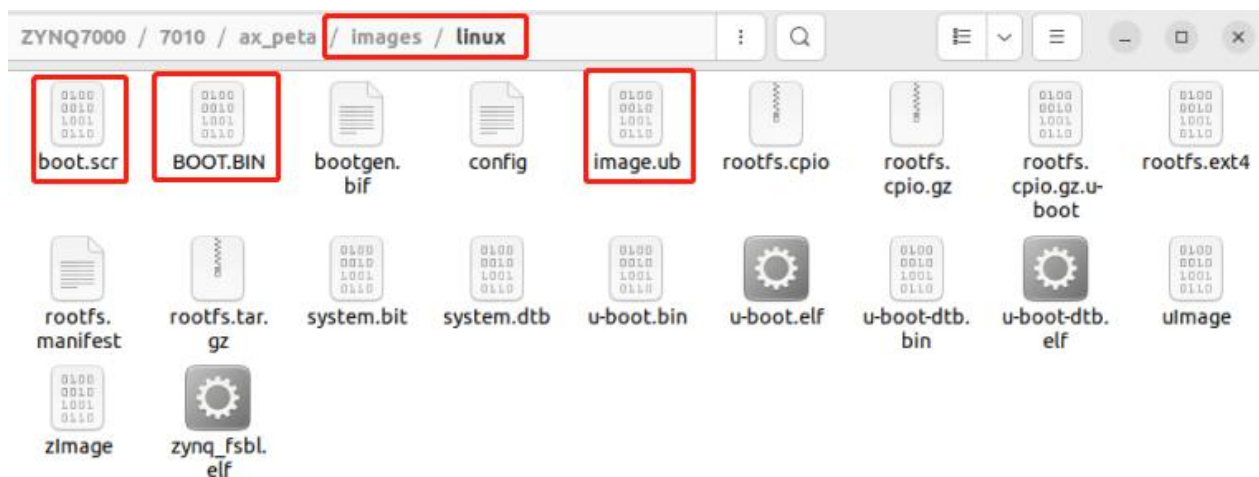
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3256 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2466 .bb files complete (2433 cached, 33 parsed). 3259 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:04
NOTE: Executing RunQueue Tasks
NOTE: Tasks Summary: Attempted 2 tasks of which 0 didn't need to be rerun and al
l succeeded.
[INFO] successfully configured kernel
webtalk failed:PetaLinux statistics:extra lines detected:notsent_nofile!
webtalk failed:Failed to get PetaLinux usage statistics!
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ petalinux-build
[INFO] building project
[INFO] sourcing bitbake
INFO: bitbake petalinux-user-image
Loading cache: 100% |#####| Time: 0:00:01
Loaded 3256 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2466 .bb files complete (2433 cached, 33 parsed). 3259 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 18% |#####| ETA: 0:00:08

```

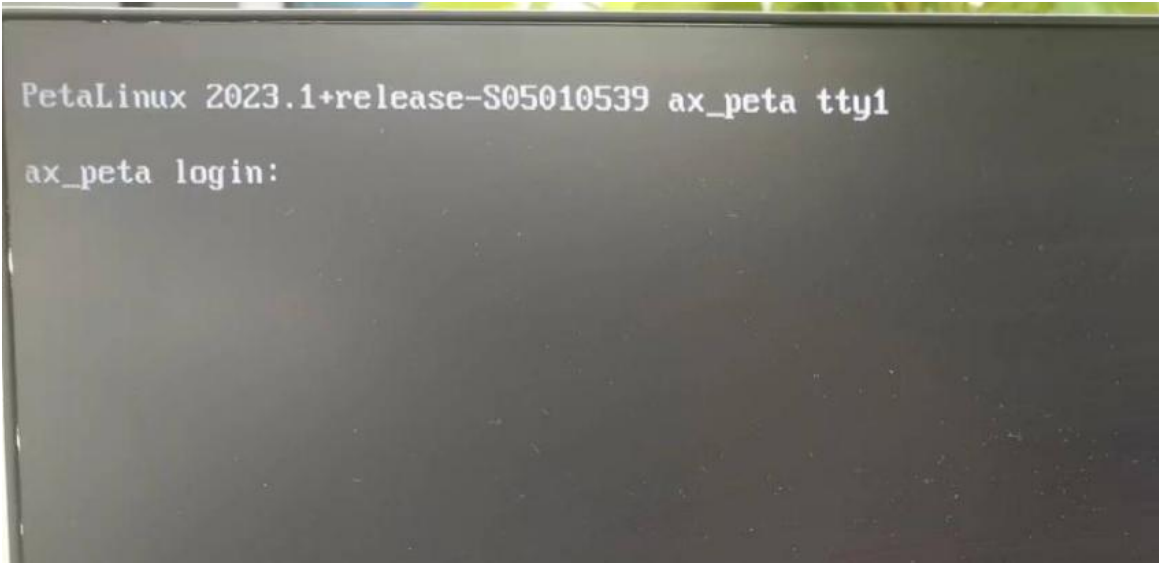
- 2) 运行下面命令生成 BOOT 文件，注意空格和短线

```
petalinux-package --boot --fsbl ./images/linux/zynq_fsbl.elf --fpga --u-boot --force
```

- 3) 把 BOOT.bin, iamge.ub 和 boot.scr 复制到 sd 中，设置开发板 sd 模式启动，插上 HDMI 显示器，启动开发板。



- 4) 显示器会显示出如下内容



```
PetaLinux 2023.1+release-S05010539 ax_peta tty1
ax_peta login:
```

8.5 常见问题

8.5.1 如何防止系统休眠

休眠之前运行命令

```
echo -e "\033[9;0]\033[?33l\033[?25h\033[?1c" > /dev/tty0
echo -e "\033[9;0]\033[?33l\033[?25h\033[?1c" > /dev/tty1
echo -e "\033[9;0]\033[?33l\033[?25h\033[?1c" > /dev/tty
echo -e "\033[9;0]\033[?33l\033[?25h\033[?1c" > /dev/console
```

第九章 使用 Debian 桌面系统

实验 Vivado 工程为 “linux_base” ,在 vivado.zip 压缩包中。

实验 petalinux 工程相关文件夹为 “ax_peta_debian”

前面的教程中我们学习了如何使用 Petalinux 定制一个嵌入式 Linux 系统以及 HDMI 显示, 本实验将做一个基于 Debian 的根文件系统, 根文件系统的制作比较复杂, 本章不再讲解, 直接使用制作好的 Debian 根文件系统。

9.1 Petalinux 配置

由于 Debian 根文件系统比较大, QSPI flash 是无法放下的, 只能放在 sd 卡或 emmc 里, 所以我们要配置 Petalinux。

本实验还是在前面 HDMI 实验的 Petalinux 工程基础上进行修改的, 需要先掌握前面的实验内容。

- 1) 打开终端, 进入前面实验中的 Petalinux 工程目录



```
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$
```

- 2) 设置 petalinux 环境变量, 运行下面命令

```
source /opt/pkg/petalinux/settings.sh
```

- 3) 运行下面命令设置 vivado 环境变量

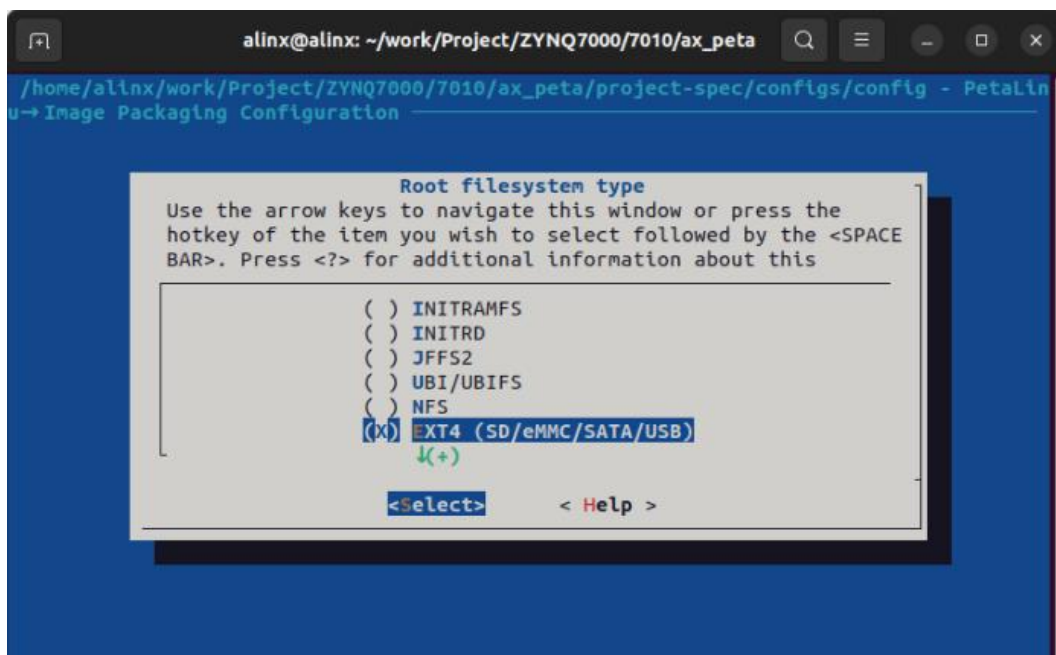
```
source /tools/Xilinx/Vivado/2023.1/settings64.sh
```

```
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ source /opt/pkg/petalinux/settings.sh
PetaLinux environment set to '/opt/pkg/petalinux'
WARNING: /bin/sh is not bash!
bash is PetaLinux recommended shell. Please set your default shell to bash.
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "PetaLinux SDK Installation Guide" for its impact and solution
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ source /opt/Xilinx/Vivado/2017.4/settings64.sh
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$
```

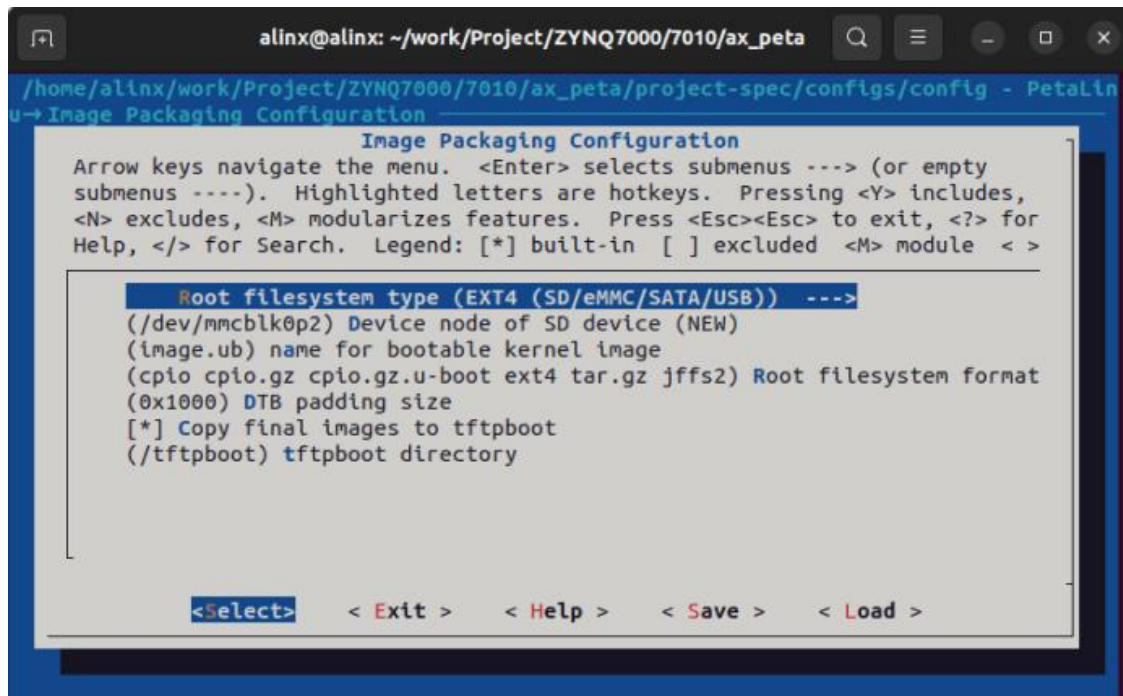
- 4) 使用下面命令重新配置 petalinux

```
petalinux-config
```

- 5) 在 Image Packaging Configuration --> Root filesystem type 选项中选择 SD card, 把根文件系统放在 SD 卡中。



- 6) 保存并退出



9.2 配置 Linux 内核

- 1) 运行下面的命令配置内核

```
petalinux-config -c kernel
```

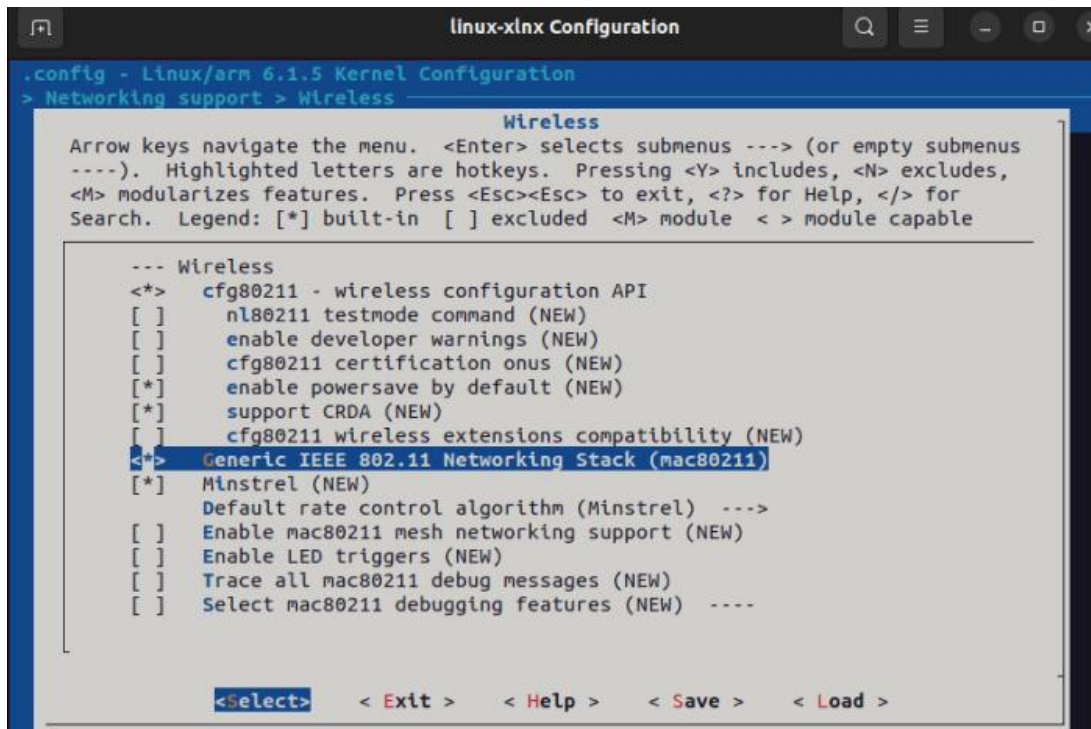
```
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
*** Execute 'make' to start the build or try 'make help'.

[INFO] sourcing bitbake
[INFO] generating plnxtool conf
[INFO] generating meta-plnx-generated layer
~/Downloads/peta_prj/ax_peta/build/misc/plnx-generated ~/Downloads/peta_prj/ax_peta
~/Downloads/peta_prj/ax_peta
[INFO] generating machine configuration
[INFO] generating bbappends for project . This may take time !
~/Downloads/peta_prj/ax_peta/build/misc/plnx-generated ~/Downloads/peta_prj/ax_peta
~/Downloads/peta_prj/ax_peta
[INFO] generating u-boot configuration files

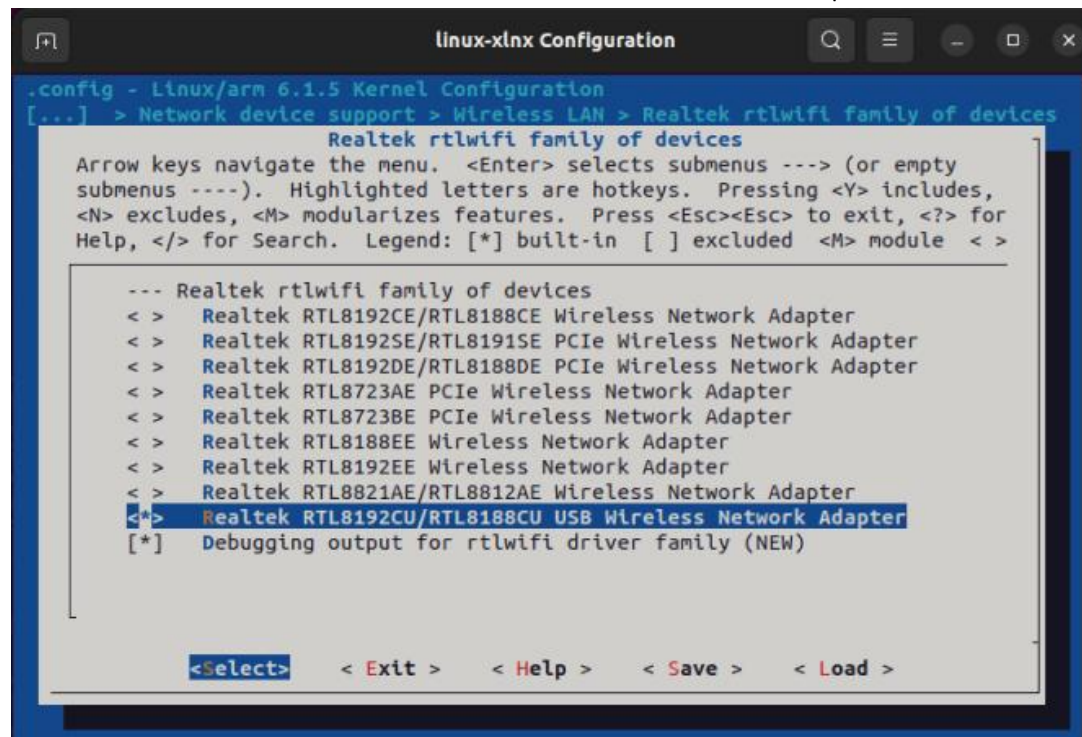
[INFO] generating kernel configuration files
[INFO] generating kconfig for Rootfs
Generate rootfs kconfig
[INFO] oldconfig rootfs
[INFO] generating petalinux-user-image.bb
[INFO] successfully configured project
weblink failed:PetaLinux statistics:extra lines detected:notsent_nofile!
weblink failed:Failed to get PetaLinux usage statistics!
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ petalinux-config -c kernel
```

9.2.1 配置 USB WIFI 模块驱动

- 1) 在选项 Networking Support -> Wireless -> 选项中先选择 cfg80211 - wireless configuration API 再选择 Generic IEEE 802.11 Networking Stack (mac80211)

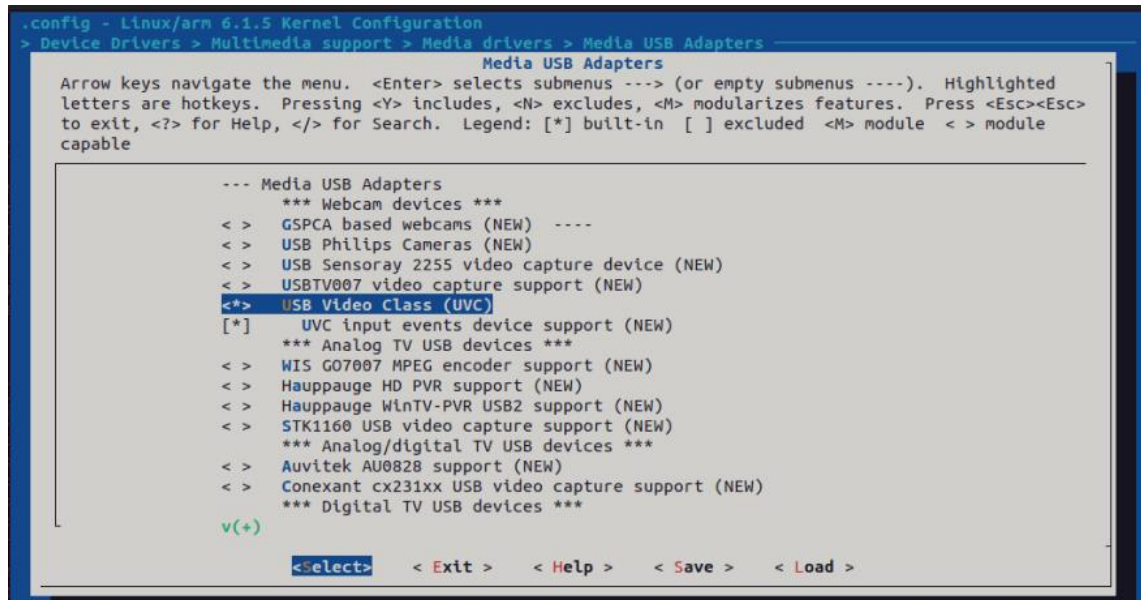


- 2) 在选项 Device Drivers -> Network device support -> Wireless LAN -> Realtek rtlwifi family of devices 选择 Realtek RTL8192CU/RTL8188CU USB Wireless Network Adapter



9.2.2 配置 USB 摄像头驱动

- 1) 在选项 Device Drivers ---> Multimedia support ---> Media drivers---> Media USB Adapters ---> 中使能 USB Video Class (UVC)



- 2) 保存并退出

9.3 编译测试 Petalinux 工程

- 1) 使用下面命令配置编译 uboot、内核、根文件系统、设备树等。

```
petalinux-build
```

```
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3256 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2466 .bb files complete (2433 cached, 33 parsed). 3259 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:04
NOTE: Executing RunQueue Tasks
NOTE: Tasks Summary: Attempted 2 tasks of which 0 didn't need to be rerun and al
l succeeded.
[INFO] successfully configured kernel
webtalk failed:PetaLinux statistics:extra lines detected:notsent_nofile!
webtalk failed:Failed to get PetaLinux usage statistics!
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ petalinux-build
[INFO] building project
[INFO] sourcing bitbake
INFO: bitbake petalinux-user-image
Loading cache: 100% |#####| Time: 0:00:01
Loaded 3256 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2466 .bb files complete (2433 cached, 33 parsed). 3259 targets, 226 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 18% |#####| ETA: 0:00:08
```

- 2) 运行下面命令生成 BOOT.BIN 文件，注意空格和短线

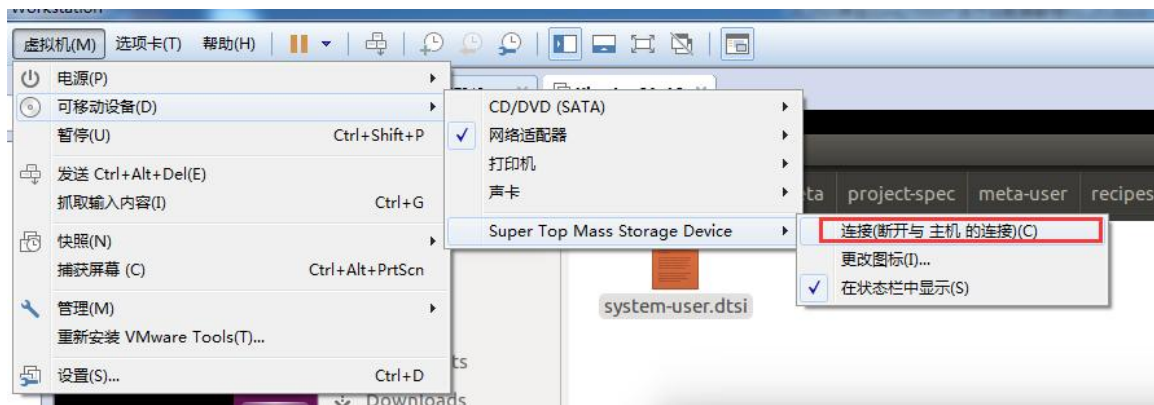
```
petalinux-package --boot --u-boot --fsbl --fpga --force
```

9.4 制作 SD 卡文件系统

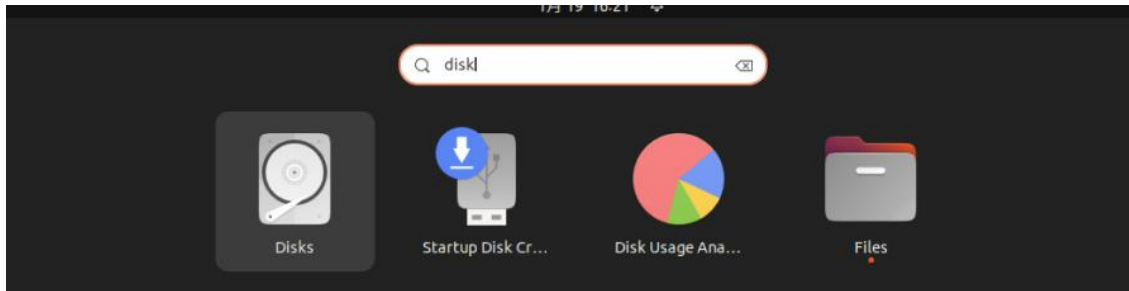
制作 SD 卡文件系统会导致 SD 卡里内容丢失，请先做好备份。

9.4.1 SD 卡修改分区

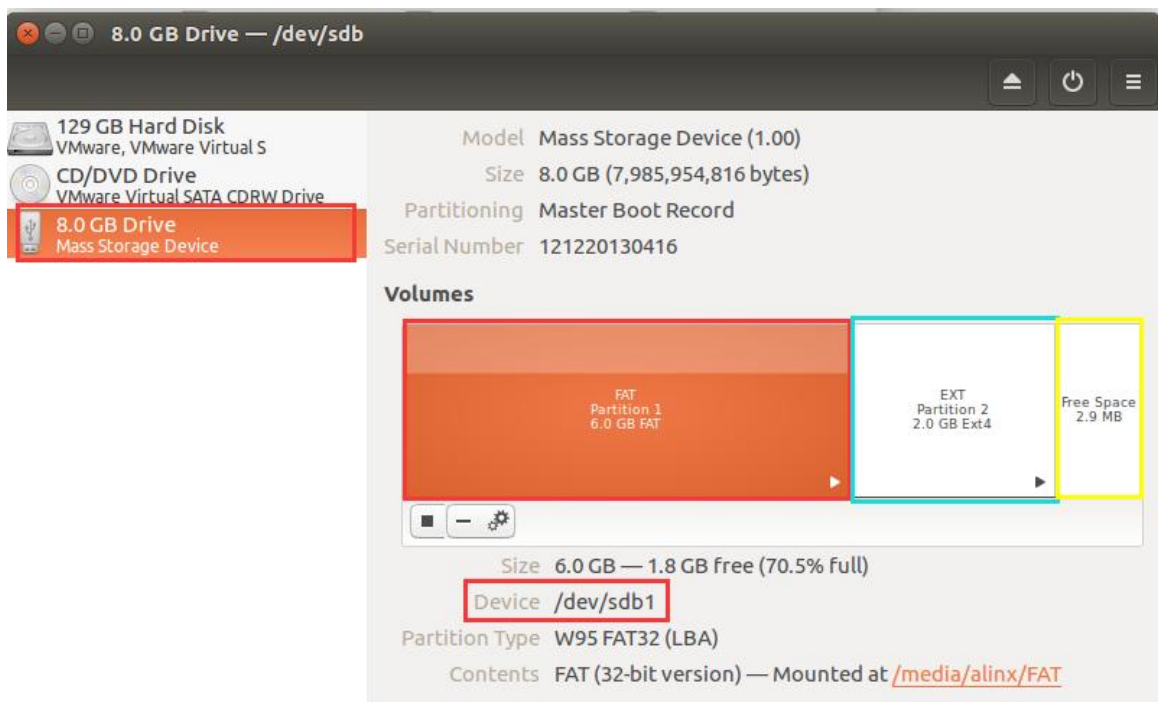
- 1) 把开发板的 sd 卡插入读卡器，然后插入电脑的 USB 口
- 2) 连接到虚拟机 Linux 中



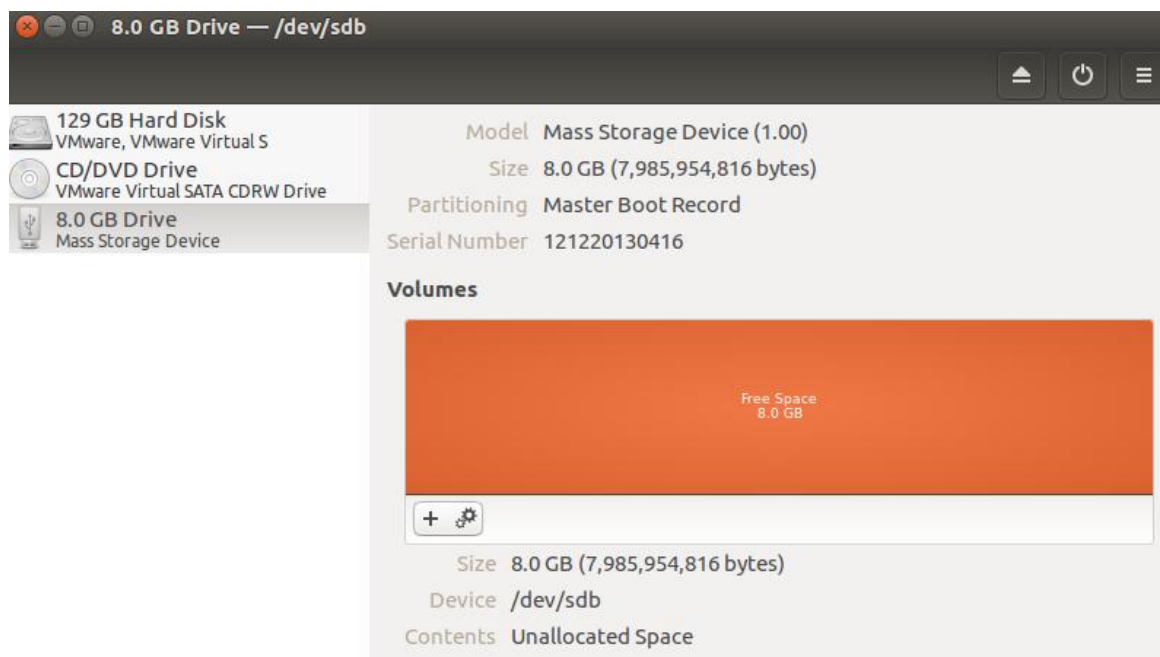
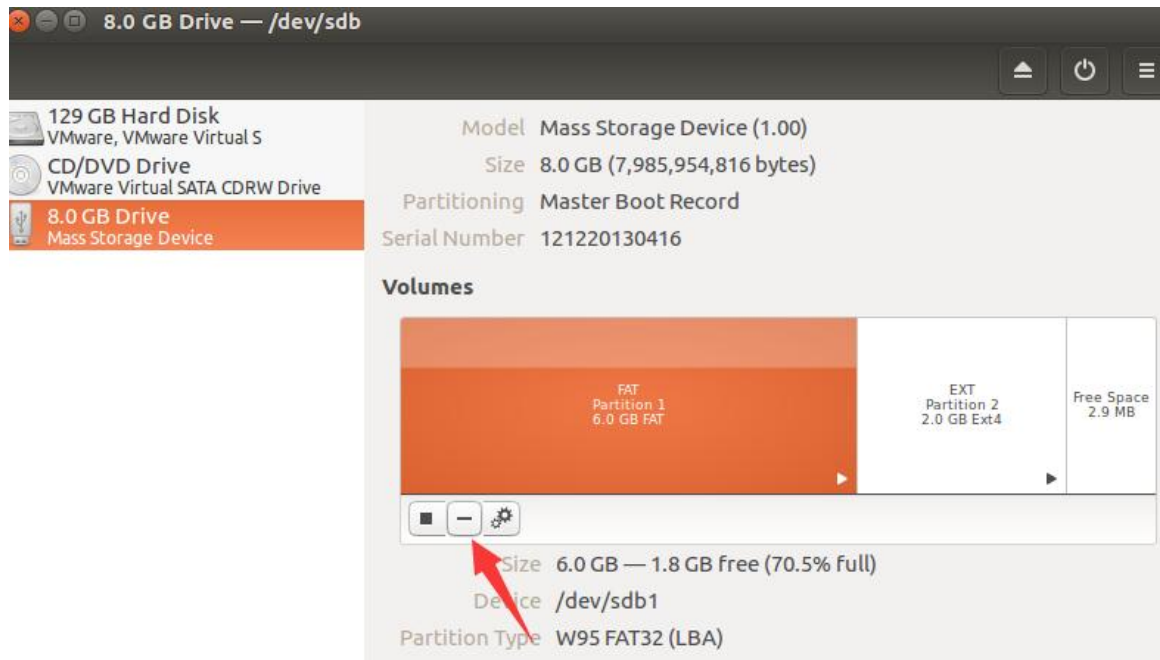
- 3) 在 ubuntu 的搜索路径中，输入 disk，会出现 Disks 的图标



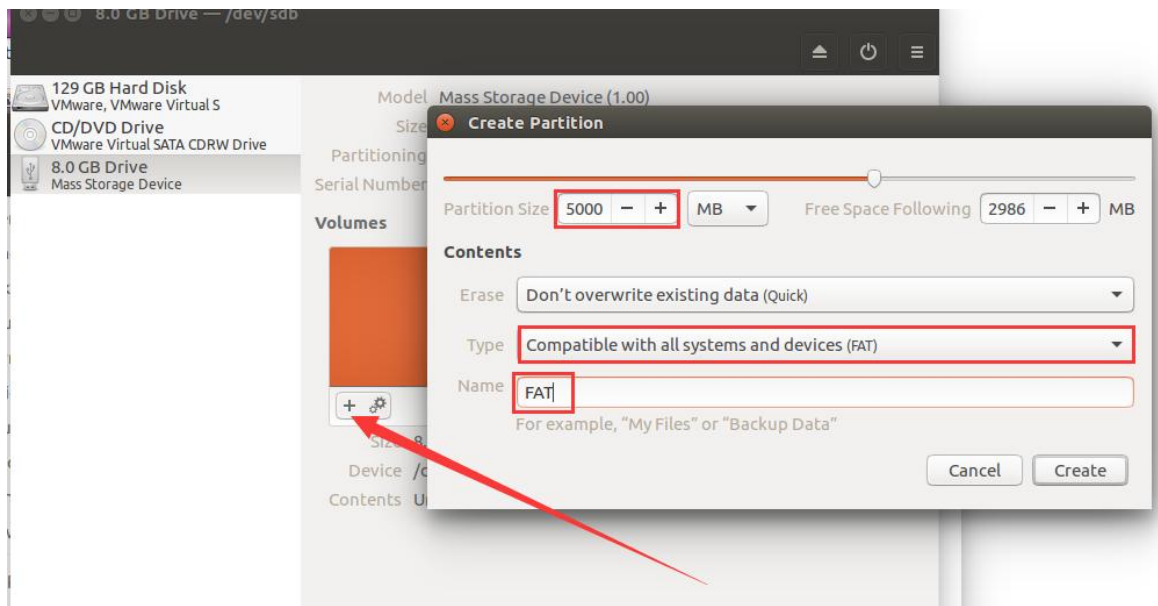
- 4) 鼠标点击 Disks 图标，出现"Disks"的对话框，本实验中 SD 卡已经分为 2 个分区，一个名为 FAT，一个名为 EXT，这里要重新分区。



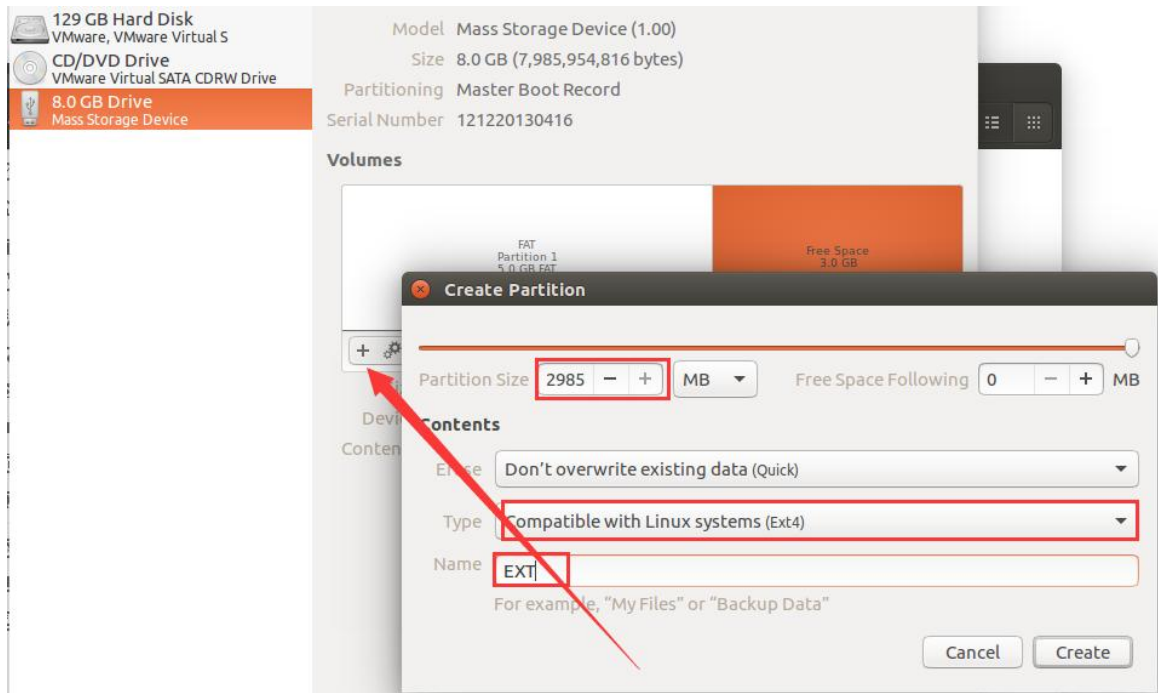
- 5) 选择每个分区下面的删除分区图标，删除所有分区。



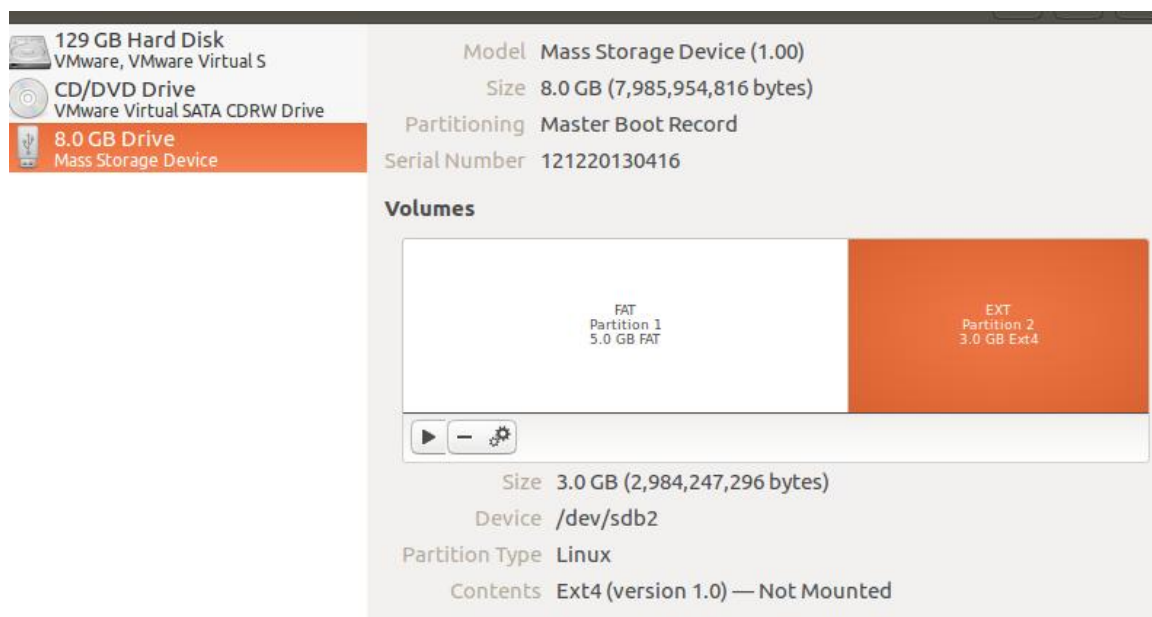
- 6) 点击添加分区的图标，添加第一个分区，本实验填写 5000MB，格式为 FAT，用于存放 ZYNQ 的启动文件 BOOT.bin, image.ub 和 boot.scr，名称为 FAT



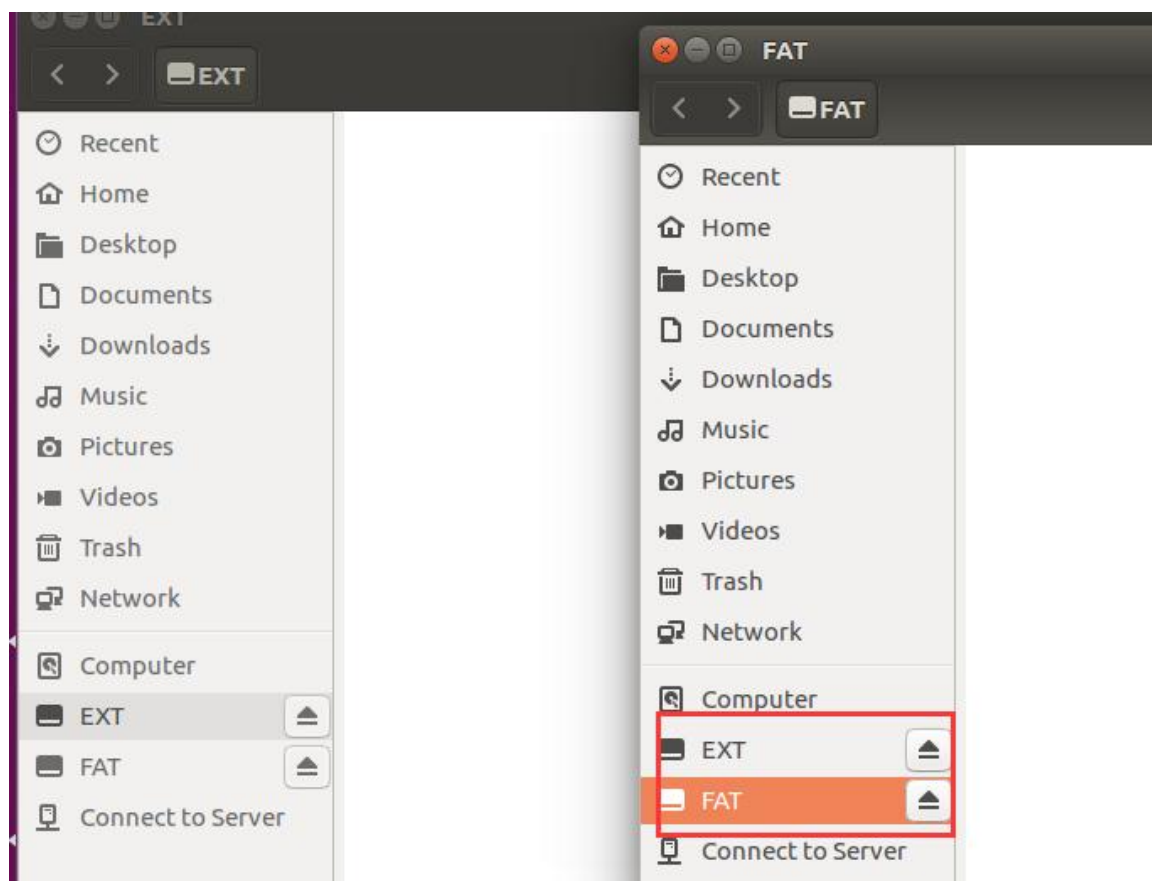
7) 创建第二个分区，用于存放根文件系统，格式为 EXT4，名称为 EXT



8) 创建分区完成后重新插入 sd 卡

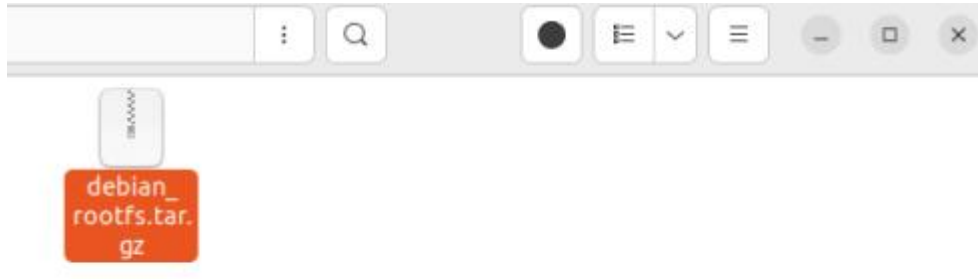


9) 系统会自动挂载分区，并弹出窗口



9.4.2 同步根文件系统到 SD 卡 EXT4 分区

- 1) 复制资料中的 debian 根文件系统的压缩包到 Linux 主机（实验复制在 “/home/alinx/work” 目录下）



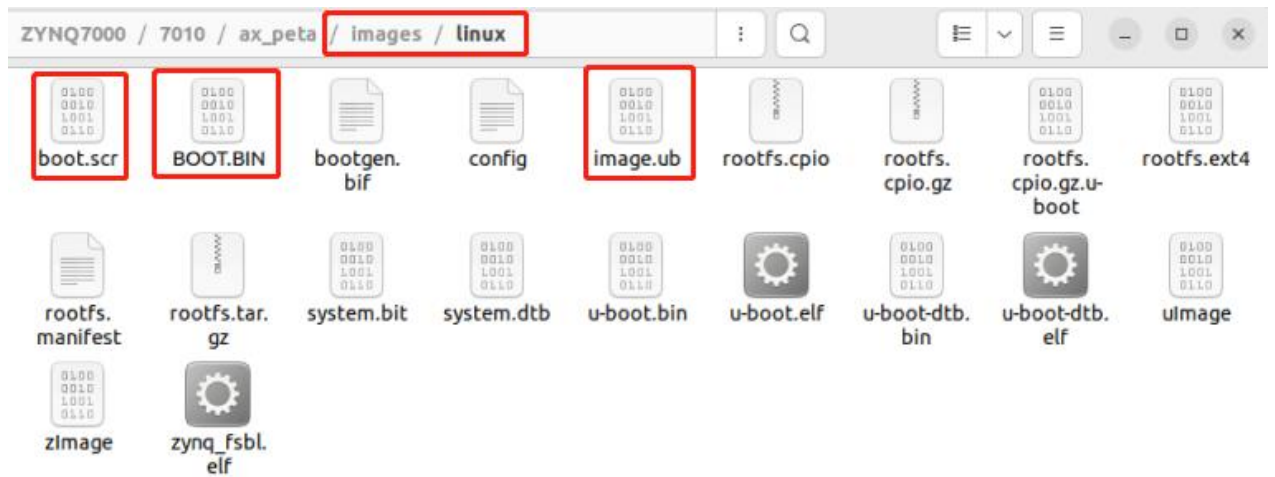
- 2) 输入解压缩命令“tar -xzvf txt.tar.gz -C /media/alinx/EXT4/ ”，解压缩文件系统，-C 选项后面的路径是 EXT 分区所在的路径。解压缩可能需要几分钟的时间。*请注意解压方式，必须和教程一致。*

```
alinx@alinx:~/work/Project$ tar -xzvf txt.tar.gz -C /media/alinx/EXT4/
```

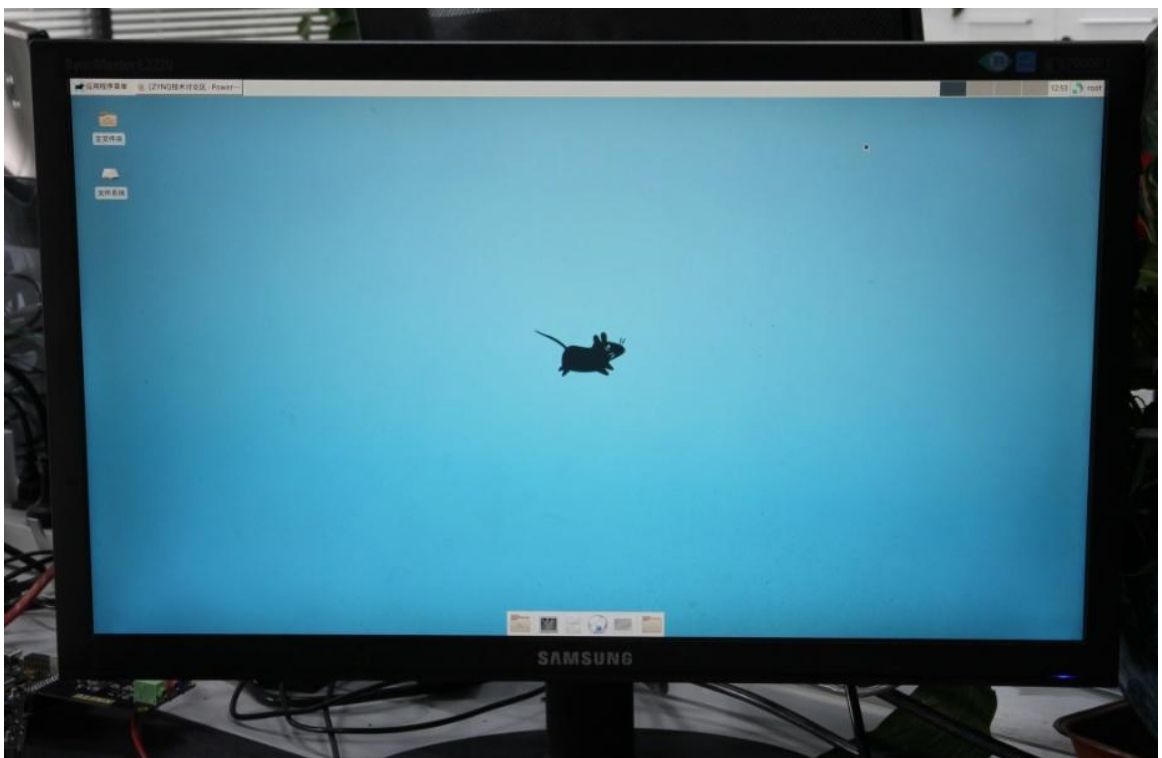
- 3) 输入 sync 命令，等待系统把解压后的文件同步到 SD 卡中，需要等待一段时间，当命令行中重新出现命令提示符时，表示同步的过程结束。

```
alinx@alinx:~/work/Project$ sync
alinx@alinx:~/work/Project$
```

- 4) 把 BOOT.bin, image.ub, boot.scr 复制到 sd 的 FAT32 分区（第一分区）中，设置开发板 sd 模式启动，插上 HDMI 显示器，启动开发板。



- 5) SD 卡制作完成后, 把制作好后的 SD 卡插入到开发板的 SD 卡插槽内。连接 USB 串口线, 连接 HDMI 显示器, 开发板上电后在 HDMI 显示器上会显示 Debian 的操作系统的界面。另外在串口工具里我们可以看到操作系统启动的过程, 运行 u-boot 之后开始运行 Linux, 账号: root, 密码: root



- 6) 进入系统后使用 ifconfig 命令查看网络连接

```
root@zynq:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0a:35:00:1e:53
          inet addr:192.168.1.46  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::20a:35ff:fe00:1e53/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:119 errors:0 dropped:0 overruns:0 frame:0
          TX packets:86 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:12231 (11.9 KiB)  TX bytes:7420 (7.2 KiB)
          Interrupt:29 Base address:0xb000

eth1      Link encap:Ethernet  HWaddr 00:0a:35:00:03:22
          inet addr:192.168.1.62  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::20a:35ff:fe00:322/64 Scope:Link
          UP BROADCAST RUNNING  MTU:1500  Metric:1
          RX packets:26 errors:0 dropped:0 overruns:0 frame:0
          TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2771 (2.7 KiB)  TX bytes:1184 (1.1 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:236 errors:0 dropped:0 overruns:0 frame:0
          TX packets:236 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:18960 (18.5 KiB)  TX bytes:18960 (18.5 KiB)
```

9.5 常见问题

第十章 制作 QSPI Flash 启动的 Linux

实验 Vivado 工程为 “linux_base” ,在 vivado.zip 压缩包中。

实验 petalinux 工程相关文件夹为 “ax_peta_qspi”

前面教程讲解的 Linux 都是在 SD 里启动，本实验讲解如何使用 Petalinux 制作一个从 QSPI Flash 启动的 Linux。

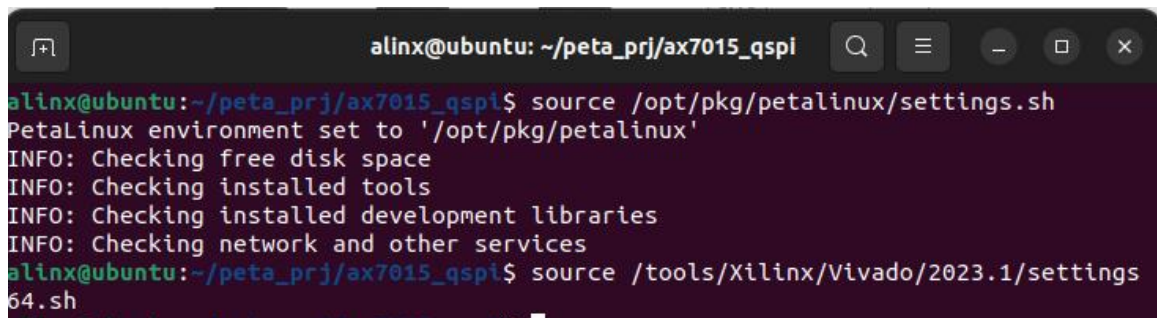
10.1 创建 Petalinux 工程

前面的 HDMI 教程中我们建立了一个基于 BSP 包创建的工程，这里我们可以直接使用该工程，后面如果需要创建新的工程，工程配置又和之前工程差别不大的情况下，就可以直接使用之前的工程，当然也可以用对应章节资料的 BSP 包重新创建工程。

10.2 配置编译 Petalinux

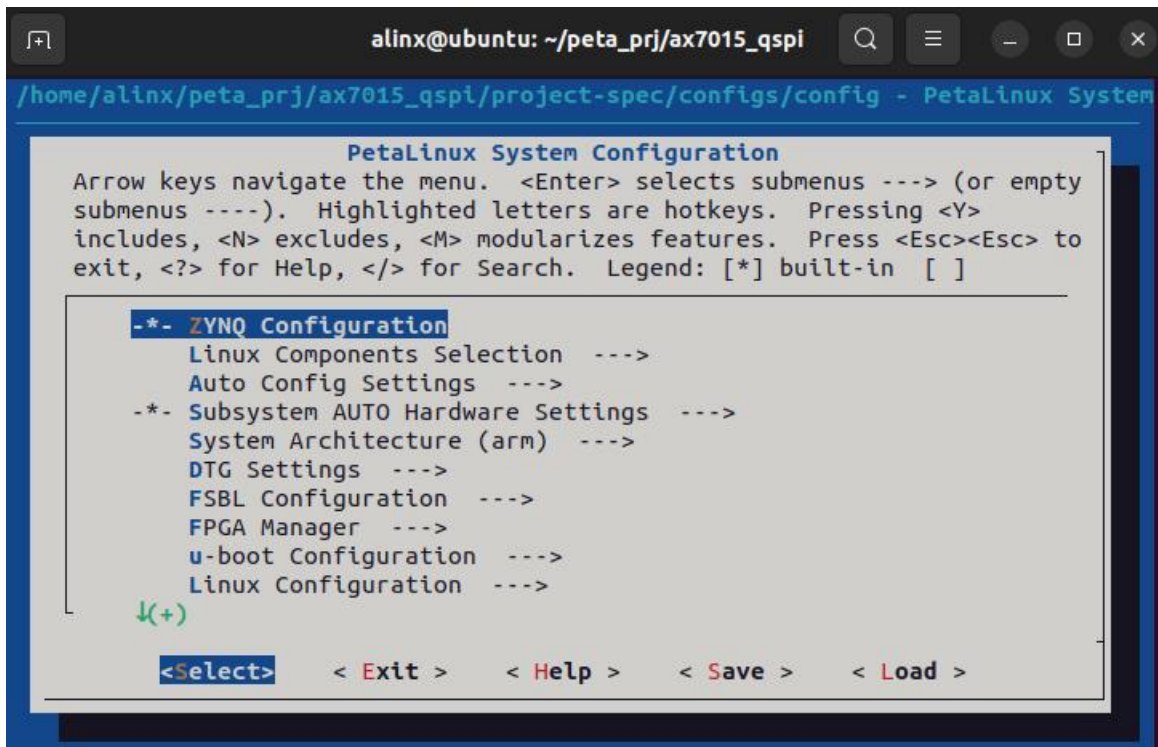
- 1) 使用下面命令设置环境变量

```
source /opt/pkg/petalinux/settings.sh
source /tools/Xilinx/Vivado/2023.1/settings64.sh
```

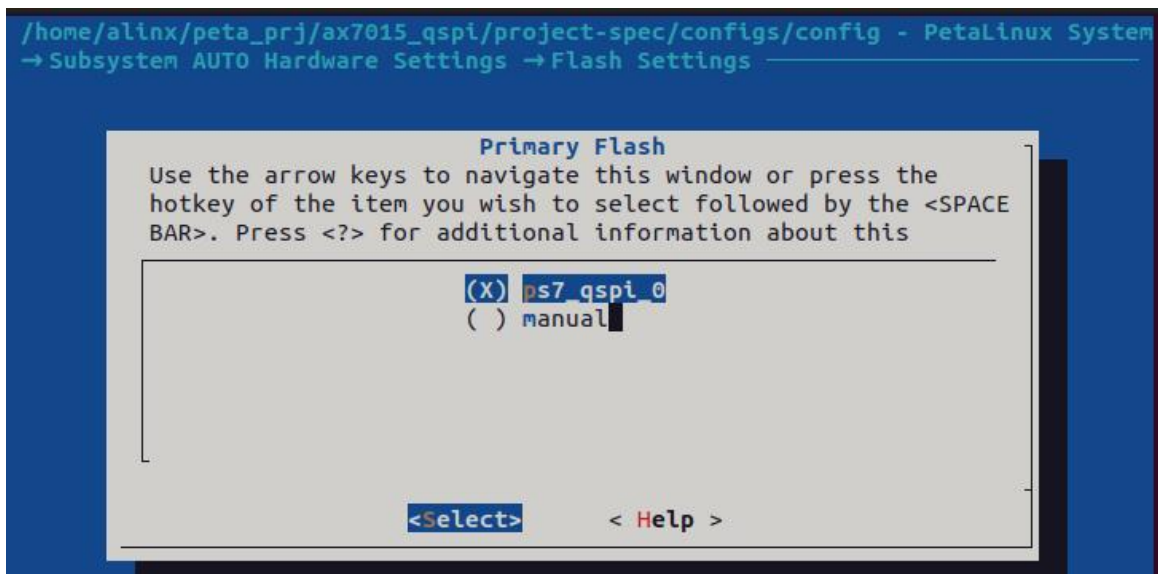


```
alinx@ubuntu: ~/peta_prj/ax7015_qspi
alinx@ubuntu:~/peta_prj/ax7015_qspi$ source /opt/pkg/petalinux/settings.sh
PetaLinux environment set to '/opt/pkg/petalinux'
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
alinx@ubuntu:~/peta_prj/ax7015_qspi$ source /tools/Xilinx/Vivado/2023.1/settings64.sh
```

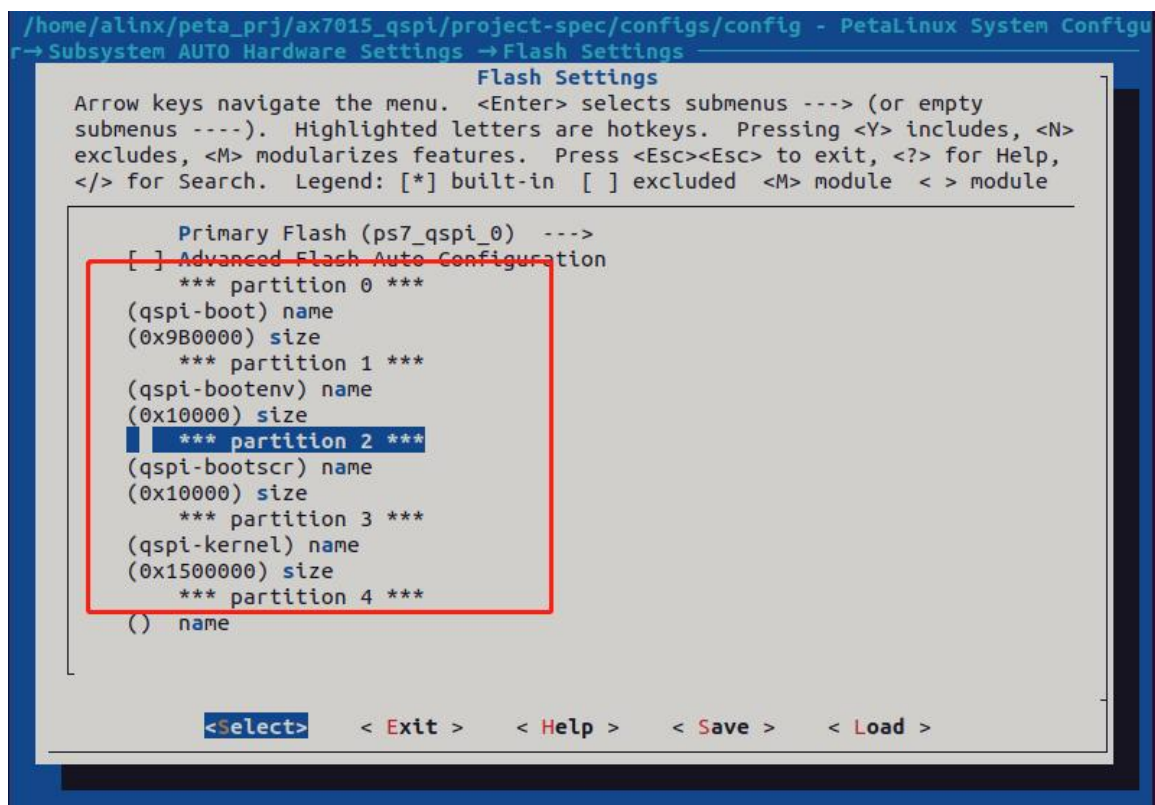
- 2) 使用 petalinux-config 命令配置 Petalinux



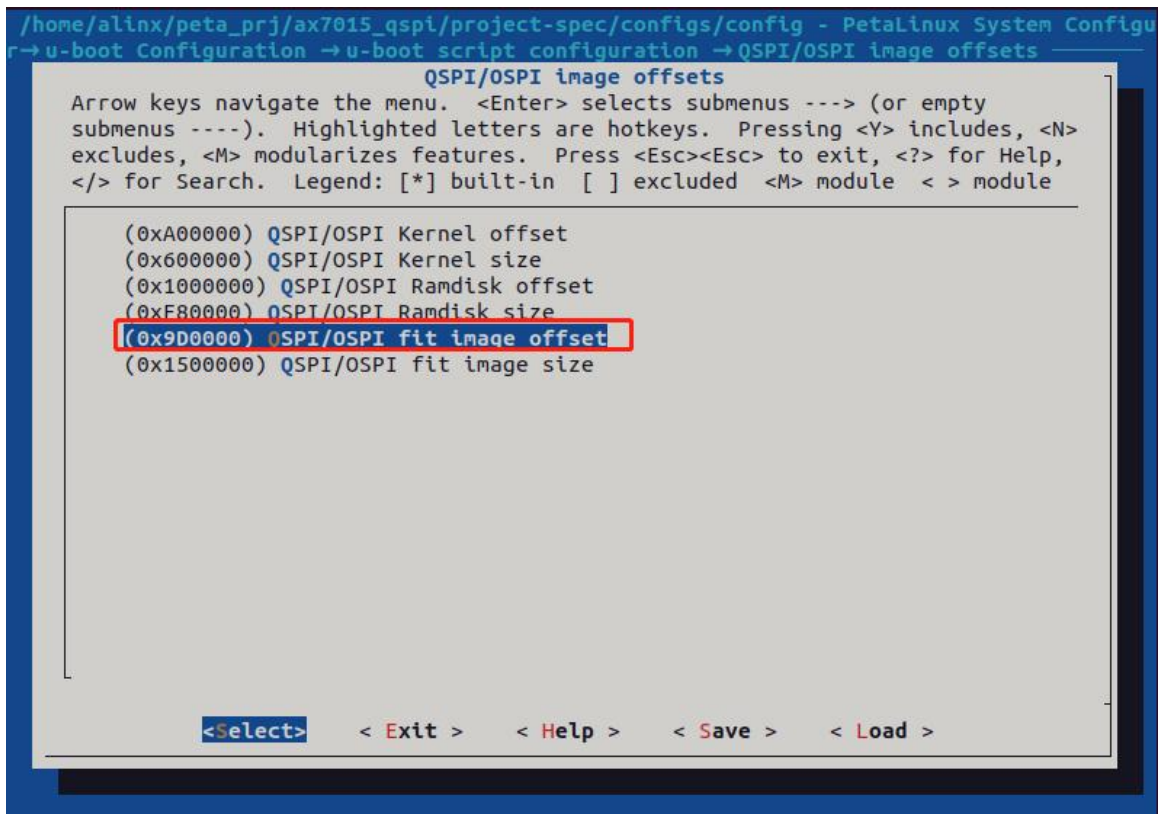
- 3) 在 Subsystem AUTO Hardware Settings ---> Flash Settings ---> Primary flash 选项中选择要使用的 flash 设备



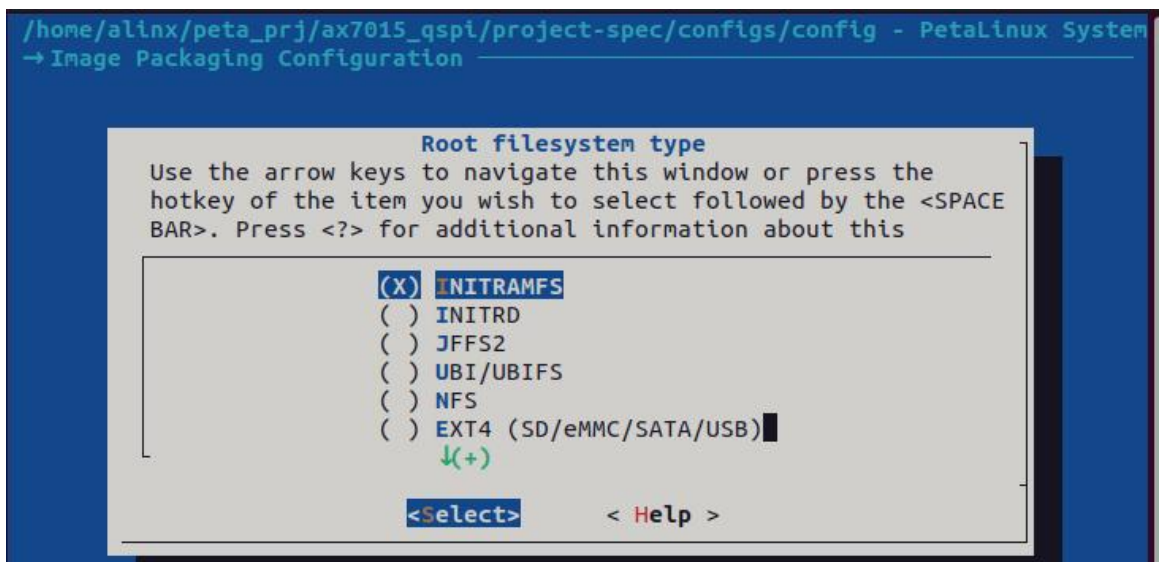
- 4) Subsystem AUTO Hardware Settings → Flash Settings 中可以修改 QSPI flash 的分区，需要按照下图调整一下大小，注意：u-boot 的默认加载地址为 0x9C0000。



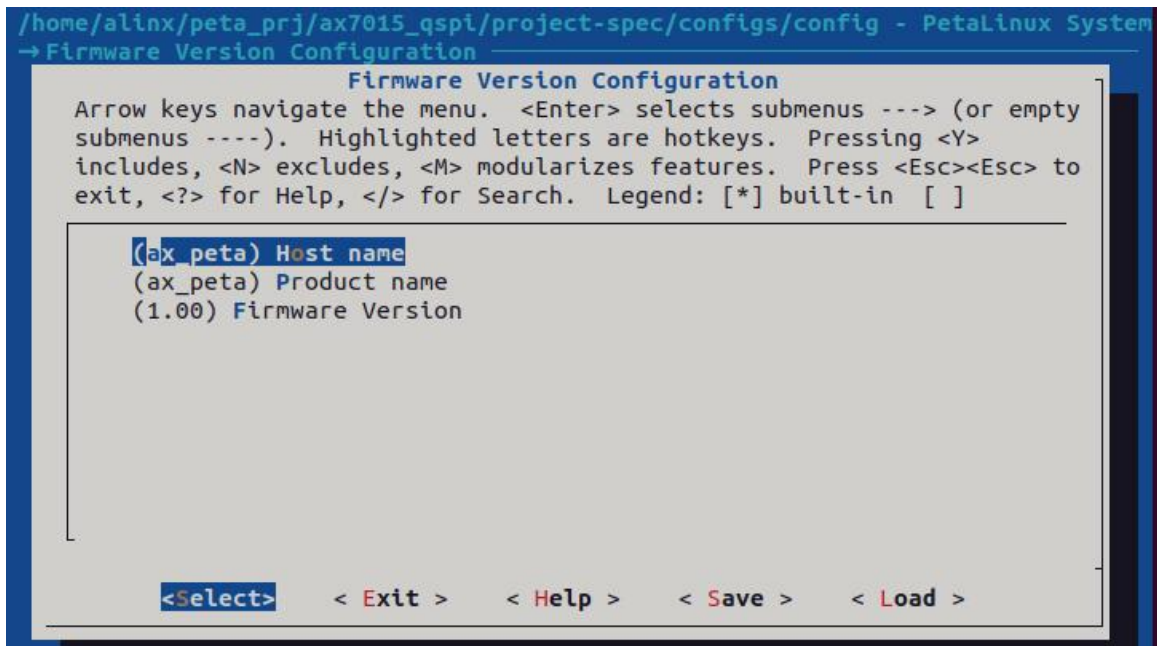
- boot 分区中存放描述 FPGA 的 .bit 文件、u-boot、fsbl、设备树.dtb 等文件。boot 分区的大小需要大于这些文件大小总和，需要根据实际情况修改。
 - bootenv 分区存放 u-boot 的环境变量，根据实际情况修改。
 - kernel 分区存放 image.ub，分区大于 image.ub 即可，需要根据实际情况修改。
 - bootscr 分区存放 boot.scr，分区大于 boot.scr 即可，需要根据实际情况修改。
 - 另外需要注意，所有打包文件的大小总和，不能超过 flash 的大小。
- 5) 在 u-boot Configuration → u-boot script configuration → QSPI/OSPI image offsets 中更改下图中的 QSPI/OSPI fit image offset 选项，将偏移量改为 0x9D0000，放在 boot.scr 文件的后面，需要与上一步中的地址对应。



- 6) 在 Image Packaging Configuration ---> Root filesystem type 选择 INITRAMFS, 使用 RAM 类型的根文件系统, 这样就可以方便打包烧写到 QSPI Flash。



- 7) 在 Firmware Version Configuration ---> 中可以修改 Host name 等信息



- 8) 保存后，使用 petalinux-build 命令编译工程
- 9) 编译成功后，使用下面命令合成 BOOT.bin，使用--kernel 选项把内核编译进 BOOT.bin，此处参照官方文档进行了内核偏移量的指定，不然会打包不成功。

```
petalinux-package --boot --u-boot --kernel --fsbl --fpga --force
```

```
alinx@ubuntu:~/peta_prj/ax7015_qspi$ petalinux-package --boot --u-boot --kernel --fsbl --fpga --force
[INFO] Sourcing buildtools
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax7015_qspi/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax7015_qspi/project-spec/hw-description/design_1_wrapper.bit"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax7015_qspi/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax7015_qspi/images/linux/system.dtb"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax7015_qspi/images/linux/boot.scr"
INFO: File in BOOT BIN: "/home/alinx/peta_prj/ax7015_qspi/images/linux/image.ub"
INFO: Generating zynq binary package BOOT.BIN...

***** Bootgen v2023.1
**** Build date : Apr 7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition design_1_wrapper.bit.0 memory range
[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition boot.scr.0 memory range
[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition image.ub.0 memory range
[WARNING]: Partition design_1_wrapper.bit.0 range is overlapped with partition boot.scr.0 memory range
[WARNING]: Partition design_1_wrapper.bit.0 range is overlapped with partition image.ub.0 memory range
[WARNING]: Partition system.dtb.0 range is overlapped with partition image.ub.0 memory range
[WARNING]: Partition boot.scr.0 range is overlapped with partition image.ub.0 memory range

[INFO] : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
```

- 10) 可以参考 S2 教程 4.5 “固化程序” 章节内容将 BOOT.bin 烧写到 QSPI Flash。
- 11) 调整启动模式为 QSPI，将跳线帽接到 QSPI 启动。

应用篇

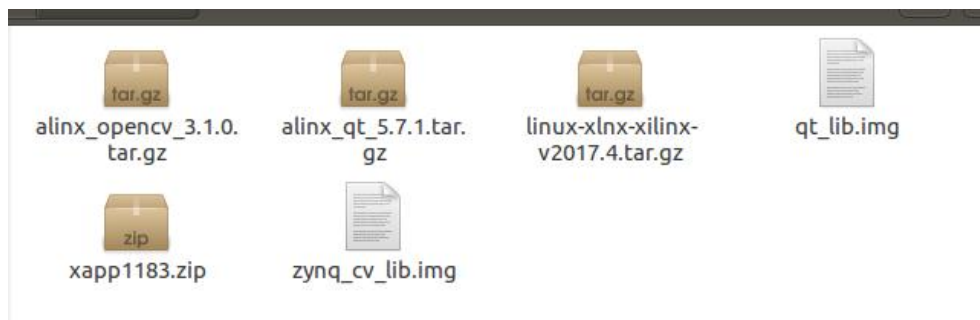
例程需要使用 petalinux 2023.1 环境, 请参考前面的教程, 安装好 petalinux, 按照教程学习好 petalinux 工具的使用。如果使用其他版本的 petalinux, 可能无法正常工作。

例程需要熟练掌握裸机实验例程, 请先学习 vitis 裸机例程。

例程需要非常好的软件开发基础, 请打好基础, 量力而行。

例程所有源文件位于名为 `course_s4_linux` 压缩包中。

例程中可能用到的源码包、安装软件在 **`zynq_soft_2017/00_resource(需单独下载)`** 目录下



例程目录一般结构: `app` 表示 Linux 下的应用软件; `petalinux` 为 petalinux 工程目录, Linux 内核配置、设备树、根文件系统配置都在这里; `sd_boot` 目录为编译好的可运行程序, 复制所有文件到 sd 卡, 可快速复现实验; `vivado` 目录是例程需要的 vivado 工程。

例程中 petalinux 工程使用的 Linux 内核使用本地源, 如果直接使用例程 petalinux 工程, 请先使用 `petalinux-config` 命令配置 Linux 内核源码路径。如何使用复制过来的 petalinux 工程, 在前面的基础篇里已经讲过, 不会的要先回去复习一下。

再次申明, 本实验内容要求熟练掌握前面所学内容, 对 vitis 裸机开发、petalinux 使用熟练掌握才能学习, 免得操作上遇到低级错误。

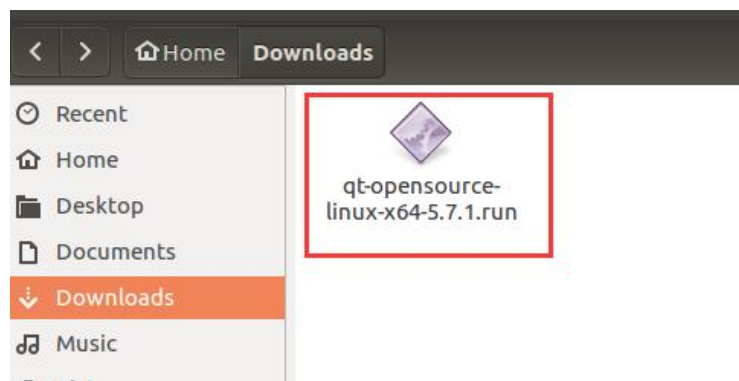
第十一章 QT 在 ZYNQ 上的应用

Qt 是跨平台 C++ 图形用户界面应用程序开发框架，可以简单的将它理解为用于图形开发的库。如果我们使用 Qt 库为板卡开发带界面的应用程序，则需要交叉编译 Qt 库。OpenCV 是视频处理中有名的库，很多应用程序需要使用 OpenCV 库，以往 QT 和 OpenCV 库都需要自己下载代码编译，非常麻烦。本章讲解如何使用 Petalinux 配置 OpenCV 和 QT 的开发环境，再次体现出 Petalinux 开发环境的优势，只需简单的配置就可以解决很复杂的问题。

11.1 QT Creator 安装

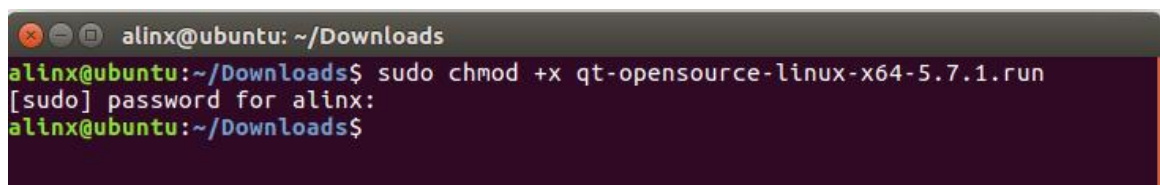
我们在 Ubuntu 中安装 QT Creator，安装文件名称为 “qt-opensource-linux-x64-5.7.1.run”，从文件名我们可以看出 QT 版本为 5.7.1，这些版本也是我们精心挑选，适合当前环境的版本。

- 1) 将安装文件复制到 Ubuntu 主机中



- 2) 添加运行权限

```
sudo chmod +x qt-opensource-linux-x64-5.7.1.run
```

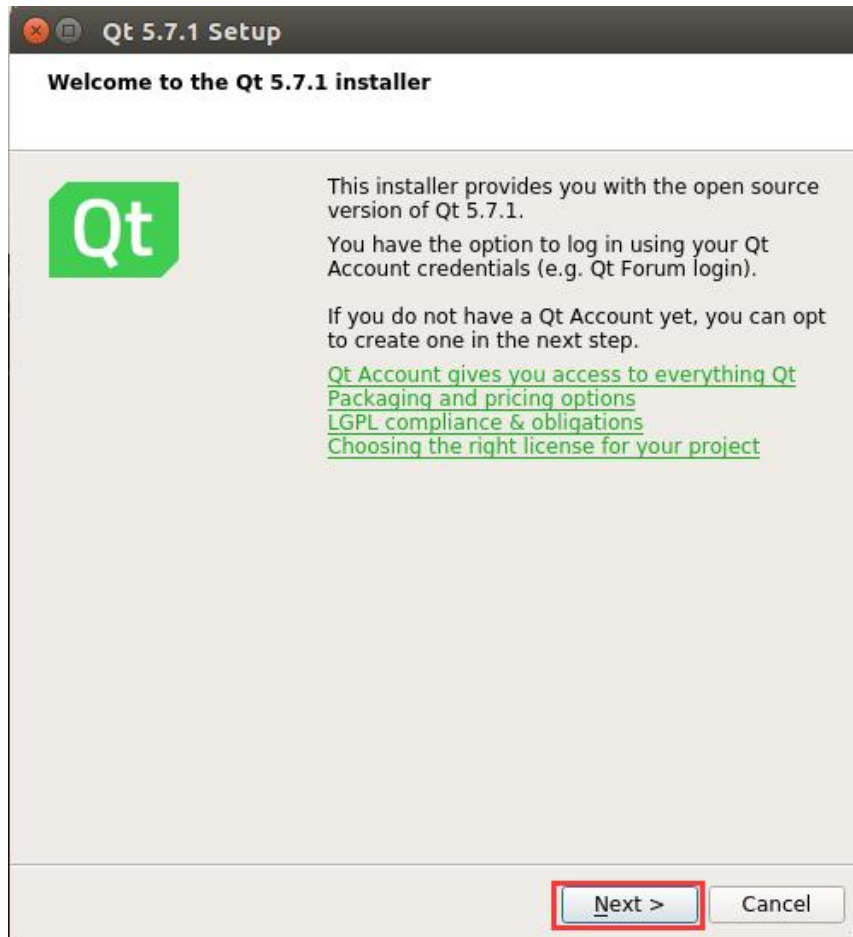


- 3) 运行安装软件

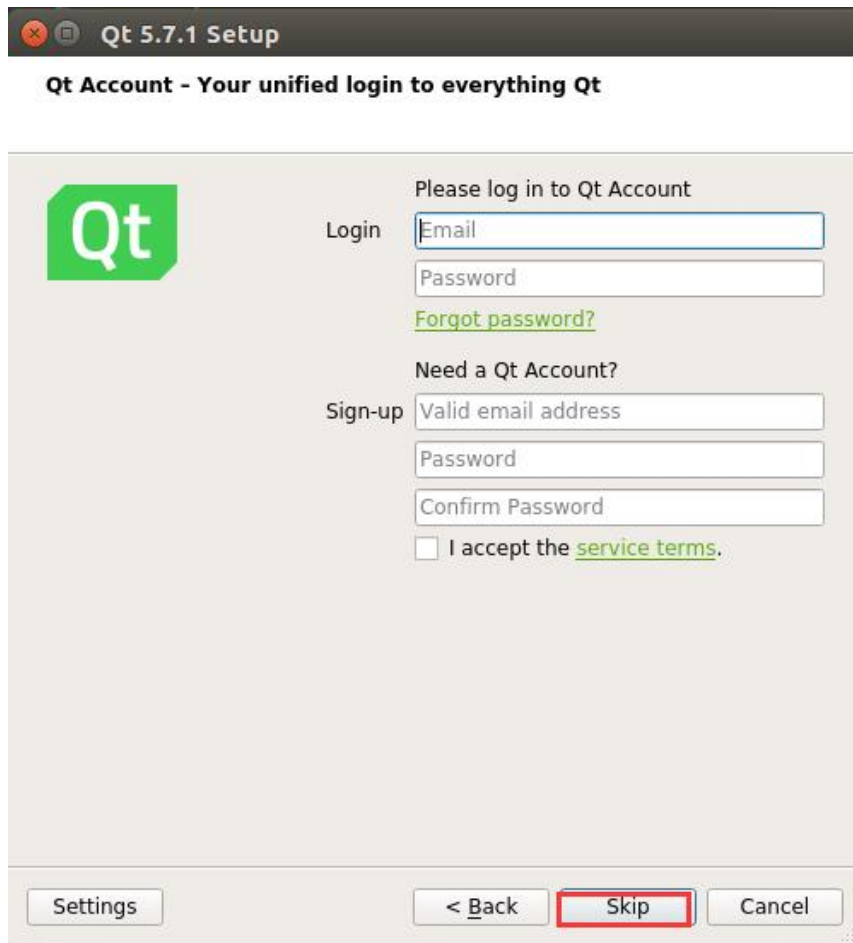
```
sudo ./qt-opensource-linux-x64-5.7.1.run
```

```
alinx@ubuntu: ~/Downloads
alinx@ubuntu:~/Downloads$ sudo chmod +x qt-opensource-linux-x64-5.7.1.run
[sudo] password for alinx:
alinx@ubuntu:~/Downloads$ sudo ./qt-opensource-linux-x64-5.7.1.run
[sudo] password for alinx:
```

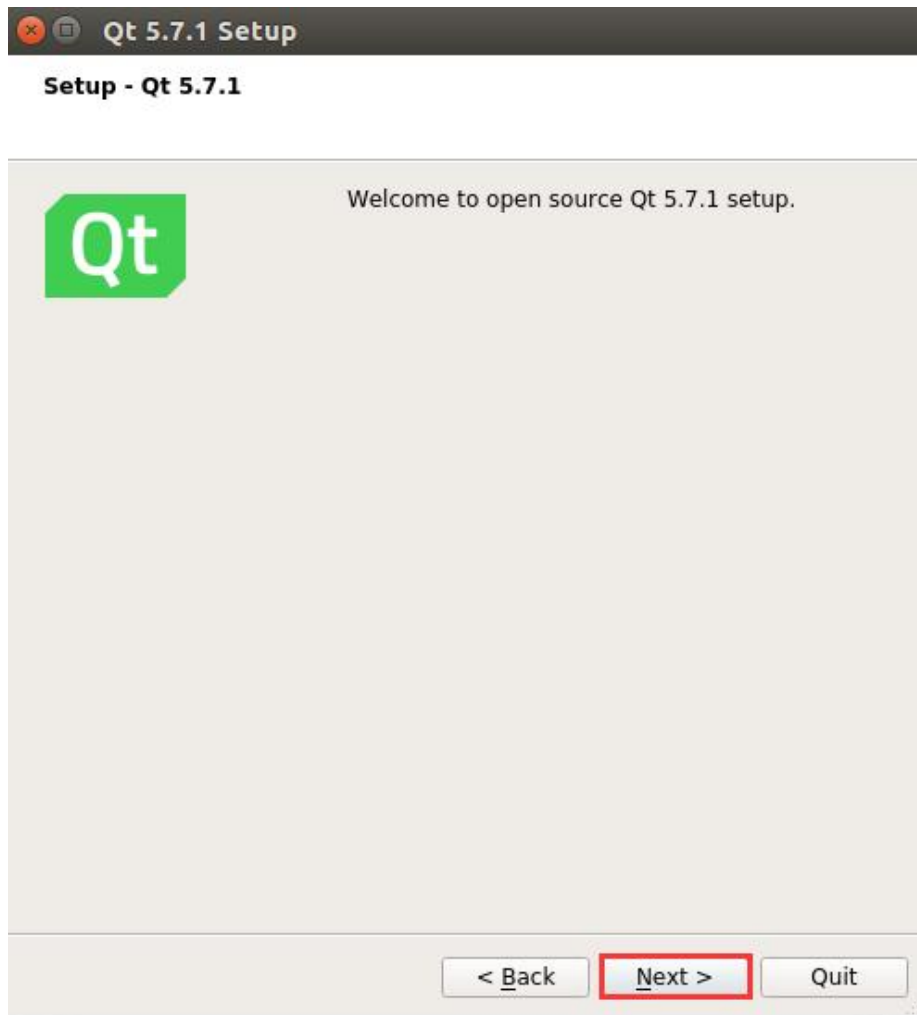
4) 点击 Next 开始安装



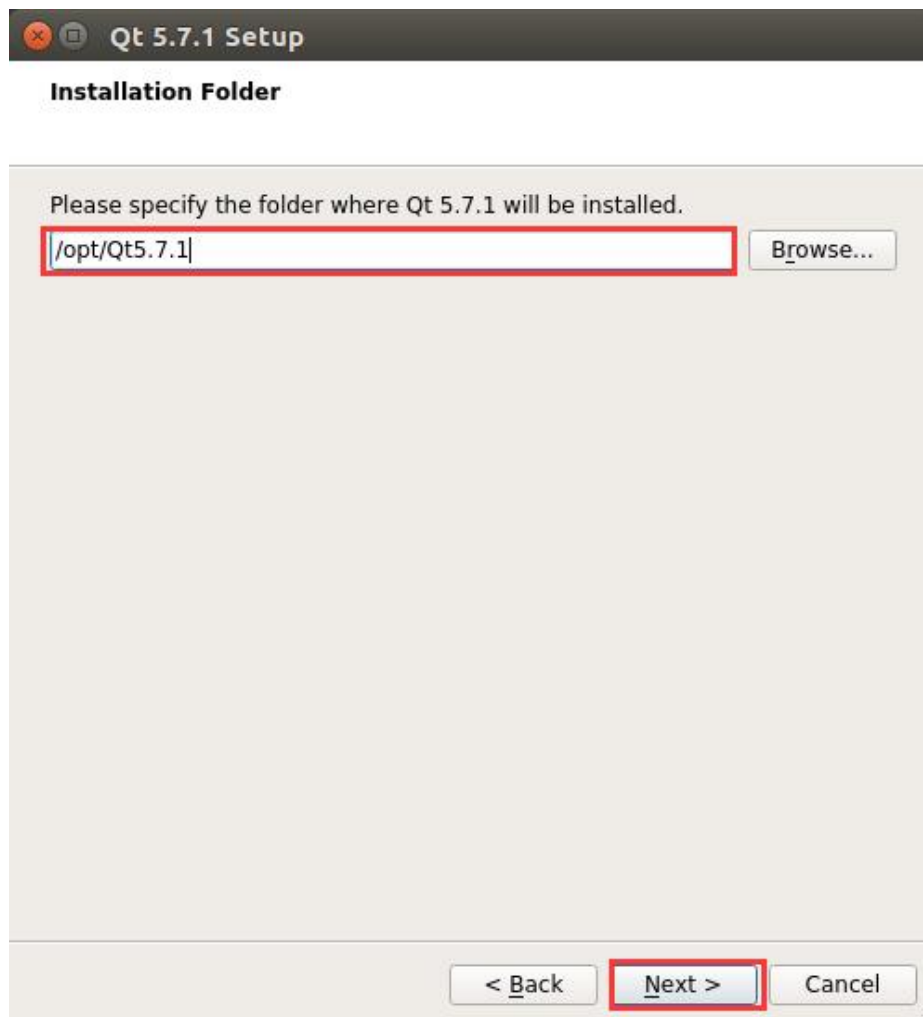
5) 点击 "Skip" 跳过账号信息填写



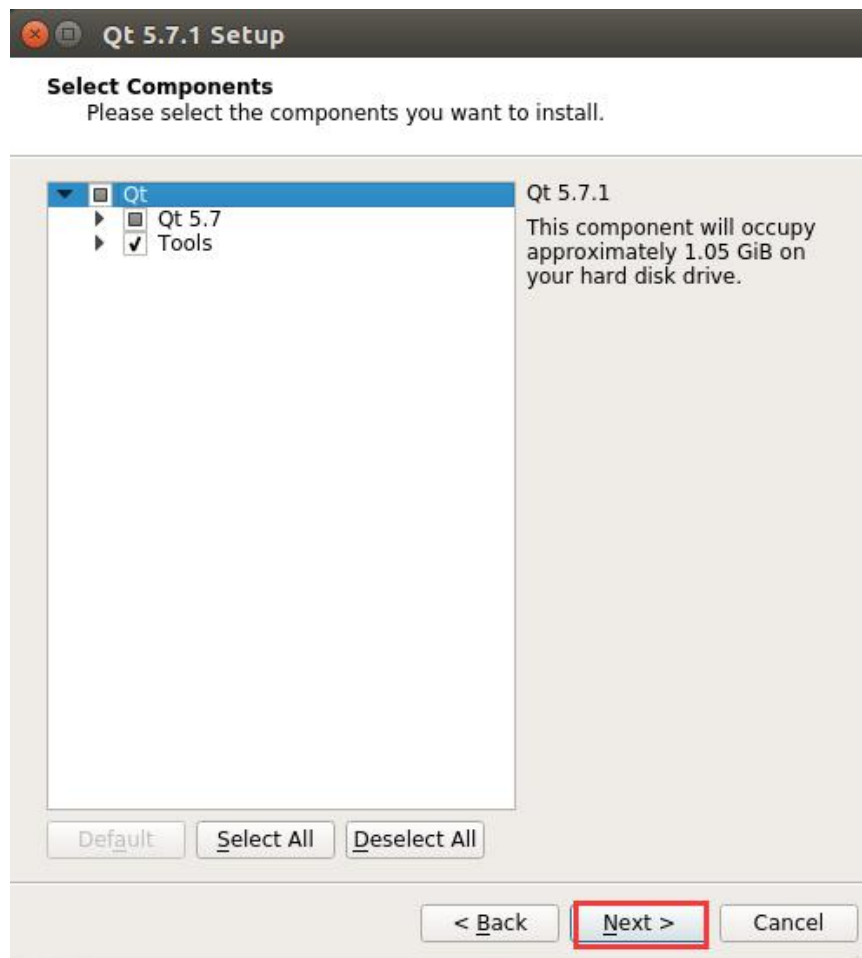
6) 点击 “Next” 继续安装



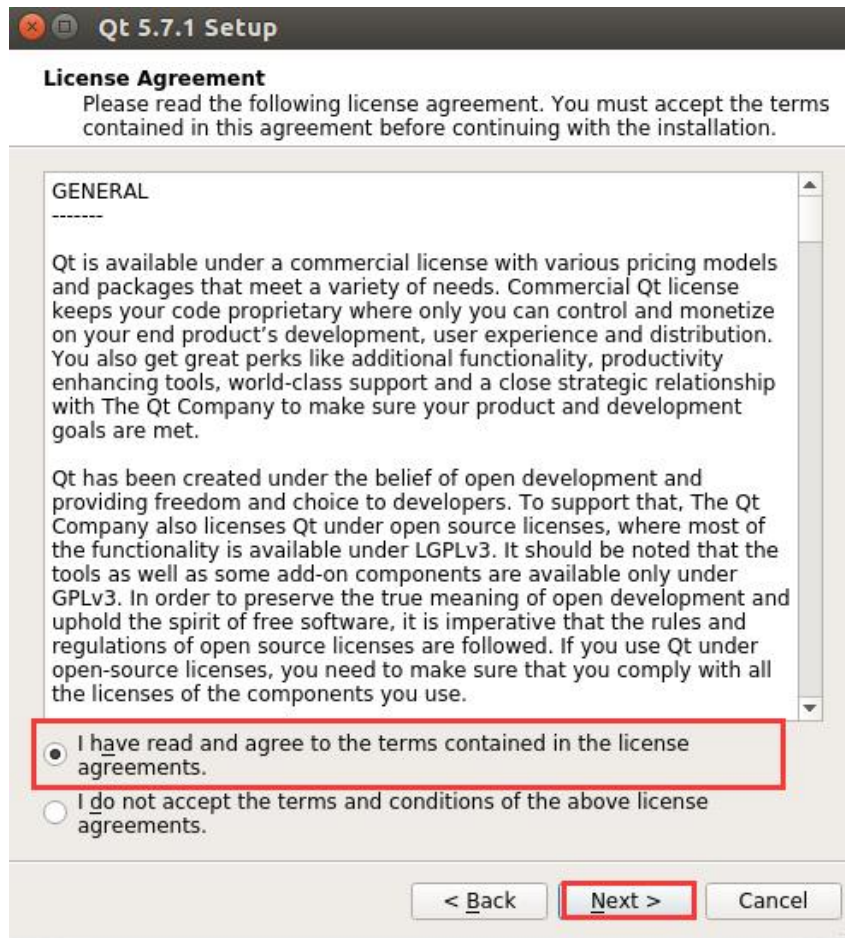
- 7) 安装路径保持默认，点击“Next”继续安装



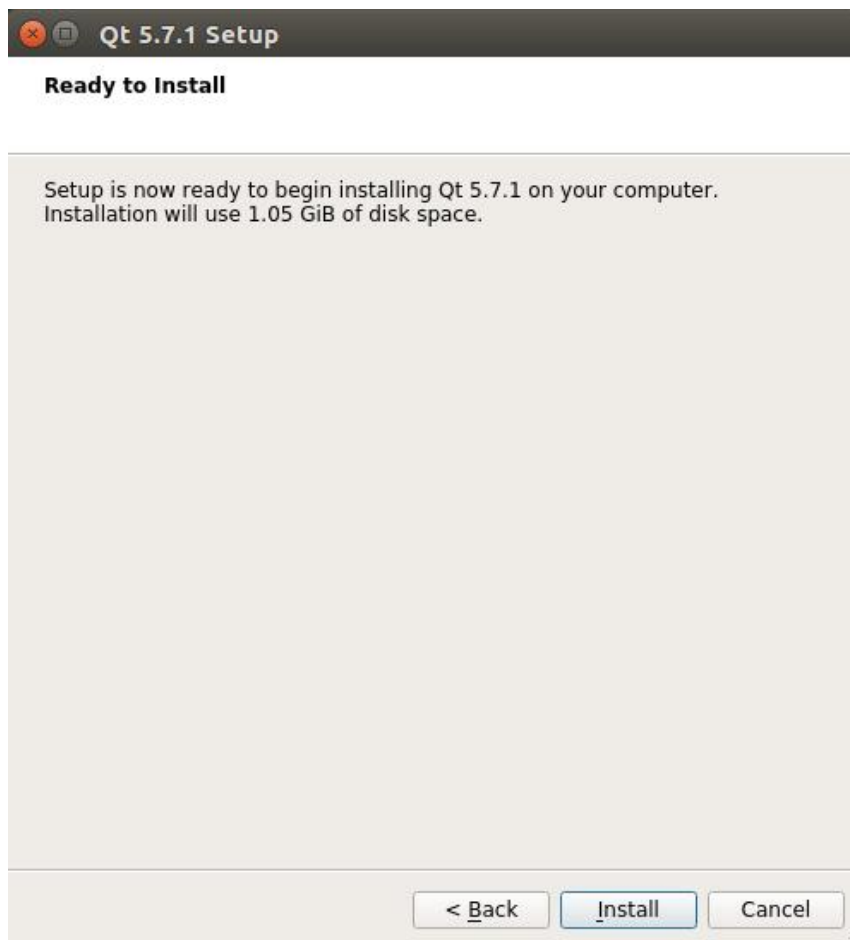
- 8) 保持默认安装选项，点击 “Next” 继续安装



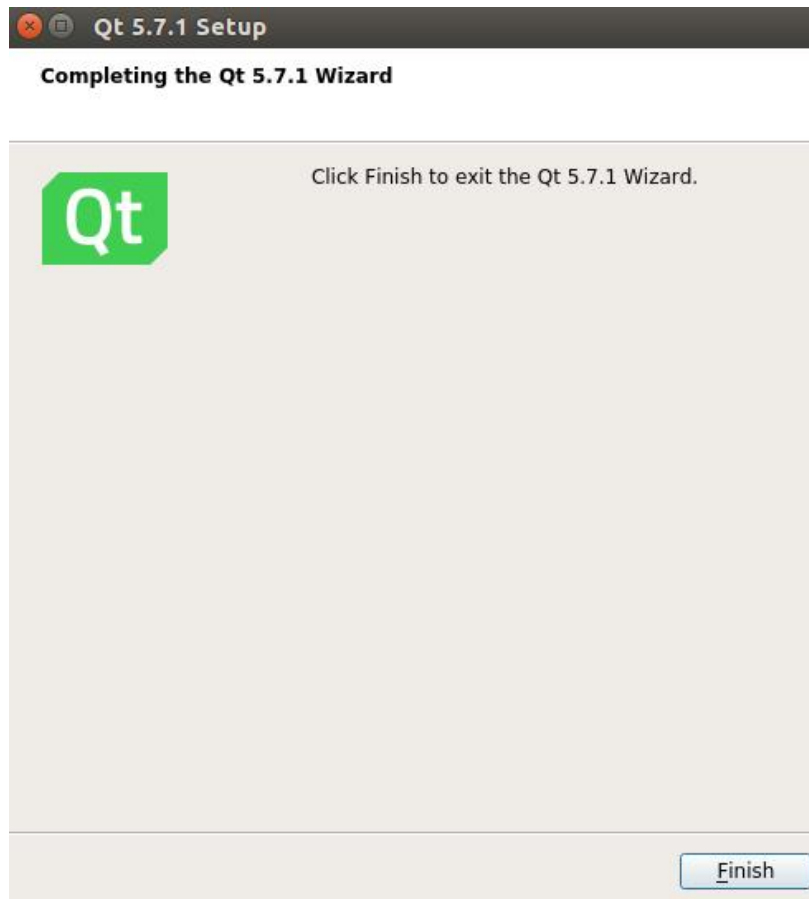
9) 选择同意 License 许可



10) 点击 “Install” 安装



11) 点击 “Finish” 完成安装



11.2 QT 库交叉编译

前面安装的 QT 可以创建在 x86 处理器上运行的应用程序，不能建立 ZYNQ 能运行的程序，我们需要交叉编译 QT 的源代码才能在 zynq 上运行。为了简化编译流程，芯驿电子准备好了 QT 5.7.1 的源代码包，并制作了一些编译脚本。

- 1) 将压缩包复制到 Ubuntu 主机中

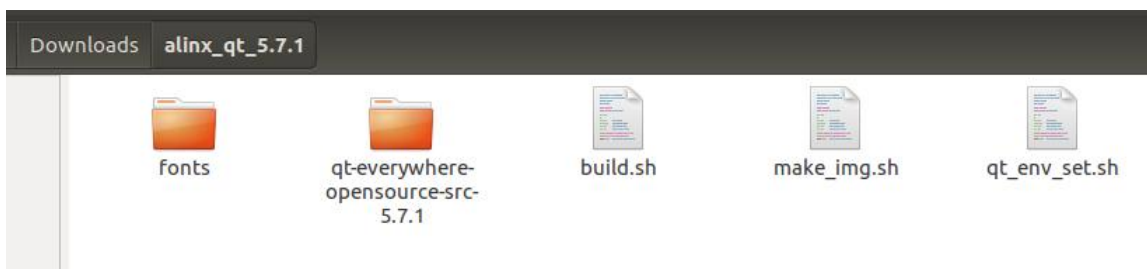


2) 将压缩包解压

```
tar -zxvf alinx_qt_5.7.1.tar.gz
```

```
alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtdatavis3d/src/datavisualization/global/datavisualizationglobal_p.h
alinx@ubuntu:~/Downloads$ tar -zxvf alinx_qt_5.7.1.tar.gz
```

- 3) 解压后的文件夹，“fonts”是QT应用程序显示需要用到的字体文件，“qt-everywhere-opensource-src-5.7.1”是QT源代码文件夹，“build.sh”是编译脚本，“make_img.sh”是把编译的库打包成img镜像文件。



- 4) “build.sh”脚本中指定了XVitis的安装目录，主要使用XVitis自带的交叉编译器，如果你的Vitis安装目录和脚本里不同，请修改XVitis路径，同时指定ZYNQ版本QT库的安装路径为“/opt/alinx/zynq_qt5.7.1”，编译完成以后可以在这个路径里找到交叉编译过的QT库，不建议修改此路径。

```
#!/bin/sh
SDK_PATH=/opt/Xilinx/SDK/2017.4
if [ ! -d $SDK_PATH ]; then
    echo "no dir ${SDK_PATH}"
    exit 1
fi
source ${SDK_PATH}/settings64.sh
export ZYNQ_QT_INSTALL=/opt/alinx/zynq_qt5.7.1
export PKG_CONFIG_LIBDIR=${SDK_PATH}/gnu/aarch32/bin/gcc-arm-linux-gnueabi/arm-linux-gnueabi/lib
export CROSS_COMPILE=arm-linux-gnueabi-

if [ ! -d $PKG_CONFIG_LIBDIR ]; then
    echo "no ${PKG_CONFIG_LIBDIR}"
    exit 1
fi

Cur_Dir=$(pwd)
QT_SOURCE=qt-everywhere-opensource-src-5.7.1
echo $Cur_Dir
export ZYNQ_QT_BUILD=$Cur_Dir
export PATH=$ZYNQ_QT_INSTALL/bin:$PATH
cd $QT_SOURCE
./configure -xplatform arm-linux-gnueabi-g++ \
    -opensource \
    -confirm-license \
```

5) 运行编译脚本

```
./build.sh
```

```
alinx@ubuntu: ~/Downloads/alinx_qt_5.7.1
alinx@ubuntu:~/Downloads/alinx_qt_5.7.1$ ./build.sh
```

- 6) 等待一段时间，编译快完成时需要输入 Ubuntu 账户密码

```
alinx@ubuntu: ~/Downloads/alinx_qt_5.7.1
\${ORIGIN}/../lib -o marshal .obj/main.o -L/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/lib -lQt5Script -L/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtbase/lib -lQt5Core -lpthread
make[4]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/marshal'
arm-linux-gnueabi-g++ -Wl,-O1 -Wl,--enable-new-dtags -Wl,-z,origin -Wl,-rpath,\${ORIGIN}/../lib -Wl,-rpath-link,/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtbase/lib -o context2d .obj/qcontext2dcanvas.o .obj/context2d.o .obj/domimage.o .obj/environment.o .obj/window.o .obj/main.o .obj/qrc_context2d.o .obj/moc_qcontext2dcanvas.o .obj/moc_context2d.o .obj/moc_environment.o .obj/moc_window.o -L/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtbase/lib -lQt5Widgets -lQt5Gui -L/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/lib -lQt5Script -lQt5ScriptTools -lQt5Core -lpthread
make[4]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/context2d'
make[3]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script'
make[2]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples'
make[1]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript'
[sudo] password for alinx:
```

- 7) 输入密码后，编译安装完成

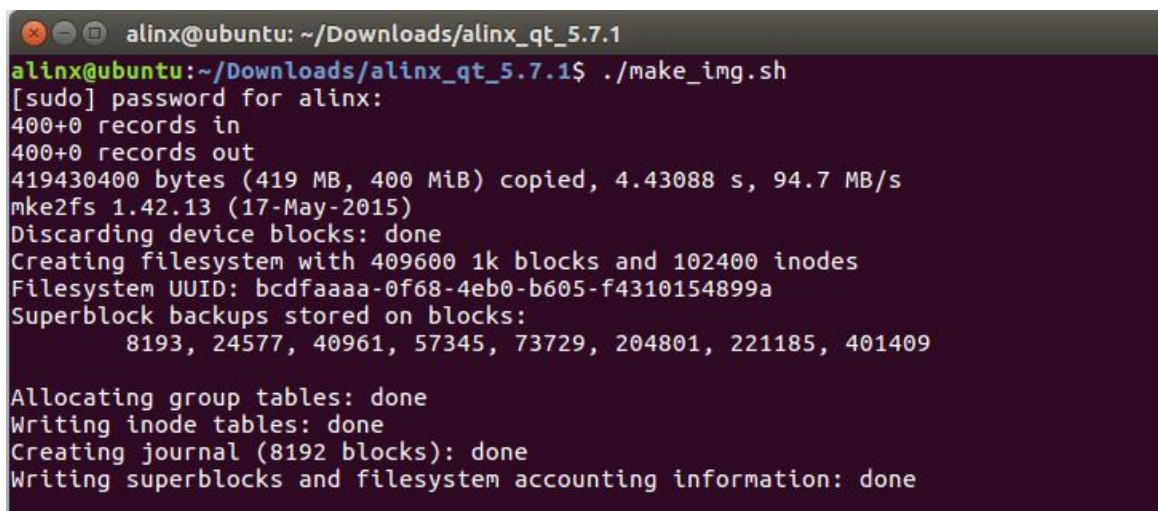
```
alinx@ubuntu: ~/Downloads/alinx_qt_5.7.1
make[4]: arm-linux-gnueabi-strip: Command not found
Makefile:786: recipe for target 'install_target' failed
make[4]: [install_target] Error 127 (ignored)
install -m 644 -p /home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/marshal/marshal.pro /opt/alinx/zynq_qt5.7.1/examples/script/marshal/
install -m 644 -p /home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/marshal/main.cpp /opt/alinx/zynq_qt5.7.1/examples/script/marshal/
make[4]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/marshal'
install -m 644 -p /home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/README /opt/alinx/zynq_qt5.7.1/examples/script/
install -m 644 -p /home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script/script.pro /opt/alinx/zynq_qt5.7.1/examples/script/
make[3]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples/script'
make[2]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript/examples'
make[1]: Leaving directory '/home/alinx/Downloads/alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtscript'
alinx@ubuntu:~/Downloads/alinx_qt_5.7.1$
```

- 8) 到目录 “/opt/alinx/zynq_qt5.7.1” 下我们可以找到 bin 目录，里面有我们要用到的 qmake 文件，和 lib 目录，这个目录就是我们需要的 QT 库。

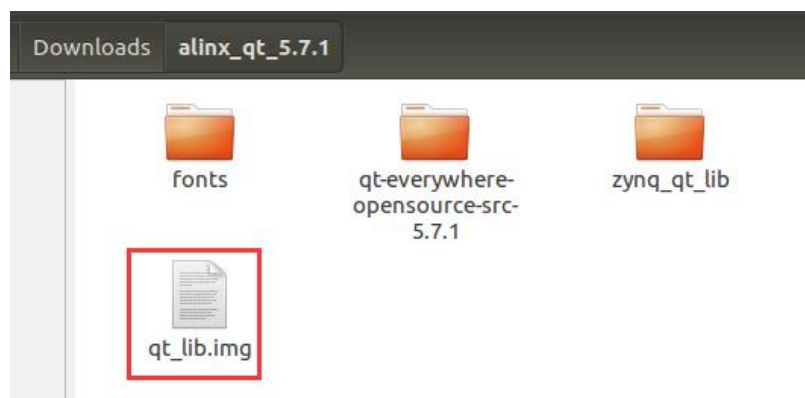


- 9) 运行脚本制作 QT 库的 img 文件，需要输入账号密码

```
./make_img.sh
```

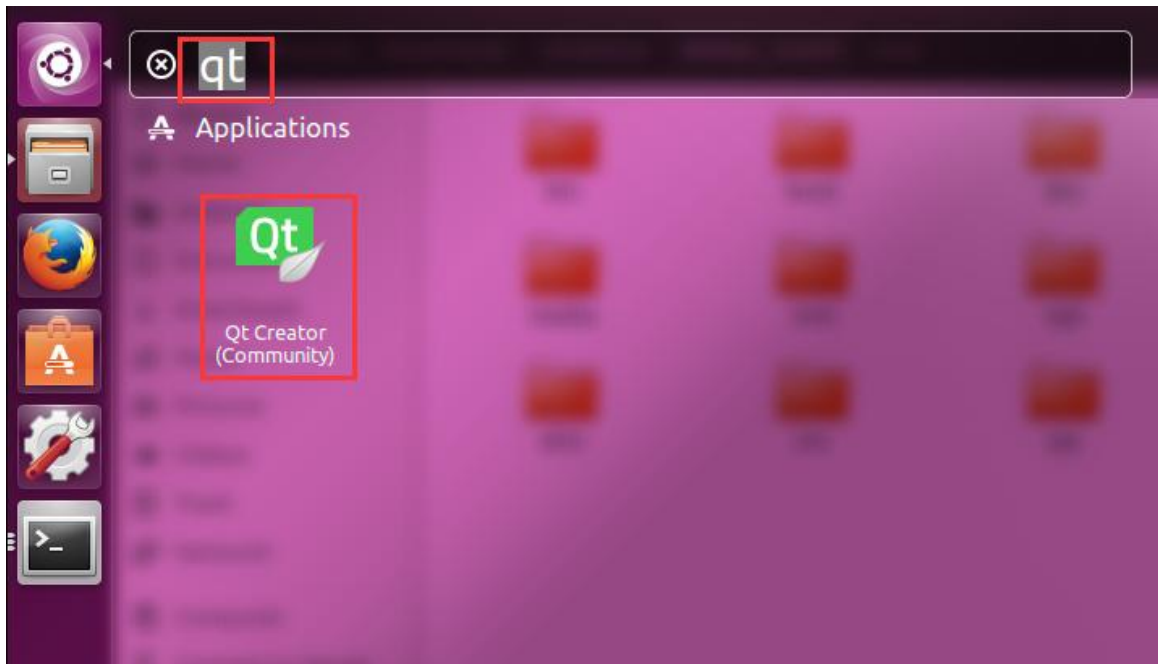


- 10) 得到 qt_lib.img 可以在 zynq 板上挂载

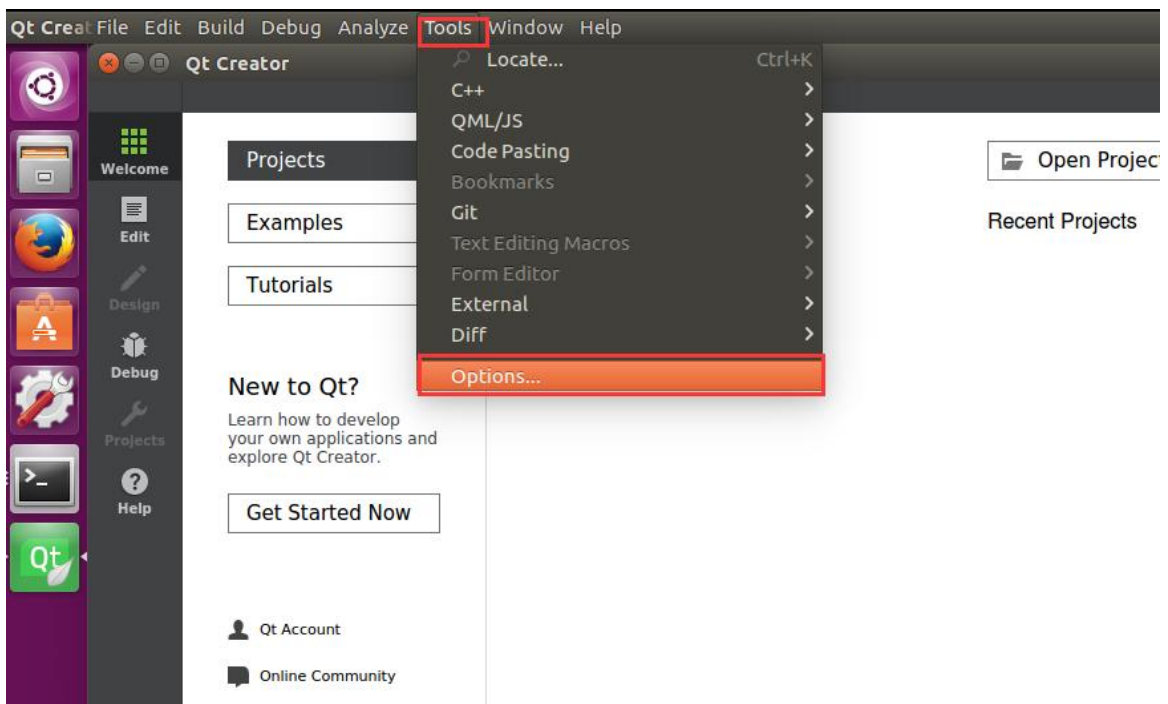


11.3 配置 ZYNQ 版 QT Kits

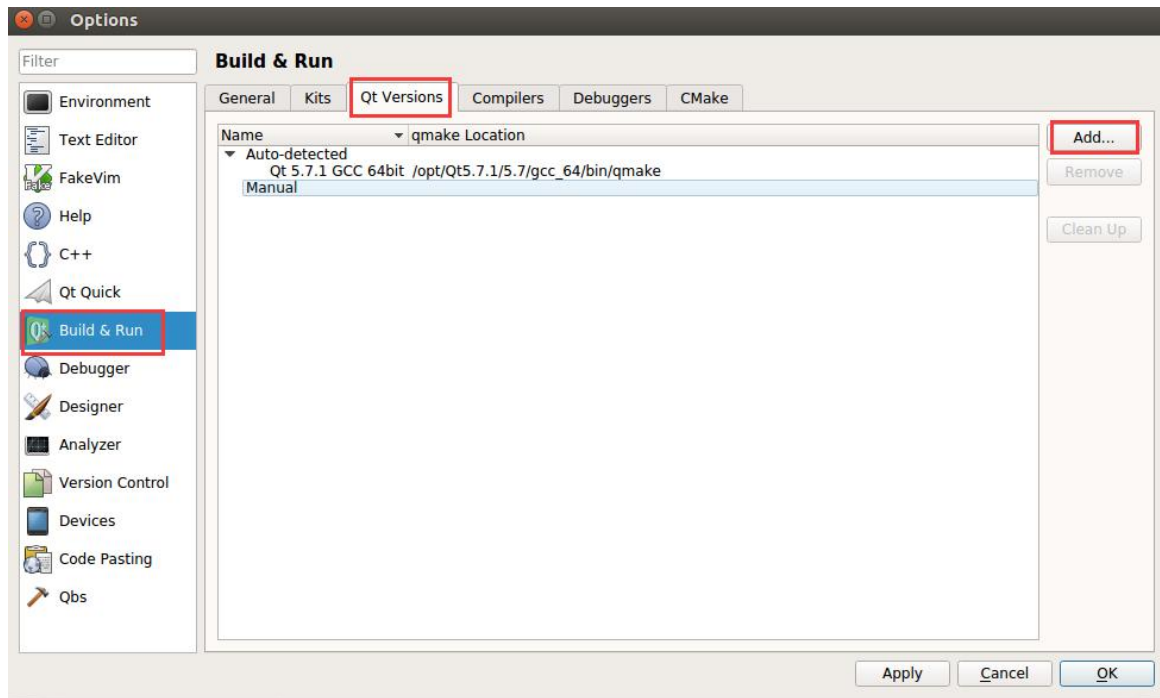
- 1) 运行 QT Creator, 找不到应用程序快捷方式, 可以搜索一下。



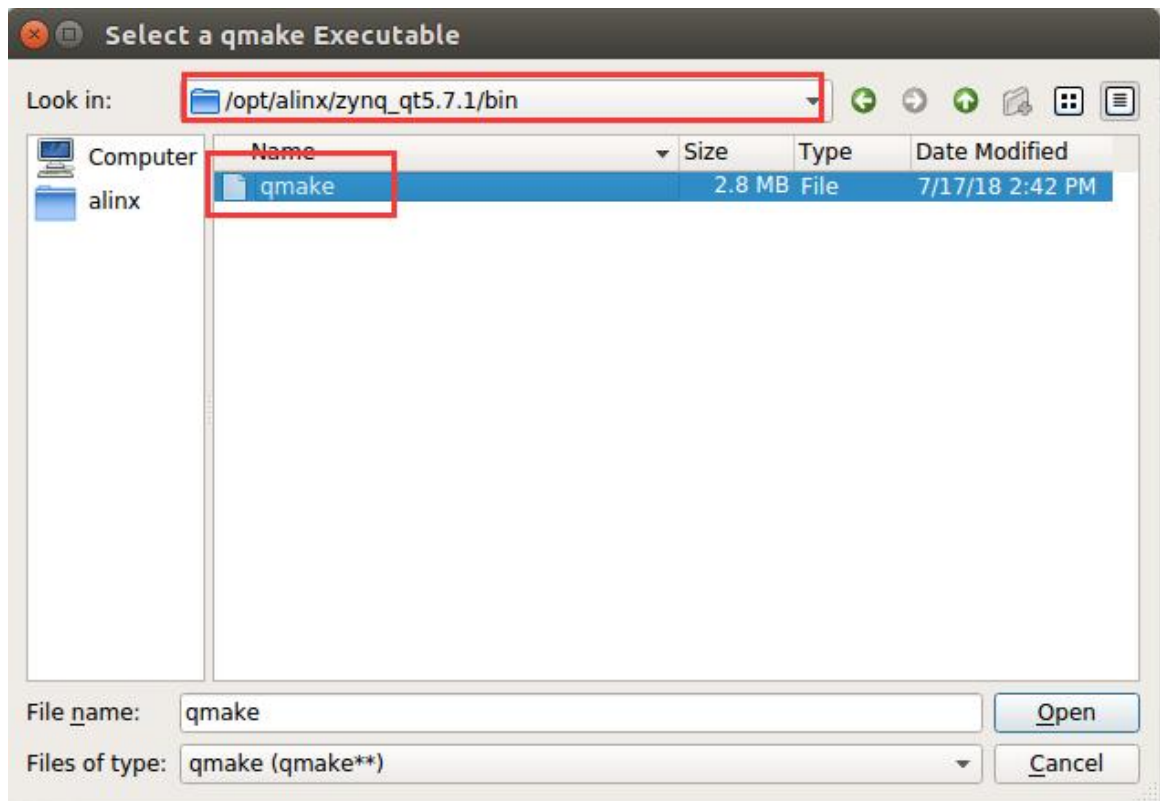
- 2) 点击 Tools -> Options



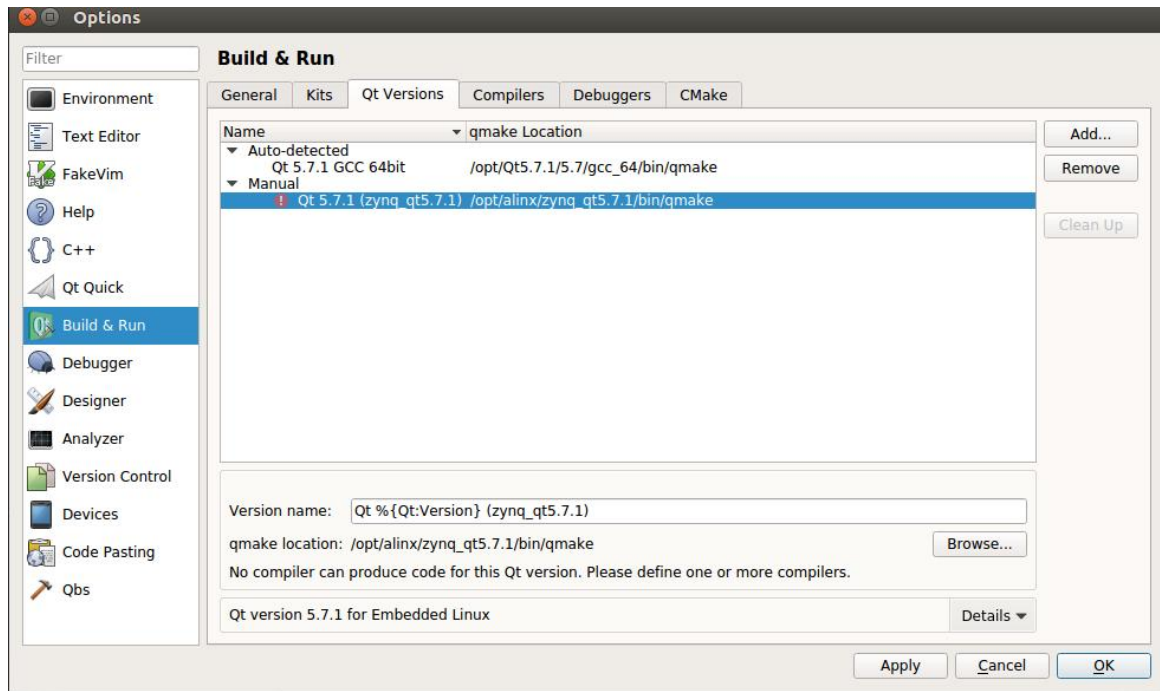
- 3) 在 Build & Run 选项的 Qt Versions 页点击 Add..



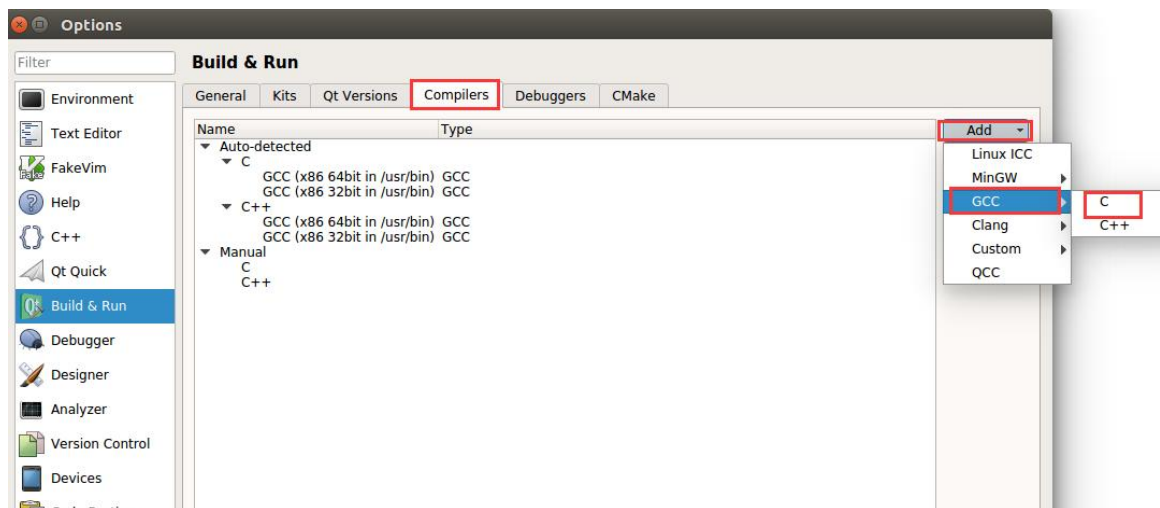
- 4) 选择前面编译生成的 qmake 文件



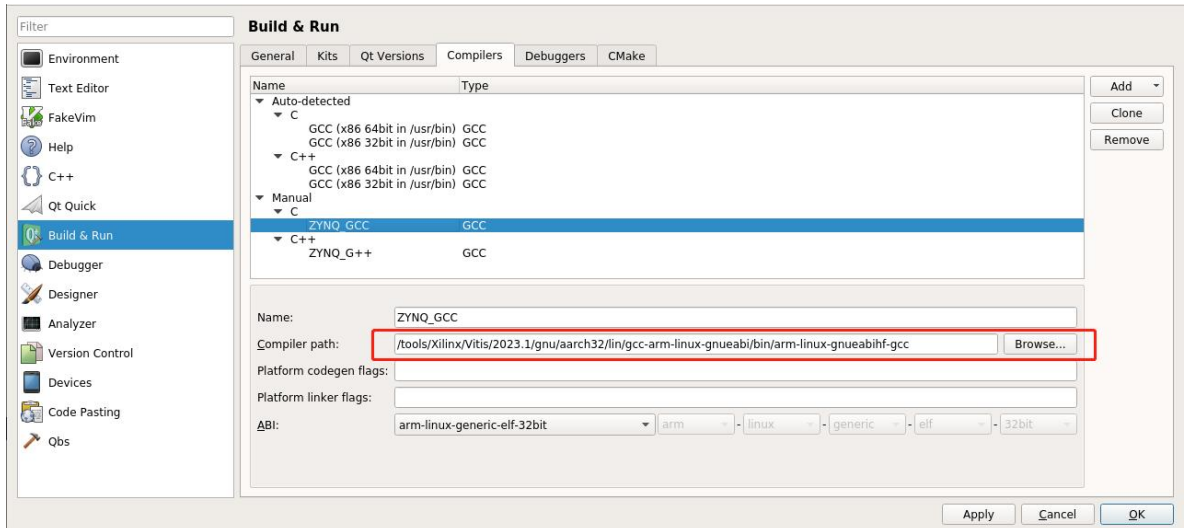
- 5) 添加 qmake 以后，保持默认 name，不修改



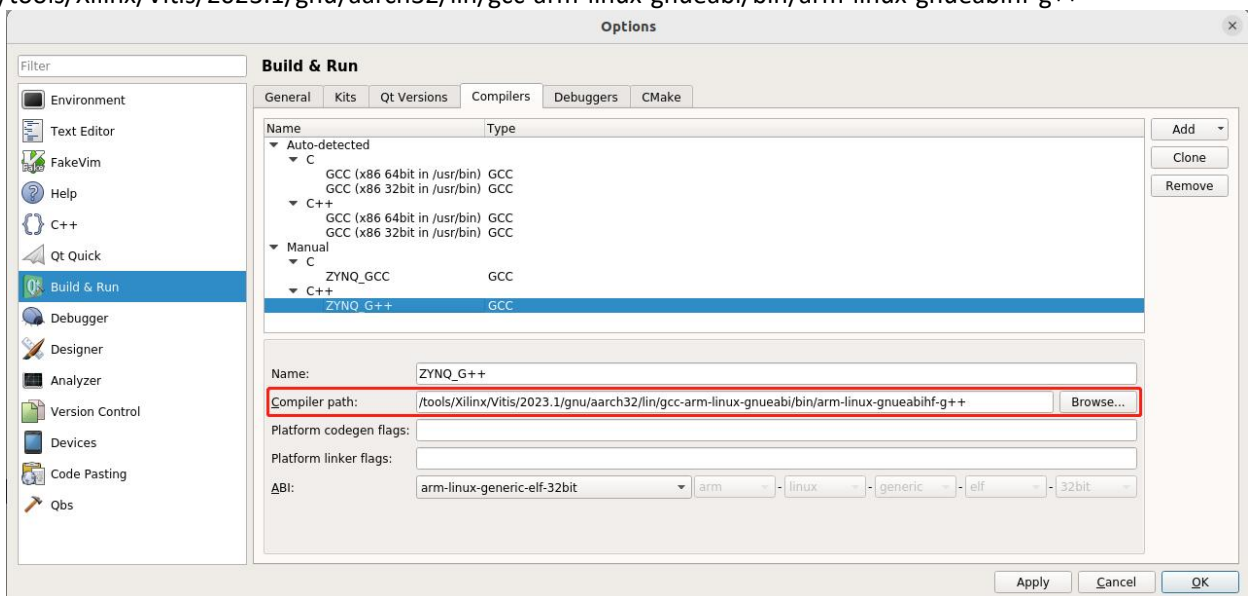
6) 在 Compilers 页面, 点击 Add., 选择 GCC -> C



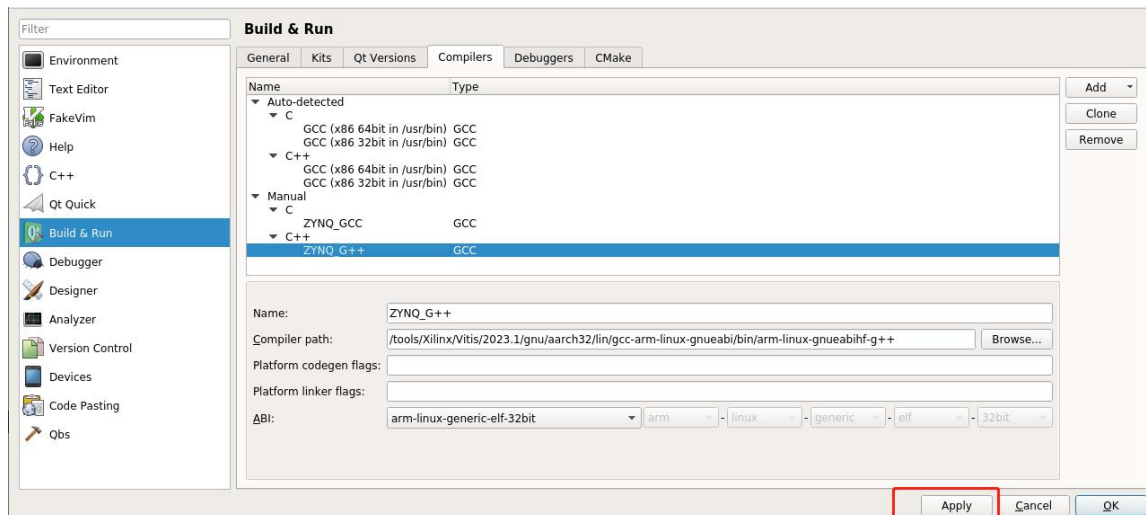
7) Name 修改为 ZYNQ_GCC, Compiler path 选择
/tools/Xilinx/Vitis/2023.1/gnu/aarch32/linux/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabi-hf-gcc



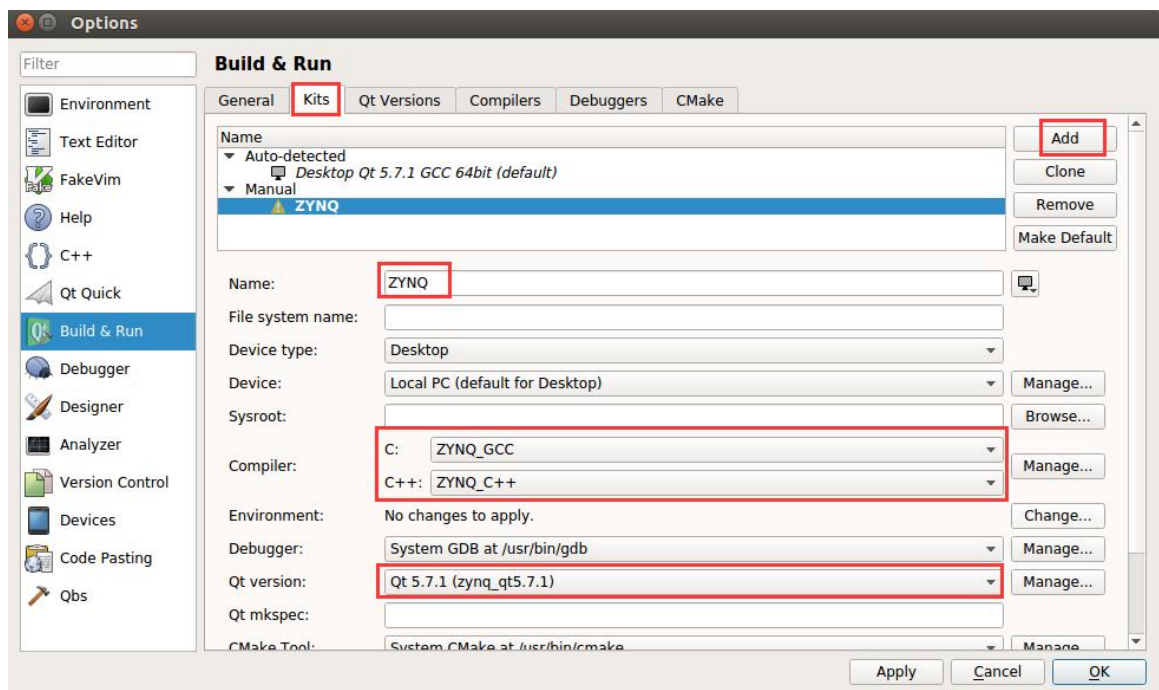
- 8) 在 Compilers 页面，点击 Add..，选择 GCC -> C++，Name 修改为 ZYNQ_C++，路径选择 `/tools/Xilinx/Vitis/2023.1/gnu/aarch32/linux/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabi-hf-g++`



- 9) 再点击 “Apply”，更新信息



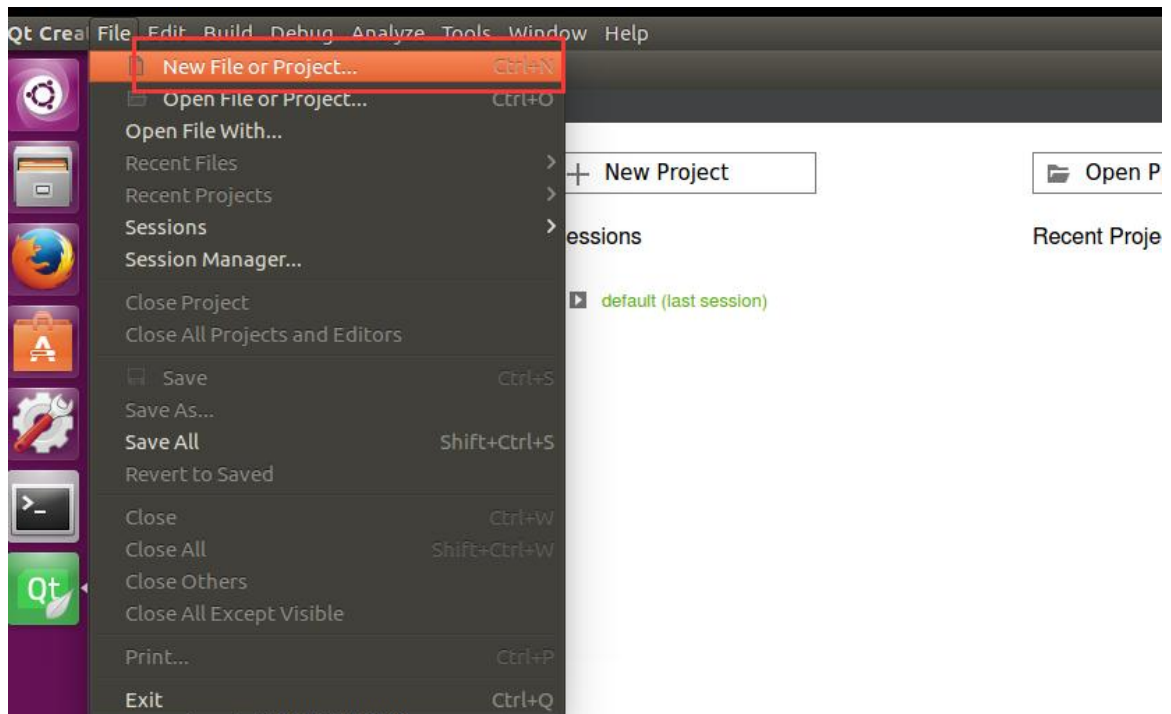
在 Kits 页，点击 Add 添加一个新的 Kit，Name 修改为 ZYNQ，Compiler C：选择 ZYNQ_GCC，Compiler C++ 选择 ZYNQ_C++，Qt version 选择 Qt 5.7.1(zynq_qt5.7.1)



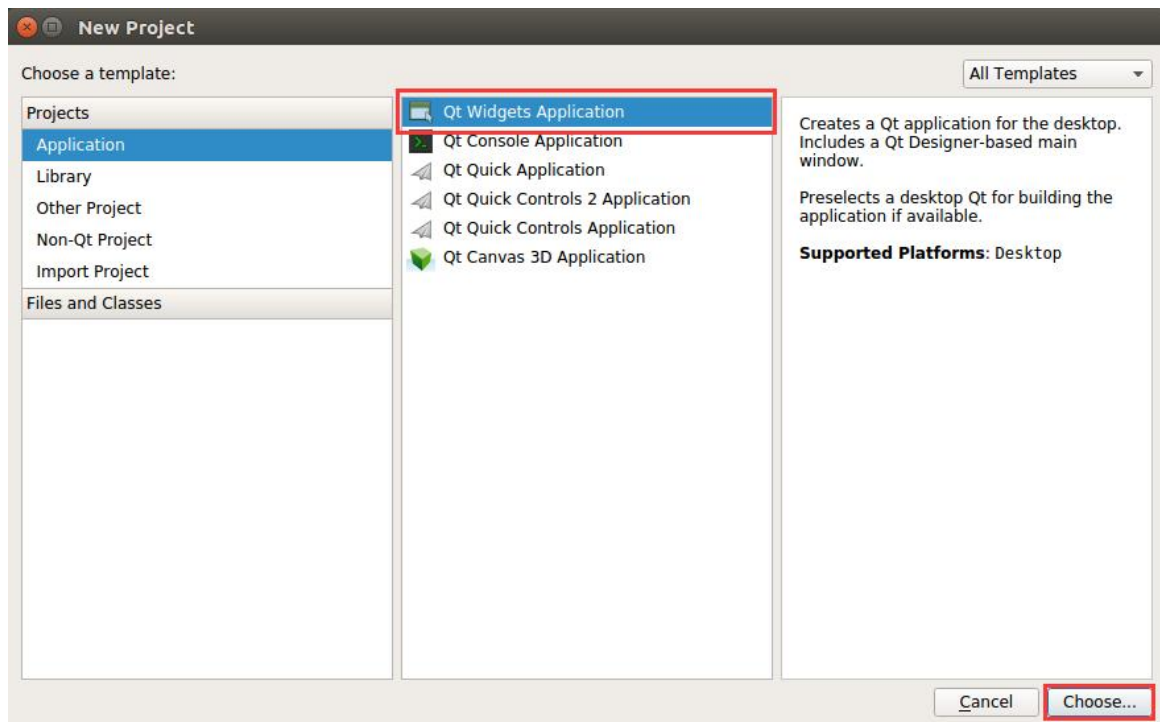
10) 点击 OK 配置 Kits 完成

11.4 创建 QT 测试工程

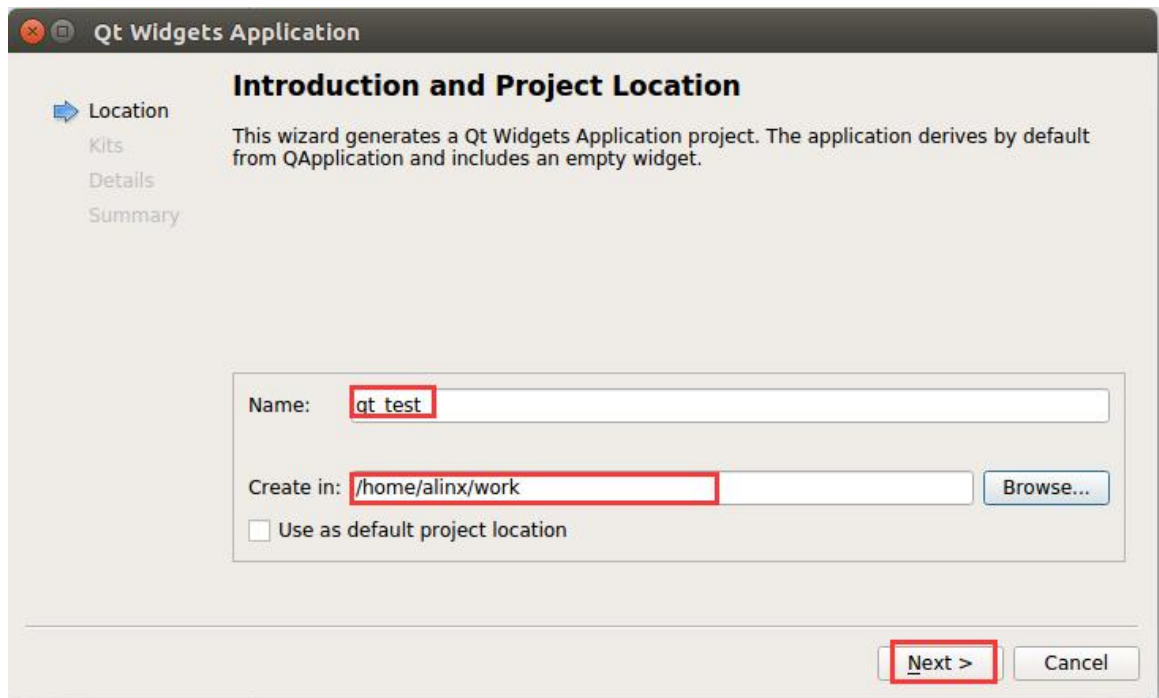
1) 点击 File -> New File or Project



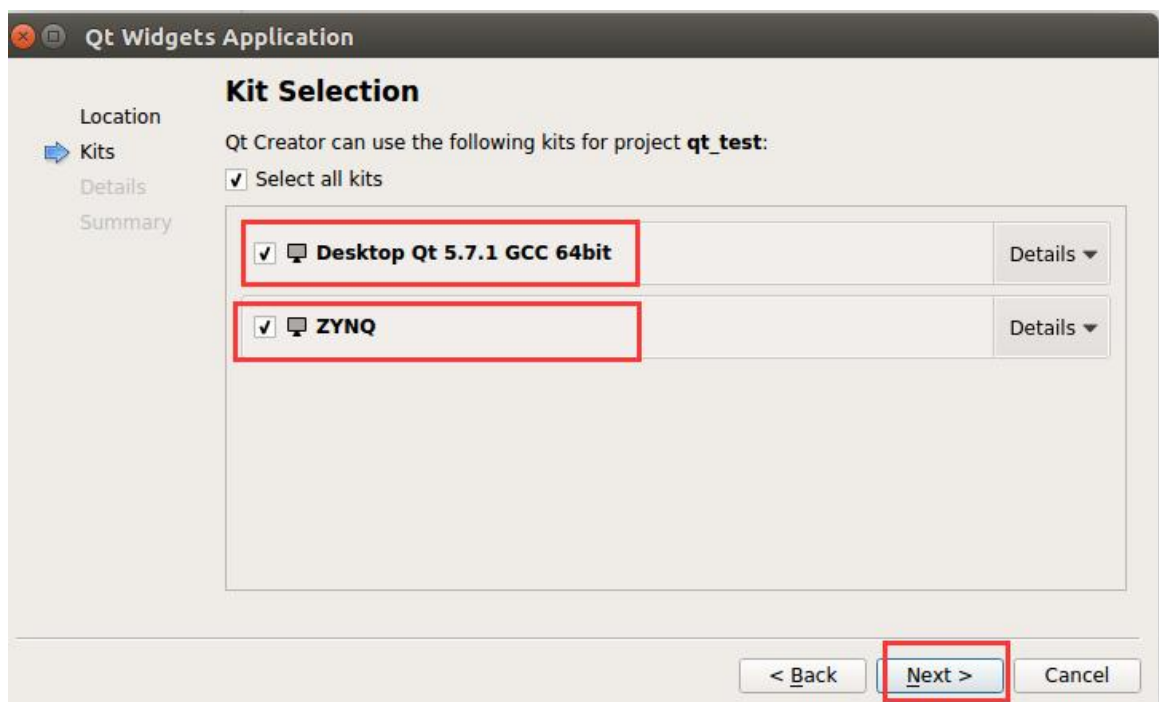
- 2) 工程模板选择 “Application” “Qt Widgets Application” ，然后点击 “Choose...”



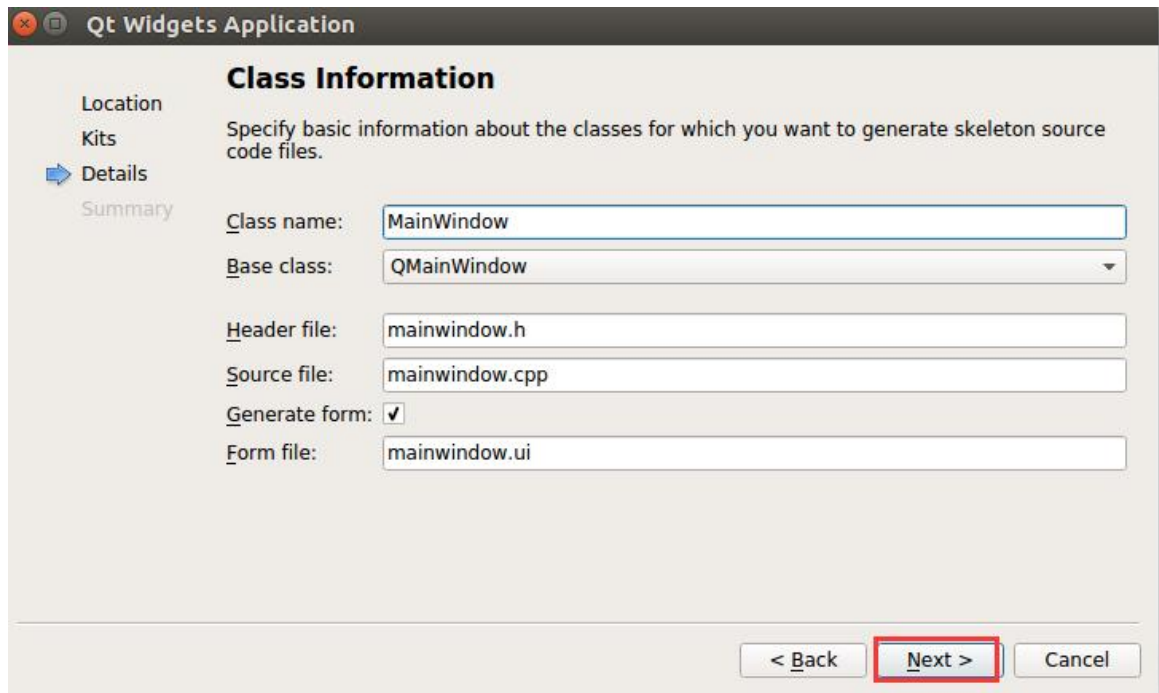
- 3) Name 修改为 qt_test, 路径本实验选择/home/alinx/work



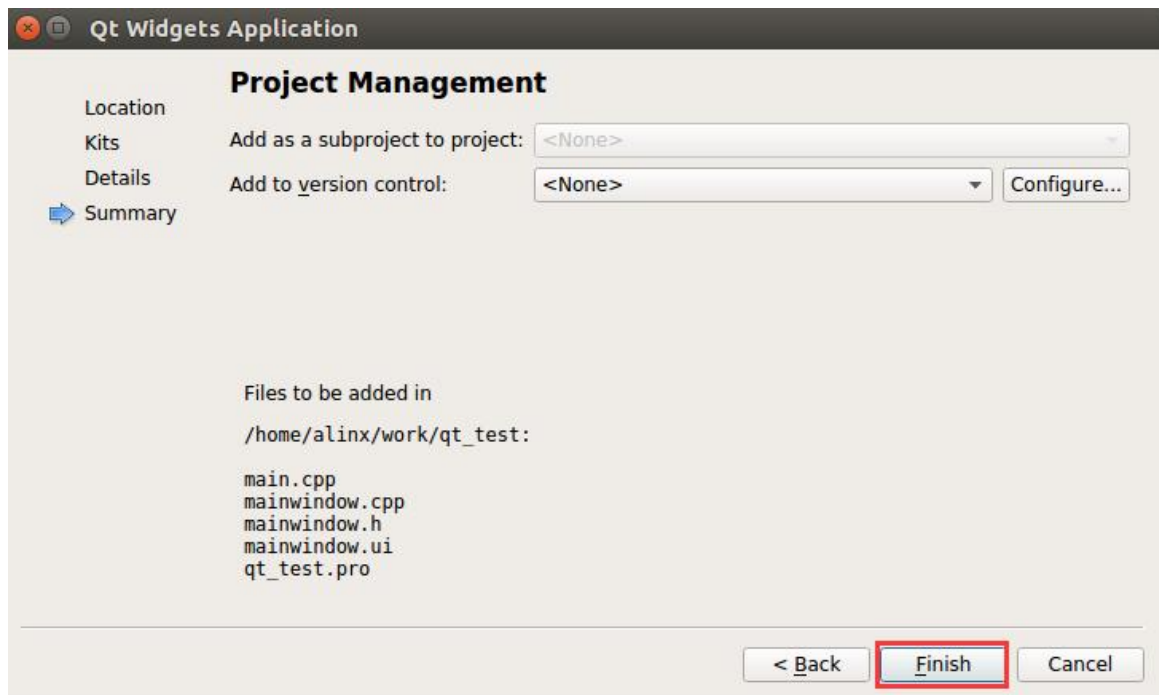
4) kit 选择, 这里选择 ZYNQ 版本



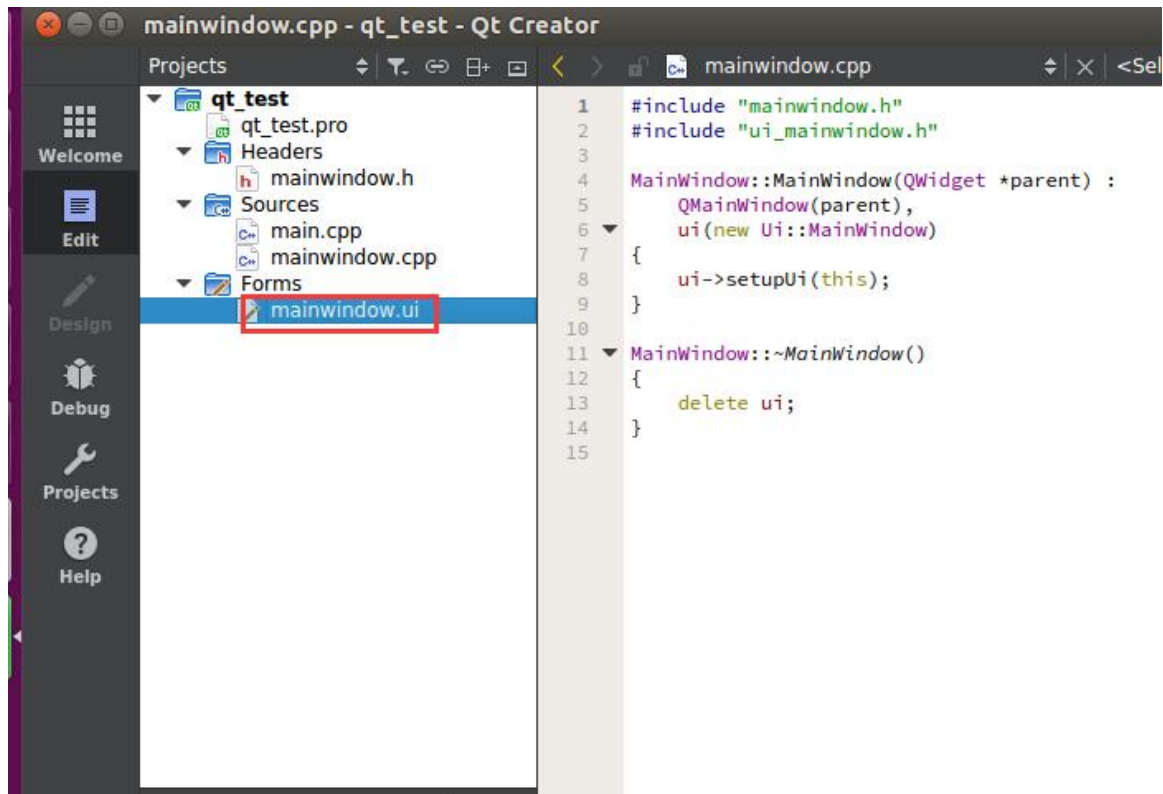
5) 选择 Next 继续



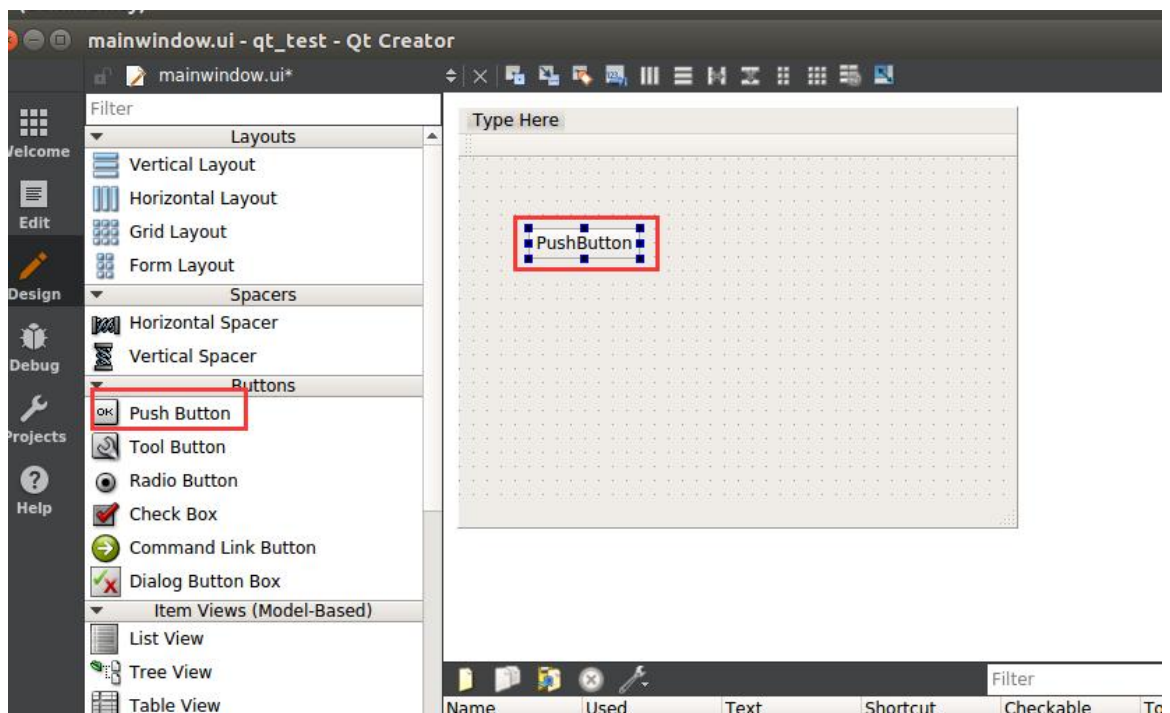
6) 选择 Finish 结束工程创建向导



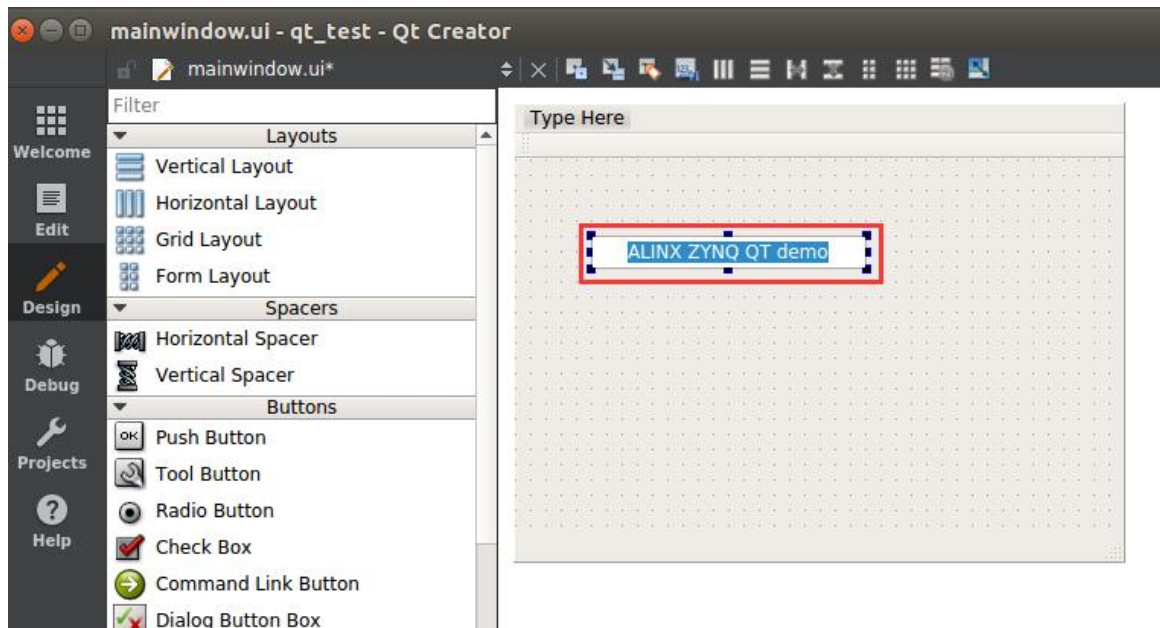
7) 双击 mainwindow.ui 文件



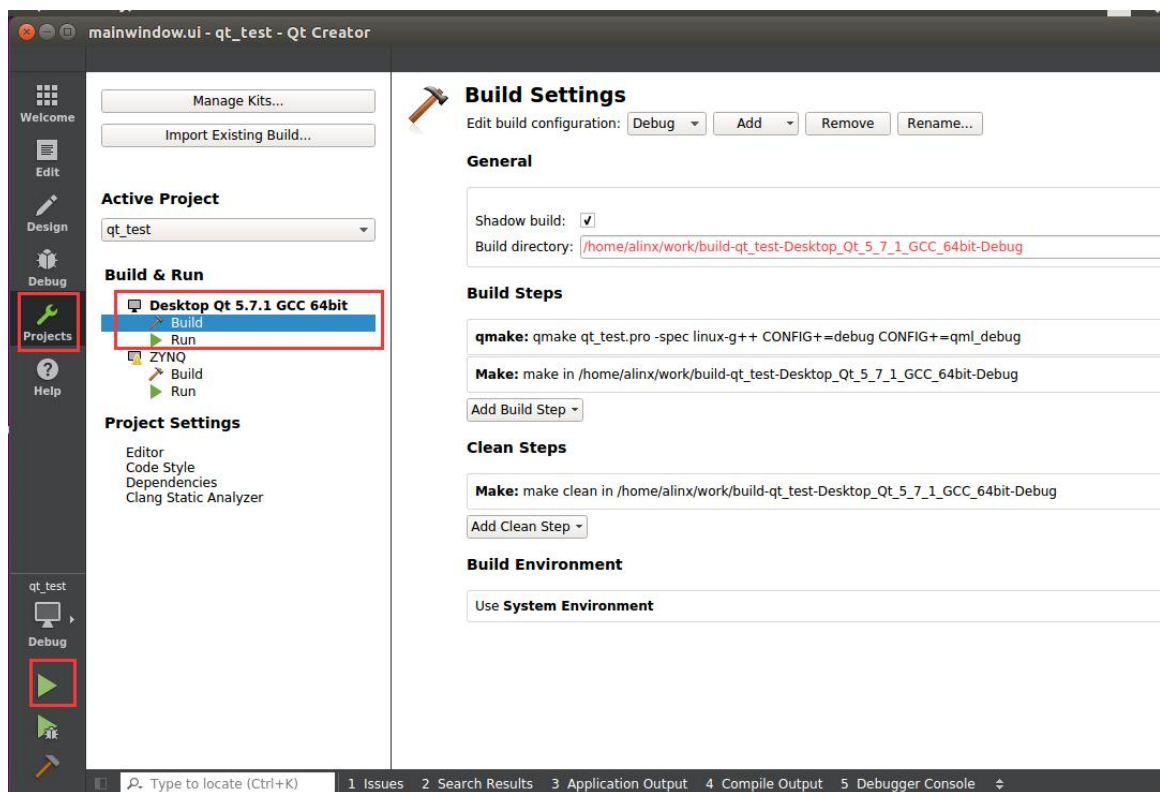
8) 拖拽一个 Push Button 到主界面中



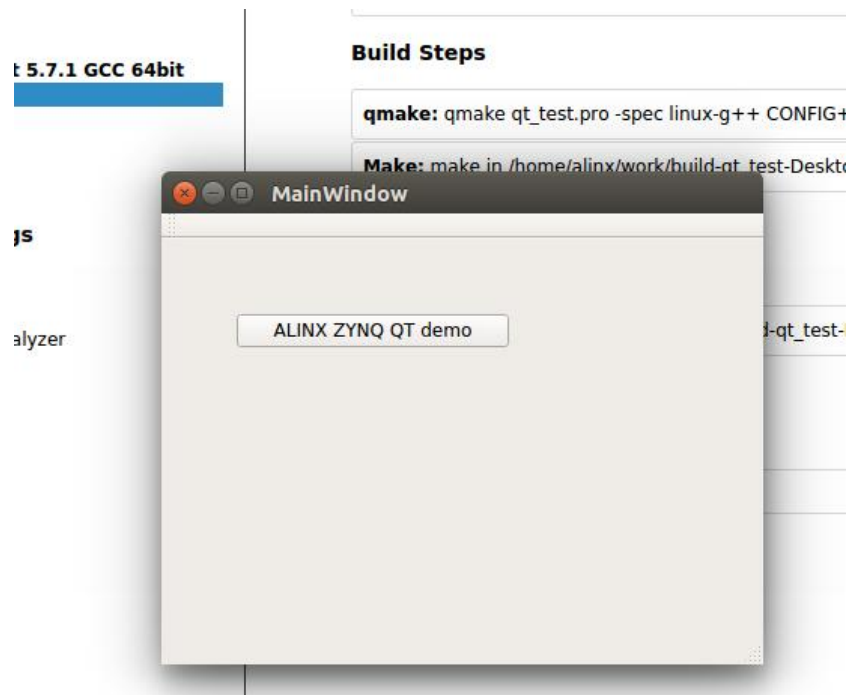
9) 双击文字部分，可以修改按钮文字



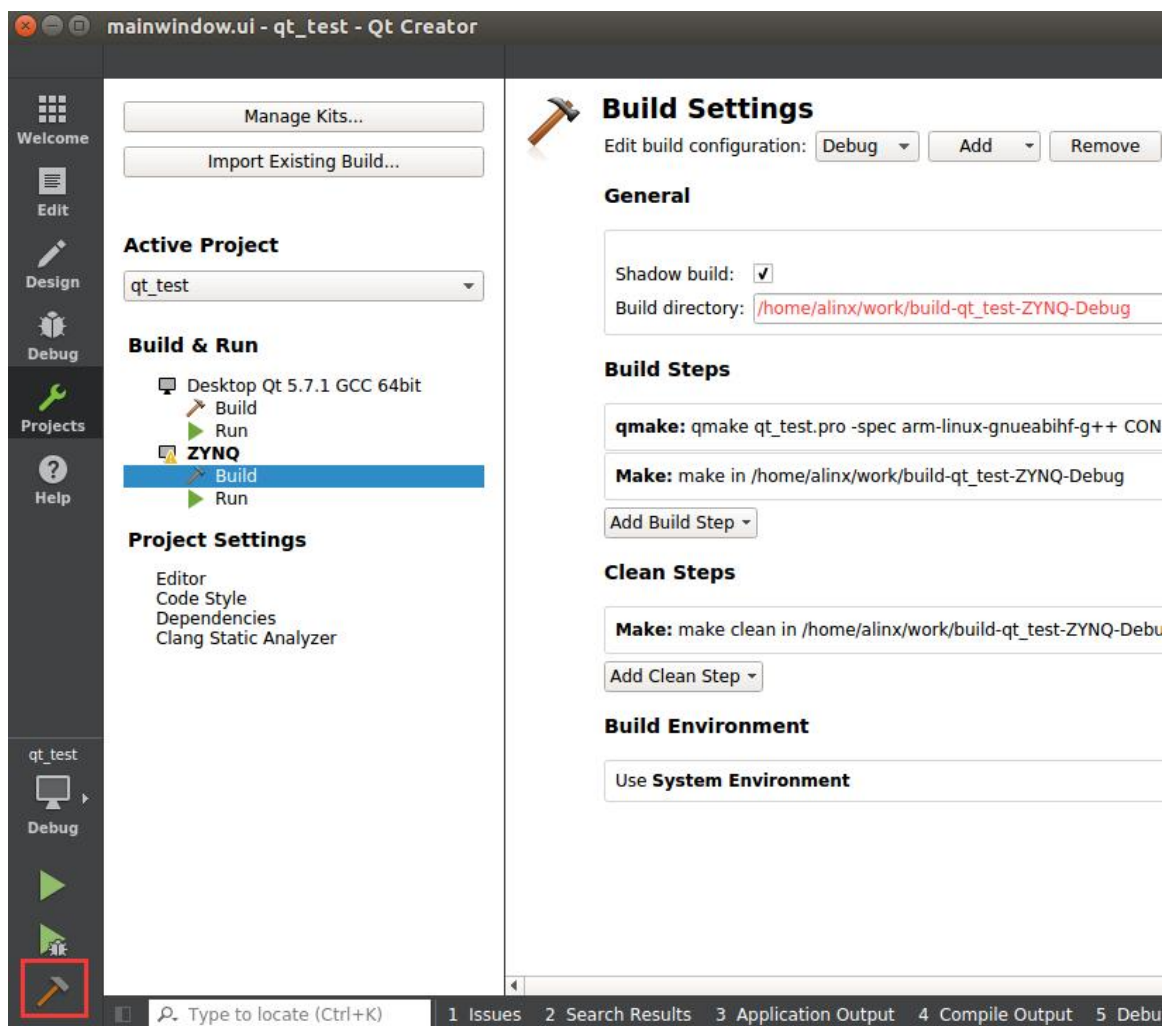
10) 在 Projects 选择, 选择 “Desktop Qt 5.7.1 GCC 64bit” ,然后点击绿色三角运行 Host 版本



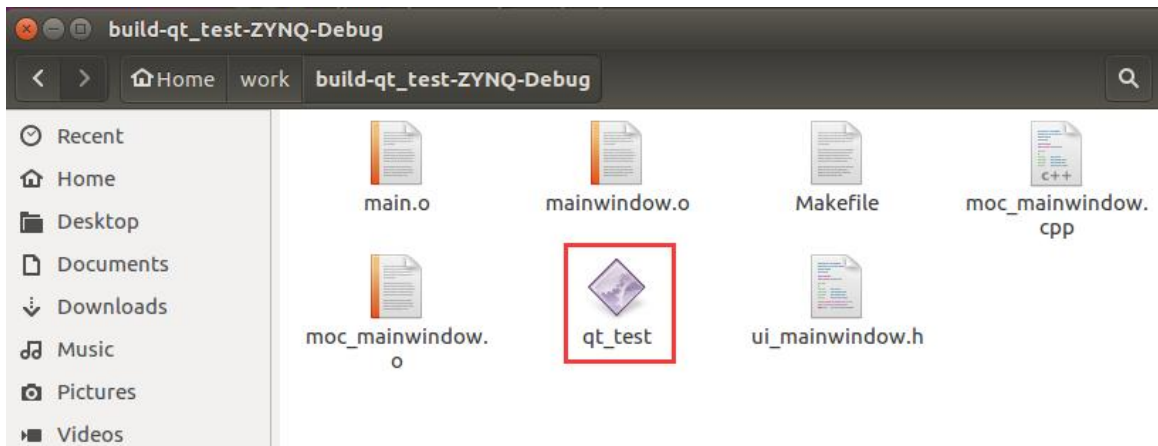
11) 运行效果



12) 选择 ZYNQ Kit, 点击锤子图标, 只编译程序, 不运行



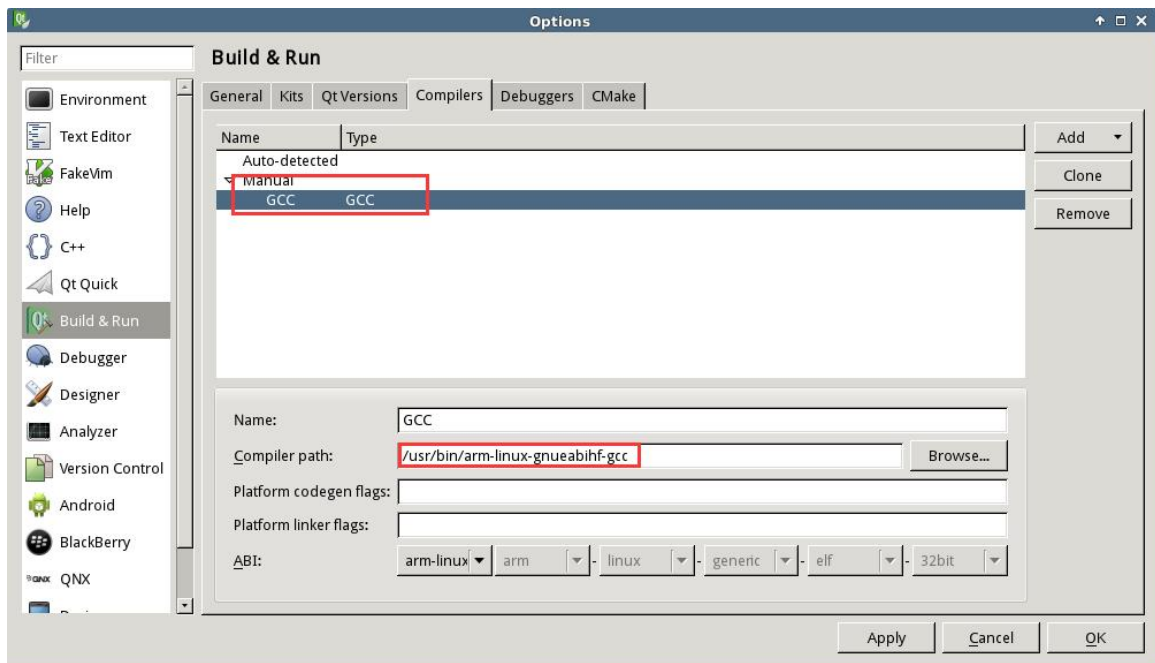
- 13) 在 build-qt_test-ZYNQ-Debug 目录可以看到生成了一个 qt_test 的文件，这个文件要在 ZYNQ 上运行。



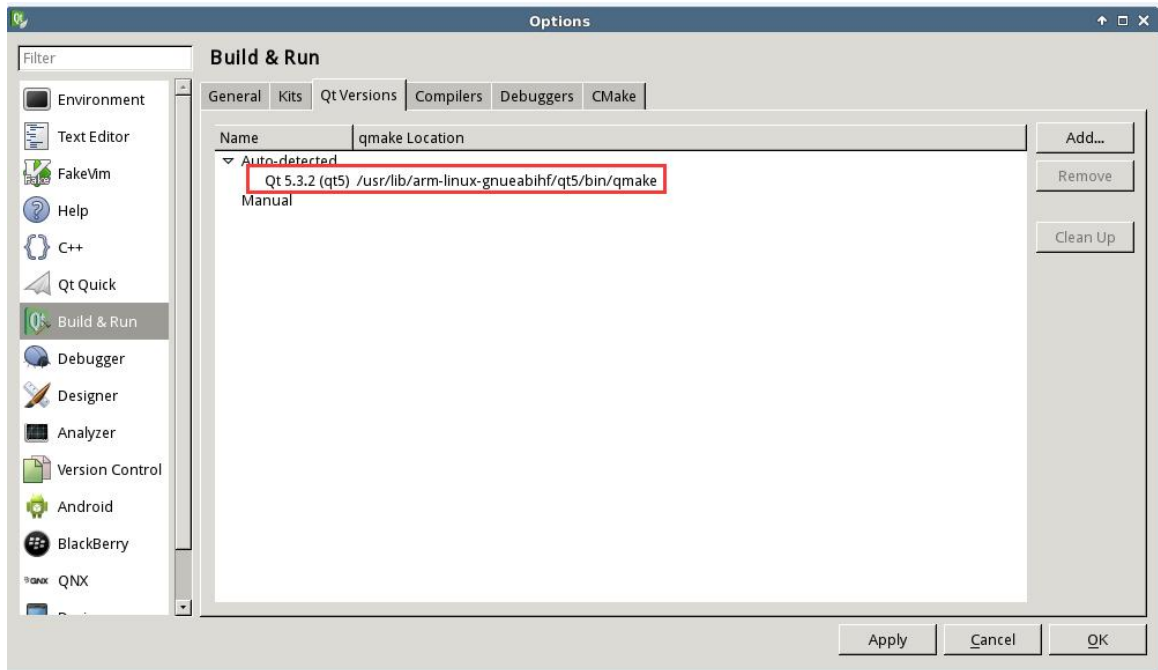
11.5 常见问题

11.5.1 如何使用开发板 Debian 系统自带的 QT Creator

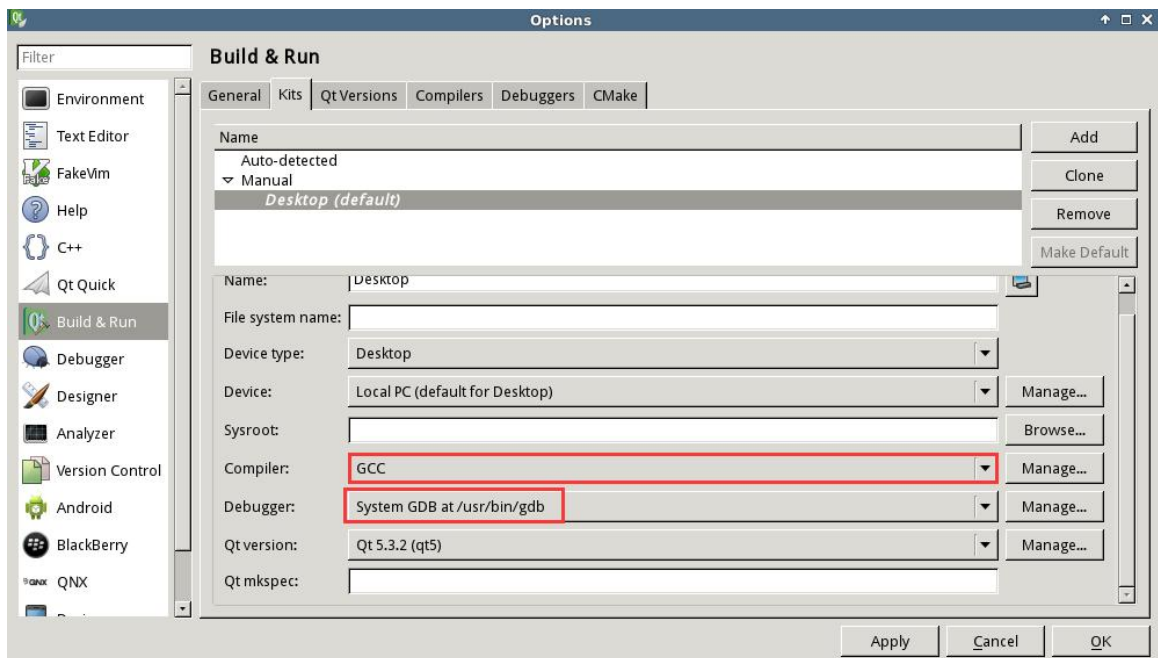
1) 配置编译器



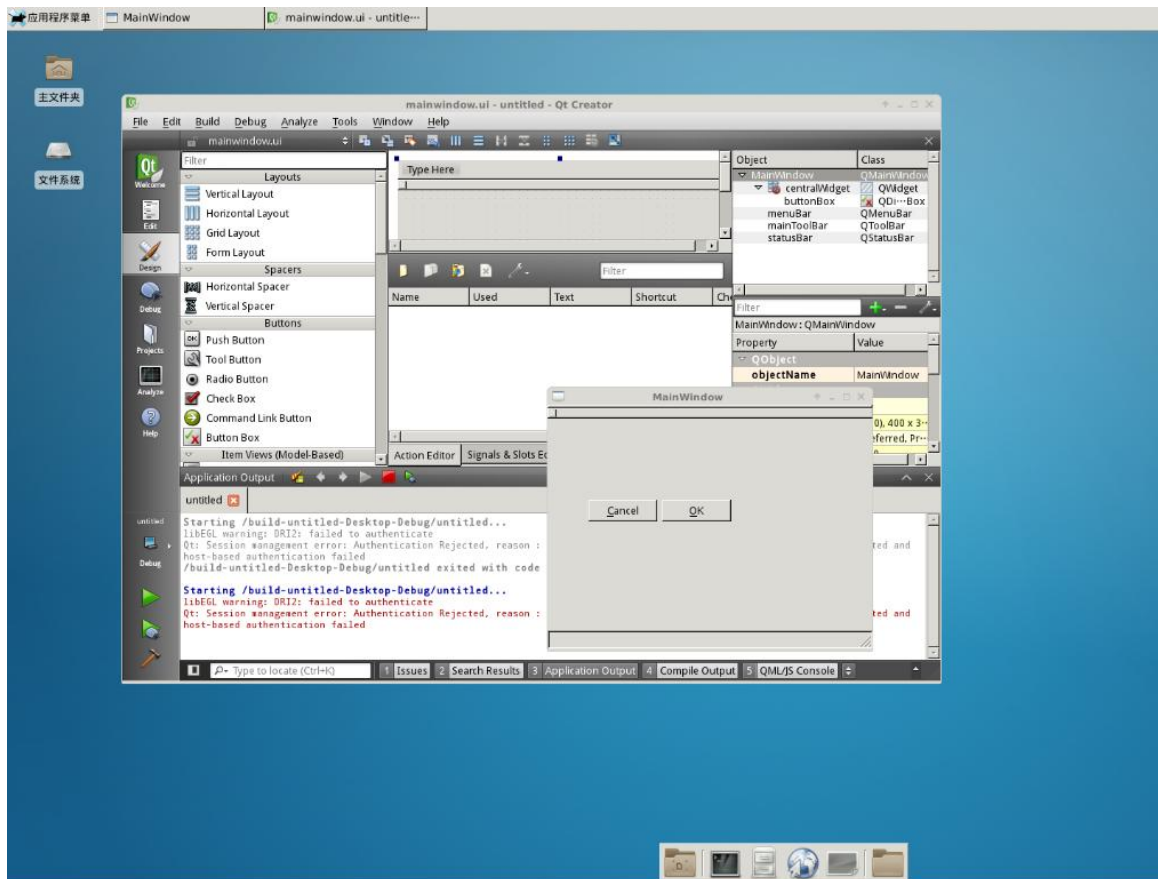
2) 查看 qmake 路径是否正确



3) 查看 Kits 设置是否和下图一致



4) 配置完成后就可以建立一个工程测试一下



11.5.2 系统运行打印 I2C 错误信息

可以忽略这个错误信息

```
+++++++reay set format 800x600+++++++
ov5640 0-003c: reg_write: i2c write error, reg: 3808
+++++++reay set format 800x600+++++++
ov5640 1-003c: reg_write: i2c write error, reg: 3808
ov5640 0-003c: reg_write: i2c write error, reg: 3809
ov5640 1-003c: reg_write: i2c write error, reg: 3809
ov5640 0-003c: reg_write: i2c write error, reg: 380a
ov5640 1-003c: reg_write: i2c write error, reg: 380a
ov5640 0-003c: reg_write: i2c write error, reg: 380b
ov5640 1-003c: reg_write: i2c write error, reg: 380b

PetaLinux 2017.4 an5642_peta /dev/ttyPS0

an5642_peta login:
PetaLinux 2017.4 an5642_peta /dev/ttyPS0
```

第十二章 OpenCV 应用（USB 摄像头显示）

例程相关文件夹： opencv_usb_camera

本章讲解如何编译 OpenCV 库，包括 Ubuntu 主机版和 ZYNQ 版。同样，芯驿电子为大家准备好了源代码包和编译脚本，从名称可以看出，OpenCV 版本为 3.1.0。本实验还是在前面 HDMI 实验的 Petalinux 工程基础上进行修改的，需要先掌握前面的实验内容。



12.1 配置 USB 摄像头驱动

- 1) 运行下面的命令配置内核

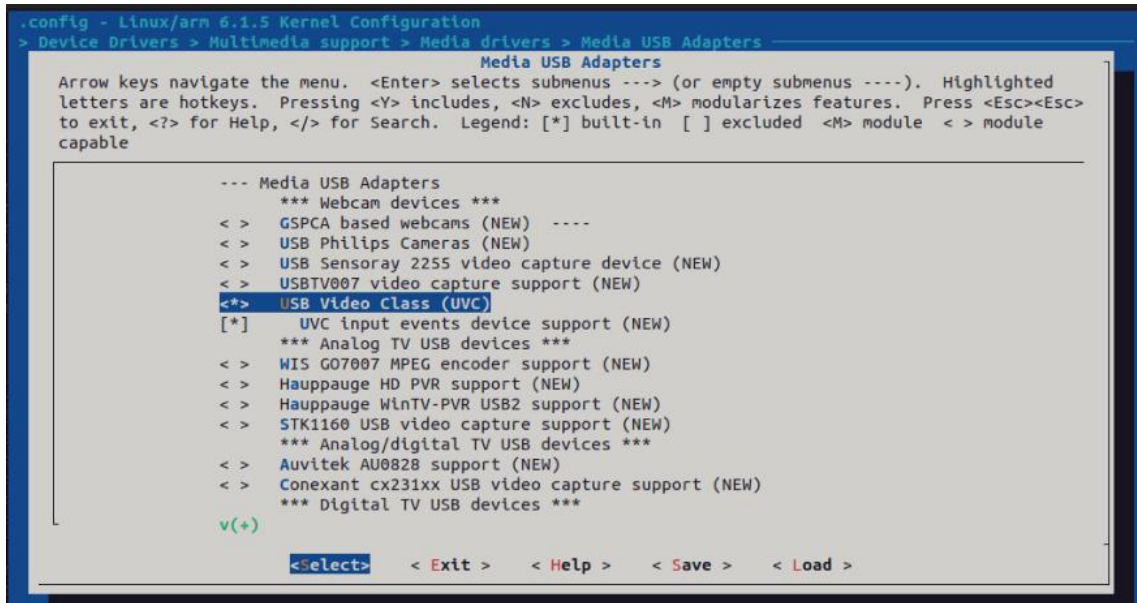
```
petalinux-config -c kernel
```

```
alinx@ubuntu: ~/Downloads/peta_prj/ax_peta
*** Execute 'make' to start the build or try 'make help'.

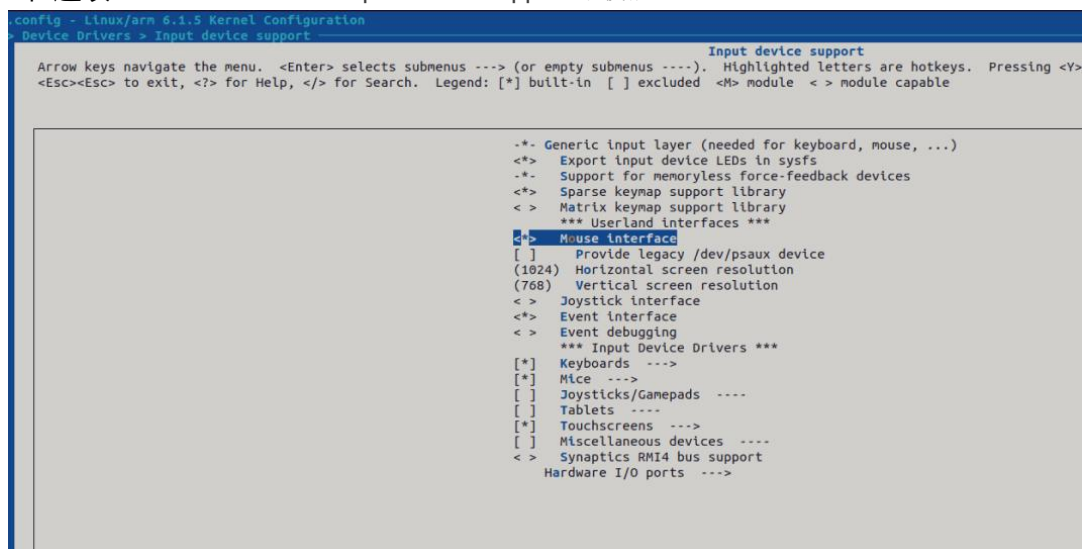
[INFO] sourcing bitbake
[INFO] generating plnxtool conf
[INFO] generating meta-plnx-generated layer
~/Downloads/peta_prj/ax_peta/build/misc/plnx-generated ~/Downloads/peta_prj/ax_peta
~/Downloads/peta_prj/ax_peta
[INFO] generating machine configuration
[INFO] generating bbappends for project . This may take time !
~/Downloads/peta_prj/ax_peta/build/misc/plnx-generated ~/Downloads/peta_prj/ax_peta
~/Downloads/peta_prj/ax_peta
[INFO] generating u-boot configuration files

[INFO] generating kernel configuration files
[INFO] generating kconfig for Rootfs
Generate rootfs kconfig
[INFO] oldconfig rootfs
[INFO] generating petalinux-user-image.bb
[INFO] successfully configured project
webtalk failed:Petalinux statistics:extra lines detected:notsent_nofile!
webtalk failed:Failed to get Petalinux usage statistics!
alinx@ubuntu:~/Downloads/peta_prj/ax_peta$ petalinux-config -c kernel
```

- 2) 在选项 Device Drivers ---> Multimedia support ---> Media drivers---> Media USB Adapters ---> 中使能 USB Video Class (UVC)



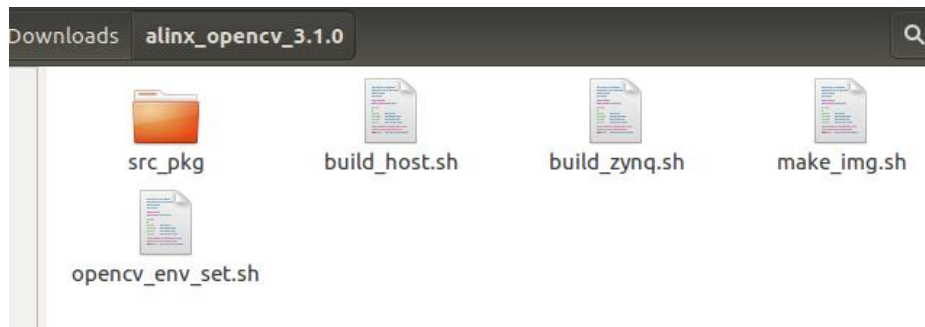
3) 在选项 Device Drivers --->Input device support 中使能 Mouse interface



4) 保存并退出

12.2 host 版 OpenCV 库编译

- 1) 解压源码包
- 2) "src_pkg" 里就是源码, "build_host.sh" 是编译 Ubuntu 版本 OpenCV 的脚本, "build_zynq.sh" 是编译 ZYNQ 版本的脚本, "make_img.sh" 是制作 ZYNQ 版本库镜像的脚本



3) 安装 cmake

```
sudo apt-get install cmake
```

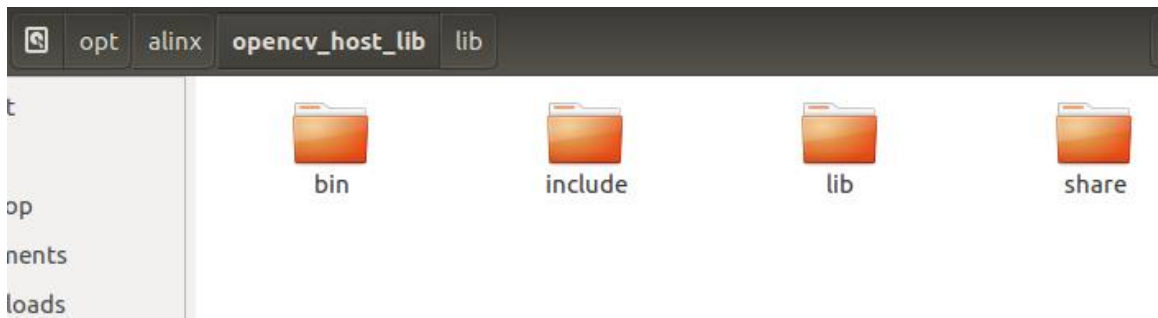
4) 在终端运行 “build_host.sh” ， 会提示输入账号密码，输入密码，继续运行

```
alinx@alinx:~/Downloads/zynq/alinx_opencv_3.4.20$ ./build_host.sh
/home/alinx/Downloads/zynq/alinx_opencv_3.4.20
[sudo] password for alinx: 
```

5) 编译完成

```
alinx@ubuntu: ~/Downloads/alinx_opencv_3.1.0
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/haarcascades/haarcascade_
frontalcatface.xml
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/haarcascades/haarcascade_
lowerbody.xml
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/haarcascades/haarcascade_
frontalcatface_extended.xml
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/lbpcascades/lbpcascade_si
lverware.xml
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/lbpcascades/lbpcascade_pr
ofileface.xml
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/lbpcascades/lbpcascade_fr
ontalface.xml
-- Installing: /opt/alinx/opencv_host_lib/share/OpenCV/lbpcascades/lbpcascade_fr
ontalcatface.xml
-- Installing: /opt/alinx/opencv_host_lib/bin/opencv_traincascade
-- Set runtime path of "/opt/alinx/opencv_host_lib/bin/opencv_traincascade" to "
/opt/alinx/opencv_host_lib/lib"
-- Installing: /opt/alinx/opencv_host_lib/bin/opencv_createsamples
-- Set runtime path of "/opt/alinx/opencv_host_lib/bin/opencv_createsamples" to "
/opt/alinx/opencv_host_lib/lib"
-- Installing: /opt/alinx/opencv_host_lib/bin/opencv_annotation
-- Set runtime path of "/opt/alinx/opencv_host_lib/bin/opencv_annotation" to "/o
pt/alinx/opencv_host_lib/lib"
```

6) 在/opt/alinx/opencv_host_lib 目录下可以看到编译好的 OpenCV 库， “include” 目录为头文件， “lib” 为库。

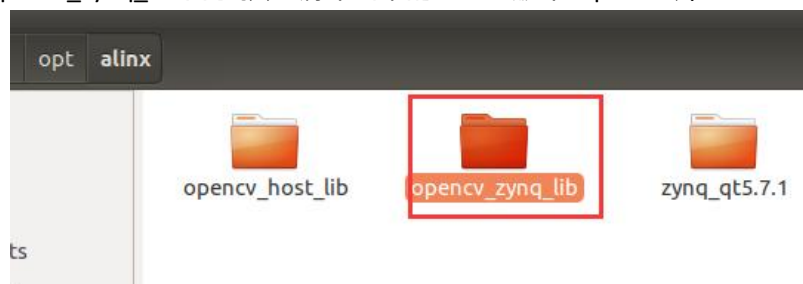


12.3 zynq 版 OpenCV 库编译

- 1) 和 host 版本编译类型，运行 “build_zynq.sh” 编译 ZYNQ 版本的 OpenCV 库，需要输入账号密码后就开始了漫长的编译等待时间

```
alinx@alinx:~/Downloads/zynq/alinx_opencv_3.4.20$ ./build_zynq.sh  
[sudo] password for alinx: 
```

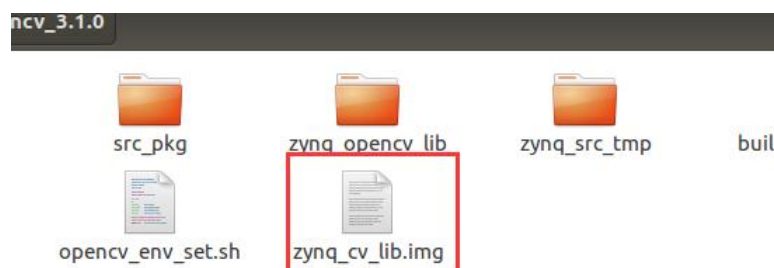
- 2) /opt/alinx/opencv_zynq_lib 目录就是编译出来的 ZYNQ 版本 OpenCV 库



- 3) 运行 make_img.sh 制作 OpenCV 库的镜像，需要输入账号密码

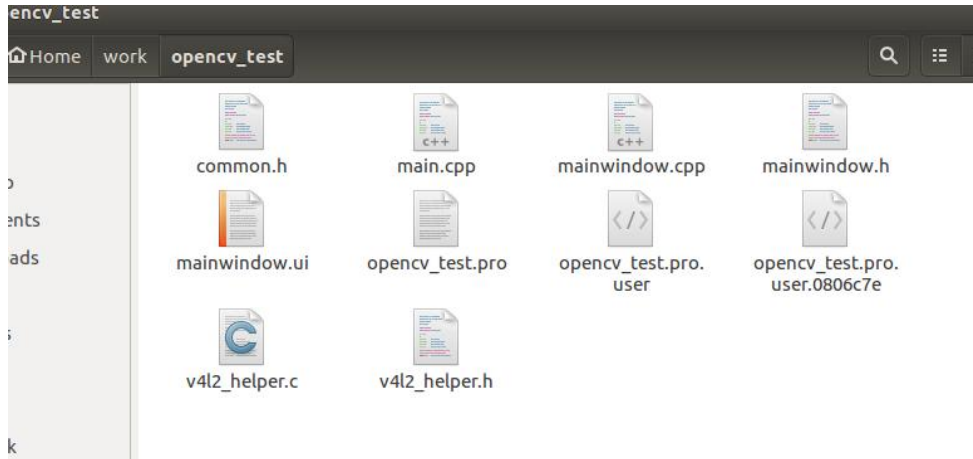
```
alinx@alinx:~/Downloads/zynq/alinx_opencv_3.4.20$ ./make_img.sh  
[sudo] password for alinx: 
```

- 4) zynq_cv_lib.img 文件就是制作好的 OpenCV 库镜像

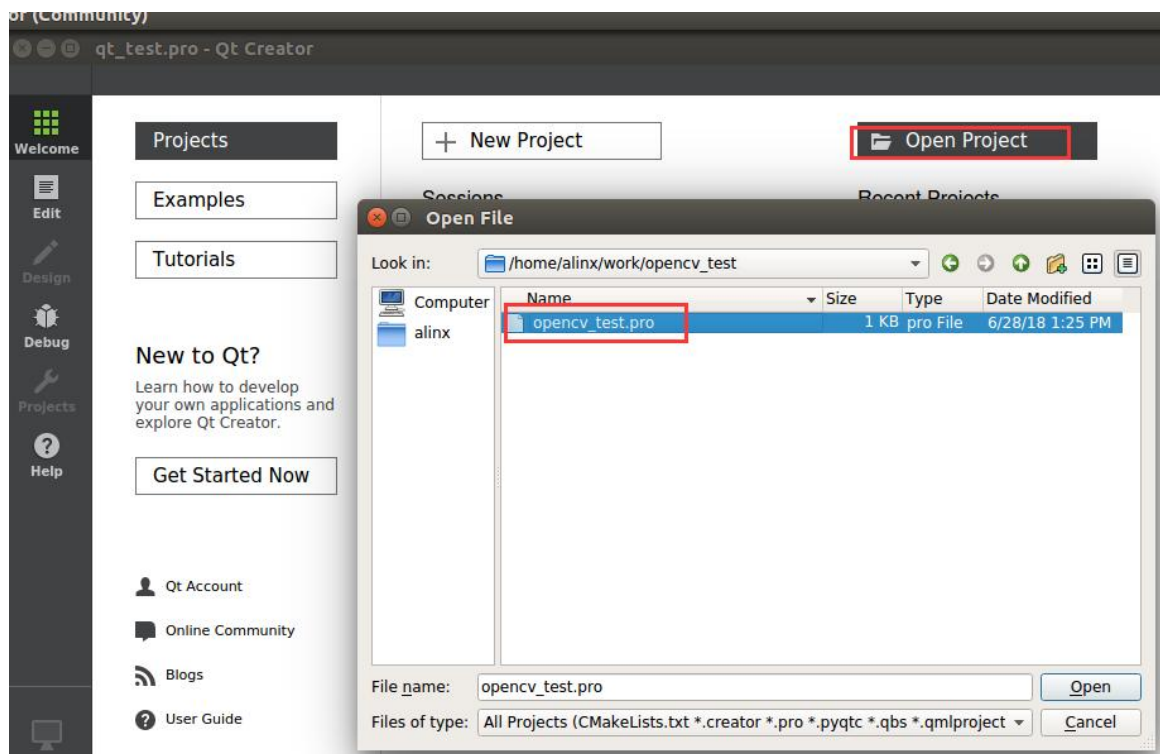


12.4 测试 host 版 OpenCV

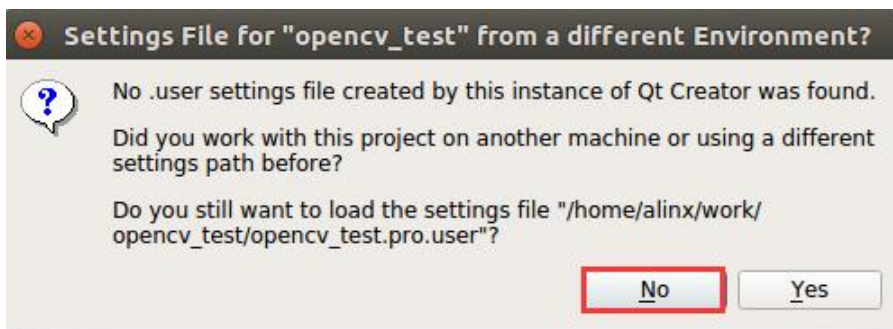
实验给出一个 QT 工程，工程中使用 OpenCV 进行 YUV 到 RGB 的颜色空间转换，其他更复杂的例子开发者可以自行开发，需要说明的是，在嵌入式上运行 OpenCV 和 Ubuntu 主机会有有效差异。



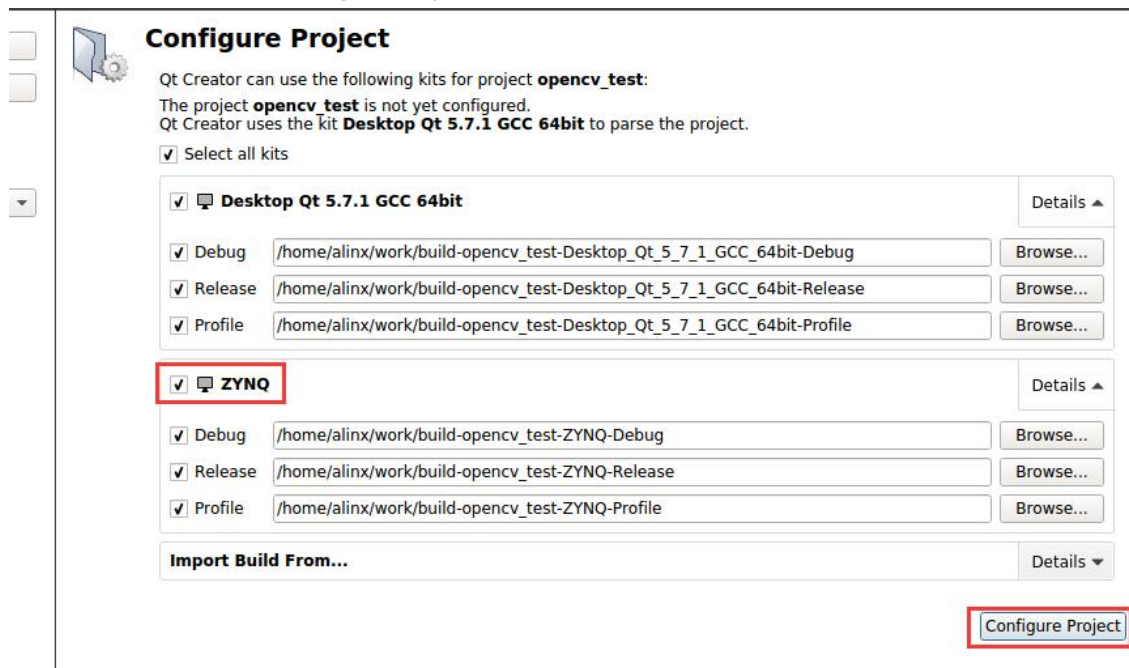
5) 运行 QT Creator，打开 opencv_test 工程



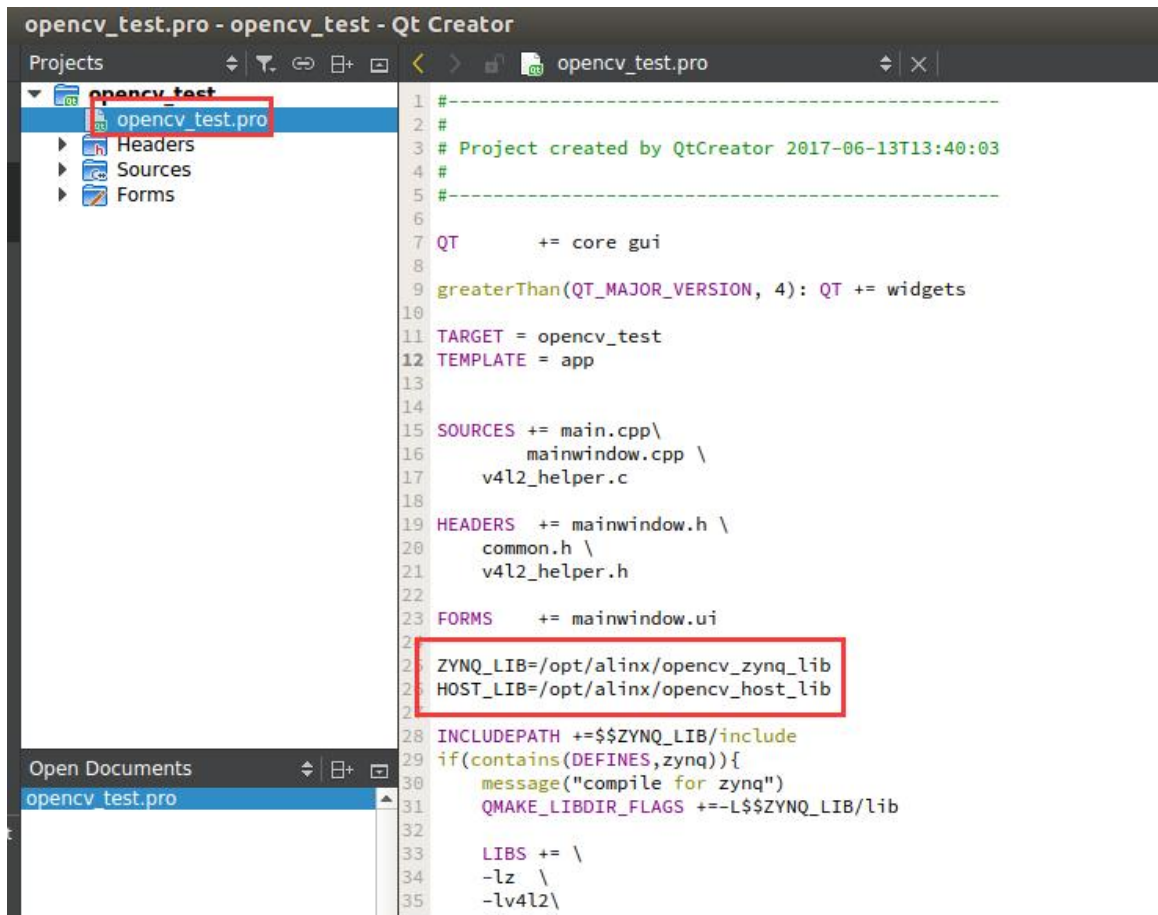
6) 如果出现下图对话框，选择 “No”



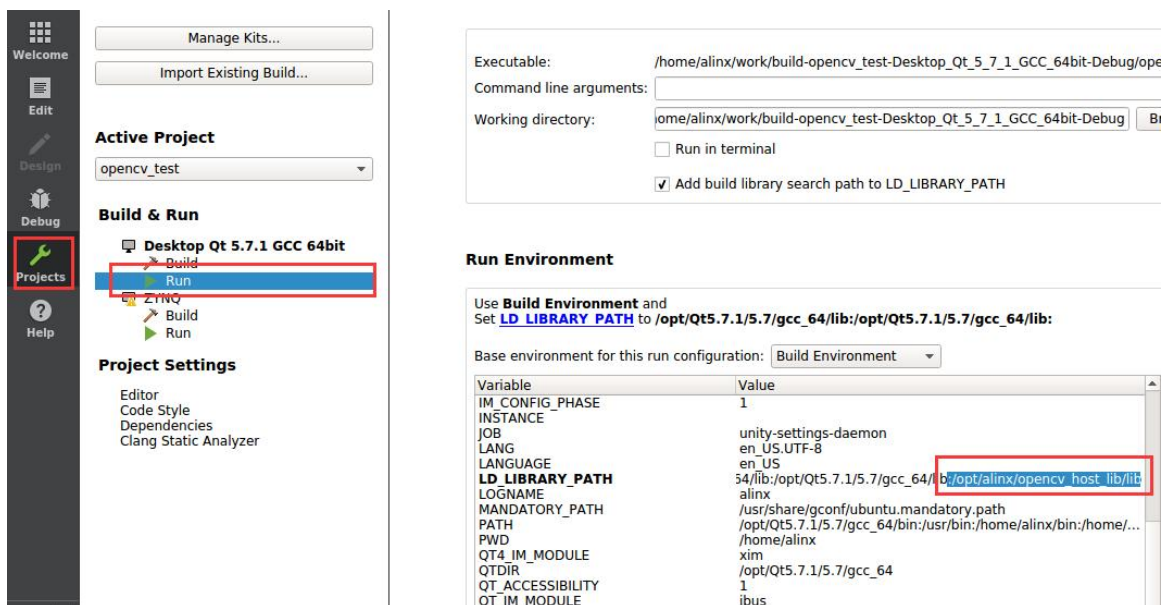
7) 选中 ZYNQ kit, 点击 "Configure Project"



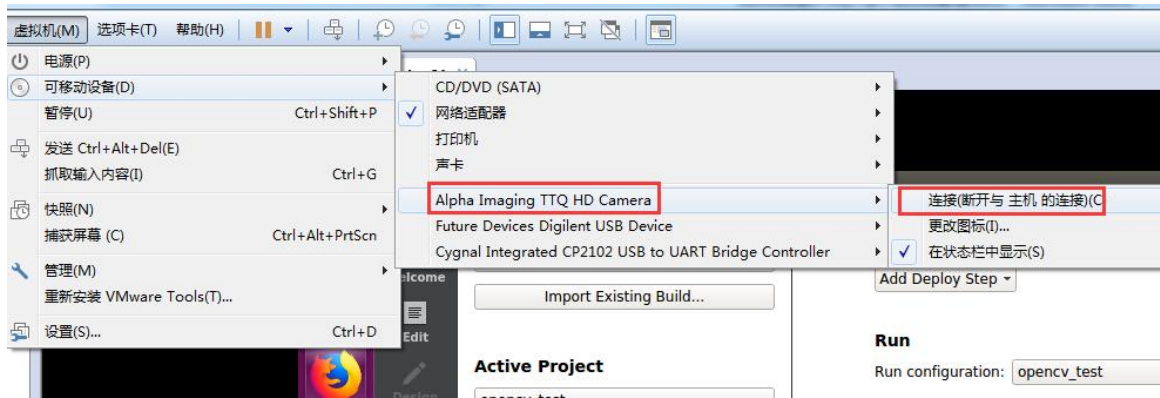
8) 双击 "opencv_test.pro" 文件可以看到设置了 ZYNQ 和 HOST 版本的 OpenCV 库的路径, 根据实际情况修改, 如果前面的库都是按默认配置编译这里不需要修改。



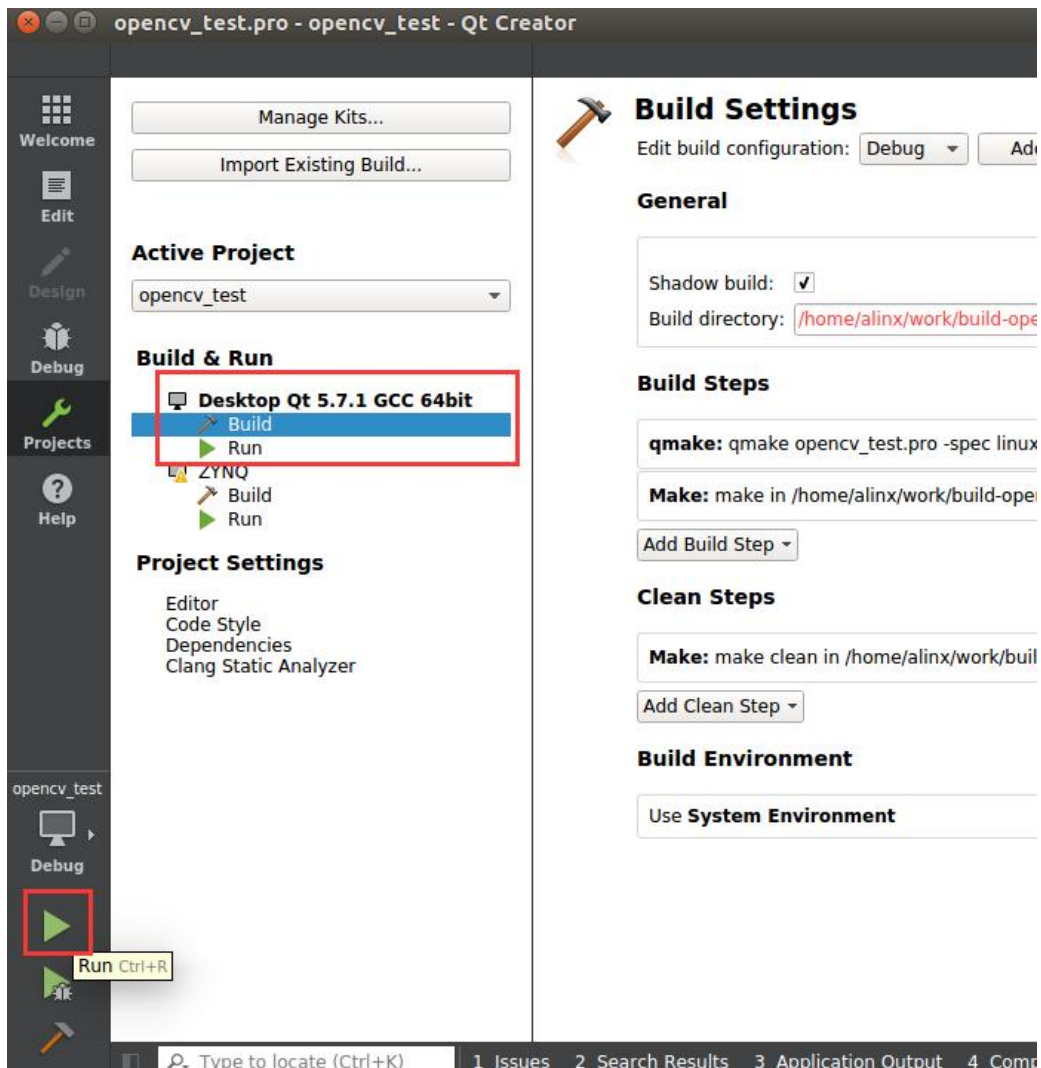
- 9) 设置 host 版运行环境变量，在 LD_LIBRARY_PATH 选项后面添加
 “:/opt/alinx/opencv_host_lib/lib”



- 10) 连接 USB 摄像头到 Ubuntu 主机中



11) 运行 host 版本测试程序

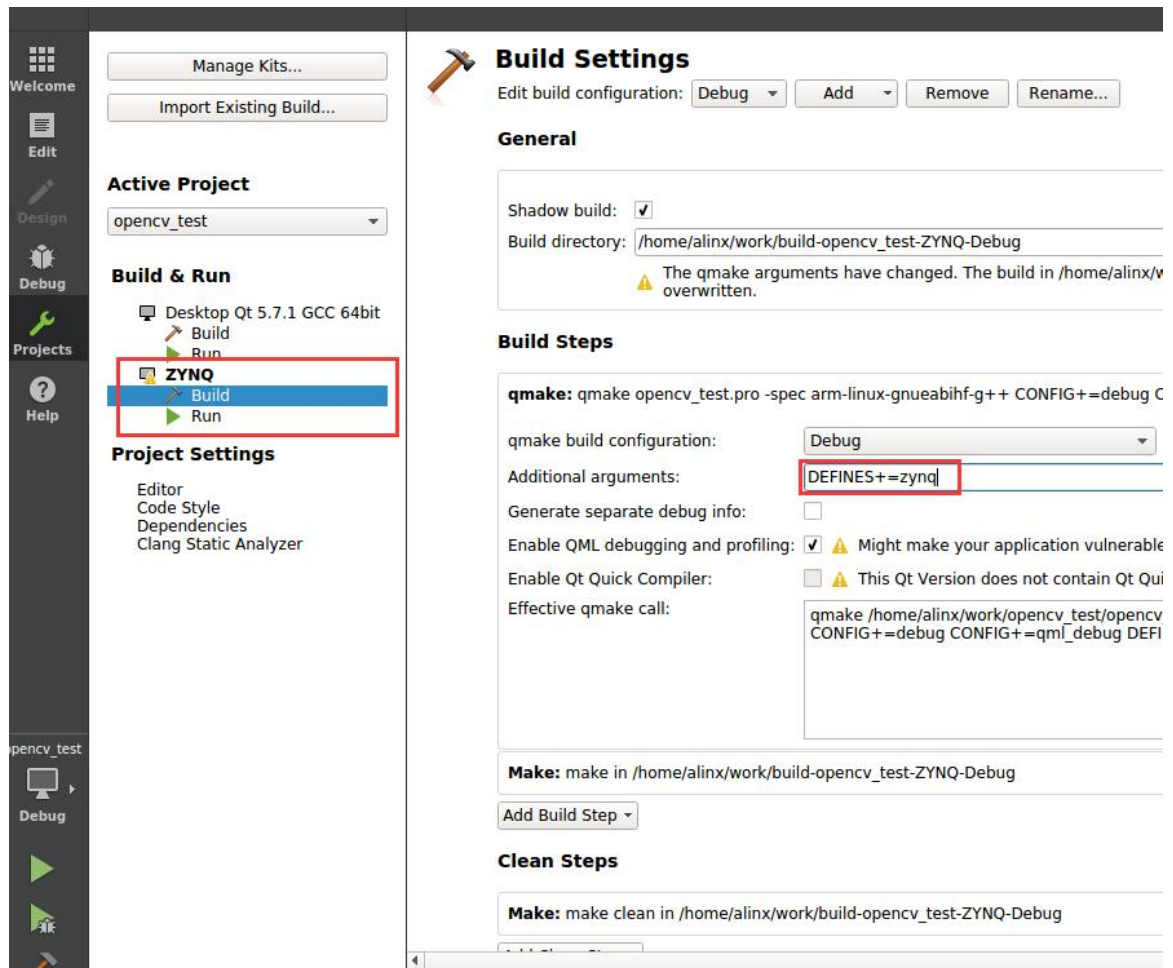


12) 点击 open 即可打开 USB 摄像头

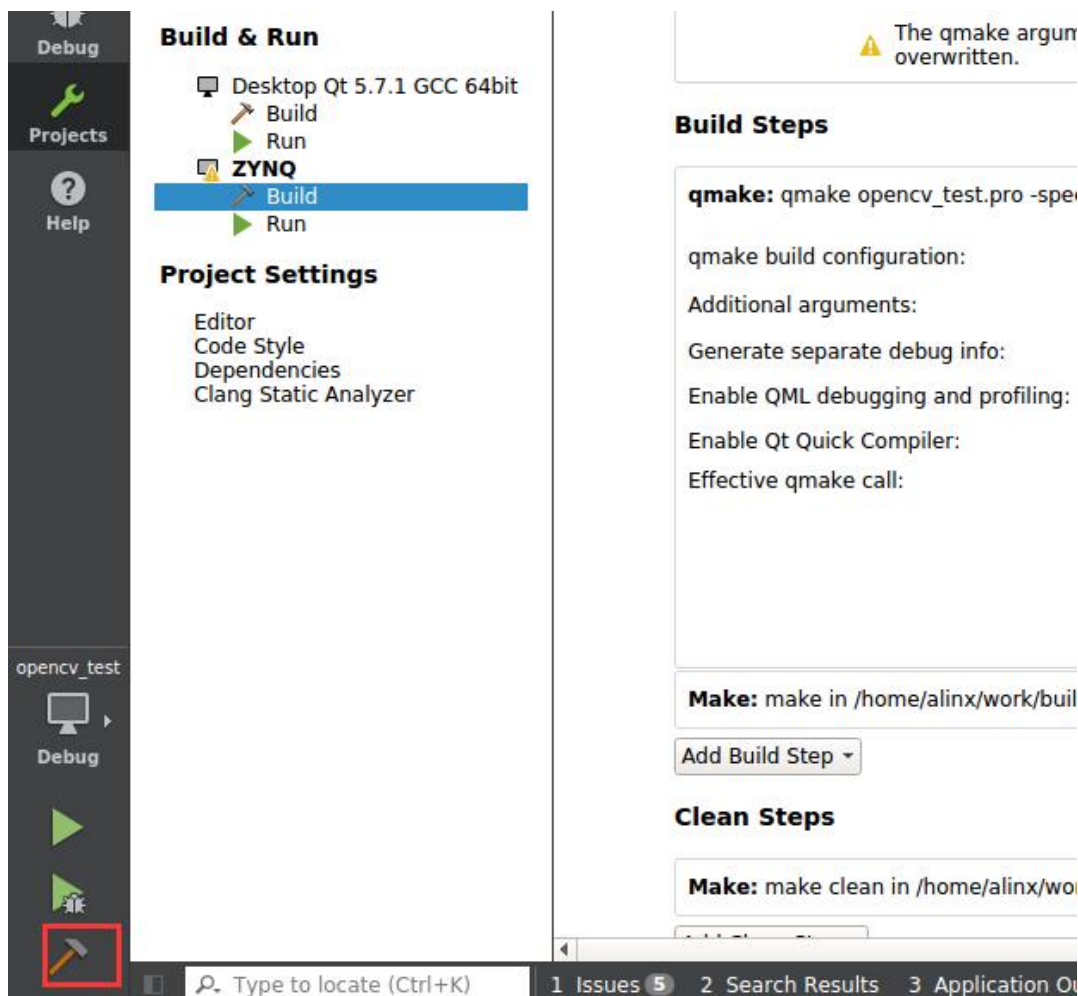


12.5 测试 zynq 版 OpenCV

- 1) 在 zynq 的 kit 设置里添加额外的编译选项 `DEFINES+=zynq`，定义一个 zynq 宏



2) 点击锤子图标编译



- 3) 准备好开发板，连接好显示器，将例程给的 BOOT.BIN，image.ub 复制到 sd 卡（sd_boot 目录），建议先格式化 sd 卡，再复制。
- 4) 使用串口终端登录系统，默认账户 root，密码 root

```
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
Sending discover...
Sending select for 192.168.1.46...
Lease of 192.168.1.46 obtained, lease time 86400
/etc/udhcpd.d/50default: Adding DNS 192.168.1.1
done.
Starting Dropbear SSH server: Generating key, this may take a while...
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDS6PnYncg3QkvRX9ykZpeL1B9GEmuj83s8PqHb+OZs
72dZpqajFznfrRtElNP7agPaD646HACC4AsFYT34ds+eAf8tx4hxNRxn8ZdTmFhONdq/A9600Z1Uy2qV
Fingerprint: md5 b8:c8:a4:a4:70:9e:ff:e4:00:15:41:d8:c2:21:f3:2d
dropbear.
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2017.4 an5642_peta /dev/ttyPS0

an5642_peta login: root
Password:
root@an5642_peta:~#
```

- 5) 挂载 NFS，设置 QT 运行环境

```
mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
```

```
root@an5642_peta:~# mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
root@an5642_peta:~# cd /mnt
root@an5642_peta:/mnt# mkdir /tmp/qt
root@an5642_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/qt
root@an5642_peta:/tmp/qt# source ./qt_env_set.sh
```

6) 复制 zynq_cv_lib.img 到/home/alinx/work 文件夹，设置 OpenCV 库环境

```
mkdir /tmp/cv
cd /mnt
mount zynq_cv_lib.img /tmp/cv
cd /tmp/cv
source ./opencv_env_set.sh
```

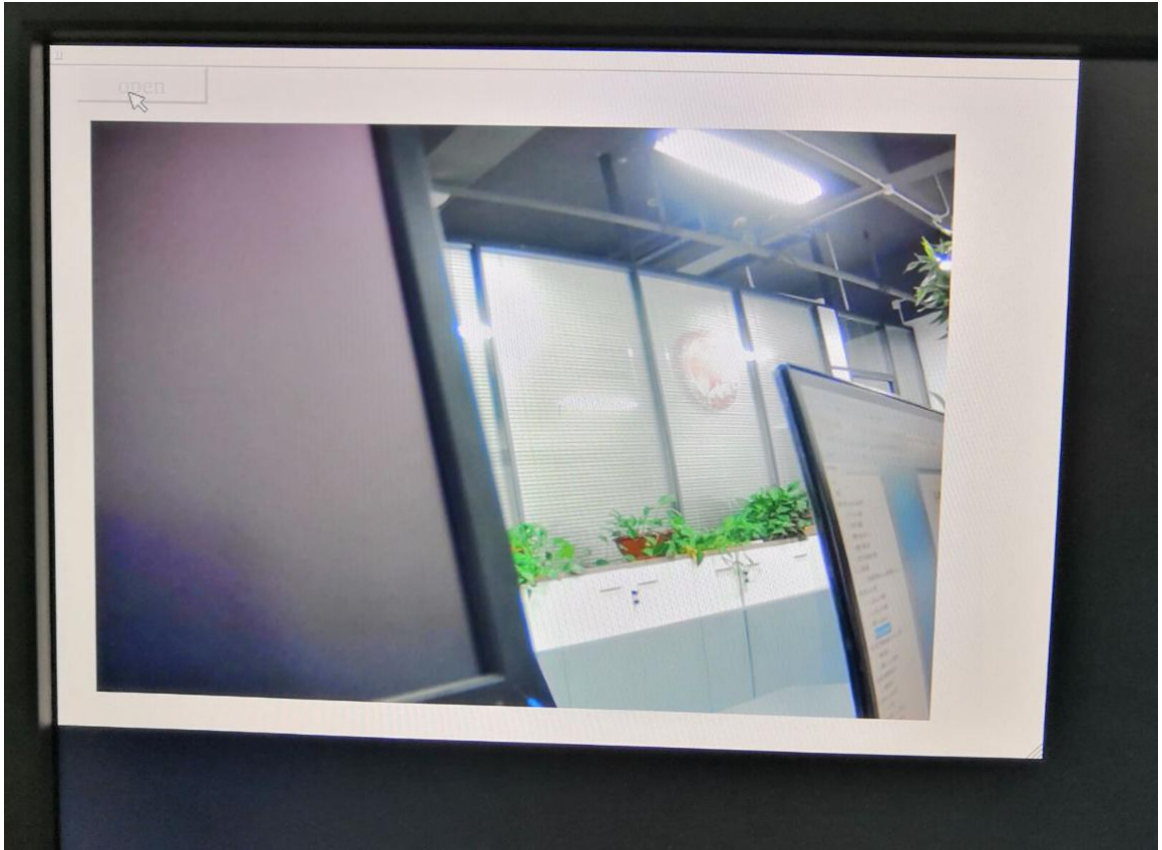
```
root@an5642_peta:/tmp/qt# mkdir /tmp/cv
root@an5642_peta:/tmp/qt# cd /mnt
root@an5642_peta:/mnt# mount zynq_cv_lib.img /tmp/cv
EXT4-fs (loop1): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/cv
root@an5642_peta:/tmp/cv# source ./opencv_env_set.sh
/tmp/cv
root@an5642_peta:/tmp/cv#
```

7) 运行测试

```
cd /mnt/build-opencv_test-ZYNQ-Debug/
./opencv_test
```

```
root@an5642_peta:/tmp/cv# cd /mnt/build-opencv_test-ZYNQ-Debug/
root@an5642_peta:/mnt/build-opencv_test-ZYNQ-Debug# ./opencv_test random: crng init done
```

8) 插入 USB 摄像头，点击 open，测试结果如下



第十三章 双目摄像头模块 OpenCV 显示

例程相关文件夹：opencv_an5642

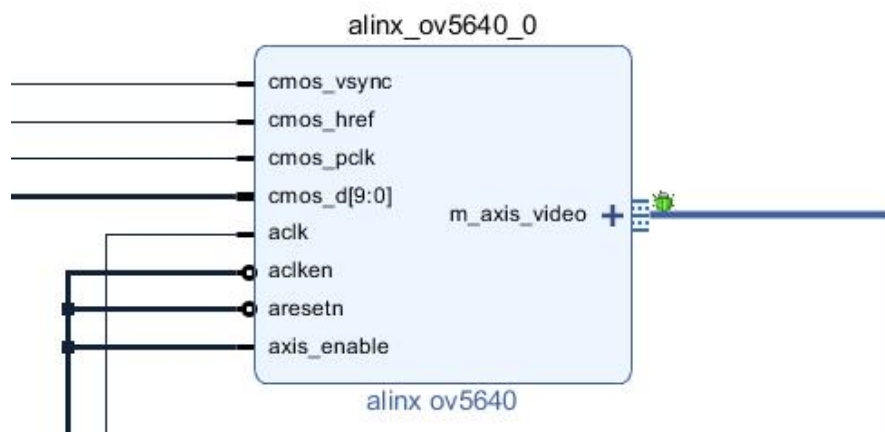
Vivado 工程在 vivado.zip 压缩包中：opencv_an5642

前面的教程中讲到如何使用 OpenCV 和 QT 显示 USB 摄像头的应用, 本章讲解如何显示芯驿电子配套模块双目摄像头 (AN5642) 的应用。

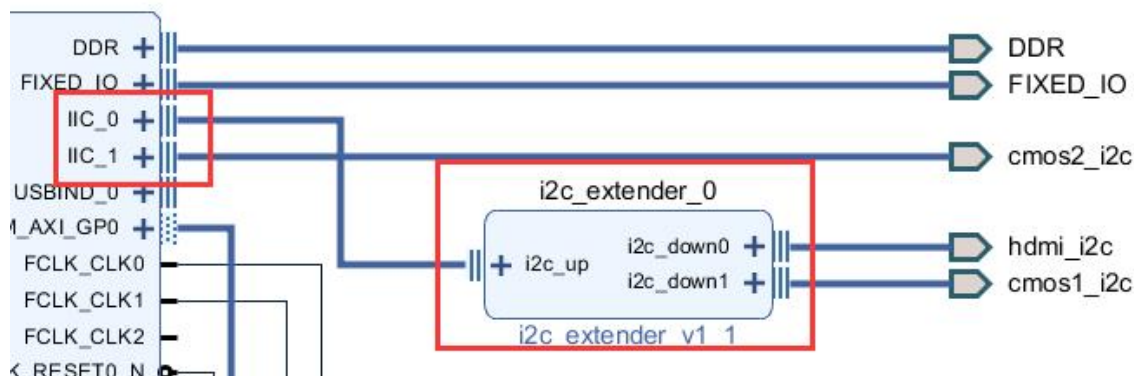
例程资料中给出了 vivado 工程, BSP 包。

13.1 vivado 工程简介

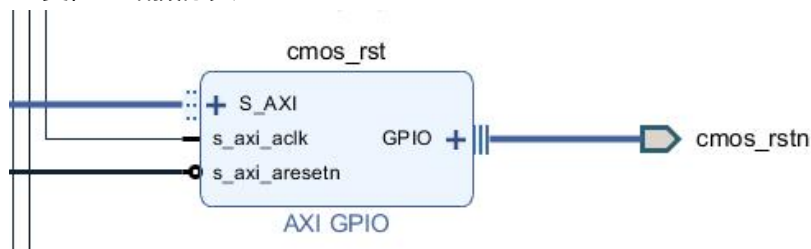
- 1) 摄像头输入转换为 AXI Stream 模块, 将摄像头输入数据转换为 16bit 的 AXI Stream, 然后通过 VDMA 写入 PS 端 DDR3。



- 2) IIC 处理, 由于双目 OV5640 的 IIC 地址完全相同, 所以这里使能 2 路 IIC, 都是通过 EMIO 引出到 PL 端, 其中一个通过 IIC 扩展模块扩展出多路。

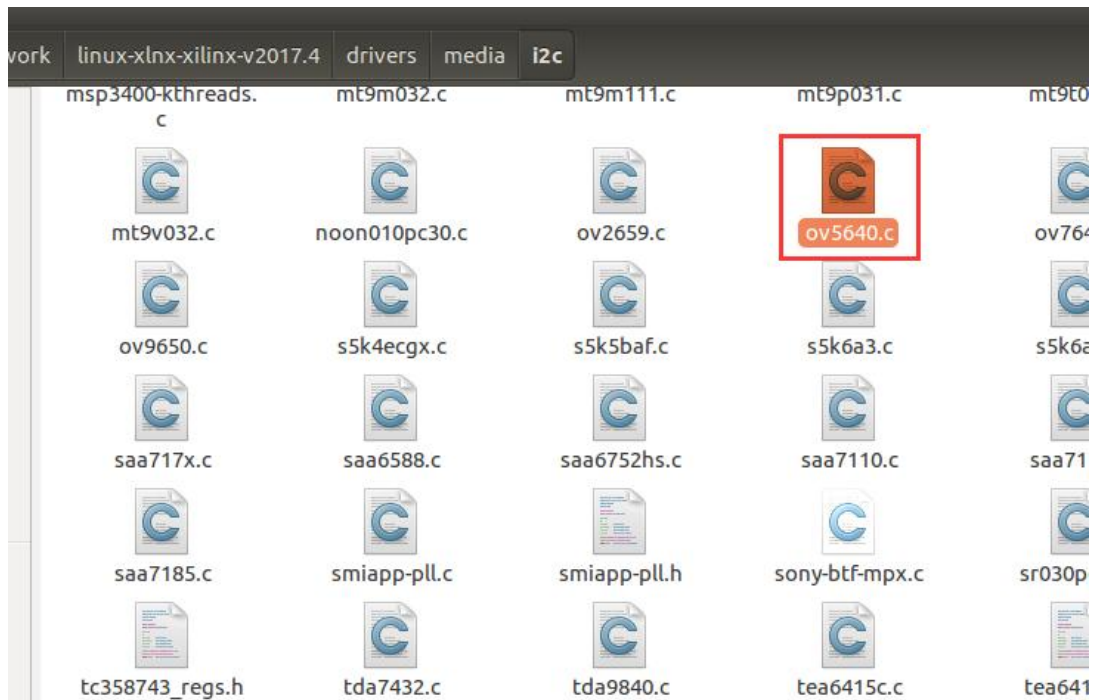


3) 通过 AXI GPIO 复位 2 路摄像头



13.2 摄像头驱动

芯骅电子为 OV5640 写了简单的驱动，由于 Linux 驱动编写涉及内容较多，大部分驱动编写都是通过改写内核中已有驱动，这个驱动也是一样。驱动主要配置 OV5640 输出 YUV422 数据，分辨率根据应用程序来修改，驱动不做过多解释，本身就是 Linux 开发中比较难的点，请直接使用芯骅电子提供的 Linux 内核源码包。

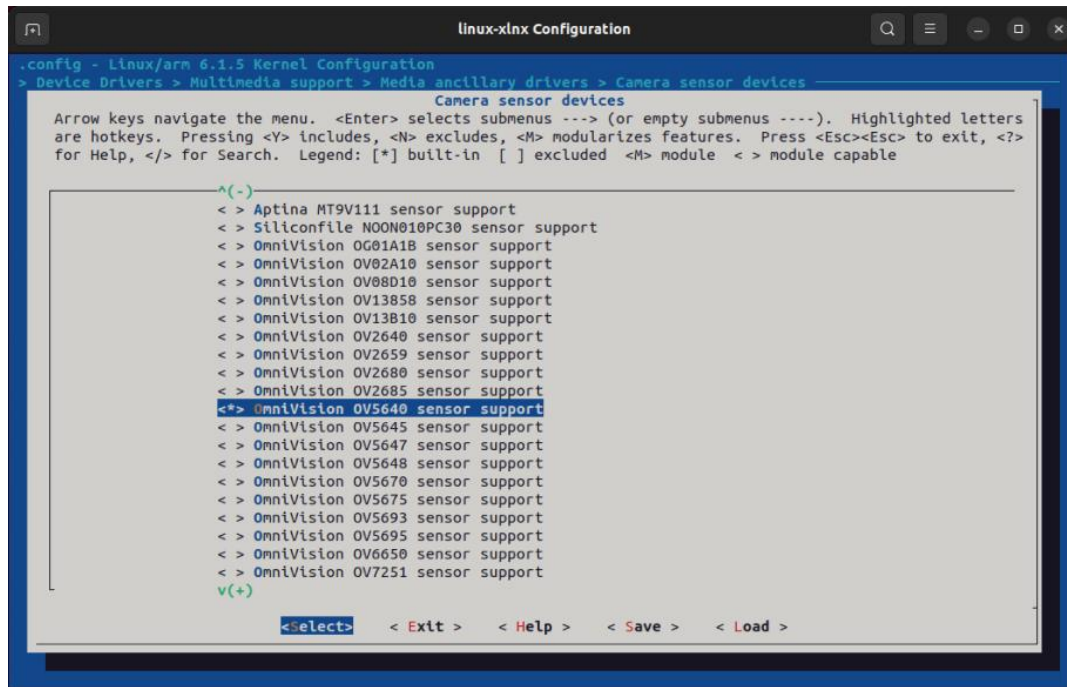


13.3 内核驱动配置

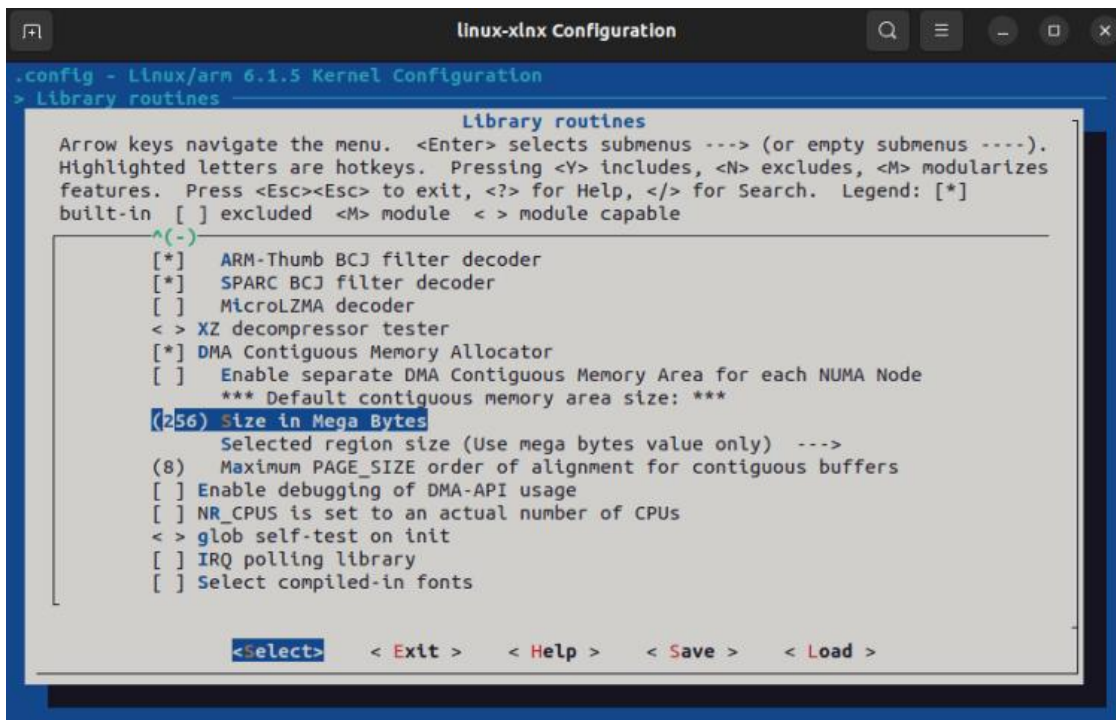
内核驱动中涉及到 drm 显示的部分请参考第八章 HDMI 显示的内核配置，可以在 HDMI 显示的工程基础上添加下列配置，也可以用 BSP 包重新生成工程。

- 1) 驱动配置, Device Drivers → Multimedia support → Media ancillary drivers → Camera sensor devices → <*> ALINX OV5640 sensor support, HDMI 等显示驱动配置在前面的教程中已经讲

解，不再复述。



- 2) Library routines → Size in Mega Bytes , CMA_SIZE_MBYTES 配置，由于视频路数增加，需要添加更多的 DMA 空间，配置 CMA_SIZE_MBYTES 大小为 256MB



13.4 设备树配置

- 1) 在 petalinux 工程 project-spec/meta-user/recipes-bsp/device-tree/files 目录下，找到 system-user.dtsi，这是我们要修改会添加的设备树，例程中已经修改好
- 2) 2 个 OV5640 分别挂载 i2c0 和 i2c1 总线下，地址为 0x3c（7bit 表示，是 8bit 表示地址的一半），详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

```
&i2c0 {
    clock-frequency = <100000>;
    sensor@3c{
        compatible = "alinx,ov5640";
        reg = <0x3c>;
        #address-cells = <1>;
        #size-cells = <0>;
        reset-gpios = <&cmos_rst 0 0>;
        port@0 {
            reg = <0>;
            xlnx,video-width = <16>;
            ov5640_out_0: endpoint {
                remote-endpoint = <&vcap_ov5640_in_0>;
            };
        };
    };
};

&i2c1 {
    clock-frequency = <100000>;
    sensor@3c{
        compatible = "alinx,ov5640";
        reg = <0x3c>;
        #address-cells = <1>;
        #size-cells = <0>;
        reset-gpios = <&cmos_rst 1 0>;
        port@0 {
            reg = <0>;
            xlnx,video-width = <16>;
            ov5640_out_1: endpoint {
                remote-endpoint = <&vcap_ov5640_in_1>;
            };
        };
    };
};
```

- 3) vcap_ov5640 设备为 Xilinx 提供 VDMA 驱动封装起来，做成 V4L2 接口

```
vcap_ov5640_0 {
    compatible = "xlnx,video";
    dmas = <&axi_vdma_1 1>;
    dma-names = "port0";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            direction = "input";
            vcap_ov5640_in_0: endpoint {
                remote-endpoint = <&ov5640_out_0>;
            };
        };
    };
};

vcap_ov5640_1 {
    compatible = "xlnx,video";
    dmas = <&axi_vdma_2 1>;
    dma-names = "port0";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            direction = "input";
            vcap_ov5640_in_1: endpoint {
                remote-endpoint = <&ov5640_out_1>;
            };
        };
    };
};
```

13.5 摄像头 Linux 应用程序

- 1) 应用程序使用 QT 编写，结合 OpenCV 做颜色空间转换，在前面的教程中已经将过，不再复述
- 2) 需要注意的是，elf 文件生成位置，和前面教程不同，这次在 app 目录里
- 3) 运行例程 Linux，可以看到双目摄像头相关驱动打印信息。

```
nub 1-U:1.U: 1 port detected
mousedev: PS/2 mouse device common for all mice
i2c /dev entries driver
cdns-i2c e0004000.i2c: 100 kHz mmio e0004000 irq 23
cdns-i2c e0005000.i2c: 100 kHz mmio e0005000 irq 24
random: fast init done
ov5640 0-003c: Chip ID 0x5640 detected
ov5640 1-003c: Chip ID 0x5640 detected
xilinx-video amba_pl:vcap_ov5640_0: device registered
xilinx-video amba_pl:vcap_ov5640_1: device registered
uvcvideo: Unable to create debugfs directory
usbcore: registered new interface driver uvcvideo
USB Video Class driver (1.1.1)
cdns-wdt f8005000.watchdog: Xilinx Watchdog Timer at f096e000 with timeout 10s
EDAC MC: ECC not enabled
Xilinx Zynq CpuIdle Driver started
sdhci: Secure Digital Host Controller Interface driver
sdhci: Copyright(c) Pierre Ossman
sdhci-pltfm: SDHCI platform and OF driver helper
usb 1-1: new high-speed USB device number 2 using ci_hdrc
mmc0: SDHCI controller on e0100000.sdhci [e0100000.sdhci] using ADMA
mmc0: new high speed SDHC card at address aaaa
mmcblk0: mmc0:aaaa SL08G 7.40 GiB
mmc1: SDHCI controller on e0101000.sdhci [e0101000.sdhci] using ADMA
mmcblk0: p1 p2
ledtrig-cpu: registered to indicate activity on CPUs
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
NET: Registered protocol family 10
sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
NET: Registered protocol family 17
```

4) 挂载 NFS, 设置 QT 和 OpenCV 运行环境

```
mount -t nfs -o nolock 192.168.1.115:/home/alinx2020/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
mkdir /tmp/cv
cd /mnt
mount zynq_cv_lib.img /tmp/cv
cd /tmp/cv
source ./opencv_env_set.sh
```

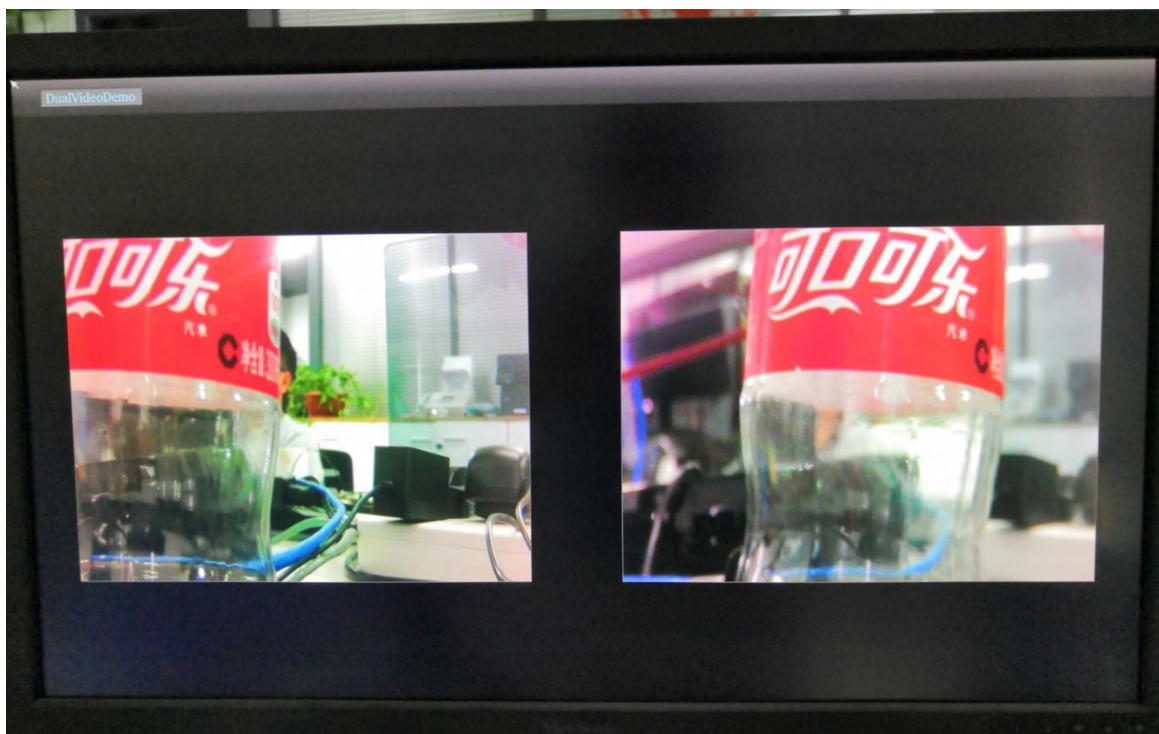
```
an5642_peta login: root
Password:
root@an5642_peta:~# mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
mount qt_lib.img /tmp/qt
cd /tmp/qt
root@an5642_peta:~# cd /mnt
root@an5642_peta:/mnt# mkdir /tmp/qt
cd /tmp/cv
root@an5642_peta:/mnt# mount qt_lib.img /tmp/qt
source ./opencv_env_set.sh
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/qt
root@an5642_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@an5642_peta:/tmp/qt# mkdir /tmp/cv
root@an5642_peta:/tmp/qt# cd /mnt
root@an5642_peta:/mnt# mount zynq_cv_lib.img /tmp/cv
EXT4-fs (loop1): recovery complete
EXT4-fs (loop1): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/cv
root@an5642_peta:/tmp/cv# source ./opencv_env_set.sh
/tmp/cv
root@an5642_peta:/tmp/cv#
```

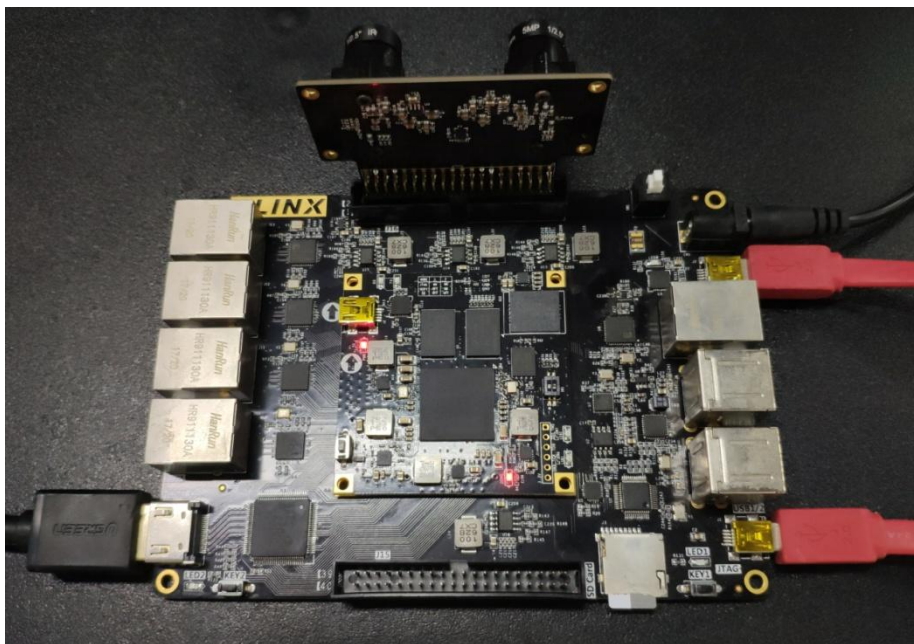
5) 运行测试程序

```
cd /mnt/app
./an5642_demo
```

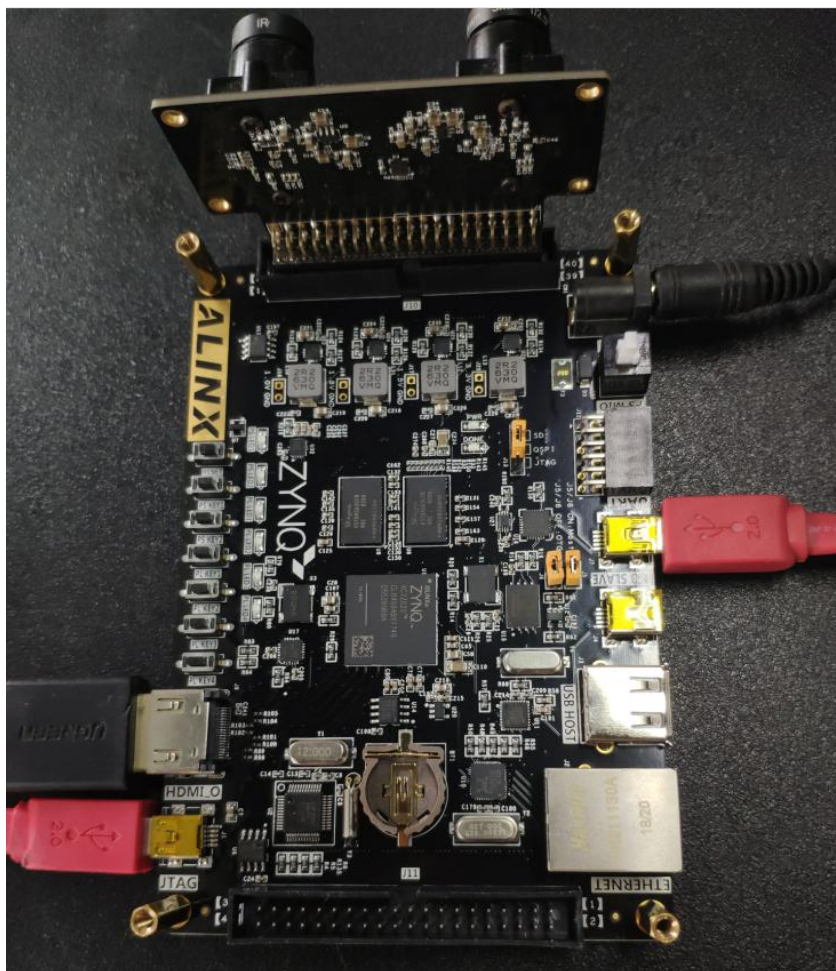
```
root@an5642_peta:/tmp/cv# cd /mnt/app
root@an5642_peta:/mnt/app# ./an5642_demo
```

- 6) 运行效果，资料包中给出了编译好的程序在 sd_boot 目录，所有文件复制到 sd 卡根目录运行即可。





AX7021 硬件连接图 (J16 扩展口)



AX7020/AX7010 硬件连接图(J10 扩展口)

第十四章 触摸屏模块应用

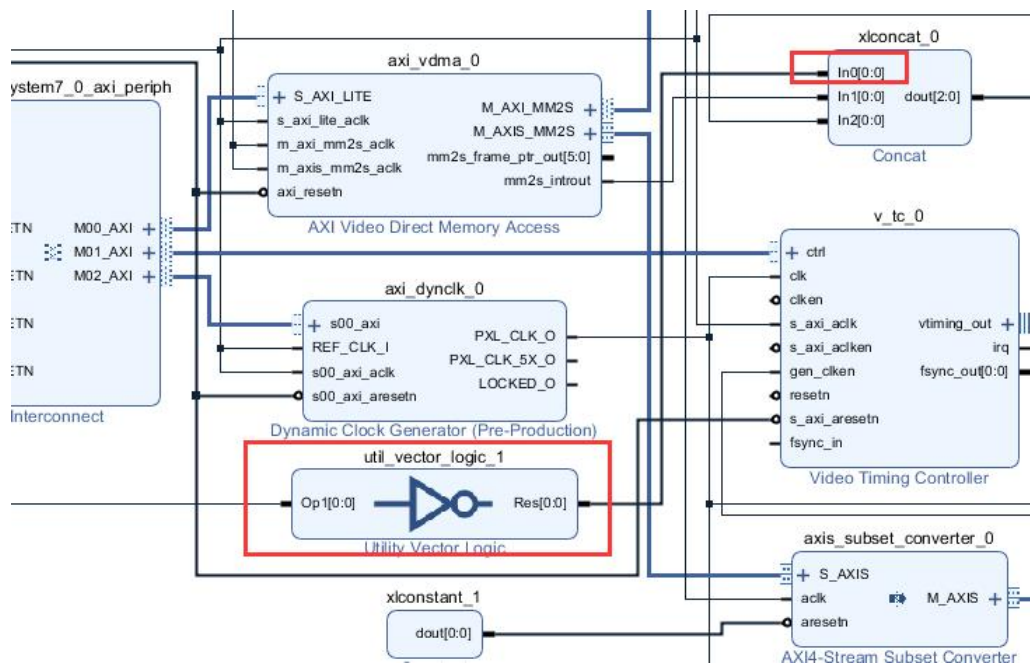
例程相关文件夹：lcd_touch

Vivado 工程在 vivado.zip 压缩包中：lcd_touch

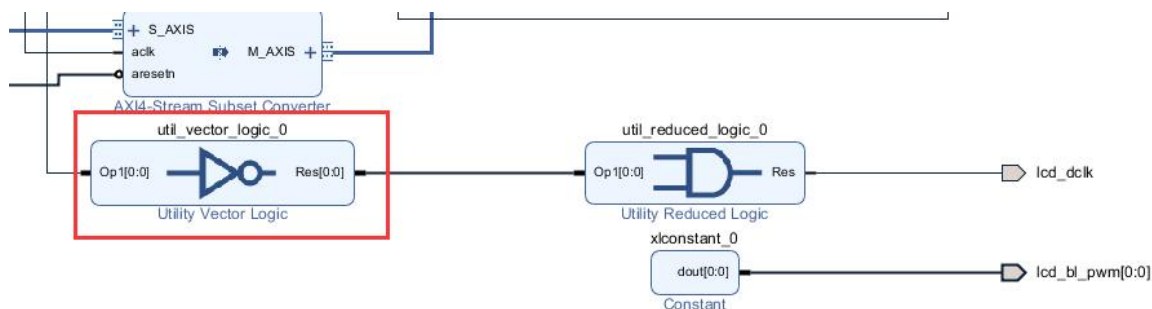
例程资料中给出了 vivado 工程，petalinux 工程，本实验还是在前面 HDMI 实验的 Petalinux 工程基础上进行修改的，需要先掌握前面的实验内容。

14.1 vivado 工程简介

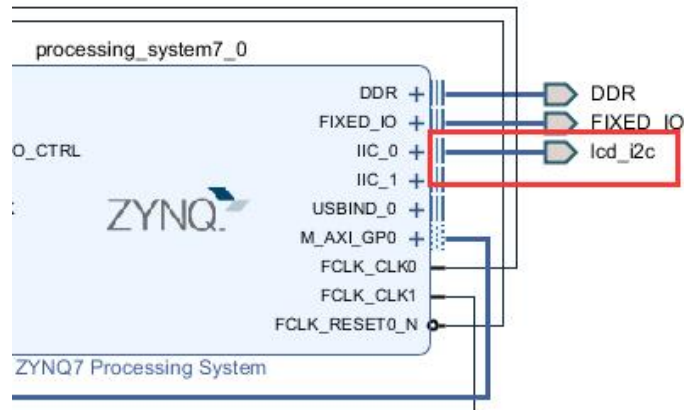
- 1) 触摸屏中断处理，由于触摸屏默认中断是低电平，而 PS 来自 PL 的中断只能是高电平或上升沿，这里加一个非门反相。中断连接到 PS 的第一个中断管脚，后面我们可以根据这些信息推断 Linux 下中断号，Linux 下中断编号可以参考 Xilinx 文档 UG585



- 2) 液晶屏时钟处理，由于液晶屏是下降沿采样，这里使用非门把时钟反相输出，后面的与门没有什么作用，仅仅用于信号的连接。

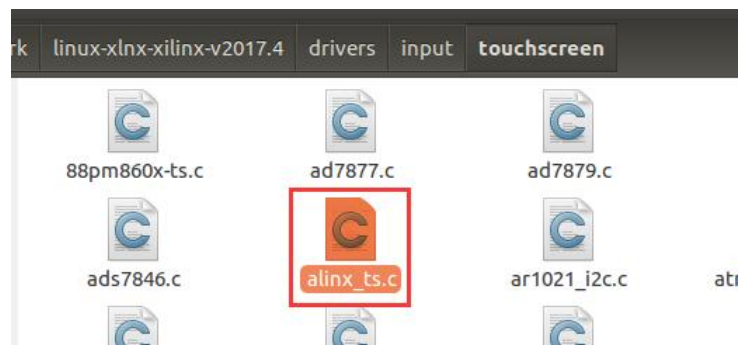


- 3) 液晶屏背光设置为一直高电平，就是最亮状态，可能会让液晶屏发热较多
- 4) IIC 用于读取液晶屏触摸信息

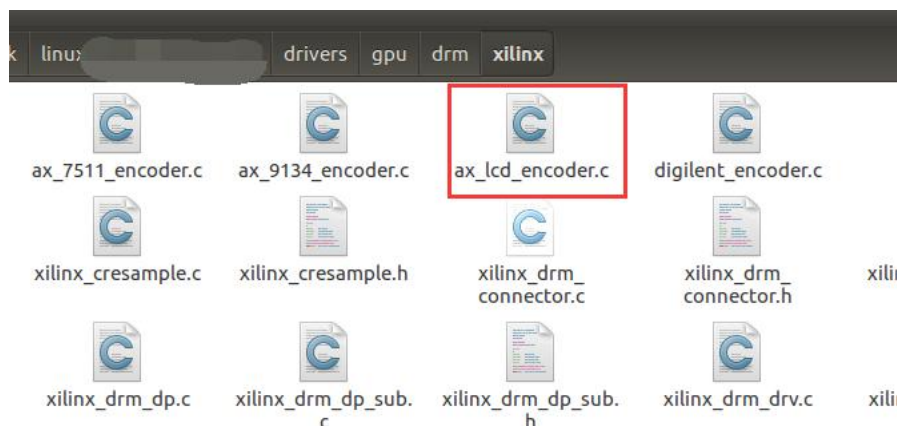


14.2 触摸屏驱动

- 1) alinx_ts.c 为芯驿电子提供的触摸屏驱动，位于内核目录 drivers/input/touchscreen 中



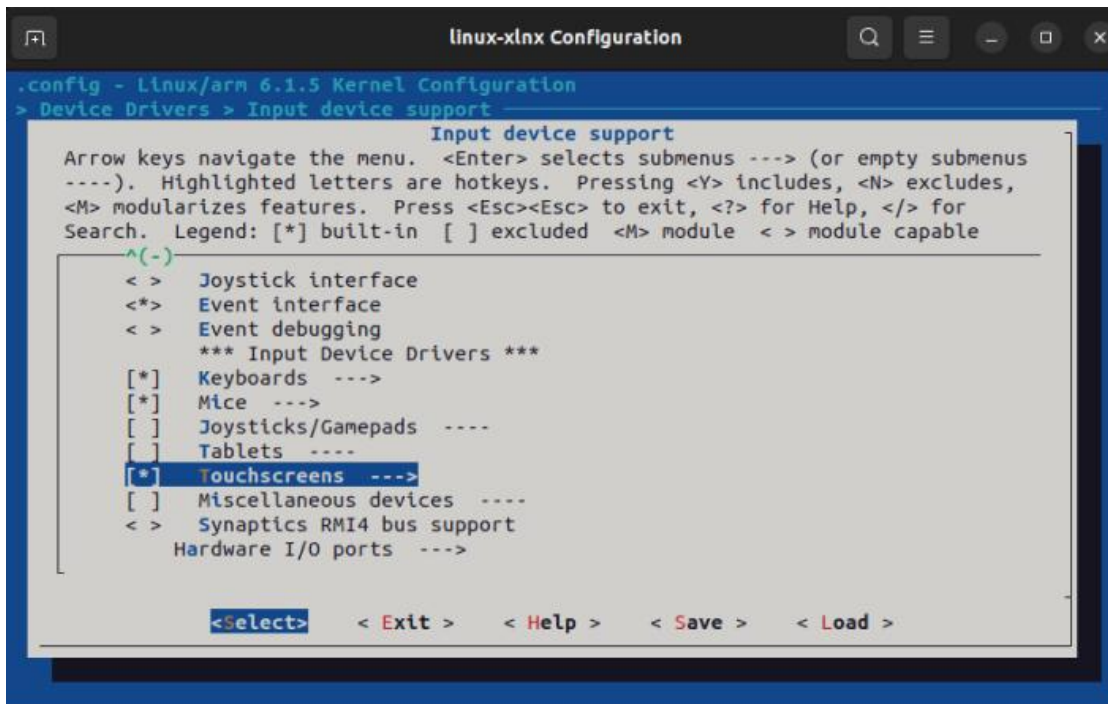
- 2) 触摸屏还要配套编码器才能工作，ax_lcd_encoder.c 是芯驿电子提供的 DRM 编码器驱动，位于内核 drivers/gpu/drm/xilinx



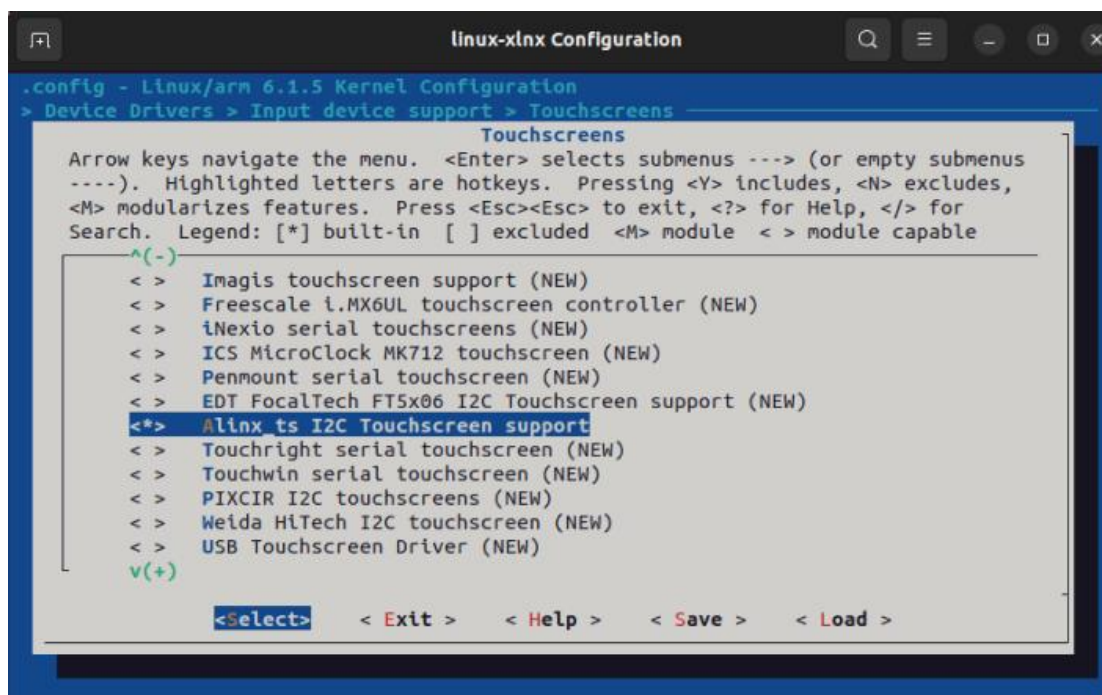
14.3 内核驱动配置

内核驱动中涉及到 drm 显示的部分请参考第八章 HDMI 显示的内核配置，可以在 HDMI 显示的工程基础上添加下列配置，也可以用 BSP 包重新生成工程。

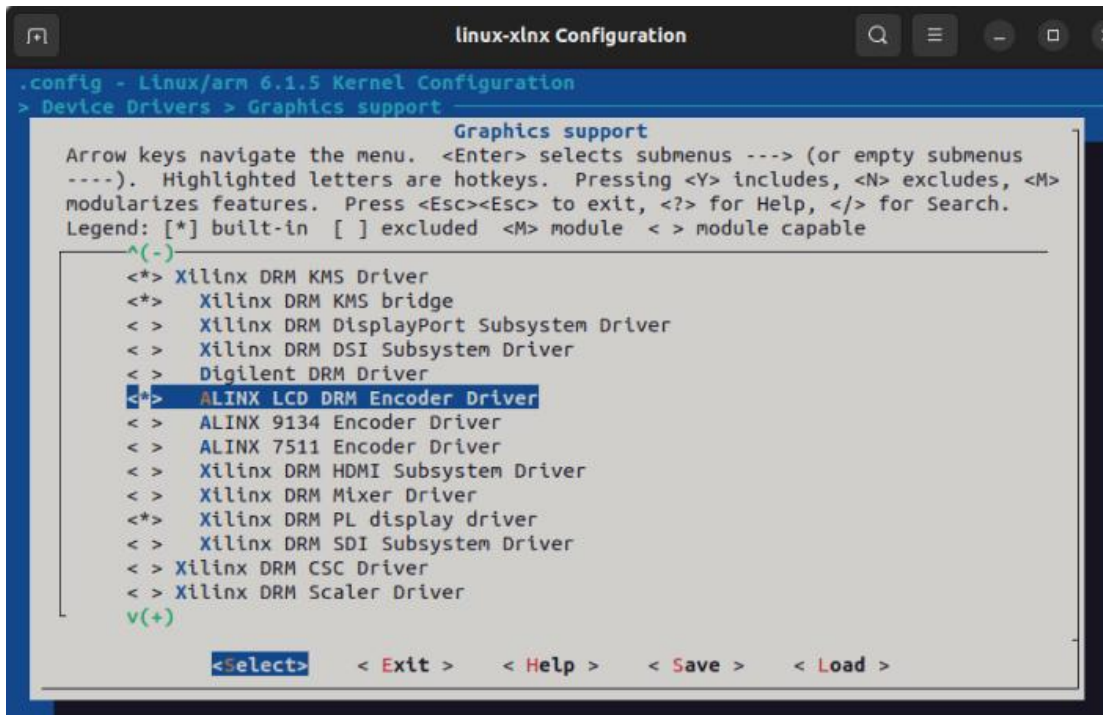
- 1) Device Drivers > Input device support > [*] Touchscreens 使能触摸屏



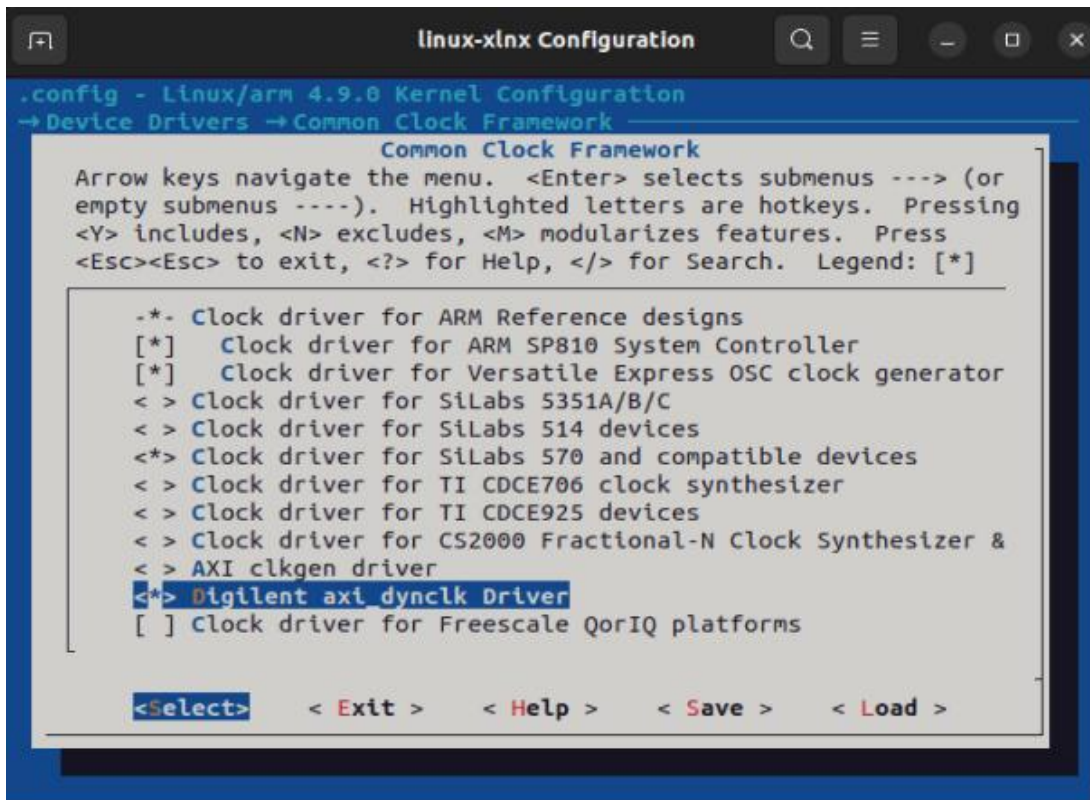
- 2) 选择 <*> Alinx_ts I2C Touchscreen support



- 3) 在 Device Drivers > Graphics support 中选择<*> ALINX LCD DRM Encoder Driver ,



- 4) 在 Device Drivers > Common Clock Framework 中选择<*> Digilent axi_dyncclk Driver



- 5) 如果用到其他外设，请更加需要配置驱动

14.4 设备树配置

在 i2c0 下挂触摸屏模块，设备地址为 0x5D (7bit 地址)。详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

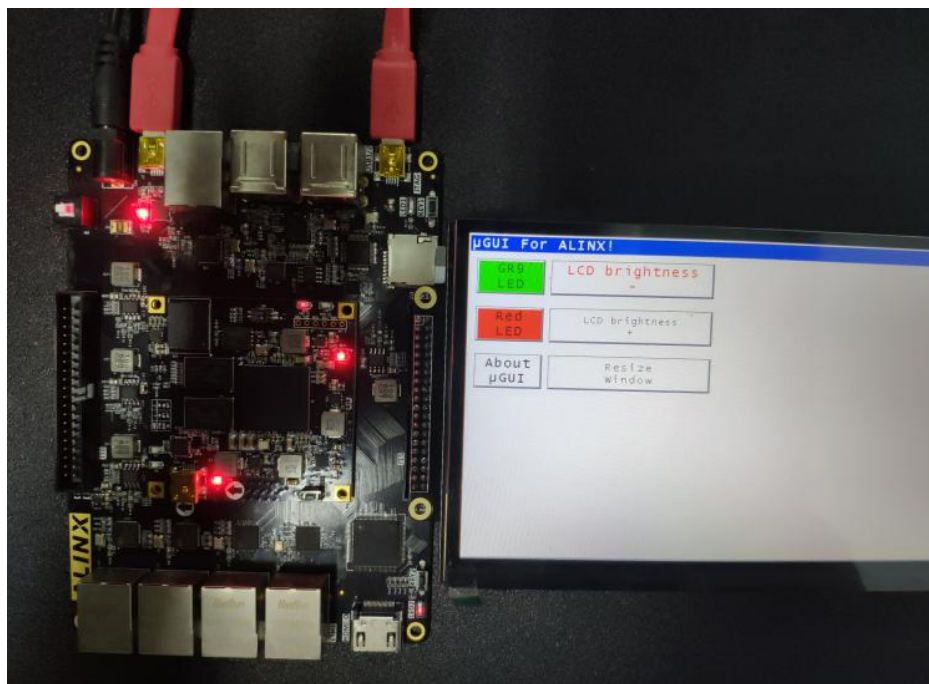
```
&i2c0 {
    clock-frequency = <100000>;
    alinx_an071@5D {
        compatible = "alinx,an071";
        reg = <0x5D>;
        interrupt-parent = <&intc>;
        interrupts = <0 29 4>;
    };
};
```

14.5 触摸屏测试

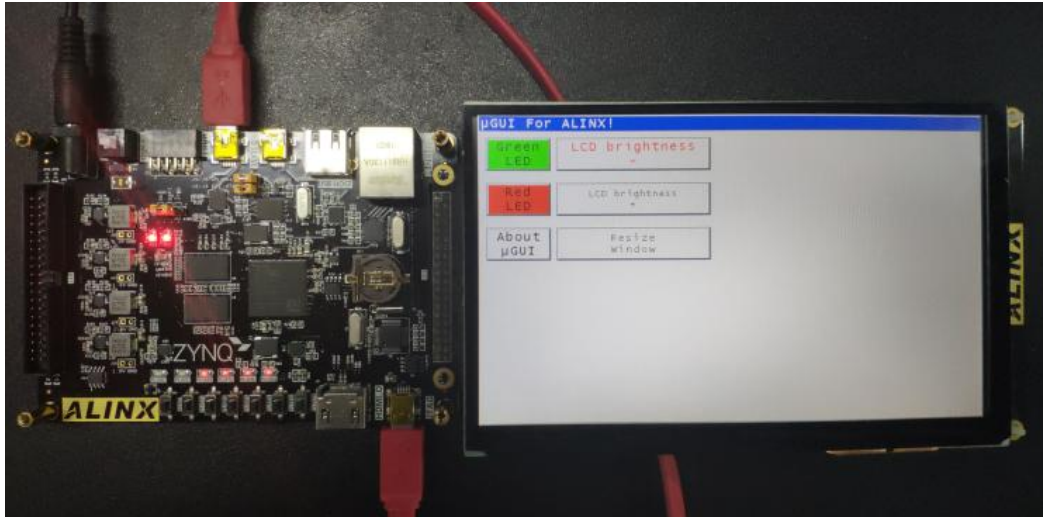
- 1) 用例程给的 petalinux 工程生成的 BOOT.BIN 和 image.ub，放入 SD 卡
- 2) 挂载 NFS，运行前面例程的 QT 测试程序

```
mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
cd build-qt_test-ZYNQ-Debug
./qt_test
```

- 3) 运行效果



AX7021 硬件连接 (J15 扩展口) 请忽略显示内容



AX7020/AX7010 硬件连接 (J11 扩展口) 请忽略显示内容

第十五章 AXI DMA 读写测试

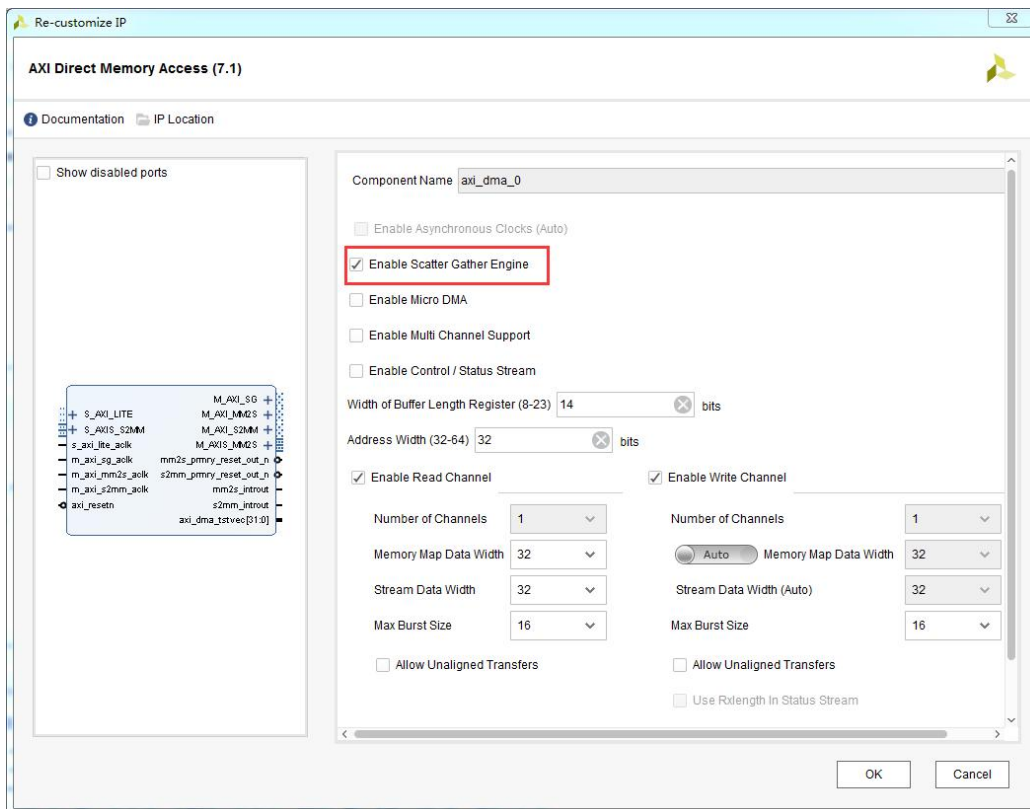
例程相关文件夹：sg_dma_test

Vivado 工程在 vivado.zip 压缩包中：sg_dma_test

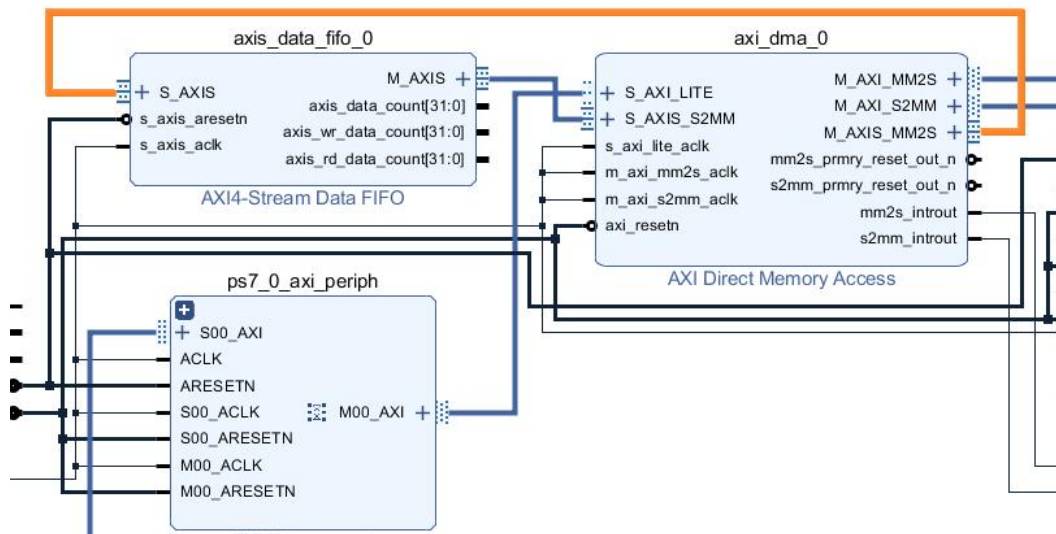
在前面的 Vitis 裸机例程中我们大量讲解了如何使用 DMA, 现在我们要在 Linux 下使用 DMA, 需要说明的是, 这里使用的 DMA 支持 “Scatter Gather”, 也就是 SG-DMA。Xilinx 提供了 DMA 驱动和 DMA 测试程序, 测试程序也是一个驱动, 从这可以看出, 如果要使用 DMA, 就算有了 DMA 驱动, 我们还是要开发一个 DMA 驱动客户端, 称为 DMA client, 所以 DMA 的使用是有一定的门槛, 对于 Linux 门外汉来说非常有挑战。

15.1 vivado 工程

- 1) 和 Vitis 下测试 DMA 类似, 添加 DMA 的 IP, 选择 “Enable Scatter Gather Engine”

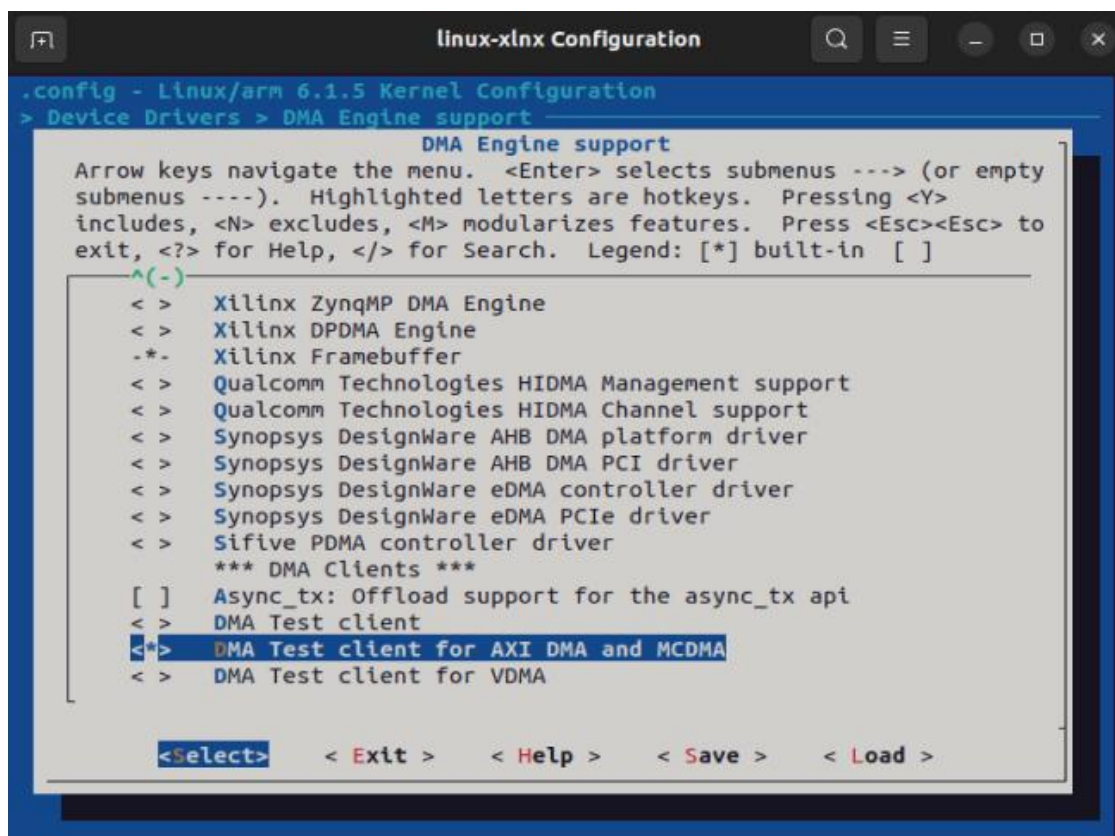


- 2) 添加一个 “AXI4-Stream Data FIFO”, 然后将 DMA 的 MM2S (读端口) 和 S2MM (写端口) 端口连接, 组成一个读写环路, 用于测试 DMA 功能。



15.2 petalinux 工程配置

- 1) 使用 Vivado 工程导出的 xsa 文件创建 petalinux 工程
- 2) 配置 DMA 测试客户端驱动，在 Device Drivers > DMA Engine support > Xilinx DMA Engines 中使能 DMA Test client for AXI DMA



- 3) 修改设备树，将 petalinux 工程中的 system-user.dtsi 文件修改如下

```
/include/ "system-conf.dtsi"
/{

    axidmatest_0: axidmatest {
        compatible = "xlnx,axi-dma-test-1.00.a";
        dmas = <&axi_dma_0 0
                &axi_dma_0 1>;
        dma-names = "axidma0", "axidma1";
    };

};
```

15.3 运行测试

- 1) 编译生成 BOOT.BIN 和 image.ub
- 2) 将 BOOT.BIN 和 image.ub 复制到 sd 卡中，开发板 sd 卡模式启动运行
- 3) 测试结果可以看到，DMA 读写环通测试 5 次，没有错误

```
dma1chan0-dma1c: terminating after 5 tests, 0 failures 632 iops 34138 KB/s (status 0)
```

```
Segment Routing with IPv6
In-situ OAM (IOAM) with IPv6
sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
NET: Registered PF_PACKET protocol family
can: controller area network core
NET: Registered PF_CAN protocol family
can: raw protocol
can: broadcast manager protocol
can: netlink gateway - max_hops=1
Registering SWP/SWPB emulation handler
dmaest: Started 1 threads using dma1chan0 dma1chan1
mmc0: new high speed SDHC card at address b36c
mmcblk0: mmc0:b36c SDABC 29.5 GiB
mmcblk0: p1 p2
dma1chan0-dma1c: terminating after 5 tests, 0 failures 632 iops 34138 KB/s (status 0)
of-fpga-region fpga-full: FPGA Region probed
of_cfs_init
of_cfs_init: OK
ALSA device list:
  No soundcards found.
Freeing unused kernel image (initmem) memory: 34816K
Run /init as init process
INIT: version 3.04 booting
Starting udev
Starting version 251.8+
ext3: Unknown parameter 'umask'
```

第十六章 基于 DMA 的 ADC 波形显示 (AN9238)

例程相关文件夹: an9238_wave

Vivado 工程在 vivado.zip 压缩包中: an9238_wave

在 Vitis 裸机例程里我们用 DMA 将 ADC 模块 AN9238 的数据采集到 ddr 中然后显示出波形, 在上面的例程用我们也学习了 DMA 的读写测试, 我们再这个读写测试的基础上写了一个 ADC 驱动, 所以请大家在学习本章内容时请熟练掌握前面的例程。

例程资料中给出了 vivado 工程, petalinux 工程, 以及波形显示 app。

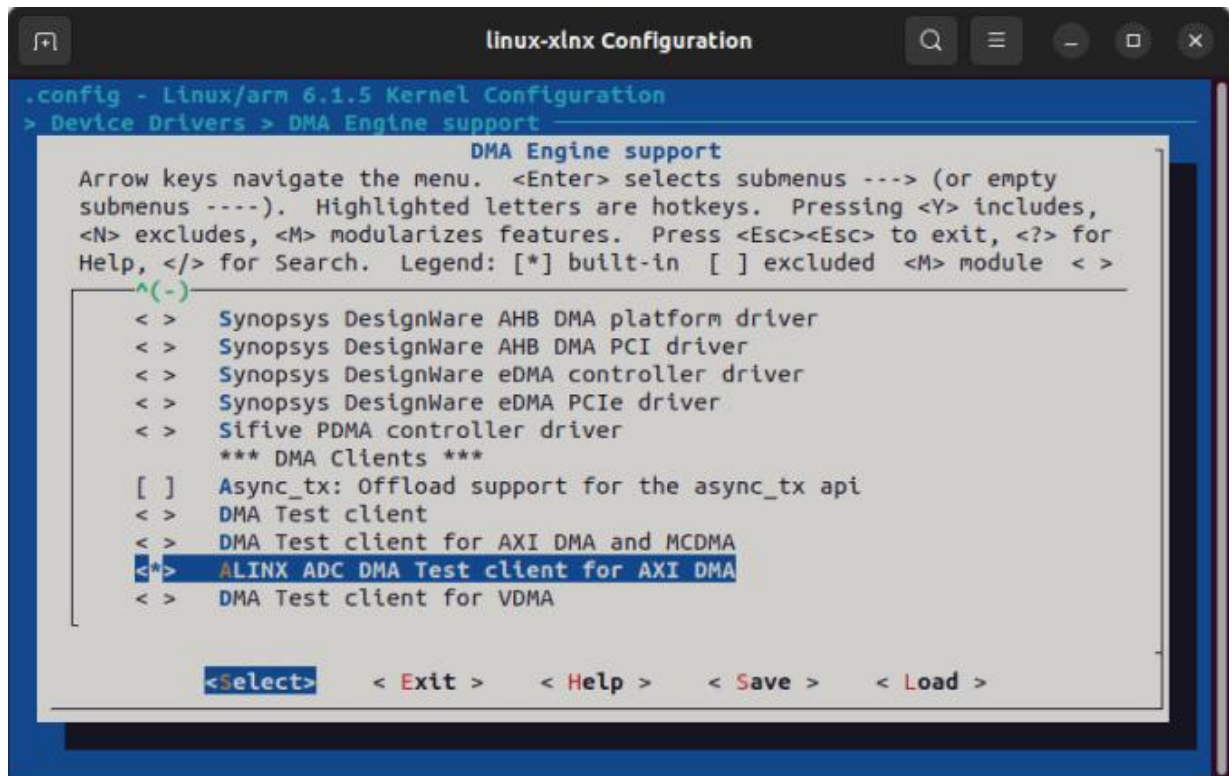
16.1 ADC 驱动

- 1) ADC 驱动是内核 drivers/dma/xilinx 目录中的 axi_adc_dma.c, 驱动编写时参考了同目录下的 axidmatest.c 文件和 Xilinx 官方例程 “xapp1183”。

16.2 内核驱动配置

内核驱动中涉及到 drm 显示的部分请参考第八章 HDMI 显示的内核配置, 可以在 HDMI 显示的工程基础上添加下列配置, 也可以用 BSP 包重新生成工程。

- 1) 和前面的例程一样要配置一些外设显示的驱动, 这里不再复述, 主要是要额外配置 ADC 的驱动, 在 Device Drivers > DMA Engine support > Xilinx DMA Engines 选项中选择<*> ALINX ADC DMA Test client for AXI DMA



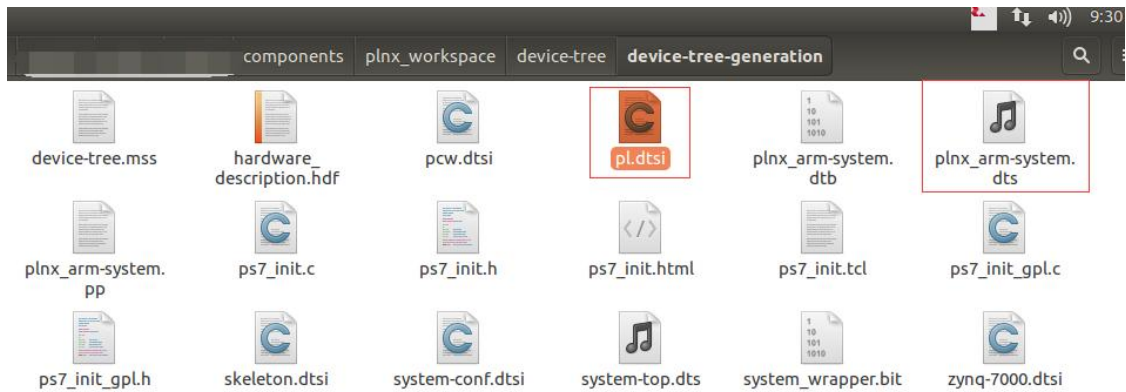
16.3 设备树配置

- 1) 这里是修改了驱动的匹配选项, compatible = "alinx,axi-adc-dma";用于匹配驱动程序。详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

```
&ad9238_sample_0 {
    compatible = "alinx,axi-adc-dma";
    dmas = <&axi_dma_0 1>;
    dma-names = "axidma1";
};

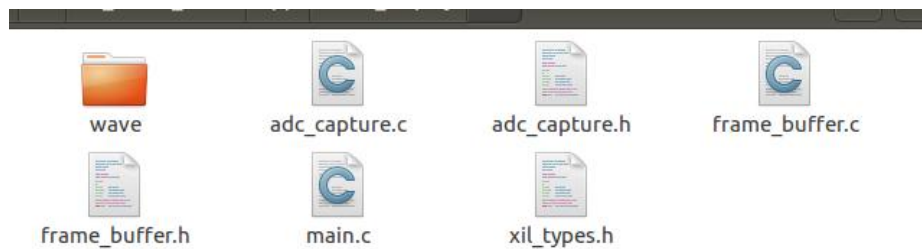
&ad9238_sample_1 {
    compatible = "alinx,axi-adc-dma";
    dmas = <&axi_dma_1 1>;
    dma-names = "axidma1";
};
```

- 2) 设备树修改技巧, 先编译一次, 然后到 components/plnx_workspace/device-tree/device-tree-generation 目录下找到 pl.dtsi 这里包含了所有 PL 端外设的设备树信息, 最好不要直接修改这个文件, plnx_arm-system.dts 是最后生成的文件, 也是非常重要的信息, 开发者可以自己好好研究一下这些文件。



16.4 应用程序

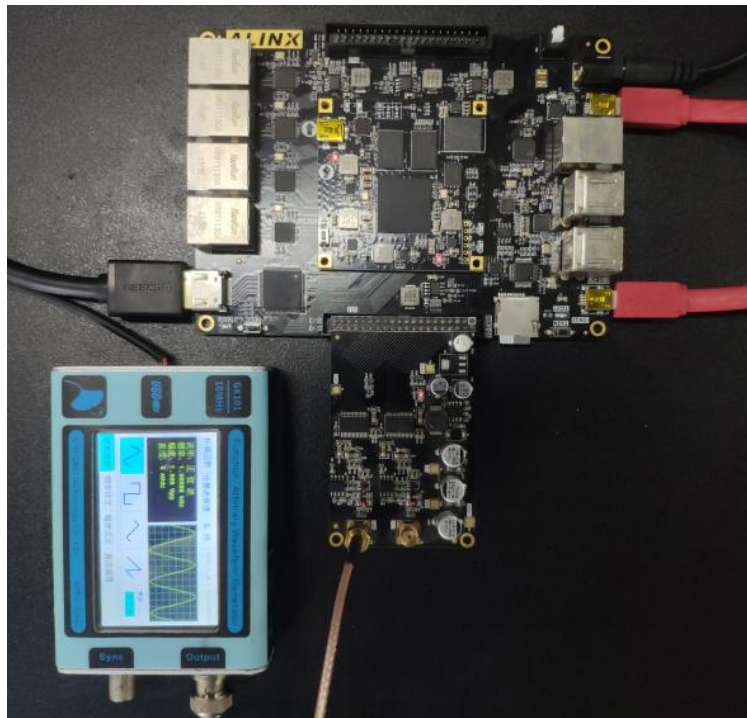
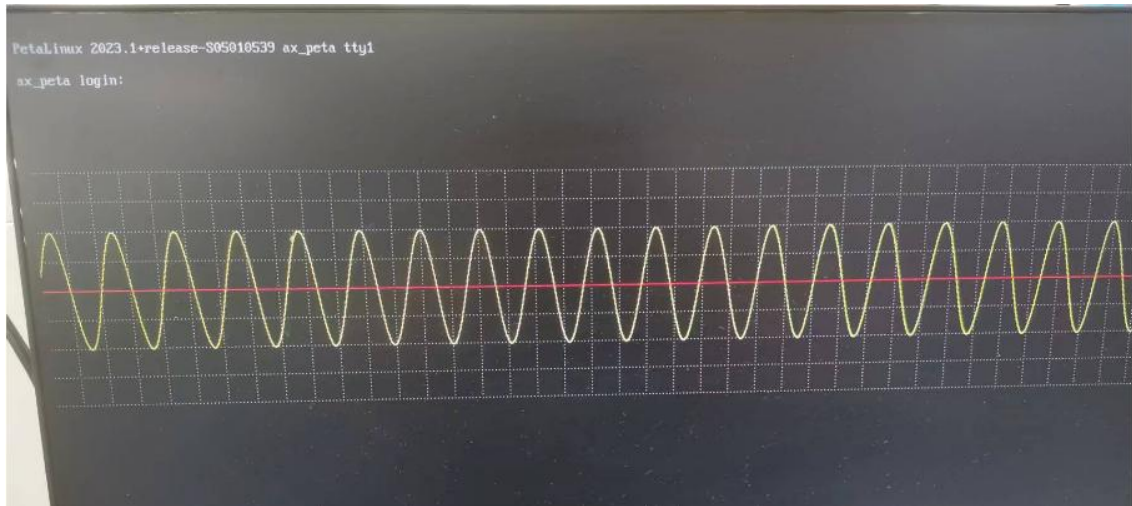
- 1) 应用程序采用 Vitis 编写，Vitis 建立 Linux 应用程序的方法在前面的教程中讲到。



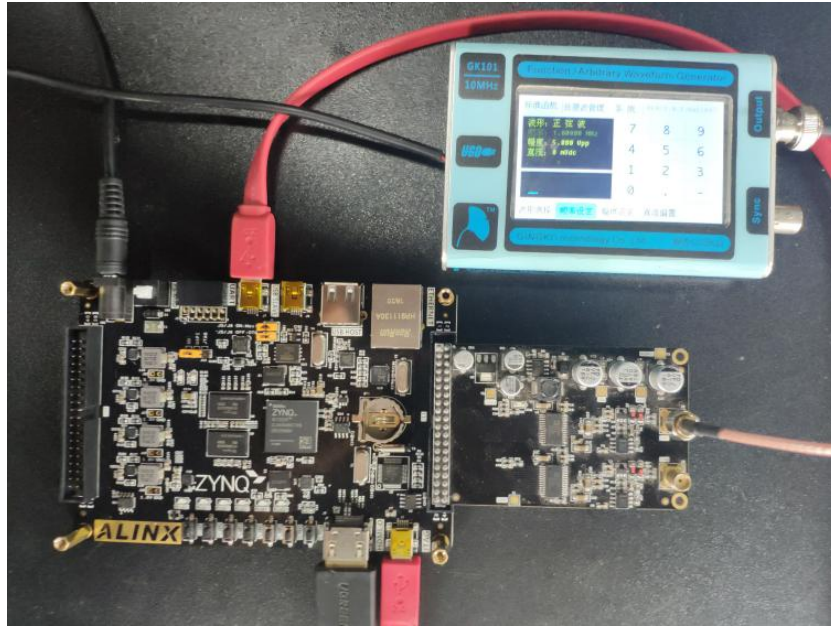
- 2) 程序没有使用 QT，而是直接绘制 framebuffer，frame_buffer.c 文件为 Linux 的 fb 操作。
- 3) adc_capture.c 为波形采集模块，主要是打开 ADC 驱动
- 4) 波形绘制函数完全来自裸机 Vitis

16.5 运行结果

- 1) 资料包中给出了编译好的程序在 sd_boot 目录，所有文件复制到 sd 卡根目录运行即可。
- 2) 给 AN9238 第一通道输入峰峰值为 5V，频率 1Mhz 的正弦波，波形显示如下。



AX7021 硬件连接图 (J15 扩展口)



AX7020/AX7010 硬件连接图 (J11 扩展口)

16.6 常见问题

16.6.1 内核崩溃

启动过程中有一定几率内核崩溃，原因未知，需要重启开发板

```

task: c0a06bc0 task.stack: c0a00000
PC is at 0xffffffff
LR is at rcu_process_callbacks+0x26c/0x41c
pc : [<fffffffe>] lr : [<c015e240>] psr: a00e0133
sp : c0a01e70 ip : ddc9fc5c fp : c09421c0
r10: c09421c0 r9 : 00000000 r8 : 00000000
r7 : debccfe0 r6 : c0a0cac0 r5 : c0a0cac0 r4 : debccfc0
r3 : 00000000 r2 : c0a00000 r1 : ffffffff r0 : ddc9fc5c
Flags: NzCv IRQs on FIQs on Mode SVC_32 ISA Thumb Segment none
Control: 18c5387d Table: 1dc6004a DAC: 00000051
Process swapper/0 (pid: 0, stack limit = 0xc0a00210)
Stack: (0xc0a01e70 to 0xc0a02000)
1e60: 00000000 debcc9c0 00000000 ddc9fc5c
1e80: 0000000a c0a0302c 00000000 c0a00000 00000003 c0a00000 40000001 00000002
1ea0: c0a32bc0 0000000a c0a020a4 00000101 c0a01eb8 c011eab0 debca768 00000001
1ec0: 00200100 ffff8b80 00000002 c0a02d00 c0a020a0 00000009 00000000 00000000
1ee0: 00000000 c093fd90 00000012 de402600 c0a00000 00000000 00000000 c011ed48
1f00: 00000000 c0154114 f8f00100 c0a03b1c c0a01f38 c0a13e00 f8f01100 c01013f8
1f20: c04b2e14 600e0013 ffffffff c0a01f6c 6962c202 c010ae8c 00000000 00000000
1f40: 1e28b000 debcc9c0 69fa3377 00000000 debcbe40 00000000 6962c202 00000000
1f60: 00000000 00000000 0000000b c0a01f88 c04b2df0 c04b2e14 600e0013 ffffffff
1f80: 00000051 00000000 debcbe40 c0a28e3c 00000000 debcbe40 c0a28e3c c0a01fb8
1fa0: c0a00000 c0a0302c 00000001 c0a0307c 00000000 c014c834 c0a03000 c0900bb4
1fc0: ffffffff ffffffff 00000000 c0900670 00000000 c0931a30 00000000 c0a324d4
1fe0: c0a03018 c0931a2c c0a07db8 0000406a 413fc090 0000807c 00000000 00000000
[<c015e240>] (rcu_process_callbacks) from [<c011eab0>] (__do_softirq+0x15c/0x1bc)
[<c011eab0>] (__do_softirq) from [<c011ed48>] (irq_exit+0x68/0xd0)
[<c011ed48>] (irq_exit) from [<c0154114>] (__handle_domain_irq+0x80/0xb4)
[<c0154114>] (__handle_domain_irq) from [<c01013f8>] (gic_handle_irq+0x50/0x94)
[<c01013f8>] (gic_handle_irq) from [<c010ae8c>] (__irq_svc+0x6c/0xa8)
Exception stack(0xc0a01f38 to 0xc0a01f80)
1f20: 00000000 00000000
1f40: 1e28b000 debcc9c0 69fa3377 00000000 debcbe40 00000000 6962c202 00000000
1f60: 00000000 00000000 0000000b c0a01f88 c04b2df0 c04b2e14 600e0013 ffffffff
[<c010ae8c>] (__irq_svc) from [<c04b2e14>] (cpuidle_enter_state+0xe4/0x1b0)
[<c04b2e14>] (cpuidle_enter_state) from [<c014c834>] (cpu_startup_entry+0x17c/0x19c)
[<c014c834>] (cpu_startup_entry) from [<c0900bb4>] (start_kernel+0x2f8/0x358)
Code: bad PC value
---[ end trace 2fddbfbf414e7a837 ]---
Kernel panic - not syncing: Fatal exception in interrupt
CPU1: stopping
CPU: 1 PID: 0 Comm: swapper/1 Tainted: G D 4.9.0-xilinx #1
Hardware name: Xilinx Zynq Platform
[<c010e244>] (unwind_backtrace) from [<c010a4a4>] (show_stack+0x10/0x14)
[<c010a4a4>] (show_stack) from [<c02dc2d0>] (dump_stack+0x80/0x9c)
[<c02dc2d0>] (dump_stack) from [<c010c9f8>] (ipi_cpu_stop+0x3c/0x70)
[<c010c9f8>] (ipi_cpu_stop) from [<c010d210>] (handle_IPI+0x64/0x84)
[<c010d210>] (handle_IPI) from [<c0101420>] (gic_handle_irq+0x78/0x94)
[<c0101420>] (gic_handle_irq) from [<c010ae8c>] (__irq_svc+0x6c/0xa8)
Exception stack(0xde471f70 to 0xde471fb8)
1f60: 00000000 00000000 1e299000 debda9c0
1f80: 792b8fc8 00000000 debd9e40 00000000 78e2382a 00000000 00000000 00000000
1fa0: 0000000b de471fc0 c04b2df0 c04b2e14 60070013 ffffffff
[<c010ae8c>] (__irq_svc) from [<c04b2e14>] (cpuidle_enter_state+0xe4/0x1b0)
[<c04b2e14>] (cpuidle_enter_state) from [<c014c834>] (cpu_startup_entry+0x17c/0x19c)
[<c014c834>] (cpu_startup_entry) from [<001014cc>] (0x1014cc)
---[ end Kernel panic - not syncing: Fatal exception in interrupt
random: fast init done

```

第十七章 基于 DMA 的 ADC 波形显示

(AN706)

例程相关文件夹: an706_wave

Vivado 工程在 vivado.zip 压缩包中: an706_wave

在 Vitis 裸机例程里我们用 DMA 将 ADC 模块 AN706 的数据采集到 ddr 中然后显示出波形, 在上面的例程用我们也学习了 DMA 的读写测试, 我们再这个读写测试的基础上写了一个 ADC 驱动, 所以请大家在学习本章内容时请熟练掌握前面的例程。

例程资料中给出了 vivado 工程, petalinux 工程, 以及波形显示 app。

本例程和前面例程非常相似, 区别在于只使用一个 DMA 传输 8 个通道的 ADC 数据, 在应用层将 8 个通道的数据分离。

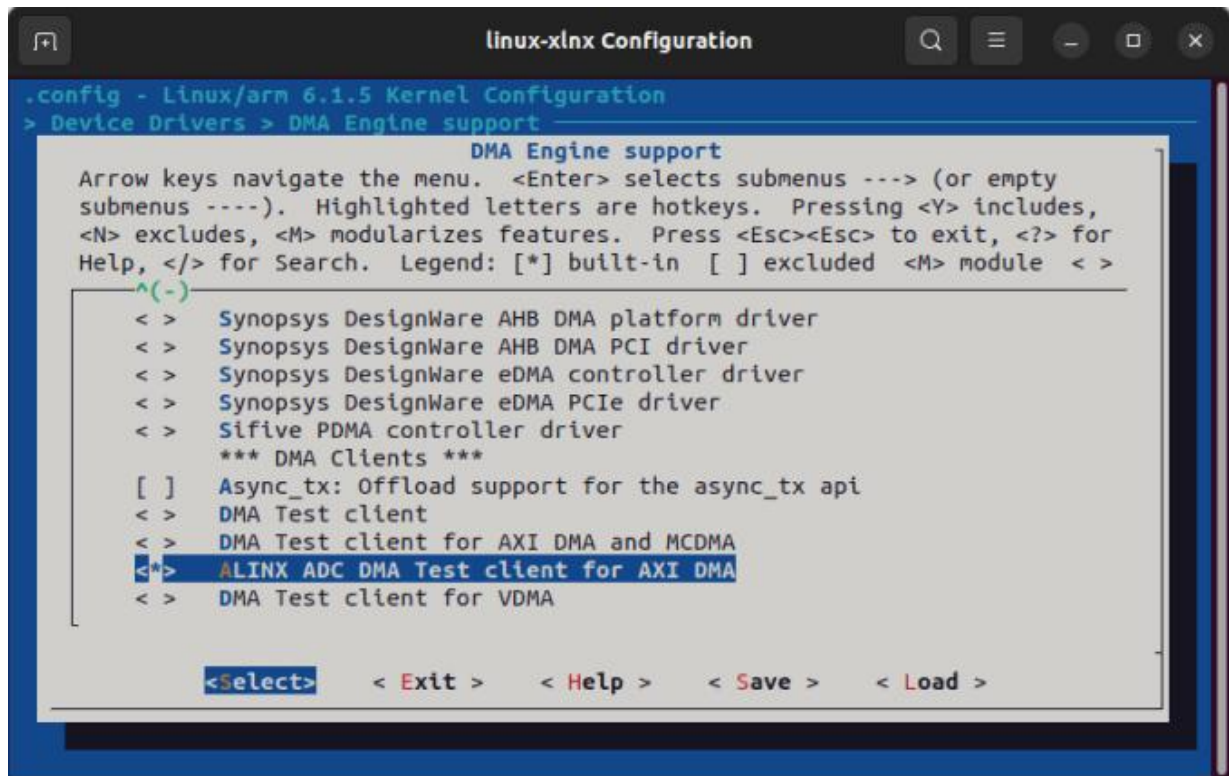
17.1 ADC 驱动

- 1) ADC 驱动是内核 drivers/dma/xilinx 目录中的 axi_adc_dma.c, 驱动编写时参考了同目录下的 axidmatest.c 文件和 Xilinx 官方例程 “xapp1183”。

17.2 内核驱动配置

内核驱动中涉及到 drm 显示的部分请参考第八章 HDMI 显示的内核配置, 可以在 HDMI 显示的工程基础上添加下列配置, 也可以用 BSP 包重新生成工程。

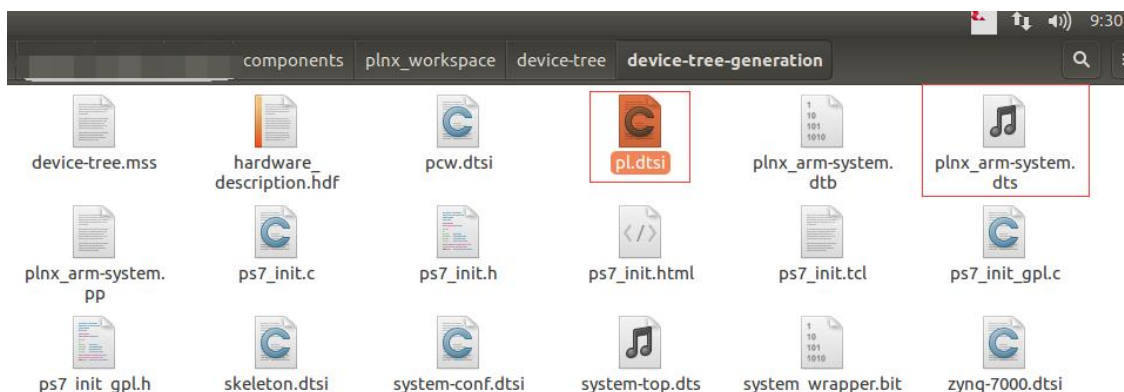
- 1) 和前面的例程一样要配置一些外设的驱动, 这里不再复述, 主要是配置 ADC 的驱动, 在 Device Drivers > DMA Engine support > Xilinx DMA Engines 选项中选择<*> ALINX ADC DMA Test client for AXI DMA



17.3 设备树配置

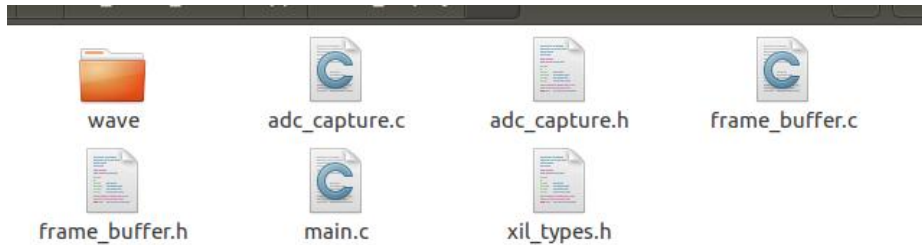
- 1) 这里是修改了驱动的匹配选项, compatible = "alinx,axi-adc-dma";用于匹配驱动程序。详细设备树可以参考例程 petalinux 工程目录下
project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

- 2) 设备树修改技巧, 先编译一次, 然后到
components/plnx_workspace/device-tree/device-tree-generation 目录下找到 pl.dtsi 这里包含了所有 PL 端外设的设备树信息, 最好不要直接修改这个文件, plnx_arm-system.dts 是最后生成的文件, 也是非常重要的信息, 开发者可以自己好好研究一下这些文件。



17.4 应用程序

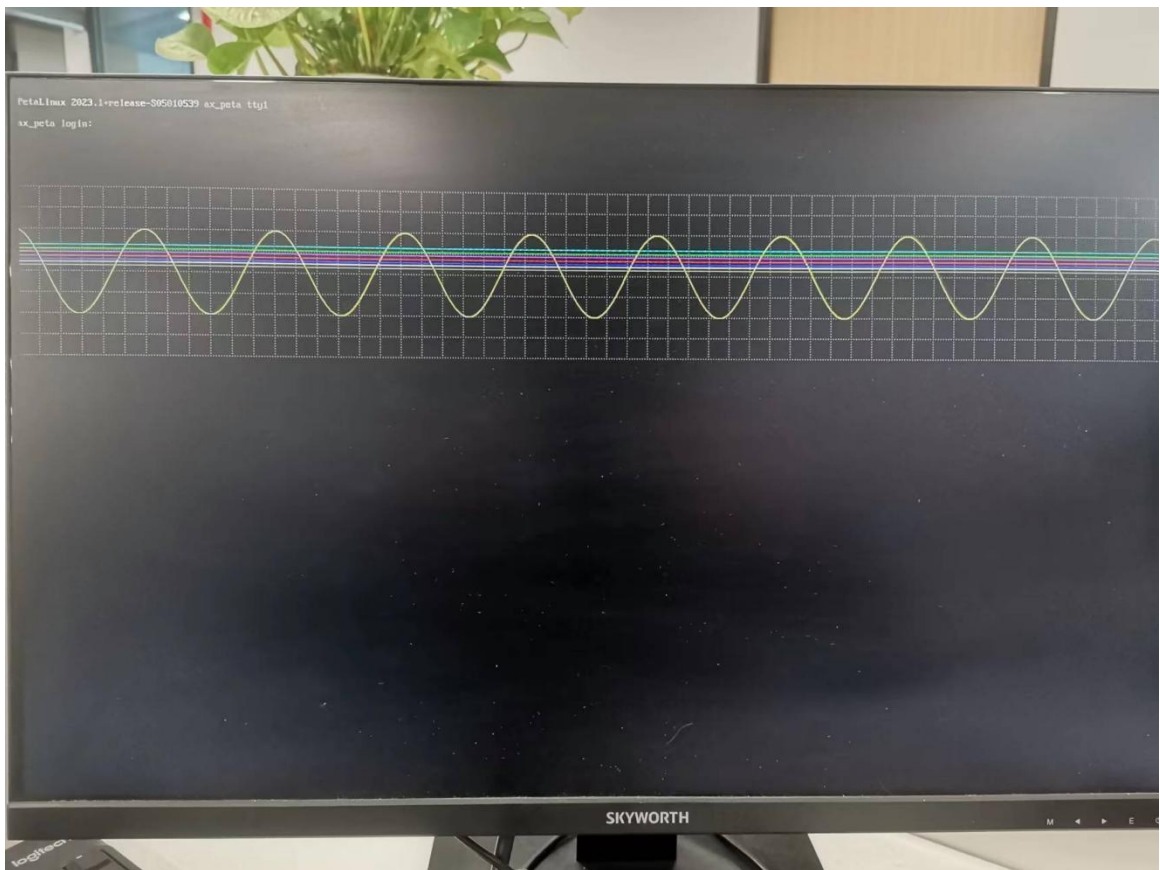
- 1) 应用程序采用 Vitis 编写，Vitis 建立 Linux 应用程序的方法在前面的教程中讲到。

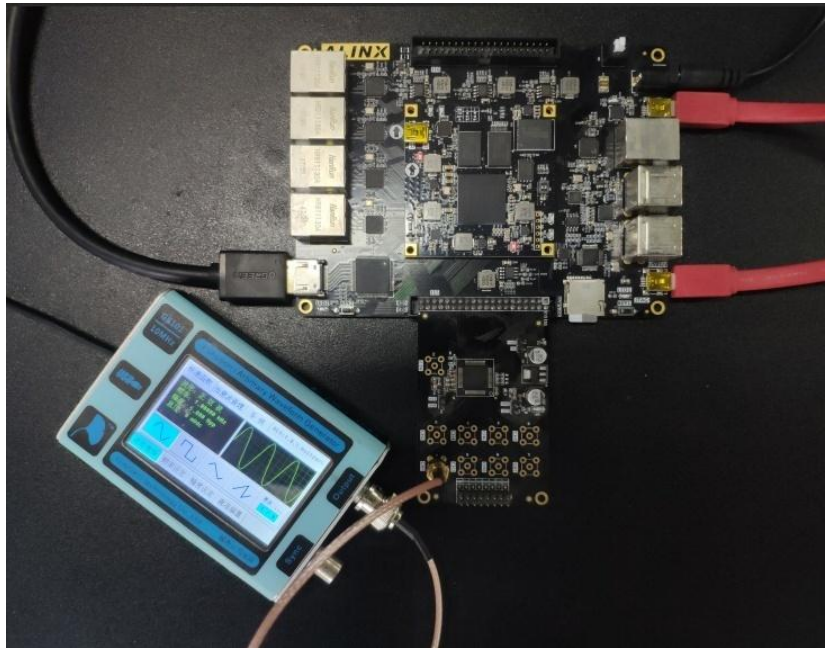


- 2) 程序没有使用 QT，而是直接绘制 framebuffer，frame_buffer.c 文件为 Linux 的 fb 操作。
- 3) adc_capture.c 为波形采集模块，主要是打开 ADC 驱动
- 4) 波形绘制函数完全来自裸机 Vitis

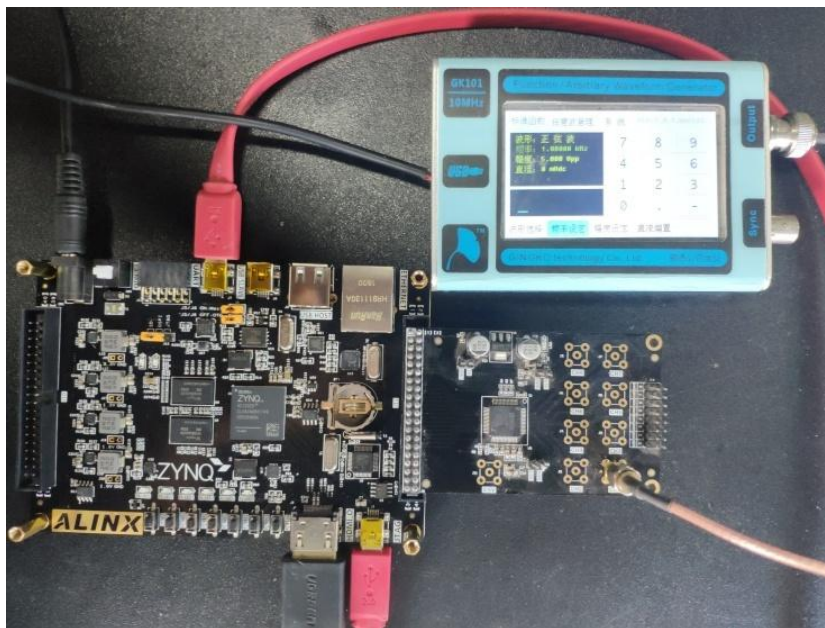
17.5 运行结果

- 1) 资料包中给出了编译好的程序在 sd_boot 目录，所有文件复制到 sd 卡根目录运行即可。
- 2) 给 AN706 第一通道输入峰峰值为 5V，频率 1khz 的正弦波，波形显示如下。





AX7021 硬件连接图 (J15 扩展口)



AX7020/AX7010 硬件连接图 (J11 扩展口)

第十八章 自动运行的 petalinux 应用程序

前面的教程中我们使用了大量 petalinux 来创建 Linux 应用，而且很多时候把根文件系统选为 ramdisk 类型，不能保存我们配置数据，所以我们需要一个上电能自动运行程序的 shell 文件。

例程中给了一个做好的应用程序，可以完成上电自动挂载 sd 卡第一分区到/sd，然后运行/sd 下的 startup.sh 文件。

18.1 准备工作

在 xilinx 提供的 petalinux 资料 [UG1144](#) 中，有个章节“Creating and Adding Custom Applications”，将如何创建一个自定义应用程序，通过下面命令来创建

```
$ petalinux-create -t apps [--template TYPE] --name --enable
```

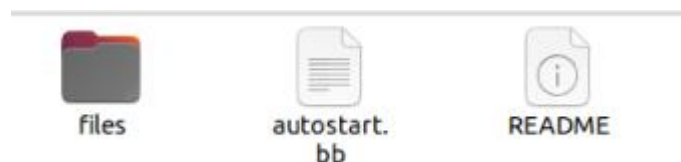
对于专业的软件开发人员来说，学习起来可能比较简单，所以这里不再描述细节，我们把已经做好的 app 提供给大家。

18.2 autostart 应用程序

- 1) 创建好的应用程序都会在文件夹 project-spec/meta-user/recipes-apps 下出现，我们也可以直接把 app 复制到这个目录下



- 2) startup 文件夹下有个 autostart.bb 文件



- 3) 用文本工具打开，do_install 函数是安装应用的 shell，我们可以大致猜测，安装过程会把 files 目录中的 autostart.sh 文件复制到根文件系统的 init.d 文件夹下，这样就可以上电自动运行这个文件。

```
1 #
2 # This file is the autostart recipe.
3 #
4
5 SUMMARY = "Simple autostart application"
6 SECTION = "PETALINUX/apps"
7 LICENSE = "MIT"
8 LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
9
10 SRC_URI = "file://autostart \
11            "
12
13 S = "${WORKDIR}"
14
15 inherit update-rc.d
16
17 INITSCRIPT_NAME = "autostart"
18 INITSCRIPT_PARAMS = "start 99 5 ."
19
20 do_install() {
21     install -d ${D}${sysconfdir}/init.d
22     install -m 0755 ${S}/autostart ${D}${sysconfdir}/init.d/autostart
23 }
24
25 RDEPENDS_${PN} += "bash"
```

- 4) files 目录中主要文件是 autostart.sh，它是系统启动后会自动运行的脚本。



- 5) autostart.sh 是先判断挂载在系统下的 SD 卡第一分区和第二分区 autostart.sh 文件是不是存在，如果存在就运行。所以只要我们在 sd 的分区任意一个放一个 startup.sh，就可以实现上电自动运行。

```
1 #!/bin/sh
2
3 echo "Hello PetaLinux World"
4 USERHOOK_SD0=/media/sd-mmcblk0p1/autostart.sh
5 USERHOOK_SD1=/media/sd-mmcblk1p1/autostart.sh
6 if [ -f $USERHOOK_SD0 ]; then
7     sh $USERHOOK_SD0 &
8 fi
9 if [ -f $USERHOOK_SD1 ]; then
10    sh $USERHOOK_SD1 &
11 fi
```

第十九章 附录： Linux 常用命令

19.1 文件和文件夹操作

19.1.1 创建一个文件夹

```
mkdir dir
```

19.1.2 创建多层目录的文件夹

```
mkdir -p dir1/dir2
```

19.1.3 改变当前目录

```
cd newdir  进入 newdir 目录
cd ..      进入上层目录
cd -       进入上一次的路径
cd         进入 home 目录
```

19.1.4 复制文件

```
cp source_file dest_file  复制 source_file 为 dest_file
cp file1 file2 dir        复制 file1 和 file2 到文件夹 dir
cp -r source_dir dest_dir 保留属性复制，等同于 rsync -a source_dir/ dest_dir/
ln -s linked_file link    创建一个软连接
```

19.1.5 文件重命名

```
mv source_file dest_file  source_file 重命名为 dest_file
```

19.1.6 删除文件和文件夹

```
rm file1 file2  删除文件或者文件链接
rmdir dir       删除一个空文件夹
rm -rf dir      删除一个非空文件夹
```

19.2 文件列表

ls 最为常用，显示当前文件夹下的所有文件，不包含隐藏的文件

ls -l 显示更多的文件信息

ls -a 显示包含隐藏文件的所有文件

19.3 压缩解压

19.3.1 创建一个压缩包

```
tar zcvf archive.tar.gz dir/
```

19.3.2 解压一个 tar.gz 文件

```
tar zxvf archive.tar.gz
```

19.4 系统管理

ifconfig -a 显示所有可用网络接口

ping 192.168.1.1 测试网络和另外的主机连通性